

งานวิจัย (บทความวิจัย) 4091606

คณิตศาสตร์สำหรับคอมพิวเตอร์ (Mathematics for Computer)

ผู้ช่วยศาสตราจารย์ ดร.พิศิษฐ์ นาคใจ

คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏอุตรดิตถ์

2568

สารบัญ

	หน้า
สารบัญ	(2)
สารบัญตาราง	(7)
สารบัญภาพ	(8)
แผนบริหารการสอนประจำวิชา	1
แผนการสอนประจำบทที่ 1	7
1 ตรรกศาสตร์ (Logic)	9
1.1 บทนำ: ทำไมตรรกศาสตร์จึงสำคัญ	9
1.2 ประพจน์ (Propositions)	9
1.3 ตัวเชื่อมทางตรรกศาสตร์ (Logical Connectives)	13
1.4 ตารางค่าความจริง (Truth Tables)	28
1.5 สมมูลทางตรรกศาสตร์ (Logical Equivalence)	33
1.6 ตรรกศาสตร์เชิงประพจน์ (Propositional Logic)	40
1.7 ประโยคเปิดและตัวบ่งปริมาณ (Open Sentences and Quantifiers)	44
1.8 ประพจน์ที่มีค่าความจริงพิเศษ (Special Truth Values)	50
1.9 การอ้างเหตุผลและกฎของการอนุมาน (Rules of Inference)	53
1.10 แบบฝึกหัดท้ายบท	60
1.11 เฉลยแนวคิด (Hints and Answers)	67
เฉลยแนวคิด	67
บรรณานุกรมและอ่านเพิ่มเติม	68
1.12 เอกสารอ้างอิง	69
แผนการสอนประจำบทที่ 2	71
2 ทฤษฎีเซต (Set Theory)	73
2.1 วัตถุประสงค์การเรียนรู้	73

สารบัญ (ต่อ)

	หน้า
2.2 บทนำ: ความเป็นมาและความสำคัญของทฤษฎีเซต	73
2.3 นิยามพื้นฐานของเซต	74
2.4 ความสัมพันธ์ระหว่างเซต	78
2.5 การดำเนินการระหว่างเซต (Set Operations)	79
2.6 กฎของเดมอร์แกน (De Morgan's Laws)	82
2.7 การนับจำนวนสมาชิกของเซต (Cardinality)	85
2.8 การประยุกต์ในวิทยาการคอมพิวเตอร์	87
2.9 แบบฝึกหัด	90
2.10 เฉลยแบบฝึกหัด	94
2.11 เซตที่นับได้และเซตที่นับไม่ได้	95
2.12 สรุป	96
2.13 เอกสารอ้างอิง	98
แผนการสอนประจำบทที่ 3	99
3 ความสัมพันธ์ (Relations)	101
3.1 วัตถุประสงค์การเรียนรู้	101
3.2 บทนำ	101
3.3 นิยามพื้นฐานของความสัมพันธ์	102
3.4 คุณสมบัติของความสัมพันธ์แบบทวิภาค	104
3.5 ความสัมพันธ์สมมูล (Equivalence Relations)	110
3.6 ความสัมพันธ์สมมาตรเชิงส่วนย่อย (Partial Ordering Relations)	113
3.7 การดำเนินการบนความสัมพันธ์	116
3.8 การปิดคุณสมบัติ (Closure Properties)	118
3.9 การประยุกต์ใช้ความสัมพันธ์ในวิทยาการคอมพิวเตอร์	120
3.10 ตัวอย่างและแบบฝึกหัดท้ายบท	123
3.11 สรุป	124

สารบัญ (ต่อ)

	หน้า
3.12 แบบฝึกหัดท้ายบท	125
3.13 เอกสารอ้างอิง	126
3.14 เอกสารอ้างอิง	127
แผนการสอนประจำบทที่ 4	129
4 ระบบเลขฐาน (Number Bases)	131
4.1 วัตถุประสงค์การเรียนรู้	131
4.2 บทนำ	131
4.3 ระบบเลขฐานสิบ (Decimal System)	131
4.4 ระบบเลขฐานสอง (Binary System)	132
4.5 ระบบเลขฐานแปด (Octal System)	133
4.6 ระบบเลขฐานสิบหก (Hexadecimal System)	134
4.7 การแปลงฐานทั่วไป	135
4.8 การแปลงจำนวนทศนิยมระหว่างฐาน	137
4.9 การบวกและลบเลขฐานสอง	138
4.10 การคูณและการหารเลขฐานสอง	140
4.11 การแทนจำนวนเต็มลบในฐานสอง	141
4.12 การบวกและลบในฐานสิบหก	143
4.13 IEEE 754: การแทนทศนิยมในคอมพิวเตอร์	144
4.14 การแทนข้อมูลในคอมพิวเตอร์	145
4.15 แบบฝึกหัด	147
4.16 เฉลยแบบฝึกหัด	149
4.17 สรุป	149
4.18 เอกสารอ้างอิง	151
แผนการสอนประจำบทที่ 5	153
5 เมทริกซ์และดีเทอร์มิแนนต์ (Matrices & Determinants)	155

สารบัญ (ต่อ)

	หน้า
5.1 วัตถุประสงค์การเรียนรู้	155
5.2 บทนำ	155
5.3 นิยามพื้นฐานของเมทริกซ์	156
5.4 การดำเนินการระหว่างเมทริกซ์	158
5.5 ดีเทอร์มิแนนต์ (Determinants)	163
5.6 อินเวอร์สของเมทริกซ์ (Matrix Inverse)	166
5.7 ระบบสมการเชิงเส้น (Systems of Linear Equations)	168
5.8 การประยุกต์ในวิทยาการคอมพิวเตอร์	171
5.9 สรุป	177
5.10 เอกสารอ้างอิง	179
แผนการสอนประจำบทที่ 6	181
6 พีชคณิตบูลีน (Boolean Algebra)	183
6.1 วัตถุประสงค์การเรียนรู้	183
6.2 บทนำ	183
6.3 นิยามพื้นฐานของพีชคณิตบูลีน	184
6.4 การดำเนินการทางบูลีน (Boolean Operations)	185
6.5 กฎของพีชคณิตบูลีน (Boolean Identities)	188
6.6 กฎของเดมอร์แกน (De Morgan's Laws)	190
6.7 นิพจน์บูลีนและฟังก์ชันบูลีน (Boolean Expressions and Functions)	191
6.8 รูปแบบปกติ (Canonical Forms)	193
6.9 การลดรูปนิพจน์บูลีน (Boolean Expression Simplification)	195
6.10 แผนที่คาร์นอ (Karnaugh Map)	197
6.11 การประยุกต์ในวงจรดิจิทัล	200
6.12 Two-level Realization	202
6.13 Half Adder และ Full Adder	202

สารบัญ (ต่อ)

	หน้า
6.14 สมมูลของพีชคณิตบูลีน (Boolean Algebra Equivalence)	203
6.15 แบบฝึกหัด	204
6.16 เฉลยแบบฝึกหัด	207
6.17 ตัวอย่างในวิทยาการคอมพิวเตอร์	208
6.18 สรุป	211
6.19 เอกสารอ้างอิง	213

สารบัญตาราง

ตารางที่	หน้า
1 ตารางความจริงของ NOT Gate	14
2 ตารางความจริงของ AND Gate	15
3 ตารางความจริงของ OR Gate	17
4 ตารางความจริงของ XOR Gate	19
5 ตารางความจริงของ Half Adder	20
6 ตารางความจริงของ Implication	22
7 ตารางความจริงของ Biconditional	25
8 ตารางค่าความจริงสำหรับแบบฝึกหัดข้อ (ข)	63
9 ความสัมพันธ์ระหว่าง Relational Algebra และ Set Theory	88
10 ประสิทธิภาพของ Hash Set Operations	90
11 สรุปคุณสมบัติของความสัมพันธ์บางชนิด	109
12 ตารางเปรียบเทียบฐานสิบ ฐานสอง และฐานสิบหก	134
13 ตาราง ASCII Code บางส่วน	146

สารบัญภาพ

ภาพที่	
1	ตัวอย่างโครงสร้างตารางค่าความจริงสำหรับ 2 ตัวแปร

หน้า

28

แผนการสอนประจำวิชา

รหัสวิชา 4091606

ชื่อวิชา คณิตศาสตร์สำหรับคอมพิวเตอร์ (Mathematics for Computer)

จำนวนหน่วยกิต 3(3-0-6)

คณาจารย์ผู้สอน

ผู้ช่วยศาสตราจารย์ ดร.พัชรี มณีรัตน์ หลักสูตรสาขาวิชาวิทยาการข้อมูล ห้อง STA110

E-mail: m.patcharee@uru.ac.th

ผู้ช่วยศาสตราจารย์ ดร.พิศิษฐ์ นาคใจ หลักสูตรสาขาวิชาวิทยาการข้อมูล ห้อง STA314

E-mail: pisit.nak@uru.ac.th

คำอธิบายรายวิชา

ความรู้พื้นฐานเกี่ยวกับตรรกศาสตร์ เซต ความสัมพันธ์และฟังก์ชัน ระบบเลขฐานต่าง ๆ โดยเฉพาะเลขฐาน 2, 8, 16 เมทริกซ์และดีเทอร์มิแนนต์ พีชคณิตบูลีน

จุดประสงค์การเรียนรู้

1. นักศึกษาสามารถอธิบายถึงความรู้พื้นฐานเกี่ยวกับตรรกศาสตร์ได้
2. นักศึกษามีความรู้ความเข้าใจในคุณลักษณะของเซต พร้อมทั้งความสัมพันธ์และฟังก์ชันได้
3. นักศึกษาสามารถแปลงและคำนวณในระบบเลขฐานต่าง ๆ โดยเฉพาะเลขฐาน 2, 8, 16 ได้
4. นักศึกษามีความรู้ความเข้าใจในเมทริกซ์และดีเทอร์มิแนนต์ได้
5. นักศึกษามีความรู้ความเข้าใจในพีชคณิตบูลีน ซึ่งเป็นเครื่องมือที่ทำให้เข้าใจการทำงานทางตรรกวิทยาภายในเครื่องคอมพิวเตอร์ได้

โครงการสอน

วิชาคณิตศาสตร์สำหรับคอมพิวเตอร์มีจำนวน 3 หน่วยกิต ใช้เวลาเรียน 3 ชั่วโมง/สัปดาห์ มีเวลาเรียนทั้งสิ้น 17 สัปดาห์ สามารถทำเป็นโครงการสอนได้ดังนี้

สัปดาห์ที่	หัวข้อ/รายละเอียด	จำนวนชั่วโมง	กิจกรรมการเรียนรู้/สื่อที่ใช้	ผู้สอน
1-2	บทที่ 1 ความรู้พื้นฐานเกี่ยวกับ ตรรกศาสตร์	6	บรรยาย สรุปบทเรียน อภิปราย ชักถาม และ แลกเปลี่ยนเรียนรู้ การตอบคำถามท้ายบท	ผศ.ดร.พัชรีย์ มณีรัตน์
3	บทที่ 2 เซต	6	บรรยาย สรุปบทเรียน อภิปราย ชักถาม และ แลกเปลี่ยนเรียนรู้ ศึกษาเอกสารประกอบการสอน การตอบคำถามท้ายบท	ผศ.ดร.พัชรีย์ มณีรัตน์
4-7	บทที่ 3 ความสัมพันธ์และฟังก์ชัน	6	บรรยาย สรุปบทเรียน อภิปราย ชักถาม และ แลกเปลี่ยนเรียนรู้ ศึกษาเอกสารประกอบการสอน การตอบคำถามท้ายบท	ผศ.ดร.พัชรีย์ มณีรัตน์
8	สอบกลางภาคเรียน			
9-11	บทที่ 4 ระบบเลขฐานต่าง ๆ โดย เฉพาะเลขฐาน 2, 8, 16	9	บรรยาย สรุปบทเรียน อภิปราย ชักถาม และ แลกเปลี่ยนเรียนรู้ ศึกษาเอกสารประกอบการสอน การตอบคำถามท้ายบท	ผศ.ดร.พิศิษฐ์ นาคใจ
12-13	บทที่ 5 เมทริกซ์ และ ดีเทอร์มิ แนนต์	9	บรรยาย สรุปบทเรียน อภิปราย ชักถาม และ แลกเปลี่ยนเรียนรู้ ศึกษาเอกสารประกอบการสอน การตอบคำถามท้ายบท	ผศ.ดร.พิศิษฐ์ นาคใจ

สัปดาห์ที่	หัวข้อ/รายละเอียด	จำนวนชั่วโมง	กิจกรรมการเรียนรู้/สื่อที่ใช้	ผู้สอน
14-16	บทที่ 6 พืชชนิดบุนนาค	6	บรรยาย สรุปบทเรียน อภิปราย ซักถาม และ แลกเปลี่ยนเรียนรู้ ศึกษาเอกสารประกอบการสอน การตอบคำถามท้ายบท	ผศ.ดร.พิศิษฐ์ นาคใจ
17	สอบปลายภาคเรียน			

วิธีสอนและกิจกรรมการเรียนรู้

1. วิธีสอน

1.1. วิธีสอนส่วนใหญ่ใช้วิธีการบรรยายประกอบเอกสารประกอบการสอน และจัดกิจกรรมให้นักศึกษาอภิปราย ชักถาม และแลกเปลี่ยนเรียนรู้ในชั้นเรียน

2. กิจกรรมการเรียนรู้

2.1. ให้นักศึกษาตอบคำถามประจำบทในเอกสารประกอบการสอน

2.2. ให้นักศึกษาศึกษาเอกสารประกอบการสอนเพิ่มเติมตามหัวข้อที่กำหนด

2.3. ให้นักศึกษาร่วมอภิปราย ชักถาม และแลกเปลี่ยนเรียนรู้ในชั้นเรียน

สื่อการเรียนรู้

1. เอกสารประกอบการสอน ตำรา และบทความวิจัยที่เกี่ยวข้อง
2. สื่ออิเล็กทรอนิกส์ เช่น เว็บไซต์และแหล่งเรียนรู้ออนไลน์ต่าง ๆ
3. คำถามประจำบท

การวัดและการประเมินผล

- | | |
|---------------------------------------|-----|
| 1. คะแนนระหว่างภาคเรียน | 70% |
| 1.1. การตรงต่อเวลาและความซื่อสัตย์ | 10% |
| 1.2. งานที่ได้รับมอบหมาย (Assignment) | 10% |
| 1.3. กิจกรรมกลุ่มและการนำเสนอ | 20% |
| 1.4. ทดสอบกลางภาคเรียน | 30% |
| 2. ทดสอบปลายภาคเรียน | 30% |

ตำรา เอกสาร และงานวิจัยประกอบการเรียนการสอน

1. เอกสารหลัก

คณะกรรมการโครงการตำราวิทยาศาสตร์และคณิตศาสตร์มูลนิธิ สอวน. (2556). *คณิตศาสตร์พื้นฐานสำหรับคอมพิวเตอร์*. พิมพ์ครั้งที่ 4. กรุงเทพฯ: บริษัท ด้านสุทธาการพิมพ์ จำกัด.

2. เอกสารเพิ่มเติม

2.1. คณะ กรรมการโครงการ ตำรา วิทยาศาสตร์ และ คณิตศาสตร์ มูลนิธิ สอวน. (2553). *คณิตศาสตร์พื้นฐานสำหรับคอมพิวเตอร์*. พิมพ์ครั้งที่ 3. กรุงเทพฯ: บริษัท ด้านสุทธาการพิมพ์ จำกัด.

คะแนน	เกรด	ความหมาย
80-100	A	ดีเยี่ยม
75-79	B ⁺	ดีมาก
70-74	B	ดี
65-69	C ⁺	ดีพอใช้
60-64	C	พอใช้
55-59	D ⁺	อ่อน
50-54	D	อ่อนมาก
0-49	F	ตก

2.2. อุ๋นใจ ลิมตระกุล. (2538). *คณิตศาสตร์คอมพิวเตอร์*. พิมพ์ครั้งที่ 1. กรุงเทพฯ: สำนักพิมพ์ภูมิบัณฑิต.

ลงชื่อผู้ช่วยศาสตราจารย์ ดร.พัชรื มณีนรตัน..... อาจารย์ผู้สอน

ลงชื่อผู้ช่วยศาสตราจารย์ ดร.พิศิษฐ์ นาคใจ..... อาจารย์ผู้สอน

แผนการสอนประจำบทที่ 1

รหัสวิชา 4121701

ชื่อวิชา คณิตศาสตร์สำหรับคอมพิวเตอร์

หน่วยกิต 3 (3-0-6)

บทที่ 1

ชื่อบท ตรรกศาสตร์ (Logic)

จำนวนชั่วโมง 6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- อธิบายความหมายและคุณสมบัติของประพจน์ (Proposition) และค่าความจริงได้
- ใช้ตัวเชื่อมทางตรรกศาสตร์ทั้ง 6 ตัว ได้แก่ $\neg$, $\wedge$, $\vee$, $\oplus$, $\rightarrow$, $\leftrightarrow$ ได้อย่างถูกต้อง
- สร้างและวิเคราะห์ตารางค่าความจริง (Truth Table) ของประพจน์เชิงประกอบได้
- ตรวจสอบสมมูลทางตรรกศาสตร์ (Logical Equivalence) และกฎพีชคณิตบูลีนได้
- อธิบายและประยุกต์ใช้การอนุมาน (Inference Rules) เช่น Modus Ponens, Modus Tollens ได้
- เชื่อมโยงหลักการตรรกศาสตร์กับการเขียนโปรแกรมและวงจรดิจิทัลได้

หัวข้อที่สอน

- บทนำ: ทำไมตรรกศาสตร์จึงสำคัญ
- ประพจน์ (Propositions)
- ตัวเชื่อมทางตรรกศาสตร์ (Logical Connectives)
- ตารางค่าความจริง (Truth Tables)
- สมมูลทางตรรกศาสตร์ (Logical Equivalence)

6. กฎการอนุมาน (Rules of Inference)

สื่อการสอน

- สไลด์ประกอบการสอน
- ตัวอย่างโปรแกรมภาษา Python / C
- แบบฝึกหัดท้ายบท

ตรรกศาสตร์ (Logic)

1.1 บทนำ: ทำไมตรรกศาสตร์จึงสำคัญ

1.1.1 ความเป็นมาของตรรกศาสตร์

ตรรกศาสตร์ (Logic) คือศาสตร์แห่งการให้เหตุผล ซึ่งมีประวัติศาสตร์ยาวนานนับพันปี เริ่มต้นจากปรัชญากรีกโบราณ โดยอริสโตเติล (384-322 ปีก่อนคริสตกาล) ได้ริเริ่มศึกษารูปแบบของการให้เหตุผลที่ถูกต้องและบันทึกไว้ในงาน *Organon* ซึ่งประกอบด้วยหนังสือหกเล่ม ในยุคกลาง นักปรัชญาอิสลามอย่าง Al-Farabi และ Ibn Sina ได้ขยายความคิดตรรกศาสตร์ของอริสโตเติล และต่อมา Leibniz (1646-1716) และ George Boole (1815-1864) พัฒนาตรรกศาสตร์เชิงสัญลักษณ์ (Symbolic Logic) วางรากฐานให้คณิตศาสตร์สมัยใหม่

ในศตวรรษที่ 20 ตรรกศาสตร์พัฒนามหาศาล โดยเฉพาะด้านวิทยาการคอมพิวเตอร์ Claude Shannon (1916-2001) แสดงให้เห็นว่าตรรกศาสตร์บูลีนใช้ในการออกแบบวงจรคอมพิวเตอร์ได้ การพัฒนาภาษาโปรแกรม ปัญญาประดิษฐ์ และทฤษฎีการคำนวณล้วนมีตรรกศาสตร์เป็นรากฐาน

1.1.2 ความสำคัญของตรรกศาสตร์ในยุคปัจจุบัน

1.2 ประพจน์ (Propositions)

วัตถุประสงค์การเรียนรู้:

- ✓ ทำความเข้าใจนิยามของประพจน์และค่าความจริง
- ✓ รู้จักประพจน์จริงและเท็จ
- ✓ แยกแยะระหว่างประพจน์กับข้อความที่ไม่เป็นประพจน์
- ✓ เข้าใจการประยุกต์ในวิทยาการคอมพิวเตอร์

ในปัจจุบัน ตรรกศาสตร์มีบทบาทสำคัญในหลากหลายสาขา โดยเฉพาะวิทยาการคอมพิวเตอร์ ถูกนำไปใช้ในการออกแบบและตรวจสอบความถูกต้องของฮาร์ดแวร์ การพัฒนาฐานข้อมูล การสร้างระบบผู้เชี่ยวชาญ การประมวลผลภาษาธรรมชาติ และการพิสูจน์ความถูกต้องของอัลกอริทึม

ตัวอย่างเช่น ในภาษาโปรแกรม คำสั่ง $\text{if } (x > 0)$ เป็นตัวอย่างการใช้ประพจน์ (Proposition) ใน

การควบคุมการทำงาน โดยประพจน์ $x > 0$ ให้ค่าความจริงเป็น True เมื่อ x มากกว่าศูนย์ และ False เมื่อ x น้อยกว่าหรือเท่ากับศูนย์ ในด้านปัญญาประดิษฐ์ ระบบใช้ Modus Ponens และการอนุมานเชิงตรรกะ เพื่อดึงข้อมูลใหม่จากความรู้ที่มี

1.2.1 นิยามและความหมายของประพจน์

ประพจน์ (Proposition) คือหน่วยพื้นฐานของการคิดเชิงตรรกะ หมายถึงข้อความที่มีค่าความจริงเป็นได้เพียงอย่างเดียว คือ **จริง (True, T)** หรือ **เท็จ (False, F)** โดยข้อความนั้นต้องมีความหมายที่ชัดเจน ไม่กำกวม ประพจน์จึงเป็นเครื่องมือจำลอง (Model) ข้อความในโลกจริงให้เป็นรูปแบบทางคณิตศาสตร์ที่ประมวลผลได้ ใช้ในการตัดสินใจของโปรแกรม การออกแบบวงจรดิจิทัล และการแทนความรู้ในปัญญาประดิษฐ์

นิยาม 1.1 (ประพจน์) **ประพจน์** คือ ข้อความ (statement) ที่มีค่าความจริง (truth value) เป็นได้เพียงอย่างเดียว คือ **จริง (True, T)** หรือ **เท็จ (False, F)** โดยข้อความนั้นต้องมีความหมายที่ชัดเจน และไม่กำกวม

1.2.2 ตัวอย่างประพจน์ในชีวิตจริง

ตัวอย่างประพจน์ในชีวิตจริง ได้แก่ “ $2 + 2 = 4$ ” เป็นประพจน์จริง (T), “กรุงเทพมหานครเป็นเมืองหลวงของประเทศไทย” เป็นประพจน์จริง (T), “5 เป็นจำนวนคู่” เป็นประพจน์เท็จ (F) เพราะ 5 หารด้วย 2 ไม่ลงตัว ($5/2 = 2.5$), และ “ $x + 1 = 5$ ” ไม่เป็นประพจน์ เพราะไม่สามารถระบุค่าความจริงได้จนกว่าจะทราบค่า x (เป็น Open Sentence)

ตัวอย่าง 1.2 พิจารณาข้อความต่อไปนี้:

- (ก) “ $2 + 2 = 4$ ” เป็นประพจน์ที่มีค่าความจริงเป็น **จริง**
- (ข) “กรุงเทพมหานครเป็นเมืองหลวงของประเทศไทย” เป็นประพจน์ที่มีค่าความจริงเป็น **จริง**
- (ค) “5 เป็นจำนวนคู่” เป็นประพจน์ที่มีค่าความจริงเป็น **เท็จ**
- (ง) “ $x + 1 = 5$ ” ไม่เป็นประพจน์ เพราะไม่สามารถระบุค่าความจริงได้จนกว่าจะทราบค่า x

1.2.3 ข้อความที่ไม่เป็นประพจน์

ข้อความที่ไม่เป็นประพจน์ ได้แก่ ประโยคคำถาม (“คุณชื่ออะไร?”), ประโยคอ้างเหตุผล (“โปรดปิดประตูด้วย”), ประโยคเปิด (“ $x > 3$ ” ซึ่งขึ้นกับค่า x), และประโยคที่ขัดแย้งในตัวเอง (“ประโยคนี้เป็นเท็จ” — Liar’s Paradox)

หมายเหตุ 1.3 ข้อความที่ไม่เป็นประพจน์ มีหลายประเภท เช่น:

1. **ประโยคคำถาม** เช่น “คุณชื่ออะไร?” — ไม่เป็นการกล่าวอ้างข้อเท็จจริง
2. **ประโยคอ้างเหตุผล** เช่น “โปรดปิดประตูด้วย” — เป็นการขอร้องหรือสั่ง ไม่ใช่ข้อเท็จจริง
3. **ประโยคเปิด (Open Sentence)** เช่น “ $x > 3$ ” ซึ่งค่าความจริงขึ้นกับค่าของ x
4. **ประโยคที่ขัดแย้งในตัวเอง** เช่น “ประโยคนี้เป็นเท็จ” — ไม่สามารถระบุค่าความจริงได้

1.2.4 การประยุกต์ในวิทยาการคอมพิวเตอร์

ประพจน์มีความสำคัญในการเขียนโปรแกรม ใน Python คำสั่ง `if (x > 0):` มีประพจน์ $x > 0$ ซึ่งให้ค่า `True` เมื่อ $x > 0$ และ `False` เมื่อ $x \leq 0$ ในภาษา C หรือ Java ตัวดำเนินการ `&&` (AND) และ `||` (OR) ใช้รวมประพจน์หลายตัว นอกจากนี้ในด้าน AI ระบบใช้ Modus Ponens เพื่ออนุมานข้อมูลใหม่จาก Knowledge Base

ในวงจรดิจิทัล ทุกวงจรสามารถแทนด้วยประพจน์ทางตรรกศาสตร์ และใช้ตารางความจริง (Truth Tables) ตรวจสอบการทำงานได้

ตัวอย่าง 1.4 (การประยุกต์ในวิทยาการคอมพิวเตอร์ — การตรวจสอบเงื่อนไข) ในภาษาโปรแกรมทุกภาษา เงื่อนไขการตัดสินใจต้องเป็นประพจน์ที่มีค่าความจริงชัดเจน:

```
// Python: proposition  $x > 0$  is True when  $x > 0$ , False when  $x \leq 0$ 
```

```
if (x > 0):
```

```
    print("Positive")
```

```
// C/Java: connective && combines multiple propositions
```

```
if (isLoggedIn && hasPermission) {
```

```
    executeAction();
```

```
}
```

เงื่อนไข `if (x > 0)` เป็นประพจน์ที่ให้ค่า `True` หรือ `False` อย่างชัดเจน และโปรแกรมจะทำงานตามเส้นทางที่กำหนดโดยอาศัยหลักการของตรรกศาสตร์

ตัวอย่าง 1.5 (AI — Knowledge Base Entailment) ในระบบปัญญาประดิษฐ์ ปัญหาหลายอย่างสามารถจำลองเป็นประพจน์ได้:

1. ให้ Knowledge Base (KB) = {"ฝนตก" $\rightarrow$ "พื้นเปียก", "ฝนตก"}
2. ต้องการตรวจสอบว่า KB $\models$ "พื้นเปียก" (KB บ่งว่าพื้นเปียก) หรือไม่
3. จากกฎ Modus Ponens: ถ้า $P \rightarrow Q$ และ P จริง แล้ว Q จริง
4. ดังนั้น KB $\models$ "พื้นเปียก" เป็น **จริง**

นี่คือหลักการพื้นฐานของ Logic Programming และ SAT Solvers

1.2.5 การจำแนกประพจน์: อะตอมิกและเชิงประกอบ

ประพจน์แบ่งเป็น 2 ประเภท คือ **ประพจน์อะตอมิก** ซึ่งไม่สามารถแยกย่อยได้ เช่น p, q, r และ **ประพจน์เชิงประกอบ** ซึ่งเกิดจากการรวมประพจน์อะตอมิกด้วยตัวเชื่อมทางตรรกศาสตร์

เมื่อต้องการตรวจสอบค่าความจริงของประพจน์เชิงประกอบ ให้แยกออกเป็นประพจน์อะตอมิก แล้วตรวจสอบค่าความจริงของแต่ละตัว จากนั้นรวมผลลัพธ์ตามตัวเชื่อมที่ใช้

ทฤษฎีบท 1.6 (การแยกประพจน์)

1. **ประพจน์อะตอมิก (Atomic Proposition)** คือ ประพจน์ที่ไม่สามารถแยกย่อยได้อีก เช่น p, q, r

2. **ประพจน์เชิงประกอบ (Compound Proposition)** คือประพจน์ที่เกิดจากการรวมประพจน์อะตอมิกด้วย **ตัวเชื่อมทางตรรกศาสตร์** (logical connectives)

ตัวอย่าง 1.7 ให้ p : "ฉันไปมหาวิทยาลัย" และ q : "วันนี้วันจันทร์"

1. p เป็นประพจน์อะตอมิก — เป็นข้อความง่ายๆ ที่ไม่มีตัวเชื่อม
2. $p \wedge q$ เป็นประพจน์เชิงประกอบ — ใช้ตัวเชื่อม "และ" เชื่อม p และ q
3. $\neg p \vee q$ เป็นประพจน์เชิงประกอบ — ใช้ตัวเชื่อม "ไม่" ($\neg$) และ "หรือ" ($\vee$)

1.3 ตัวเชื่อมทางตรรกศาสตร์ (Logical Connectives)

วัตถุประสงค์การเรียนรู้:

- ✓ ทำความเข้าใจตัวเชื่อมทางตรรกศาสตร์ทั้ง 6 ตัว
- ✓ ใช้ตัวเชื่อมสร้างประพจน์เชิงประกอบ
- ✓ เข้าใจการประยุกต์ในวงจรดิจิทัลและภาษาโปรแกรม

1.3.1 บทนำ: ตัวเชื่อมทางตรรกศาสตร์คืออะไร

ในชีวิตประจำวัน เรามักใช้คำเชื่อมต่างๆ เมื่อพูดหรือเขียนประโยคที่ซับซ้อน เช่น "ฝนตกและถนนเปียก" หรือ "ถ้าฝนตกแล้วถนนเปียก" ในทางตรรกศาสตร์ คำเชื่อมเหล่านี้เรียกว่า "ตัวเชื่อมทางตรรกศาสตร์" (Logical Connectives) ซึ่งเป็นสัญลักษณ์ที่ใช้ในการรวมประพจน์เข้าด้วยกัน

ตัวเชื่อมทางตรรกศาสตร์มีทั้งหมด 6 ตัว ได้แก่ นิเสธ ($\neg$), การเลือก ($\wedge$), การแยก ($\vee$), การแยกแบบเอกซ์คลูซีฟ ($\oplus$), ประโยคบทนิเสธ ($\rightarrow$), และนิเสธสองทิศทาง ($\leftrightarrow$) ในการเขียนโปรแกรมคอมพิวเตอร์ ภาษา Python ใช้ `and`, `or`, `not` สำหรับ $\wedge$, $\vee$, $\neg$ ตามลำดับ

1.3.2 นิเสธ (Negation): $\neg$

นิเสธหรือ Negation เป็นตัวเชื่อมที่กลับค่าความจริงของประพจน์ ถ้า p เป็นจริง นิเสธ $\neg p$ จะเป็นเท็จ และในทางกลับกัน ในการเขียนโปรแกรม Python ใช้ `not` สำหรับนิเสธ ส่วนในภาษา C/Java ใช้ `!`

นิยาม 1.8 (นิเสธ) ให้ p เป็นประพจน์ นิเสธ ของ p เขียนแทนด้วย $\neg p$ มีค่าความจริงตรงข้ามกับ p :

1. ถ้า p จริง แล้ว $\neg p$ เท็จ
2. ถ้า p เท็จ แล้ว $\neg p$ จริง

หมายเหตุ 1.9 (การอ่านและการเขียน $\neg p$)

1. ภาษาอังกฤษ: "not p ", "It is not the case that p "
2. ภาษาโปรแกรม: `!p` (C/Java/Python), `p` (C/C++ bitwise NOT)
3. ในวงจรดิจิทัล: สัญลักษณ์ NOT Gate หรือที่เรียกว่า Inverter

ตัวอย่าง 1.10 (วงจร NOT Gate ในวงจรดิจิทัล) ในวงจรดิจิทัล NOT Gate รับสัญญาณอินพุตหนึ่งตัวและส่งสัญญาณเอาต์พุตที่เป็นค่าตรงข้าม ถ้าอินพุตเป็น 1 เอาต์พุตจะเป็น 0 และในทางกลับกัน

ตารางความจริงของ NOT Gate แสดงในตารางที่ 1:

ตารางที่ 1 ตารางความจริงของ NOT Gate

p	$\neg p$
T (1)	F (0)
F (0)	T (1)

NOT Gate เป็นองค์ประกอบพื้นฐานที่ใช้ในการสร้างวงจรดิจิทัลที่ซับซ้อน

ตัวอย่าง 1.11 (การประยุกต์ในภาษาโปรแกรม: การตรวจสอบเงื่อนไข) นิเสธถูกใช้บ่อยมากในการเขียนโปรแกรม โดยเฉพาะในการตรวจสอบเงื่อนไขที่ต้องการค่าตรงข้าม

```
// Python: validating input values
```

```
if not is_valid:
```

```
    print("Invalid data!")
```

```
// Python: preventing division by zero
```

```
// condition denominator != 0 is negation of denominator == 0
```

```
if denominator != 0 and numerator / denominator > 0:
```

```
    print("Result is positive")
```

```
// C language: checking null pointer before accessing members
```

```
if (ptr != NULL && ptr->value > 0) {
```

```
    process(ptr->value);
```

```
}
```

ใน ตัวอย่าง การ ป้องกัน การ หาร ด้วย ศูนย์ เราใช้ `denominator != 0` ซึ่งเป็น นิเสธ ของ `denominator == 0` ถ้า `denominator == 0` เป็นจริง (หารด้วยศูนย์) แล้ว `denominator != 0` จะเป็นเท็จ และเงื่อนไขทั้งหมดจะเป็นเท็จทำให้ไม่เข้าสู่บล็อก ซึ่งป้องกันการเกิด runtime error ได้

1.3.3 การเลือก (Conjunction): $\wedge$

การเลือกหรือ Conjunction เป็นตัวเชื่อมที่ใช้เชื่อมประพจน์สองตัว โดย $p \wedge q$ จะเป็นจริงก็ต่อเมื่อ p และ q เป็นจริงทั้งคู่ ในการเขียนโปรแกรม Python ใช้ `and` ส่วนในภาษา C/Java ใช้ `&&` ในวงจรดิจิทัล AND Gate ทำงานเหมือนการเลือก

นิยาม 1.12 (การเลือก) ให้ p และ q เป็นประพจน์ การเลือก ของ p และ q เขียนแทนด้วย $p \wedge q$ มีค่าความจริงเป็นจริง เฉพาะเมื่อ ทั้ง p และ q จริง ทุกกรณีอื่นเป็นเท็จ

ตารางความจริงของการเลือก แสดงในตารางที่ 2:

ตารางที่ 2 ตารางความจริงของ AND Gate

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

หมายเหตุ 1.13 (การเรียกและการใช้งาน)

1. ภาษาอังกฤษ: “ p and q ”
2. ภาษาโปรแกรม: $p \ \&\& \ q$ (C/Java), $p \ \text{and} \ q$ (Python)
3. วงจรดิจิทัล: AND Gate ซึ่งมีสัญลักษณ์ที่คล้ายกับตัวเชื่อมตัว D ในลอจิกเกต

ตัวอย่าง 1.14 (การประยุกต์ในภาษา SQL: การค้นหาข้อมูล) การเลือกถูกใช้บ่อยในการเขียนคำสั่ง SQL เพื่อกรองข้อมูลที่ต้องการ:

```
SELECT * FROM users
```

```
WHERE age > 18 AND status = 'active';
```

ผลลัพธ์:

1. Alice (อายุ 25, status: 'active') → ☐ ทั้งสองเงื่อนไขจริง
2. Bob (อายุ 17, status: 'active') → ☐ อายุไม่เกิน 18

3. Charlie (อายุ 20, status: 'inactive') $\rightarrow \square$ สถานะไม่ใช่ 'active'

ตัวอย่าง 1.15 (Algorithm Correctness — Preconditions และ Postconditions) ในการพิสูจน์ความถูกต้องของอัลกอริทึม Precondition และ Postcondition เป็นประพจน์ที่ต้องเป็นจริงก่อนและหลังการทำงานของอัลกอริทึม ตามลำดับ

Example: division function

function divide(a, b):

 # Precondition: $b \neq 0$ $\leftarrow$ proposition that must be true before function call

 # Postcondition: result is a / b $\leftarrow$ proposition that will be true after function execution

 assert $b \neq 0$: "Division by zero"

 return a / b

Precondition คือประพจน์ที่ต้องเป็นจริงก่อนการทำงานของอัลกอริทึม ซึ่งกำหนดขอบเขตของข้อมูลนำเข้าที่ถูกต้อง

1.3.4 การแยก (Disjunction): $\vee$

การแยกหรือ Disjunction เป็นตัวเชื่อมที่ $p \vee q$ จะเป็นเท็จก็ต่อเมื่อทั้ง p และ q เป็นเท็จ (Inclusive OR) สัญลักษณ์ $\vee$ ใน Python ใช้ `or` และในภาษา C/Java ใช้ `||` ตัวอย่างเช่น p : "ฉันจะไปดูหนัง", q : "ฉันจะไปอ่านหนังสือ" แล้ว $p \vee q$: "ฉันจะไปดูหนังหรืออ่านหนังสือ" ซึ่งจะเป็นจริงถ้าอย่างน้อยหนึ่งกิจกรรมเกิดขึ้น สิ่งสำคัญคือการแยกในทางตรรกศาสตร์เป็น Inclusive OR คือถ้าทั้งสองประพจน์เป็นจริง การแยกก็ยังคงเป็นจริง ต่างจาก Exclusive OR (XOR) ซึ่งจะเป็นเท็จเมื่อทั้งสองประพจน์เป็นจริง

นิยาม 1.16 (การแยก) ให้ p และ q เป็นประพจน์ การแยก ของ p และ q เขียนแทนด้วย $p \vee q$ มีค่าความจริงเป็นเท็จ เฉพาะเมื่อ ทั้ง p และ q เท็จ (Inclusive OR)

ตารางความจริงของการแยก แสดงในตารางที่ 3:

ตารางที่ 3 ตารางความจริงของ OR Gate

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

หมายเหตุ 1.17 (Inclusive OR vs Exclusive OR)

1. $\vee$ คือ Inclusive OR: จริงเมื่อ p จริง

หรือ q จริง หรือทั้งคู่จริง

2. $\oplus$ คือ Exclusive OR (XOR): จริงเมื่อ p จริง หรือ q จริง แต่ ไม่ใช่ทั้งคู่

ตัวอย่าง 1.18 (การประยุกต์: การควบคุมการเข้าถึง (Access Control)) การแยกถูกใช้บ่อยในระบบควบคุมการเข้าถึง:

```
// C/Java: Admin or permission-based access control
if (isAdmin || hasPermission) {
    grantAccess();
}
```

```
// Python: checking multiple user roles

if user_role == "admin" or user_role == "moderator":

    allowModeration()


// SQL: querying data matching at least one condition

SELECT * FROM products

WHERE category = 'Electronics' OR category = 'Books';
```

ตัวอย่าง 1.19 (Short-circuit Evaluation — ประโยชน์ในทางปฏิบัติ) Short-circuit evaluation ช่วยเพิ่มประสิทธิภาพและความปลอดภัย สำหรับการเลือก ($p \wedge q$): ถ้า p เป็นเท็จ ผลลัพธ์จะเป็นเท็จโดยไม่ต้องประเมิน q สำหรับการแยก ($p \vee q$): ถ้า p เป็นจริง ผลลัพธ์จะเป็นจริงโดยไม่ต้องประเมิน q

```
// C: null pointer check before accessing members

// Short-circuit: if ptr == NULL (false), skip ptr->value check
if (ptr != NULL && ptr->value > 0) {

    // ptr is not NULL and ptr->value > 0

    process(ptr->value);

}


// Java: multi-level condition checking

if (isLoggedIn && hasPermission && !isBanned) {

    // if isLoggedIn is false, skip everything

    allowAccess();

}
```

1.3.5 การแยกแบบเอกซ์คลูซีฟ (Exclusive OR): $\oplus$

การแยกแบบเอกซ์คลูซีฟ (XOR) เป็นตัวเชื่อมที่ $p \oplus q$ จะเป็นจริงก็ต่อเมื่อประพจน์สองตัวมีค่าต่างกัน (หนึ่งจริง หนึ่งเท็จ) และเป็นเท็จเมื่อทั้งคู่มีค่าเหมือนกัน สัญลักษณ์ $\oplus$ ตัวอย่างเช่น "คุณสามารถเลือกของหวานหรือผลไม้" ในความหมาย Exclusive หมายความว่าต้องเลือกอย่างใดอย่างหนึ่งเท่านั้น ในการเขียนโปรแกรม XOR ถูกใช้ในการสลับค่าตัวแปร (Swapping) โดยไม่ต้องใช้ตัวแปรชั่วคราว การเข้ารหัสข้อมูล (Encryption) และการตรวจสอบความถูกต้อง (Parity Check) ในวงจรดิจิทัล XOR Gate เป็นองค์ประกอบสำคัญในการสร้างวงจรบวกเลขฐานสอง (Adder Circuits)

นิยาม 1.20 (การแยกแบบเอกซ์คลูซีฟ) ให้ p และ q เป็นประพจน์ XOR ของ p และ q เขียนแทนด้วย $p \oplus q$ มีค่าความจริงเป็นจริง เมื่อ p และ q มีค่าต่างกัน (ต่างกันพอดี)

ตารางความจริงของ XOR แสดงให้เห็นว่า XOR เป็นเหมือน "ตัวตรวจสอบความแตกต่าง" แสดงในตารางที่ 4:

ตารางที่ 4 ตารางความจริงของ XOR Gate

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

ดังแสดงในรูปที่ ?? วงจรดิจิทัล (Logic Gates) เป็นสัญลักษณ์มาตรฐานที่ใช้ในการแทนการดำเนินการทางตรรกศาสตร์ในวงจรไฟฟ้า การเข้าใจความสัมพันธ์ระหว่างตรรกศาสตร์และวงจรดิจิทัลมีความสำคัญในการออกแบบระบบคอมพิวเตอร์และอุปกรณ์อิเล็กทรอนิกส์

ตัวอย่าง 1.21 (การประยุกต์: Half Adder — การบวกเลขฐานสอง) Half Adder เป็นวงจรดิจิทัลพื้นฐานที่ใช้ในการบวกเลขฐานสองขนาด 1 บิต วงจรนี้รับอินพุตสองตัวคือ A และ B และให้เอาต์พุตสองตัวคือ Sum (ผลรวม) และ Carry (ตัวทด)

หลักการทำงานของ Half Adder:

1. ผลรวม (Sum) จะเป็น 1 เมื่อ A และ B มีค่าต่างกัน — นี่คือการใช้ XOR

2. ตัวทด (Carry) จะเป็น 1 เมื่อ A และ B เป็น 1 ทั้งคู่ — นี่คือการใช้ AND

ตารางความจริง Half Adder แสดงในตารางที่ 5:

ตารางที่ 5 ตารางความจริงของ Half Adder

A	B	$\text{Sum}(= A \oplus B)$	$\text{Carry}(= A \wedge B)$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

ดังนั้น สมการของ Half Adder คือ:

1. $S = A \oplus B$ (Sum ใช้ XOR Gate)
2. $C = A \wedge B$ (Carry ใช้ AND Gate)

ตัวอย่าง 1.22 (การประยุกต์: การสลับค่าตัวแปรโดยไม่ใช้ตัวแปรชั่วคราว (XOR Swap)) XOR Swap เป็นเทคนิคที่ใช้คุณสมบัติของ XOR ในการสลับค่าของตัวแปรสองตัวโดยไม่ต้องใช้ตัวแปรชั่วคราว ซึ่งประหยัดหน่วยความจำได้ แม้ว่าในทางปฏิบัติคอมพิวเตอร์สมัยใหม่มักจะปรับปรุงโค้ดให้เหมาะสมอยู่แล้ว แต่การเข้าใจ XOR Swap เป็นการแสดงให้เห็นคุณสมบัติทางคณิตศาสตร์ที่สำคัญของ XOR

คุณสมบัติพื้นฐานของ XOR ที่ใช้ในการ Swap:

1. $a \oplus a = 0$ — ค่าใด XOR กับตัวเองจะได้ 0
2. $a \oplus 0 = a$ — ค่าใด XOR กับ 0 จะได้ค่าเดิม
3. $(a \oplus b) \oplus b = a$ — XOR สองครั้งกับค่าเดียวจะได้ค่าเดิม

ขั้นตอนการ Swap:

```
// C: swapping two values using XOR
```

```
int a = 5, b = 3;
```

```
a = a ^ b; // a = 5 ^ 3 = 6
```

```
b = a ^ b; // b = (5 ^ 3) ^ 3 = 5 (because (a^b)^b = a)
```

```
a = a ^ b; // a = (5 ^ 3) ^ 5 = 3 (because (a^b)^a = b)
```

```
# after this: a = 3, b = 5 (values swapped)
```

1.3.6 ประโยคบทนิเสธ (Implication): $\rightarrow$

ประโยคบทนิเสธ (Implication) เป็นตัวเชื่อมที่แสดงความสัมพันธ์เชิงเงื่อนไข $p \rightarrow q$ อ่านว่า "ถ้า p แล้ว q " โดย p คือ "เหตุ" (Antecedent) และ q คือ "ผล" (Consequent) หมายความว่า "ถ้า p เป็นจริง แล้ว q ต้องเป็นจริง" ซึ่งจะเป็นเท็จเฉพาะเมื่อ p จริงและ q เท็จ ตัวอย่างเช่น "ถ้า $2+2=5$ แล้ว กรุงเทพฯ เป็นเมืองหลวงของไทย" เป็นประพจน์ที่มีค่าความจริงเป็นจริง เพราะเมื่อเหตุ (p) เป็นเท็จ Implication จะเป็นจริงเสมอ

นิยาม 1.23 (ประโยคบทนิเสธ) ให้ p และ q เป็นประพจน์ Implication $p \rightarrow q$ อ่านว่า "ถ้า p แล้ว q " มีค่าความจริงเป็นเท็จ เฉพาะเมื่อ p จริงและ q เท็จ

1. p เรียกว่า เหตุ (Antecedent/Hypothesis)
2. q เรียกว่า ผล (Consequent/Conclusion)

ตารางความจริงของ Implication แสดงในตารางที่ 6:

ตารางที่ 6 ตารางความจริงของ Implication

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

สังเกตได้ว่า $p \rightarrow q$ เป็นเท็จเฉพาะกรณีเดียวคือ "จริงนำ เท็จตาม" ซึ่งเป็นกรณีที่เหตุเป็นจริงแต่ผลเป็นเท็จ ซึ่งขัดแย้งกับความหมายของประโยคว่า "ถ้าเหตุเป็นจริงแล้วผลต้องเป็นจริง"

หมายเหตุ 1.24 (ความเข้าใจผิดที่พบบ่อยเกี่ยวกับ Implication) นักศึกษามักเข้าใจผิดว่า $p \rightarrow q$ หมายถึง " p ทำให้ q เป็นจริง" แต่ในทางตรรกศาสตร์ **ไม่จำเป็นต้องมีความสัมพันธ์เชิงเหตุผล** เช่น "ถ้า $2 + 2 = 5$ แล้ว กรุงเทพฯ เป็นเมืองหลวงของไทย" $\equiv$ จริง เพราะเหตุเป็นเท็จ นอกจากนี้ $p \rightarrow q$ และ $q \rightarrow p$ มีความหมายต่างกัน ตัวอย่างเช่น: "ถ้าเป็นสัตว์ แล้วมีชีวิต" — จริง แต่ "ถ้ามีชีวิต แล้วเป็นสัตว์" — เท็จ (พืชมีชีวิตแต่ไม่ใช่สัตว์)

ตัวอย่าง 1.25 (Converse, Inverse, Contrapositive) จาก $p \rightarrow q$ เราสามารถสร้างประโยคที่เกี่ยวข้องได้หลายรูปแบบ:

1. **Converse (บทกลับ):** $q \rightarrow p$ — สลับตำแหน่งของเหตุและผล
2. **Inverse (ผกผัน):** $\neg p \rightarrow \neg q$ — นิเสธทั้งเหตุและผล
3. **Contrapositive (ตรงข้าม):** $\neg q \rightarrow \neg p$ — นิเสธและสลับตำแหน่ง

ตัวอย่างเช่น จาก "ถ้าฝนตก แล้วถนนเปียก" ($p \rightarrow q$):

1. Converse: "ถ้าถนนเปียก แล้วฝนตก" ($q \rightarrow p$) — อาจไม่จริงเพราะถนนอาจเปียกจากสาเหตุอื่น
2. Inverse: "ถ้าฝนไม่ตก แล้วถนนไม่เปียก" ($\neg p \rightarrow \neg q$) — อาจไม่จริงเพราะถนนอาจเปียกจากสาเหตุอื่น
3. Contrapositive: "ถ้าถนนไม่เปียก แล้วฝนไม่ตก" ($\neg q \rightarrow \neg p$) — จริง

ทฤษฎีบท 1.26 (ความสมมูลของ Contrapositive)

$$(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$$

นี่คือหลักการที่ใช้ในการพิสูจน์แบบ Contraposition

Proof. พิสูจน์โดยตารางค่าความจริง:

p	q	$\neg p$	$\neg q$	$p \rightarrow q$	$\neg q \rightarrow \neg p$
T	T	F	F	T	T
T	F	F	T	F	F
F	T	T	F	T	T
F	F	T	T	T	T

คอลัมน์ที่ 5 และ 6 มีค่าเดียวกันทุกแถว ดังนั้น $(p \rightarrow q) \equiv (\neg q \rightarrow \neg p)$ □

ตัวอย่าง 1.27 (การประยุกต์: Preconditions และ Postconditions) ในการพิสูจน์ความถูกต้องของอัลกอริทึม เราใช้ Implication ในการอธิบายความสัมพันธ์ระหว่างสถานะก่อนและหลังการทำงานของโปรแกรม

Precondition $\rightarrow$ Postcondition เป็นความสัมพันธ์ที่ต้องเป็นจริง ถ้า Precondition เป็นจริงก่อนการทำงานของโปรแกรม แล้ว Postcondition ต้องเป็นจริงหลังการทำงานเสร็จสิ้น


```
# Example: maximum finding algorithm
```

```
function findMax(a, b):
```

```
    # Precondition: inputs a, b are real numbers
```

```
    # Postcondition: returns the greater of a and b
```

```
    if a > b:
```

```
        return a    # if a > b, return a (which is greater)
```

```
    else:
```

```
        return b    # if a <= b, return b (which is greater or equal)
```

1.3.7 นิเสธสองทิศทาง (Biconditional): $\leftrightarrow$

นิเสธสองทิศทาง (Biconditional) $p \leftrightarrow q$ อ่านว่า " p ก็ต่อเมื่อ q " หมายความว่า p และ q มีค่าความจริงเหมือนกันเสมอ ถ้า p เป็นจริง แล้ว q ต้องเป็นจริง และถ้า p เป็นเท็จ แล้ว q ต้องเป็นเท็จ สัญลักษณ์ $\leftrightarrow$ หรือ $\equiv$ ตัวอย่างเช่น "จำนวนเต็ม n เป็นจำนวนคู่ ก็ต่อเมื่อ n หารด้วย 2 ลงตัว" ในการเขียนโปรแกรม Biconditional ถูกใช้ในการตรวจสอบความเท่าเทียม คำสั่ง `if (x == y)` จะทำงานเมื่อ x และ y มีค่าเท่ากัน

นิยาม 1.28 (นิเสธสองทิศทาง) ให้ p และ q เป็นประพจน์ **Biconditional** $p \leftrightarrow q$ อ่านว่า " p ก็ต่อเมื่อ q " มีค่าความจริงเป็นจริง เมื่อ p และ q มีค่าเดียวกัน (ทั้งจริงหรือทั้งเท็จ)

ตารางความจริงของ Biconditional แสดงในตารางที่ 7:

ตารางที่ 7 ตารางความจริงของ Biconditional

p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

สังเกตได้ว่า Biconditional เป็นจริงเมื่อทั้ง p และ q มีค่าเดียวกัน และเป็นเท็จเมื่อค่าต่างกัน

ทฤษฎีบท 1.29 (ความสัมพันธ์ระหว่าง Biconditional กับ Implication)

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$$

นี้หมายความว่า Biconditional เป็นได้ทั้ง Implication ในทิศทาง $p \rightarrow q$ และ Implication ในทิศทาง $q \rightarrow p$ พร้อมกัน

Proof. พิสูจน์โดยตารางค่าความจริง:

p	q	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$	$p \leftrightarrow q$
T	T	T	T	T	T
T	F	F	T	F	F
F	T	T	F	F	F
F	F	T	T	T	T

คอลัมน์ที่ 5 และ 6 มีค่าเดียวกันทุกแถว ดังนั้น $p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$ □

ตัวอย่าง 1.30 (การประยุกต์: การตรวจสอบความเท่าเทียมในโปรแกรม) Biconditional ถูกใช้ในการตรวจสอบว่าสองค่ามีค่าเท่ากันหรือไม่

// Python: checking if two values are equal

```
if (x == y):    // x if and only if y (x == y)
```

```
    print("Equal")
```

// Java: checking if both conditions have the same truth value

```
boolean sameSign = (x > 0) == (y > 0);
```

// Check if x and y have the same sign

// meaning both positive, both negative, or both zero

ตัวอย่าง 1.31 (Algorithm — Loop Invariant) ในการพิสูจน์ความถูกต้องของลูป ต้องแสดงว่า Loop Invariant เป็นจริงตลอดการทำงานของลูป

Loop Invariant คือประพจน์ที่เป็นจริง:

1. ก่อนเข้าลูปครั้งแรก (Initialization)
2. หลังจากทำงานในแต่ละรอบ (Maintenance)
3. เมื่อลูปจบ (Termination)

Example: sum loop

```
sum = 0
```

```
i = 0
```

```
# Loop Invariant: sum = sum of a[0] + a[1] + ... + a[i-1]
```

```
while i < n:

    sum = sum + a[i]    # Invariant still holds after execution

    i = i + 1

# when loop ends (i == n):

# Invariant says sum = a[0] + a[1] + ... + a[n-1]

# which is the desired sum
```

1.4 ตารางค่าความจริง (Truth Tables)

วัตถุประสงค์การเรียนรู้:

- ✓ สร้างตารางค่าความจริงสำหรับประพจน์ทุกรูปแบบ
- ✓ ใช้ตารางค่าความจริงพิสูจน์สมมูลทางตรรกศาสตร์
- ✓ วิเคราะห์ประพจน์เชิงประกอบซับซ้อน

1.4.1 บทนำ: ตารางค่าความจริงคืออะไร

p	q	$p \wedge q$	$p \vee q$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

ภาพที่ 1 ตัวอย่างโครงสร้างตารางค่าความจริงสำหรับ 2 ตัวแปร

ทฤษฎีบท 1.32 (ตารางค่าความจริงของตัวเชื่อมทั้ง 6) ตารางค่าความจริง (Truth Table) เป็นเครื่องมือพื้นฐานที่สำคัญที่สุดในการศึกษาตรรกศาสตร์เชิงประพจน์ ตารางค่าความจริงเป็นตารางที่แสดงค่าความจริงของประพจน์เชิงประกอบ สำหรับทุกกรณีที่เป็นไปได้ของประพจน์อะตอมิกที่ประกอบกัน ตารางนี้ช่วยให้เราสามารถวิเคราะห์ประพจน์ที่ซับซ้อนได้อย่างเป็นระบบ โดยไม่ต้องพึ่งพาสัญชาตญาณหรือการคาดเดา

หลักการทำงานของตารางค่าความจริงคือการแจกแจง (Enumerate) ทุกกรณีของค่าความจริงของประพจน์อะตอมิก จากนั้นค่อยๆ คำนวณค่าความจริงของประพจน์ย่อยแต่ละตัวจนได้ค่าความจริงของประพจน์เชิงประกอบทั้งหมด สำหรับประพจน์ที่มี n ตัวแปรอะตอมิก จะมีทั้งหมด 2^n กรณี ตัวอย่างเช่น ถ้ามีประพจน์ 1 ตัวแปร (p) จะมี $2^1 = 2$ กรณี ถ้ามี 2 ตัวแปร (p, q) จะมี $2^2 = 4$ กรณี ถ้ามี 3 ตัวแปร (p, q, r) จะมี $2^3 = 8$ กรณี และถ้ามี 4 ตัวแปรจะมี $2^4 = 16$ กรณี ในการสร้างตารางค่าความจริง เรามักจะใช้รูปแบบมาตรฐานในการเรียงลำดับกรณีต่างๆ โดยเริ่มจากกรณีที่ประพจน์ทั้งหมดเป็นจริง แล้วลดลงทีละบิตจนถึงกรณีที่ประพจน์ทั้งหมดเป็นเท็จ ตารางค่าความจริงมีประโยชน์มากใน

หลายด้าน ประการแรก มันช่วยให้เราตรวจสอบค่าความจริงของประพจน์ที่ซับซ้อนได้อย่างถูกต้อง ประการที่สอง มันช่วยในการพิสูจน์สมมูลทางตรรกศาสตร์ (Logical Equivalence) ระหว่างประพจน์สองตัว ถ้าตารางค่าความจริงของประพจน์สองตัวมีค่าเดียวกันทุกกรณี แล้วประพจน์ทั้งสองสมมูลกัน ประการที่สาม มันช่วยในการวิเคราะห์ว่าประพจน์ใดเป็น Tautology (จริงทุกกรณี), Contradiction (เท็จทุกกรณี), หรือ Contingency (จริงบ้างเท็จบ้าง)

1. $\neg p$ มีค่าตรงข้ามกับ p เสมอ
2. $p \wedge q$ เป็นจริงเฉพาะเมื่อทั้งคู่จริง
3. $p \vee q$ เป็นเท็จเฉพาะเมื่อทั้งคู่เท็จ
4. $p \oplus q$ เป็นจริงเมื่อค่าต่างกัน
5. $p \rightarrow q$ เป็นเท็จเฉพาะเมื่อ p จริงและ q เท็จ
6. $p \leftrightarrow q$ เป็นจริงเมื่อค่าเหมือนกัน

p	q	$\neg p$	$p \wedge q$
T	T	F	T
T	F	F	F
F	T	T	F
F	F	T	F

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \oplus q$	$p \rightarrow q$	$p \leftrightarrow q$
T	T	F	T	T	F	T	T
T	F	F	F	T	T	F	F
F	T	T	F	T	T	T	F
F	F	T	F	F	F	T	T

ตารางค่าความจริงจะมีคอลัมน์สำหรับแต่ละประพจน์อะตอมิก และเพิ่มคอลัมน์สำหรับประพจน์ย่อยแต่ละตัวที่คำนวณไปเรื่อยๆ จนได้ประพจน์เชิงประกอบสุดท้าย

ตัวอย่าง 1.33 (การสร้างตารางค่าความจริงของประพจน์เชิงประกอบ) จงหาค่าความจริงของ $(p \wedge q) \rightarrow (p \vee q)$

ขั้นตอนการสร้างตาราง:

1. กำหนดประพจน์อะตอมิก: p, q
2. คำนวณ $p \wedge q$: การเลือกของ p และ q
3. คำนวณ $p \vee q$: การแยกของ p และ q
4. คำนวณ $(p \wedge q) \rightarrow (p \vee q)$: Implication จาก $p \wedge q$ ไป $p \vee q$

p	q	$p \wedge q$	$p \vee q$	$(p \wedge q) \rightarrow (p \vee q)$
T	T	T	T	T
T	F	F	T	T
F	T	F	T	T
F	F	F	F	T

สังเกตว่า $(p \wedge q) \rightarrow (p \vee q)$ เป็น **Tautology เสมอ** (ค่าความจริงเป็น T ทุกกรณี) เพราะเมื่อ $p \wedge q$ เป็นจริง (กรณีแรกเท่านั้น) $p \vee q$ ก็เป็นจริงด้วย และเมื่อ $p \wedge q$ เป็นเท็จ (กรณีที่ 2, 3, 4) Implication เป็นจริงเสมอตามนิยาม

ตัวอย่าง 1.34 (Full Adder — วงจรบวกเลขฐานสองแบบเต็ม) Full Adder เป็นวงจรดิจิทัลที่บวกเลขฐานสองขนาด 1 บิตโดยคำนึงถึงตัวทดจากบิตก่อนหน้า (Carry-in) วงจรนี้มีอินพุต 3 ตัว ได้แก่ A, B, และ Carry-in (C_{in}) และมีเอาต์พุต 2 ตัว ได้แก่ Sum (S) และ Carry-out (C_{out})

สมการของ Full Adder:

1. $S = A \oplus B \oplus C_{in}$ (ผลรวมเป็น XOR ของทั้งสามค่า)
2. $C_{out} = (A \wedge B) \vee (C_{in} \wedge (A \oplus B))$ (ตัวทดเป็น AND หรือ OR ของค่าต่างๆ)

ตารางค่าความจริง Full Adder:

A	B	C_{in}	S	C_{out}	Binary Sum
0	0	0	0	0	$000_2 = 0_{10}$
0	0	1	1	0	$001_2 = 1_{10}$
0	1	0	1	0	$010_2 = 2_{10}$
0	1	1	0	1	$011_2 = 3_{10}$
1	0	0	1	0	$100_2 = 4_{10}$
1	0	1	0	1	$101_2 = 5_{10}$
1	1	0	0	1	$110_2 = 6_{10}$
1	1	1	1	1	$111_2 = 7_{10}$

Full Adder สามารถนำมาต่อกันเพื่อสร้างวงจรบวกเลขฐานสองหลายบิตได้ โดย Carry-out จากบิตก่อนหน้าจะเป็น Carry-in ของบิตถัดไป วงจรที่สร้างจาก Full Adder หลายตัวเรียกว่า Ripple Carry Adder ซึ่งเป็นพื้นฐานของการสร้าง ALU (Arithmetic Logic Unit) ในหน่วยประมวลผลกลาง (CPU)

ตัวอย่าง 1.35 (SAT Solver — 2-SAT และ 3-SAT) ปัญหา SAT หรือ Boolean Satisfiability Problem เป็นปัญหาพื้นฐานที่สำคัญที่สุดในทฤษฎีการคำนวณ ปัญหานี้ถามว่า: กำหนดประพจน์บูลีน (Boolean Formula) มีการกำหนดค่าความจริงให้กับตัวแปรหรือไม่ ที่ทำให้ประพจน์ทั้งหมดเป็นจริง?

ปัญหา SAT ถูกจัดอยู่ในกลุ่ม NP-Complete ซึ่งหมายความว่าไม่มีอัลกอริทึมที่มีประสิทธิภาพ (Polynomial Time) ในการแก้ปัญหานี้ในกรณีทั่วไป อย่างไรก็ตาม ในทางปฏิบัติ SAT Solver สมัยใหม่สามารถแก้ปัญห SAT ที่มีตัวแปรหลายพันตัวได้อย่างรวดเร็วในหลายกรณี

ปัญหา SAT มีหลายรูปแบบย่อยที่มีความซับซ้อนต่างกัน:

1. **2-SAT**: ประพจน์อยู่ในรูป Conjunction ของ Clauses ที่มี 2 Literals ต่อ Clause: $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge \dots$

1.1. แก้ได้ในเวลาเชิงเส้น $O(n)$ โดยใช้ Graph algorithm

1.2. ใช้หลักการของ Implication Graph และ Strongly Connected Components

2. **3-SAT**: ประพจน์อยู่ในรูป Conjunction ของ Clauses ที่มี 3 Literals ต่อ Clause: $(x_1 \vee x_2 \vee \neg x_3) \wedge (x_2 \vee \neg x_1 \vee x_4)$

2.1. เป็น NP-Complete problem

2.2. ปัญหาหลายอย่างสามารถ reduce เป็น 3-SAT ได้

2.3. SAT Solver สมัยใหม่ใช้เทคนิค CDCL (Conflict-Driven Clause Learning) แก้ได้เร็วมากในทางปฏิบัติ

```
# 2-SAT example:
# (p  ¬q)  (q  r)  (¬p  ¬r)
#
# Check if there exists an assignment (p,q,r) that makes all true?
#
# Try p=T, q=T, r=F:
#   Clause 1: (T  ¬T) = (T  F) = T
#   Clause 2: (T  F) = T
#   Clause 3: (¬T  ¬F) = (F  T) = T
#   all Clauses are T  $\rightarrow$  entire proposition is T  $\rightarrow$  SAT
#
# Solution method: use Implication Graph
# p  $\rightarrow$  q means ¬p  q
# Build graph from Implications and check SCCs
```

1.5 สมมูลทางตรรกศาสตร์ (Logical Equivalence)

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจนิยามของสมมูลทางตรรกศาสตร์
- ✓ จำและใช้กฎของ De Morgan, Identity, Domination เป็นต้น
- ✓ พิสูจน์สมมูลโดยใช้ตารางค่าความจริงและการแปลงรูป

1.5.1 บทนำ: สมมูลทางตรรกศาสตร์คืออะไร

ในทางคณิตศาสตร์และตรรกศาสตร์ เรามักสนใจว่าประโยคสองประโยคมีความหมายเหมือนกันหรือไม่ หรือมีค่าความจริงเหมือนกันทุกกรณีหรือไม่ ถ้าประโยคสองประโยคมีค่าความจริงเหมือนกันทุกกรณี เราก็สามารถใช้ประโยคหนึ่งแทนอีกประโยคหนึ่งได้โดยไม่เปลี่ยนความหมาย ในทางตรรกศาสตร์ เราเรียกความสัมพันธ์เช่นนี้ว่า "สมมูลทางตรรกศาสตร์" (Logical Equivalence)

สมมูลทางตรรกศาสตร์มีความสำคัญมากในการ simplify ประพจน์ที่ซับซ้อน ตัวอย่างเช่น ในวงจรดิจิทัล การลดจำนวน Gate ที่ใช้จะทำให้วงจรมีขนาดเล็กลงและใช้พลังงานน้อยลง โดยเราสามารถแปลงประพจน์ให้อยู่ในรูปที่เทียบเท่ากันแต่ใช้ตัวเชื่อมน้อยลงได้ ในการเขียนโปรแกรม การใช้สมมูลทางตรรกศาสตร์ช่วยให้เราเขียนเงื่อนไขได้อย่างกระชับและเข้าใจง่ายขึ้น และในการพิสูจน์ทางคณิตศาสตร์ การใช้สมมูลทางตรรกศาสตร์ช่วยให้เราสามารถแปลงประโยคที่ต้องการพิสูจน์ให้อยู่ในรูปที่ง่ายต่อการพิสูจน์มากขึ้น

นอกจากนี้ สมมูลทางตรรกศาสตร์ยังมีคุณสมบัติที่สำคัญสามประการ ได้แก่ คุณสมบัติสะท้อน (Reflexive) ซึ่งหมายความว่าทุกประพจน์สมมูลกับตัวมันเอง คุณสมบัติสมมาตร (Symmetric) ซึ่งหมายความว่าถ้า p สมมูลกับ q แล้ว q ก็สมมูลกับ p และคุณสมบัติถ่ายทอด (Transitive) ซึ่งหมายความว่าถ้า p สมมูลกับ q และ q สมมูลกับ r แล้ว p ก็สมมูลกับ r คุณสมบัติเหล่านี้ทำให้สมมูลทางตรรกศาสตร์เป็นความสัมพันธ์ที่มีความสมบูรณ์ (Equivalence Relation)

นิยาม 1.36 (สมมูลทางตรรกศาสตร์) ประพจน์ p และ q กล่าวว่า **สมมูลกัน** เขียนแทนด้วย $p \equiv q$ เมื่อ p และ q มีค่าความจริงเหมือนกันทุกกรณี (ในตารางค่าความจริงทุกแถว)

ทฤษฎีบท 1.37 (คุณสมบัติของสมมูลทางตรรกศาสตร์) สมมูลทางตรรกศาสตร์มีคุณสมบัติ:

1. สะท้อน (Reflexive): $p \equiv p$ — ทุกประพจน์สมมูลกับตัวมันเอง
2. สมมาตร (Symmetric): ถ้า $p \equiv q$ แล้ว $q \equiv p$ — สมมูลกันแปลงกลับได้
3. ถ่ายทอด (Transitive): ถ้า $p \equiv q$ และ $q \equiv r$ แล้ว $p \equiv r$ — การสมมูลถ่ายทอดได้

ทฤษฎีบท 1.38 (กฎของ De Morgan) กฎของ De Morgan เป็นหนึ่งในกฎที่สำคัญที่สุดในตรรกศาสตร์ กฎนี้แสดงความสัมพันธ์ระหว่างนิเสธและตัวเชื่อม $\square$ และ $\square$:

$$\neg(p \wedge q) \equiv \neg p \vee \neg q \quad \text{นิเสธของการเลือกเท่ากับการแยกของนิเสธ}$$

$$\neg(p \vee q) \equiv \neg p \wedge \neg q \quad \text{นิเสธของการแยกเท่ากับการเลือกของนิเสธ}$$

กฎเหล่านี้สามารถขยายไปถึงประพจน์ที่มีตัวแปรมากกว่าสองตัวได้ ตัวอย่างเช่น:

$$\neg(p \wedge q \wedge r) \equiv \neg p \vee \neg q \vee \neg r$$

Proof. พิสูจน์โดยตารางค่าความจริง:

p	q	$p \wedge q$	$\neg(p \wedge q)$	$\neg p$	$\neg q$	$\neg p \vee \neg q$
T	T	T	F	F	F	F
T	F	F	T	F	T	T
F	T	F	T	T	F	T
F	F	F	T	T	T	T

คอลัมน์ที่ 4 และ 7 มีค่าเดียวกันทุกแถว ดังนั้น $\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\square$

ตัวอย่าง 1.39 (การประยุกต์ De Morgan ในการเขียนโปรแกรม) กฎของ De Morgan ถูกใช้บ่อยมากในการเขียนโปรแกรมเพื่อแปลงเงื่อนไขให้อยู่ในรูปที่เข้าใจง่ายขึ้นหรือมีประสิทธิภาพมากขึ้น

```
// Goal: select when NOT (A AND B)
```

```
// meaning not both A and B at the same time
```

```
// Form 1: using NOT and AND
```

```
if (!(isValid && hasData)) {
```

```

    // executes when isValid is false or hasData is false
}

// Form 2: using De Morgan:  $\neg(p \wedge q) \equiv \neg p \vee \neg q$ 
if (!isValid || !hasData) {
    // executes when isValid is false or hasData is false
    // same result, but Form 2 may be more efficient in some languages
}

// SQL example: find users who are not both admin and active
SELECT * FROM users
WHERE NOT (role = 'admin' AND status = 'active');

-- using De Morgan:  $\neg(p \wedge q) \equiv \neg p \vee \neg q$ 
WHERE role != 'admin' OR status != 'active';

-- same result

```

ทฤษฎีบท 1.40 (ตารางกฎของสมมูลทางตรรกศาสตร์ที่สำคัญ) นอกจากกฎของ De Morgan แล้ว ยังมีกฎสมมูลที่สำคัญอีกมาก:

ชื่อกฎ	สมมูล	อธิบาย
Identity (เอกลักษณ์)	$p \wedge T \equiv p$ $p \vee F \equiv p$	การเลือกกับ T จะได้ p การแยกกับ F จะได้ p
Domination (เด่นดัง)	$p \wedge F \equiv F$ $p \vee T \equiv T$	การเลือกกับ F จะได้ F การแยกกับ T จะได้ T
Idempotent (กรณีย)	$p \wedge p \equiv p$ $p \vee p \equiv p$	การเลือก p กับ p จะได้ p การแยก p กับ p จะได้ p
Double Negation (นิเสธซ้อน)	$\neg\neg p \equiv p$	นิเสธนิเสธจะได้ p
Complement (เติมเต็ม)	$p \wedge \neg p \equiv F$ $p \vee \neg p \equiv T$	p และ ไม่ p เป็นเท็จ p หรือ ไม่ p เป็นจริง
Absorption (ดูดกลืน)	$p \wedge (p \vee q) \equiv p$ $p \vee (p \wedge q) \equiv p$	p ดูด p หรือ q p หรือ (p และ q) จะได้ p
Associative (เชื่อมกลุ่ม)	$(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ $(p \vee q) \vee r \equiv p \vee (q \vee r)$	การเลือกเชื่อมกลุ่มได้ การแยกเชื่อมกลุ่มได้
Commutative (สลับที่)	$p \wedge q \equiv q \wedge p$ $p \vee q \equiv q \vee p$	การเลือกสลับที่ได้ การแยกสลับที่ได้
Distributive (แจกแจง)	$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$ $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$	☐ แจกแจงบน ☐ ☐ แจกแจงบน ☐

ทฤษฎีบท 1.41 (สมมูลที่เกี่ยวกับ Implication) สมมูลที่เกี่ยวกับ Implication เป็นสิ่งที่ต้องจำเพราะถูกใช้บ่อยมาก:

$$p \rightarrow q \equiv \neg p \vee q \quad (\text{Implication แปลงเป็น OR})$$

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p) \quad (\text{Biconditional เป็น AND ของ Implication})$$

$$p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q) \quad (\text{Biconditional เป็น XNOR})$$

$$\neg(p \rightarrow q) \equiv p \wedge \neg q \quad (\text{นิเสธของ Implication})$$

สมมูล $p \rightarrow q \equiv \neg p \vee q$ เป็นที่นิยมใช้มากที่สุด เพราะช่วยให้เราแปลง Implication ให้อยู่ในรูปที่ใช้เฉพาะ $\square, \square, \neg$ ได้

Proof. พิสูจน์ $p \rightarrow q \equiv \neg p \vee q$ โดยตารางค่าความจริง:

p	q	$p \rightarrow q$	$\neg p$	$\neg p \vee q$
T	T	T	F	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

คอลัมน์ที่ 3 และ 5 มีค่าเดียวกันทุกแถว ดังนั้น $p \rightarrow q \equiv \neg p \vee q$ □

ตัวอย่าง 1.42 (การประยุกต์: การแปลงเงื่อนไข IF) การแปลง Implication เป็น OR มีประโยชน์ในการเขียนโปรแกรมเพื่อให้เงื่อนไขซับซ้อนง่ายขึ้น

```
// Goal: if x > 0, then result = sqrt(x)
// equivalent to: if x <= 0 or result = sqrt(x)

# Form 1: using if-then
if (x > 0) {
    result = sqrt(x);
}

# Form 2: using Implication  $\rightarrow$  OR
#  $x > 0 \rightarrow \text{result} = \text{sqrt}(x)$ 
#  $\neg(x > 0) \quad \text{result} = \text{sqrt}(x)$ 
#  $x \leq 0 \quad \text{result} = \text{sqrt}(x)$ 

if (!(x > 0) || sqrt(x)) {
```

```
// executes when x <= 0 or sqrt(x) is computed successfully
// which is equivalent to Form 1
result = sqrt(x);
}
```

ทฤษฎีบท 1.43 (การพิสูจน์สมมูลโดยใช้การแปลงรูป) นอกจากการใช้ตารางค่าความจริงแล้ว เรายังสามารถพิสูจน์สมมูลได้โดยการแปลงรูปทีละขั้น โดยใช้กฎสมมูลที่เราารู้จัก

พิสูจน์ว่า: $(p \rightarrow q) \equiv (p \wedge q) \vee \neg p$

Proof.

$$\begin{aligned}
 (p \rightarrow q) &\equiv \neg p \vee q && \text{(Implication as OR)} \\
 &\equiv (\neg p \wedge \mathbf{T}) \vee q && \text{(Identity: } \neg p \equiv \neg p \wedge \mathbf{T} \text{)} \\
 &\equiv (\neg p \wedge (q \vee \neg q)) \vee q && \text{(Complement: } q \vee \neg q \equiv \mathbf{T} \text{)} \\
 &\equiv (\neg p \wedge q) \vee (\neg p \wedge \neg q) \vee q && \text{(Distributive)} \\
 &\equiv (\neg p \wedge \neg q) \vee (\neg p \wedge q) \vee q && \text{(Commutative)} \\
 &\equiv (\neg p \wedge \neg q) \vee q && \text{(Absorption: } (\neg p \wedge q) \vee q \equiv q \text{)} \\
 &\equiv (p \wedge q) \vee \neg p && \text{(Consensus / De Morgan กลับ)}
 \end{aligned}$$

ดังนั้น $(p \rightarrow q) \equiv (p \wedge q) \vee \neg p$



หมายเหตุ 1.44 (ข้อแนะนำในการเลือกวิธีพิสูจน์สมมูล) ในการ พิสูจน์ สมมูล ทาง ตรรกศาสตร์ มี สอง วิธี หลักๆ:

1. ใช้ตารางค่าความจริง:

- 1.1. เหมาะเมื่อประพจน์มีตัวแปรน้อย (1-3 ตัว)
- 1.2. ให้ผลลัพธ์แน่นอนและตรงไปตรงมา
- 1.3. ใช้เวลาเพิ่มขึ้นอย่างมากเมื่อมีตัวแปรมาก

2. ใช้การแปลงรูป:

- 2.1. เหมาะเมื่อต้องการแสดงโครงสร้างทางพีชคณิต
- 2.2. สามารถใช้ได้กับประพจน์ที่มีตัวแปรมากโดยไม่เพิ่มความซับซ้อนมาก

2.3. ต้องจำกฎพื้นฐานให้ขึ้นใจ

กฎที่ควรจำเป็นพื้นฐาน:

1. De Morgan: $\neg(p \wedge q) \equiv \neg p \vee \neg q$
2. Implication: $p \rightarrow q \equiv \neg p \vee q$
3. Double Negation: $\neg\neg p \equiv p$
4. Absorption: $p \wedge (p \vee q) \equiv p$

1.6 ตรรกศาสตร์เชิงประพจน์ (Propositional Logic)

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจองค์ประกอบของ Well-Formed Formulas
- ✓ ใช้กฎวงเล็บและลำดับการทำงานของตัวเชื่อม
- ✓ แยกแยะประโยคที่สร้างได้ถูกต้องและไม่ถูกต้อง

1.6.1 บทนำ: ไวยากรณ์ของตรรกศาสตร์เชิงประพจน์

ในการศึกษาตรรกศาสตร์เชิงประพจน์ เราไม่เพียงแต่ต้องเข้าใจความหมายของประพจน์และตัวเชื่อม แต่ยังต้องเข้าใจว่าประโยคใดสามารถสร้างได้ถูกต้องตามหลักไวยากรณ์ (Grammar) ของตรรกศาสตร์ ในทำนองเดียวกับภาษาธรรมชาติที่มีหลักไวยากรณ์กำหนดว่าประโยคใดถูกต้องตามหลักภาษา ตรรกศาสตร์เชิงประพจน์ก็มีหลักไวยากรณ์ที่กำหนดว่าสูตร (Formula) ใดถูกต้อง

สูตรที่สร้างได้ถูกต้องเรียกว่า "Well-Formed Formula" หรือ WFF (อ่านว่า "วฟ") ซึ่งเป็นสูตรที่สร้างตามกฎเกณฑ์ที่กำหนดไว้ สูตรที่ไม่เป็น WFF จะไม่มีความหมายในทางตรรกศาสตร์ เหมือนกับประโยคที่ไม่ถูกหลักไวยากรณ์ในภาษาธรรมชาติ การเข้าใจ WFF มีความสำคัญในหลายด้าน ประการแรก ในการเขียนโปรแกรมคอมพิวเตอร์ เราต้องรู้ว่าสูตรใดถูกต้องเพื่อให้คอมพิวเตอร์หรืออินเทอร์พรีเตอร์สามารถประมวลผลได้ ประการที่สอง ในการใช้ SAT Solver เราต้องสร้างสูตรในรูปแบบที่ Solver เข้าใจได้ ประการที่สาม ในการพิสูจน์ทางคณิตศาสตร์ เราต้องรู้ว่าประโยคที่เขียนมีความหมายที่ถูกต้องหรือไม่

หลักไวยากรณ์ของตรรกศาสตร์เชิงประพจน์สามารถเขียนได้ในรูปแบบ Backus-Naur Form (BNF) ซึ่งเป็นรูปแบบมาตรฐานในการนิยามไวยากรณ์ของภาษาโปรแกรม ดังนี้:

$$\text{Formula} ::= \text{AtomicFormula} \mid \neg \text{Formula} \mid (\text{Formula} \wedge \text{Formula}) \mid (\text{Formula} \vee \text{Formula}) \\ \mid (\text{Formula} \rightarrow \text{Formula}) \mid (\text{Formula} \leftrightarrow \text{Formula})$$

$$\text{AtomicFormula} ::= p \mid q \mid r \mid \dots$$

นี่หมายความว่า สูตรอะตอมิก (เช่น p, q, r) เป็น WFF และถ้า E เป็น WFF แล้ว $\neg E$ ก็เป็น WFF และถ้า E_1, E_2 เป็น WFF แล้ว $(E_1 \wedge E_2), (E_1 \vee E_2), (E_1 \rightarrow E_2), (E_1 \leftrightarrow E_2)$ ก็เป็น WFF ด้วย

นิยาม 1.45 (Well-Formed Formula (WFF)) สูตรที่สร้างได้ถูกต้อง หรือ WFF คือสตริงของสัญลักษณ์ที่สร้างตาม **ไวยากรณ์** (grammar) ของตรรกศาสตร์เชิงประพจน์

กำหนดให้:

1. $p, q, r, \dots$ เป็น **ประพจน์อะตอมิก** (atomic formulas)
2. ถ้า E เป็น WFF แล้ว $\neg E$ เป็น WFF
3. ถ้า E_1, E_2 เป็น WFF แล้ว $(E_1 \wedge E_2), (E_1 \vee E_2), (E_1 \rightarrow E_2), (E_1 \leftrightarrow E_2)$ เป็น WFF

ตัวอย่าง 1.46 (WFF ที่ถูกต้อง vs ไม่ถูกต้อง) ตัวอย่างประโยคที่ถูกต้องและไม่ถูกต้อง:

1. $p \wedge q \square$ (ถูกต้อง: ประพจน์อะตอมิกเชื่อมด้วย $\square$)
2. $(p \rightarrow q) \vee r \square$ (ถูกต้อง: วงเล็บกำกับเพื่อความชัดเจน)
3. $(p \wedge \neg q) \rightarrow (r \vee s) \square$ (ถูกต้อง: ชับซ้อนแต่ถูกไวยากรณ์)
4. $\wedge pq \square$ (ไม่ถูกต้อง: $\square$ ต้องมี operands ทั้งสองข้าง ต้องเขียนว่า $p \wedge q$)
5. $p \rightarrow \rightarrow q \square$ (ไม่ถูกต้อง: $\rightarrow$ ซ้ำกัน ไม่มี operand ระหว่าง)
6. $(p \wedge q \square$ (ไม่ถูกต้อง: วงเล็บไม่ครบ ต้องมีวงเล็บปิด)
7. $p \vee q \rightarrow r \square$ หรือ $\square$? (ถูกต้องตาม precedence แต่ควรเขียนว่า $(p \vee q) \rightarrow r$ เพื่อความชัดเจน)

ทฤษฎีบท 1.47 (ลำดับการทำงานของตัวเชื่อม (Operator Precedence)) เมื่อไม่มีวงเล็บ ลำดับการทำงาน (Precedence) ของตัวเชื่อมจะเป็นดังนี้ (จากสูงไปต่ำ):

$$\neg > \wedge > \vee > \oplus > \rightarrow > \leftrightarrow$$

นี้เหมือนกับลำดับการทำงานของตัวดำเนินการทางคณิตศาสตร์ ที่การคูณทำก่อนการบวก ดังนั้น:

1. $\neg$ ทำก่อน $\wedge$ เหมือนกับ $\neg$ มี precedence สูงสุด
2. $\wedge$ ทำก่อน $\vee$
3. $\vee$ ทำก่อน $\rightarrow$
4. $\rightarrow$ ทำก่อน $\leftrightarrow$

หลักการจำง่าย ๆ: "N A D O X I B" ซึ่งอ่านว่า "Not And Or Xor Imply Biconditional"

ตัวอย่าง 1.48 (การตีความประโยคโดยไม่มีวงเล็บ) จงตีความ $p \vee q \rightarrow r$:

1. ใช้ลำดับ Precedence: $p \vee q \rightarrow r \equiv (p \vee q) \rightarrow r$
2. ถ้าต้องการให้ $\vee$ ทำทีหลัง (กล่าวคือ $p \vee (q \rightarrow r)$) ต้องเขียนวงเล็บชัดเจน

ความแตกต่างมีผลต่อความหมาย:

1. $(p \vee q) \rightarrow r$: "ถ้า p หรือ q แล้ว r "
2. $p \vee (q \rightarrow r)$: " p หรือ (ถ้า q แล้ว r)"

ตัวอย่าง 1.49 (SAT Solver — การแปลงประโยคเป็น CNF) ใน SAT Solver ปัญหาจะถูกแปลงให้อยู่ในรูปแบบ Conjunctive Normal Form (CNF) ซึ่งเป็นรูปแบบมาตรฐานที่ SAT Solver เข้าใจได้

CNF คือ Conjunction ของ Clauses โดยแต่ละ Clause เป็น Disjunction ของ Literals:

$$(\neg x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_2 \vee x_3 \vee x_4)$$

ขั้นตอนการแปลงเป็น CNF:

1. **ลบ Biconditional ($\leftrightarrow$):** $p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
2. **ลบ Implication ($\rightarrow$):** $p \rightarrow q \equiv \neg p \vee q$
3. **ย้าย Negation เข้าข้างใน:** ใช้กฎของ De Morgan และ Double Negation
4. **แจกแจง OR บน AND:** ใช้ Distributive Law

Example: transform $(p \leftrightarrow q)$ into CNF

Step 1: $p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$

Step 2: $(\neg p \vee q) \wedge (\neg q \vee p)$

Result: $(\neg p \vee q) \wedge (\neg q \vee p)$ # CNF ready for SAT Solver

DIMACS CNF format example (standard format for SAT problems):

p cnf 2 2

-1 2 0

```
# -2 1 0
```

```
# This means:
```

```
# - Clause 1: ( $\neg x$   $x$ )
```

```
# - Clause 2: ( $\neg x$   $x$ )
```

```
# - Therefore the formula is ( $\neg x$   $x$ ) ( $\neg x$   $x$ )
```

1.7 ประโยคเปิดและตัวบ่งปริมาณ (Open Sentences and Quantifiers)

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจความแตกต่างระหว่างประโยคเปิดและประพจน์
- ✓ ใช้ตัวบ่งปริมาณ $\forall$ และ $\exists$
- ✓ เข้าใจการนิเสธของประโยคที่มีตัวบ่งปริมาณ

1.7.1 บทนำ: ความต้องการตัวบ่งปริมาณ

1. ตัวบ่งปริมาณอวกาศ (Universal Quantifier) $\forall$ อ่านว่า "สำหรับทุก" หรือ "ทุก"
2. ตัวบ่งปริมาณนิเสธ (Existential Quantifier) $\exists$ อ่านว่า "มี" หรือ "มีอย่างน้อยหนึ่ง"

ในชีวิตจริง เรามักพูดถึงประโยคที่มีตัวแปรอยู่เสมอ ตัวอย่างเช่น เมื่อเราพูดว่า "ทุกคนต้องหายใจ" เรากำลังพูดถึงคนทุกคนในโลก ไม่ใช่คนเฉพาะคนใดคนหนึ่ง หรือเมื่อเราพูดว่า "มีคนบางคนที่ฉลาดกว่าฉัน" เรากำลังพูดถึงอย่างน้อยหนึ่งคนที่มีคุณสมบัติบางอย่าง ในทางตรรกศาสตร์เชิงประพจน์ เราไม่สามารถแทนประโยคเหล่านี้ได้โดยตรง เพราะประโยคเหล่านี้มีตัวแปร ("คน") อยู่

ตัวบ่งปริมาณ (Quantifiers) เป็นเครื่องมือที่ใช้ในการแปลงประโยคที่มีตัวแปรให้เป็นประพจน์ที่มีค่าความจริงชัดเจน มีตัวบ่งปริมาณหลักสองตัว:

การเข้าใจตัวบ่งปริมาณมีความสำคัญมากในการเขียนโปรแกรมคอมพิวเตอร์ โดยเฉพาะในภาษาที่สนับสนุนการเขียนโปรแกรมเชิงฟังก์ชัน (Functional Programming) หรือการเขียน SQL queries ที่ซับซ้อน นอกจากนี้ ตัวบ่งปริมาณยังถูกใช้ในการนิยามความถูกต้องของอัลกอริทึม (Algorithm Correctness) และในการพิสูจน์ทางคณิตศาสตร์

นิยาม 1.50 (ประโยคเปิด) ประโยคเปิด หรือ Open Sentence คือข้อความที่มี **ตัวแปร (variable)** และไม่สามารถระบุค่าความจริงได้จนกว่าจะแทนค่าของตัวแปร

เมื่อกำหนดค่าให้ตัวแปรครบทุกตัวแล้ว ประโยคเปิดจะกลายเป็นประพจน์

ในทางคณิตศาสตร์ ประโยคเปิดเรียกว่า "Predicate" และมักเขียนเป็น $P(x)$, $Q(x, y)$ เป็นต้น

ตัวอย่าง 1.51 (ประโยคเปิดในความหมายทางคณิตศาสตร์) ให้ $P(x, y) : "x + y = 10"$

1. $P(3, 7)$ เป็นประพจน์ที่มีค่าความจริงเป็น **จริง** (เพราะ $3 + 7 = 10$)

2. $P(4, 5)$ เป็นประพจน์ที่มีค่าความจริงเป็น **เท็จ** (เพราะ $4 + 5 = 9 \neq 10$)

3. $P(x, y)$ เป็นประโยคเปิดที่มี 2 ตัวแปร

ในภาษาโปรแกรม:

```
# Python: Predicate Function
```

```
def P(x, y):
```

```
    return x + y == 10 # returns True/False
```

```
>>> P(3, 7)    #  $\rightarrow$  True
```

```
>>> P(4, 5)    #  $\rightarrow$  False
```

```
>>> P(x, y)    #  $\rightarrow$  NameError: x, y are not defined
```

1.7.2 ตัวบ่งปริมาณอวกาศ (Universal Quantifier): $\forall$

ตัวบ่งปริมาณอวกาศ $\forall$ ใช้เมื่อเราต้องการพูดถึง "ทุกสมาชิก" ในโดเมนที่กำหนด $\forall x : P(x)$ อ่านว่า "สำหรับทุก x , $P(x)$ เป็นจริง" หรือ "ทุก x ที่มีคุณสมบัติ P " สิ่งสำคัญที่ต้องเข้าใจคือ $\forall x : P(x)$ จะเป็นจริงก็ต่อเมื่อ $P(x)$ เป็นจริงสำหรับทุกค่า x ในโดเมน ถ้าโดเมนว่างเปล่า $\forall x : P(x)$ จะเป็นจริงโดยนิยาม (vacuous truth) เพราะไม่มี x ที่ต้องตรวจสอบ ตัวอย่างเช่น ถ้าโดเมนคือเซตของจำนวนเต็ม $\mathbb{Z}$ แล้ว $\forall n \in \mathbb{Z} : n^2 \geq 0$ เป็นจริง เพราะกำลังสองของทุกจำนวนเต็มไม่เป็นลบ ในทางกลับกัน $\forall n \in \mathbb{Z} : n > 0$ เป็นเท็จ เพราะ $-1, -2, -3$ เป็นจำนวนเต็มที่ไม่มากกว่าศูนย์

นิยาม 1.52 (ตัวบ่งปริมาณอวกาศ) $\forall x : P(x)$ อ่านว่า "สำหรับทุก x , $P(x)$ เป็นจริง" หรือ "ทุก x ที่มีคุณสมบัติ P "

$\forall x : P(x) \equiv$ จริง เมื่อ $P(x)$ เป็นจริงสำหรับทุกค่า x ในโดเมน

ตัวอย่าง 1.53 (การใช้ $\forall$ ในชีวิตจริงและคณิตศาสตร์) ให้ $\mathbb{Z}$ เป็นโดเมน (เซตของจำนวนเต็ม)

1. $\forall n \in \mathbb{Z} : n^2 \geq 0$ — จริง (กำลังสองของทุกจำนวนเต็มไม่เป็นลบ)

2. $\forall n \in \mathbb{Z} : n > 0$ — เท็จ (จำนวนเต็มลบไม่ > 0)

3. $\forall x \in \mathbb{R} : x^2 = x$ — เท็จ (เช่น $2^2 = 4 \neq 2$)

4. $\forall x \in \mathbb{R} : x^2 \geq 0$ — จริง (กำลังสองของทุกจำนวนจริงไม่เป็นลบ)

ในภาษาโปรแกรม:

```
# Python: checking

def check_universal(prop, domain):

    """Check if prop(x) is true for all x in domain"""

    return all(prop(x) for x in domain)

# Example:

>>> check_universal(lambda n: n**2 >= 0, range(-10, 11))

True

>>> check_universal(lambda n: n > 0, range(-10, 11))

False
```

1.7.3 ตัวบ่งปริมาณนิเสธ (Existential Quantifier): $\exists$

ตัวบ่งปริมาณนิเสธ $\exists$ ใช้เมื่อเราต้องการพูดถึง "มีอย่างน้อยหนึ่งสมาชิก" ในโดเมนที่กำหนด $\exists x : P(x)$ อ่านว่า "มี x อย่างน้อยหนึ่งตัวที่ทำให้ $P(x)$ เป็นจริง" หรือ "มี x ที่มีคุณสมบัติ P " สิ่งสำคัญที่ต้องเข้าใจคือ $\exists x : P(x)$ จะเป็นจริงก็ต่อเมื่อ มี x อย่างน้อยหนึ่งตัวในโดเมนที่ทำให้ $P(x)$ เป็นจริง ถ้าโดเมนว่างเปล่า $\exists x : P(x)$ จะเป็นเท็จโดยนิยาม เพราะไม่มี x ให้ตรวจสอบ ตัวอย่างเช่น $\exists n \in \mathbb{Z} : n^2 = 4$ เป็นจริง เพราะมี $n = 2$ หรือ $n = -2$ ที่ยกกำลังสองเท่ากับ 4 ในทางกลับกัน $\exists n \in \mathbb{Z} : n^2 = -1$ เป็นเท็จ เพราะไม่มีจำนวนเต็มใดยกกำลังสองได้ -1

นิยาม 1.54 (ตัวบ่งปริมาณนิเสธ) $\exists x : P(x)$ อ่านว่า "มี x อย่างน้อยหนึ่งตัว ที่ทำให้ $P(x)$ เป็นจริง" หรือ "มี x ที่มีคุณสมบัติ P "

$\exists x : P(x) \equiv$ จริง เมื่อ มี x อย่างน้อยหนึ่งตัวในโดเมน ที่ทำให้ $P(x)$ จริง

ตัวอย่าง 1.55 (การใช้ $\exists$ ในชีวิตจริงและคณิตศาสตร์) ให้ $\mathbb{Z}$ เป็นโดเมน

1. $\exists n \in \mathbb{Z} : n^2 = 4$ — จริง (มี $n = 2$ หรือ $n = -2$)
2. $\exists n \in \mathbb{Z} : n^2 = -1$ — เท็จ (ไม่มีจำนวนเต็มใดยกกำลังสองได้ -1)

3. $\exists x \in \mathbb{R} : x^2 = 2$ — จริง (มี $x = \sqrt{2}$ หรือ $x = -\sqrt{2}$)

ในภาษาโปรแกรม:

```
# Python: checking
```

```
def check_existential(prop, domain):
```

```
    """Check if there exists at least one x in domain such that prop(x) is true"""
```

```
    return any(prop(x) for x in domain)
```

```
# Example:
```

```
>>> check_existential(lambda n: n**2 == 4, range(-10, 11))
```

```
True
```

```
>>> check_existential(lambda n: n**2 == -1, range(-10, 11))
```

```
False
```

```
>>> check_existential(lambda x: x**2 == 2, [1.4, 1.41, 1.414, 1.4142])
```

```
True
```

ทฤษฎีบท 1.56 (การนิเสธของตัวบ่งปริมาณ) การนิเสธของตัวบ่งปริมาณเป็นกฎที่สำคัญมากที่ต้องจำ:

$\neg(\forall x : P(x)) \equiv \exists x : \neg P(x)$ “ไม่ใช่ทุก x มี P ” $\equiv$ “มี x ที่ไม่มี P ”

$\neg(\exists x : P(x)) \equiv \forall x : \neg P(x)$ “ไม่มี x ที่มี P ” $\equiv$ “ทุก x ไม่มี P ”

วิธีจำ: $\neg\forall \equiv \exists\neg$ และ $\neg\exists \equiv \forall\neg$

นี่คือกฎที่สำคัญมากในการพิสูจน์และการเขียนโปรแกรม

Proof. พิสูจน์โดยตารางค่าความจริง (สมมูลในกรณี $\forall$):

สมมติโดเมนมี 3 ค่า: $\{a, b, c\}$

$$\neg(\forall x : P(x)) \equiv \neg(P(a) \wedge P(b) \wedge P(c))$$

$$\equiv \neg P(a) \vee \neg P(b) \vee \neg P(c) \quad (\text{De Morgan})$$

$$\equiv \exists x : \neg P(x)$$

สำหรับ $\neg\exists$:

$$\neg(\exists x : P(x)) \equiv \neg(P(a) \vee P(b) \vee P(c))$$

$$\equiv \neg P(a) \wedge \neg P(b) \wedge \neg P(c) \quad (\text{De Morgan})$$

$$\equiv \forall x : \neg P(x)$$



ตัวอย่าง 1.57 (การนิเสธของตัวบ่งปริมาณในโปรแกรม) การใช้ Quantifiers กับ List ใน Python:

```
students = [
    {"name": "Alice", "grade": 85, "active": True},
    {"name": "Bob", "grade": 72, "active": True},
    {"name": "Charlie", "grade": 90, "active": False},
]
```

```
# : everyone passed the criteria (grade >= 60)
all_passed = all(s["grade"] >= 60 for s in students)
# equivalent to: x: (grade(x) >= 60)
```

```
# Negation:  $\neg x: P(x)$      $x: \neg P(x)$ 
# "not everyone passed"    "someone failed"
has_failed = any(s["grade"] < 60 for s in students)
```

```
# : someone got grade A (>= 80)
has_A = any(s["grade"] >= 80 for s in students)
# equivalent to: x: (grade(x) >= 80)
```

```
# Negation:  $\neg x: P(x)$      $x: \neg P(x)$ 
# "nobody got grade A"    "everyone got grade < 80"
no_A = all(s["grade"] < 80 for s in students)
```

ตัวอย่าง 1.58 (ตัวบ่งปริมาณซ้อนกัน (Nested Quantifiers)) ให้ $L(x, y)$: “ x ชอบ y ” โดยที่ x, y อยู่ใน

เซตของคน $\{A, B, C\}$

ลำดับของตัวบ่งปริมาณมีความสำคัญมาก! $\forall\exists$ ไม่เท่ากับ $\exists\forall$ เสมอไป

1. $\forall x\forall y : L(x, y)$ — ทุกคนชอบทุกคน (รวมตัวเอง)
2. $\exists x\exists y : L(x, y)$ — มีคนที่ชอบอีกคนอย่างน้อยหนึ่งคู่
3. $\forall x\exists y : L(x, y)$ — ทุกคนมีคนที่ชอบอย่างน้อยหนึ่งคน (อาจเป็นคนต่างกันในแต่ละกรณี)
4. $\exists x\forall y : L(x, y)$ — มีคนที่ชอบทุกคน (คนดังที่ทุกคนชอบ)

ในภาษาโปรแกรม:

```
# Python: nested quantifiers
```

```
people = ["Alice", "Bob", "Charlie"]
```

```
likes = { # dict of dict: likes[a][b] = True if a likes b
```

```
    "Alice": {"Bob": True, "Charlie": True},
```

```
    "Bob": {"Alice": False, "Charlie": True},
```

```
    "Charlie": {"Alice": True, "Bob": True},
```

```
}
```

```
# x y: L(x,y)
```

```
all_like_all = all(likes[x][y] for x in people for y in people)
```

```
# x y: L(x,y) - everyone likes someone
```

```
everyone_likes_someone = all(any(likes[x][y] for y in people) for x in people)
```

```
# x y: L(x,y) - someone liked by everyone (celebrity)
```

```
someone_liked_by_all = any(all(likes[y][x] for y in people) for x in people)
```

1.8 ประพจน์ที่มีค่าความจริงพิเศษ (Special Truth Values)

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจแนวคิดของ Tautology, Contradiction และ Contingency
- ✓ ใช้หลักการของ Tautology ในการพิสูจน์
- ✓ วิเคราะห์ Contingency ในการประยุกต์

1.8.1 บทนำ: ประพจน์พิเศษในทางตรรกศาสตร์

ในการศึกษาตรรกศาสตร์เชิงประพจน์ เราแบ่งประพจน์ออกเป็นสามประเภทตามรูปแบบของค่าความจริงที่ปรากฏในตารางความจริง ประเภทแรกคือ Tautology ซึ่งเป็นประพจน์ที่เป็นจริงทุกกรณีโดยไม่ขึ้นกับค่าของตัวแปรใดๆ ประเภทที่สองคือ Contradiction ซึ่งเป็นประพจน์ที่เป็นเท็จทุกกรณี และประเภทที่สามคือ Contingency ซึ่งเป็นประพจน์ที่มีค่าความจริงเป็นจริงบ้างเท็จบ้างขึ้นอยู่กับค่าของตัวแปร

การเข้าใจ ประเภทของ ประพจน์มีความสำคัญมากในการ ประยุกต์ ตัวอย่างเช่น ในการเขียนโปรแกรม ถ้าเรารู้ว่าเงื่อนไขบางอย่างเป็น Tautology (จริงเสมอ) เราสามารถลบเงื่อนไขนั้นออกได้เพราะไม่มีผลต่อการทำงาน ในทางกลับกัน ถ้าเรารู้ว่าเงื่อนไขบางอย่างเป็น Contradiction (เท็จเสมอ) เราสามารถลบโค้ดในบล็อกนั้นออกได้เพราะโค้ดนั้นจะไม่มีวันถูก ☐☐

ในด้านการพิสูจน์ทางคณิตศาสตร์ Tautology มีความสำคัญมากเพราะเป็นรูปแบบของการให้เหตุผลที่ถูกต้อง ถ้าเราสามารถแสดงได้ว่าการอ้างเหตุผล (Argument) สามารถเขียนเป็น Tautology ได้ แล้วการอ้างเหตุผลนั้นถูกต้อง (Valid) เสมอ

นิยาม 1.59 (ประพจน์พิเศษทางตรรกศาสตร์)

1. Tautology (กฎ ตรรกะ แม่นยำ):

ประพจน์ที่มีค่าความจริงเป็น **จริงทุกกรณี** ขึ้นอยู่กับค่าของตัวแปร

2. Contradiction (ขัดแย้ง): ประพจน์ที่มีค่าความจริงเป็น **เท็จทุกกรณี** ขึ้นอยู่กับค่าของตัวแปร

3. Contingency (ตามมูลบท): ประพจน์ที่มีค่าความจริงเป็น **จริงบ้าง เท็จบ้าง** ขึ้นอยู่กับค่าของตัวแปร

ทฤษฎีบท 1.60 (ตัวอย่าง Tautology และ Contradiction ที่สำคัญ)

ชื่อ	สมมูล
Law of Excluded Middle (กฎตัดค่ากลาง)	$p \vee \neg p$
Law of Non-Contradiction (กฎไม่ขัดแย้ง)	$\neg(p \wedge \neg p)$
Identity	$p \rightarrow p$
Modus Ponens	$[(p \rightarrow q) \wedge p] \rightarrow q$
Modus Tollens	$[(p \rightarrow q) \wedge \neg q] \rightarrow \neg p$
Hypothetical Syllogism	$[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$
Absorption	$p \rightarrow (p \vee q)$

Contradiction ตัวอย่าง: $p \wedge \neg p$ (ประโยคที่ขัดแย้งในตัวเอง — ไม่มีทางเป็นจริงได้)

ตัวอย่าง 1.61 (การพิสูจน์ Tautology โดยการแปลงรูป) พิสูจน์ว่า $(p \rightarrow q) \vee (q \rightarrow p)$ เป็น Tautology

Proof.

$$(p \rightarrow q) \vee (q \rightarrow p) \equiv (\neg p \vee q) \vee (\neg q \vee p) \quad (\text{Implication as OR})$$

$$\equiv \neg p \vee q \vee \neg q \vee p \quad (\text{Associative})$$

$$\equiv \neg p \vee p \vee q \vee \neg q \quad (\text{Commutative})$$

$$\equiv \mathbf{T} \vee \mathbf{T} \quad (\text{Complement: } p \vee \neg p \equiv \mathbf{T})$$

$$\equiv \mathbf{T} \quad (\text{Domination: } \mathbf{T} \vee \mathbf{T} \equiv \mathbf{T})$$

เป็น Tautology เพราะแปลงได้เป็น **T** (จริงเสมอ) ไม่ว่า p และ q จะมีค่าอย่างไร □

ตัวอย่าง 1.62 (SAT Problem — Satisfiability และ Validity) ในทฤษฎีการคำนวณ ประพจน์สามารถจำแนกได้เป็น:

1. **Satisfiable:** ประพจน์ที่มีอย่างน้อยหนึ่งกรณีที่เป็นจริง

1.1. $p \vee q$ เป็น Satisfiable (เช่น $p = \mathbf{T}, q = \mathbf{F}$)

1.2. $p \wedge \neg p$ ไม่เป็น Satisfiable (เพราะเป็น Contradiction)

2. **Unsatisfiable (Contradiction):** ประพจน์ที่เป็นเท็จทุกกรณี

2.1. $p \wedge \neg p$ เป็น Unsatisfiable

2.2. $\neg(p \vee q) \wedge p \wedge q$ เป็น Unsatisfiable

3. **Valid (เทียบเท่า Tautology):** ประพจน์ที่เป็นจริงทุกกรณี

3.1. $p \rightarrow (q \rightarrow p)$ เป็น Valid/Tautology

3.2. $p \vee \neg p$ เป็น Valid/Tautology

ความสัมพันธ์ระหว่าง Satisfiability และ Validity:

1. Valid $\square$ นิเสธเป็น Unsatisfiable

2. Satisfiable $\square$ นิเสธไม่เป็น Valid

1.9 การอ้างเหตุผลและกฎของการอนุมาน (Rules of Inference)

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจการอ้างเหตุผล (Argument) และความสมเหตุสมผล (Validity)
- ✓ เรียนรู้กฎของการอนุมานพื้นฐาน
- ✓ รู้จัก Fallacies และการหลีกเลี่ยง

1.9.1 บทนำ: การอ้างเหตุผลคืออะไร

$$\frac{P_1, P_2, \dots, P_n}{\therefore C}$$

ในชีวิตประจำวัน เรามักได้ยินการโต้แย้งหรือการให้เหตุผล ตัวอย่างเช่น นักการเมืองอาจพูดว่า "ถ้าเราเพิ่มภาษี แล้วเศรษฐกิจจะดีขึ้น" หรือนักวิทยาศาสตร์อาจพูดว่า "ถ้าสารเคมีนี้ทำปฏิกิริยากับออกซิเจน แล้วจะเกิดการเผาไหม้" ในทางตรรกศาสตร์ เราเรียกการให้เหตุผลเหล่านี้ว่า "การอ้างเหตุผล" (Argument)

การอ้างเหตุผลประกอบด้วยสองส่วนหลัก ส่วนแรกคือ "เหตุ" (Premises) ซึ่งเป็นข้อความที่เรายอมรับว่าจริงหรือสมมติว่าจริง และส่วนที่สองคือ "ผล" (Conclusion) ซึ่งเป็นข้อความที่เราต้องการอนุมานจากเหตุ การอ้างเหตุผลเขียนอยู่ในรูปแบบ: โดยที่ $P_1, P_2, \dots, P_n$ คือเหตุ และ C คือผล ความสำคัญของการอ้างเหตุผลอยู่ที่ความถูกต้องของรูปแบบ ไม่ใช่ความจริงของเนื้อหา ในทางตรรกศาสตร์ เราสนใจว่า "ถ้าเหตุทั้งหมดเป็นจริง ผลจะต้องเป็นจริงหรือไม่" ไม่ใช่ว่าเหตุจริงหรือเท็จในโลกจริง ถ้าการอ้างเหตุผลมีรูปแบบที่ถูกต้อง เราเรียกว่า "สมเหตุสมผล" (Valid) ถ้าไม่ เราเรียกว่า "ไม่สมเหตุสมผล" หรือเป็น "เหตุผลวิบัติ" (Fallacy)

นิยาม 1.63 (การอ้างเหตุผล) **Argument** คือลำดับของประพจน์ที่เรียกว่า **เหตุ (Premises)** ตามด้วยประพจน์สุดท้ายที่เรียกว่า **ผล (Conclusion)**

เขียนแทนด้วย:

$$\frac{P_1, P_2, \dots, P_n}{\therefore C}$$

1. **Valid:** ถ้าเหตุทั้งหมดจริง แล้วผลต้องจริง
2. **Invalid (Fallacy):** ถ้ามีกรณีเหตุจริงทั้งหมดแต่ผลเท็จ

3. Sound: Valid + เหตุทั้งหมดเป็นจริงในโลกจริง

ทฤษฎีบท 1.64 (Modus Ponens (การยืนยัน)) Modus Ponens เป็นกฎของการอนุมานที่ใช้บ่อยที่สุดในการให้เหตุผล กฎนี้กล่าวว่า:

$$\frac{p \rightarrow q, \quad p}{\therefore q}$$

อ่านว่า: ถ้า $p \rightarrow q$ เป็นจริง และ p เป็นจริง แล้ว q ต้องเป็นจริง

Modus Ponens เป็น Valid. ใช้ตารางค่าความจริง:

p	q	$p \rightarrow q$	$(p \rightarrow q) \wedge p$	$[(p \rightarrow q) \wedge p] \rightarrow q$
T	T	T	T	T
T	F	F	F	T
F	T	T	F	T
F	F	T	F	T

คอลัมน์สุดท้ายเป็น T ทุกแถว $\rightarrow$ เป็น Tautology $\rightarrow$ Valid $\square$

$\square$

ตัวอย่าง 1.65 (การประยุกต์ Modus Ponens: Logic Programming) Modus Ponens เป็นหลักการพื้นฐานของ Logic Programming และ Automated Reasoning

Prolog uses Modus Ponens simply:

Rule: $\text{rain}(X) \rightarrow \text{wet}(X)$ (if rain(X), then floor wet(X))

Fact: $\text{rain}(\text{beijing})$ (rain in Beijing)

Conclusion: $\text{wet}(\text{beijing})$ (floor in Beijing is wet)

Python example (Simple KB):

knowledge_base = {

$\text{"rain_} \rightarrow \text{wet_}"$, # if rain anywhere, floor is wet there

"rain(beijing)" # rain in Beijing

```
}
```

```
def query(KB, target):
```

```
    # Modus Ponens: if  $P \rightarrow Q$  and  $P$  is true, then  $Q$  is true
```

```
    for rule in KB:
```

```
        if match(rule.antecedent, target):
```

```
            return True
```

```
    return False
```

```
>>> query(KB, "wet(beijing)")
```

```
True # because rain(beijing) matches rain(X)  $\rightarrow$  wet(beijing)
```

ใน AI: KB $\Box$ wet(beijing) เป็นจริง (Entailment) — ระบบสามารถอนุมาน wet(beijing) จาก KB ได้

ทฤษฎีบท 1.66 (Modus Tollens (การปฏิเสธ)) Modus Tollens เป็นกฎของการอนุมานที่คล้ายกับ Modus Ponens แต่ใช้นิเสธของผลในการอนุมานนิเสธของเหตุ:

$$\frac{p \rightarrow q, \quad \neg q}{\therefore \neg p}$$

อ่านว่า: ถ้า $p \rightarrow q$ เป็นจริง และ q เป็นเท็จ แล้ว p ต้องเป็นเท็จ

Modus Tollens เป็น Valid. ใช้ Contrapositive: $p \rightarrow q \equiv \neg q \rightarrow \neg p$

$$\frac{\neg q \rightarrow \neg p, \quad \neg q}{\therefore \neg p}$$

ซึ่งเป็น Modus Ponens ในรูปแบบ (ด้วยนิเสธ) ดังนั้น Valid $\Box$

ตัวอย่าง 1.67 (การประยุกต์ Modus Tollens: Debugging) Modus Tollens เป็นเครื่องมือที่มีประโยชน์ในการ debug โปรแกรมหรือระบบ

```
# Example: checking if input value is valid
```

```
# Fact: if input > 0 then function returns positive
```



```
# Fact: function returns negative
# Conclusion: input 0

# If we get result < 0  $\rightarrow$  use Modus Tollens:
# p = input > 0
# q = result > 0
# p  $\rightarrow$  q (if input > 0 then result > 0)
#  $\neg$ q (result < 0)
#  $\neg$ p (input 0)
```

ทฤษฎีบท 1.68 (รูปแบบ Syllogisms ที่สำคัญ) นอกจาก Modus Ponens และ Modus Tollens แล้ว ยังมี Syllogisms ที่สำคัญหลายรูปแบบ:

Hypothetical Syllogism (ต่อเนื่อง):

$$\frac{p \rightarrow q, \quad q \rightarrow r}{\therefore p \rightarrow r}$$

อ่านว่า: ถ้า p แล้ว q และถ้า q แล้ว r แล้ว ถ้า p แล้ว r

Disjunctive Syllogism (แยก):

$$\frac{p \vee q, \quad \neg p}{\therefore q}$$

อ่านว่า: ถ้า p หรือ q และไม่ใช่ p แล้ว q

Constructive Dilemma (สองเงื่อนไข):

$$\frac{(p \rightarrow q) \wedge (r \rightarrow s), \quad p \vee r}{\therefore q \vee s}$$

อ่านว่า: ถ้า p แล้ว q และถ้า r แล้ว s และ (p หรือ r) แล้ว (q หรือ s)

ตัวอย่าง 1.69 (Disjunctive Syllogism ใน SQL) Disjunctive Syllogism ถูกใช้ในการ filter ข้อมูลในฐานข้อมูล

```
--      :

--      : status = 'working' OR status = 'overtime'
--      : status != 'working' (NOT      )
--      : status = 'overtime'
```

```
-- SQL:

SELECT * FROM employees

WHERE status = 'overtime';

-- Logic: (p → q) → ¬p → q
--      "      " + "      " → "      "
-- p = status = 'working'
-- q = status = 'overtime'
-- p → q = status = 'working' OR status = 'overtime'
-- ¬p = status != 'working'
-- q = status = 'overtime'
```

ทฤษฎีบท 1.70 (Fallacies (เหตุผลวิบัติ)) เหตุผลวิบัติ (Fallacy) คือการอ้างเหตุผลที่ไม่สมเหตุสมผล แม้ว่าอาจจะดูสมเหตุสมผลในเบื้องต้น

Fallacy 1: Affirming the Consequent (ยืนยันผล)

$$\frac{p \rightarrow q, \quad q}{\therefore p}$$

ผิด! q อาจเกิดจากสาเหตุอื่นที่ไม่ใช่ p

Fallacy 2: Denying the Antecedent (ปฏิเสธเหตุ)

$$\frac{p \rightarrow q, \quad \neg p}{\therefore \neg q}$$

ผิด! q อาจเกิดจากเงื่อนไขอื่นที่ไม่ใช่ p

ตัวอย่าง 1.71 (Fallacy ในชีวิตจริง)

Fallacy 1 (Affirming Consequent):

เหตุ:

“ถ้าอยู่ในกรุงเทพ แล้วอยู่ในประเทศไทย”

เหตุ: “อยู่ในประเทศไทย”

ผล(ผิด): “อยู่ในกรุงเทพ”

เพราะ: อาจอยู่ที่เชียงใหม่, ภูเก็ต, ฯลฯ (ประเทศไทยมีหลายที่)

Fallacy 2 (Denying Antecedent):

เหตุ: “ถ้าเป็นหวัด แล้วจะมีไข้”

เหตุ: “ไม่เป็นหวัด”

ผล(ผิด): “ไม่มีไข่”

เพราะ: ไข่อาจเกิดจากสาเหตุอื่น (ไข่หัวโตใหญ่, ตีตเชื้อ, ไข่เลื้อยตอก, ฯลฯ)

ทฤษฎีบท 1.72 (การตรวจสอบ Validity ด้วย Truth Table) Argument $\frac{P_1, P_2, \dots, P_n}{\therefore C}$ เป็น Valid ก็ต่อเมื่อ $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow C$ เป็น Tautology

วิธีตรวจสอบ:

1. สร้างตารางค่าความจริงของ $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow C$
2. ถ้าผลลัพธ์เป็น T ทุกแถว $\rightarrow$ Valid
3. ถ้ามีแถวใดที่เหตุทั้งหมดเป็น T แต่ผลเป็น F $\rightarrow$ Invalid

ตัวอย่าง 1.73 (การตรวจสอบ Valid ด้วย Tautology) ตรวจสอบว่า $\frac{(p \rightarrow q) \wedge (q \rightarrow r)}{\therefore p \rightarrow r}$ Valid หรือไม่

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$(p \rightarrow q) \wedge (q \rightarrow r)$	$p \rightarrow r$	$[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$
T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	T
T	F	T	F	T	F	T	T
T	F	F	F	T	F	F	T
F	T	T	T	T	T	T	T
F	T	F	T	F	F	T	T
F	F	T	T	T	T	T	T
F	F	F	T	T	T	T	T

คอลัมน์สุดท้ายเป็น T ทุกแถว $\rightarrow$ เป็น Tautology $\rightarrow$ Valid $\square$ (Hypothetical Syllogism)

หมายเหตุ 1.74 (การประยุกต์: การพิสูจน์ Algorithm Correctness) การพิสูจน์ว่า Algorithm ถูกต้อง = การตรวจสอบ Validity ของ Argument

Algorithm: Binary Search

Argument:

Fact: Precondition LoopInvariant TerminatingCondition

Conclusion: Postcondition

```
#  
  
# If (Precondition  LoopInvariant  TerminatingCondition)  $\rightarrow$  Postcondition is a Ta  
# and every iteration maintains LoopInvariant  
# and TerminatingCondition is guaranteed to occur  
#  $\rightarrow$  Algorithm is correct  
  
# This is the principle of Hoare Logic and Formal Verification  
# Used for proving program correctness formally
```

1.10 แบบฝึกหัดท้ายบท

แบบฝึกหัด 1.1 จงระบุว่าข้อความต่อไปนี้ประพจน์หรือไม่ พร้อมระบุค่าความจริง:

- (ก) $3 + 5 = 8$
- (ข) $x > 2$ (เมื่อ x ไม่ระบุค่า)
- (ค) กรุณาปิดประตู
- (ง) จำนวนเฉพาะที่เล็กที่สุดคือ 2
- (จ) ข้าวเป็นอาหารหลักของคนไทย

แบบฝึกหัด 1.2 จงสร้างตารางค่าความจริงของตัวเชื่อมต่อไปนี้:

- (ก) $\neg p \vee q$
- (ข) $p \oplus q$ (XOR)
- (ค) $(p \wedge q) \rightarrow r$
- (ง) $\neg p \leftrightarrow (q \wedge r)$

แบบฝึกหัด 1.3 จงใช้กฎของ De Morgan แปลงประโยคต่อไปนี้:

- (ก) $\neg(p \wedge q) \rightarrow r$
- (ข) $\neg(\forall x : P(x))$
- (ค) “ไม่มีนักศึกษาที่สอบตกทุกวิชา”

แบบฝึกหัด 1.4 (ก) จงออกแบบวงจร Half Adder โดยใช้ AND, OR, XOR gates

- (ข) ถ้าอินพุตคือ $A = 1, B = 1$ ผลลัพธ์ Sum และ Carry คืออะไร?
- (ค) จงเขียนตารางค่าความจริงของ Full Adder (มี 3 อินพุต)

แบบฝึกหัด 1.5 พิสูจน์ว่าประพจน์ต่อไปนี้สมมูลกัน (โดยตารางหรือการแปลงรูป):

(ก) $p \rightarrow q \equiv \neg p \vee q$

(ข) $\neg(p \rightarrow q) \equiv p \wedge \neg q$

(ค) $p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$

(ง) $(p \rightarrow q) \wedge (q \rightarrow p) \equiv p \leftrightarrow q$

แบบฝึกหัด 1.6 ให้ $E(x)$: “ x มีค่ามากกว่า 10”

(ก) เขียนประโยค “ทุกจำนวนมากกว่า 10 มากกว่า 5” ด้วย $\forall$

(ข) เขียนประโยค “ไม่มีจำนวนมากกว่า 10 ที่น้อยกว่า 5” ด้วย $\neg, \exists$

(ค) เขียนนิเสธของ $\forall x : E(x)$

(ง) เขียนนิเสธของ $\exists x : \neg E(x)$

แบบฝึกหัด 1.7 พิสูจน์โดยตรงว่า: ถ้า n เป็นจำนวนคี่ แล้ว n^3 เป็นจำนวนคี่

แบบฝึกหัด 1.8 พิสูจน์โดย Contraposition ว่า: ถ้า $3n + 2$ เป็นจำนวนคี่ แล้ว n เป็นจำนวนคี่

แบบฝึกหัด 1.9 พิสูจน์โดย Contradiction ว่า: ไม่มีจำนวนเต็ม n ที่ทำให้ $n^2 = 2$

แบบฝึกหัด 1.10 พิสูจน์โดย Cases ว่า: $|ab| = |a| \cdot |b|$ สำหรับทุกจำนวนจริง a, b (แนะนำ: แยกเป็นกรณี $a \geq 0$ และ $a < 0$)

แบบฝึกหัด 1.11 จงระบุว่าการอ้างเหตุผลต่อไปนี้ใช้รูปแบบใด และสมเหตุสมผลหรือไม่:

เหตุ: ถ้าฝนตกแล้วถนนเปียก

(ก) เหตุ: ฝนตก

ผล: ถนนเปียก

เหตุ: ถ้าวันนี้วันจันทร์แล้วมีเรียน

(ข) เหตุ: มีเรียน

ผล: วันนี้วันจันทร์

เหตุ: ถ้าตอบข้อสอบถูกหมดแล้วได้เต็ม

(ค) เหตุ: ไม่ได้เต็ม

ผล: ไม่ได้ตอบถูกหมด

แบบฝึกหัด 1.12 จงระบุว่าการอ้างเหตุผลต่อไปนี้เป็น Fallacy ชนิดใด:

เหตุ: ถ้าเป็นแมวแล้วเป็นสัตว์

(ก) เหตุ: เป็นสัตว์

ผล: เป็นแมว

เหตุ: ถ้าสอบไม่ได้แล้วจะไม่ผ่าน

(ข) เหตุ: สอบได้

ผล: จะผ่าน

แบบฝึกหัด 1.13 (ก) เขียนเงื่อนไข Python ที่เทียบเท่ากับ $(p \rightarrow q) \wedge (r \rightarrow s)$

(ข) อธิบายว่า short-circuit evaluation ทำงานอย่างไรใน $(p \wedge q)$ และ $(p \vee q)$

(ค) เขียน SQL query ที่เลือกข้อมูลจากตาราง `students` โดยมีเงื่อนไข:

0.1. grade มากกว่า 80

0.2. และ enrollment status เป็น 'active'

0.3. และ major ไม่เป็น 'undecided'

แบบฝึกหัด 1.14 จงระบุว่าการประพจน์ต่อไปนี้เป็น Tautology, Contradiction หรือ Contingency:

(ก) $p \vee \neg p$ (Law of Excluded Middle)

(ข) $p \wedge \neg p$ (Law of Non-Contradiction)

(ค) $(p \rightarrow q) \leftrightarrow (\neg p \vee q)$

(ง) $(p \wedge q) \rightarrow p$

แบบฝึกหัด 1.15 (ก) จงตรวจสอบว่า $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$ เป็น SAT หรือ UNSAT

(ข) อธิบายความแตกต่างระหว่าง 2-SAT และ 3-SAT

(ค) ทำไม 3-SAT จึงเป็น NP-Complete?

แบบฝึกหัด 1.16 (ก) วาดวงจรของ $(A \wedge B) \vee \neg C$ โดยใช้ AND, OR, NOT gates

(ข) จงหาสูตรบูลีนจากตารางค่าความจริง ดังแสดงในตารางที่ 8:

ตารางที่ 8 ตารางค่าความจริงสำหรับแบบฝึกหัดข้อ (ข)

A	B	C	Output
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

แบบฝึกหัด 1.17 จงตรวจสอบว่า Argument ต่อไปนี้สมเหตุสมผลหรือไม่:

$$\frac{(p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r)}{\therefore q \vee s}$$

(แนะนำ: ใช้ตารางค่าความจริง หรือ Constructive Dilemma)

แบบฝึกหัด 1.18 พิจารณาอัลกอริทึมต่อไปนี้:

```
function factorial(n):
    # Precondition: n >= 0
    result = 1
    i = 1
    while i <= n:
        result = result * i
        i = i + 1
    return result
    # Postcondition: result = n!
```

- (ก) จงเสนอ Loop Invariant ที่เหมาะสม
- (ข) พิสูจน์ว่า Loop Invariant ถูกรักษาไว้ในทุก iteration
- (ค) ใช้ Loop Invariant พิสูจน์ว่า Postcondition ถูกต้องเมื่อลูปจบ

แบบฝึกหัด 1.19 ให้ Knowledge Base (KB) มีประพจน์ต่อไปนี้:

1. $p \rightarrow q$
2. $q \rightarrow r$
3. p
4. $s \rightarrow \neg r$

จงตรวจสอบว่า KB นำไปสู่ (entails) ประพจน์ใดบ้างต่อไปนี้:

- (ก) q
- (ข) r
- (ค) $\neg s$
- (ง) $\neg r \vee s$

แบบฝึกหัด 1.20 จงระบุว่าสูตรต่อไปนี้เป็น WFF ที่ถูกต้องหรือไม่:

- (ก) $(p \rightarrow q) \wedge r$
- (ข) $p \rightarrow q \rightarrow r$ (ไม่มีวงเล็บ)
- (ค) $(p \wedge (q \vee r) \rightarrow s)$
- (ง) $\neg(p \rightarrow q) \leftrightarrow (r \wedge \neg s)$

แบบฝึกหัด 1.21 พิสูจน์สมมูลต่อไปนี้โดยใช้การแปลงรูป (ไม่ใช่ตารางค่าความจริง):

- (ก) Absorption: $p \vee (p \wedge q) \equiv p$
- (ข) $p \wedge (p \vee q) \equiv p$
- (ค) $(p \wedge q) \vee (\neg p \wedge q) \equiv q$ (Consensus)

แบบฝึกหัด 1.22 (ก) หาตัวอย่างค้าน (counterexample) ของประโยค: “ถ้า n เป็นจำนวนเฉพาะ แล้ว n เป็นจำนวนคี่”

(ข) พิสูจน์ว่าประโยค “ถ้า n^2 เป็นจำนวนคู่ แล้ว n เป็นจำนวนคู่” เป็นจริง (ใช้ Contraposition หรือ Direct)

แบบฝึกหัด 1.23 สรุปเทคนิคการพิสูจน์ที่เหมาะสมสำหรับแต่ละรูปแบบ:

- (ก) พิสูจน์ $p \rightarrow q$ เมื่อมีข้อมูลโดยตรงเกี่ยวกับ p
- (ข) พิสูจน์ $p \rightarrow q$ เมื่อยากพิสูจน์โดยตรง แต่ง่ายกว่าถ้ารู้ว่า q เท็จนำไปสู่ p เท็จ
- (ค) พิสูจน์ $\neg p$ เมื่อ p นำไปสู่ข้อขัดแย้ง
- (ง) พิสูจน์ $\exists x : P(x)$ โดยการหาค่า x ที่ทำให้ $P(x)$ จริง
- (จ) พิสูจน์ $p \vee q$ โดยแยกพิสูจน์ p หรือ q แยกกัน

แบบฝึกหัด 1.24 ให้ $P(x, y)$: “ $x + y = 10$ ” โดยที่ x, y เป็นจำนวนจริง
จงระบุค่าความจริงของประโยคต่อไปนี้:

$$(ก) \quad \forall x \exists y : P(x, y)$$

$$(ข) \quad \exists x \forall y : P(x, y)$$

$$(ค) \quad \forall x \forall y : P(x, y)$$

$$(ง) \quad \exists x \exists y : P(x, y)$$

แบบฝึกหัด 1.25 จงแปลงประโยคต่อไปนี้เป็น CNF (Conjunctive Normal Form):

$$(ก) \quad (p \leftrightarrow q)$$

$$(ข) \quad \neg(p \rightarrow q) \rightarrow (r \vee s)$$

$$(ค) \quad (p \rightarrow q) \wedge \neg q$$

1.11 เฉลยแนวคิด (Hints and Answers)

แบบฝึกหัด 1.26 1.1 (แนวคำตอบ)

- (ก) เป็นประพจน์ — จริง (T)
- (ข) ไม่เป็นประพจน์ — เป็นประโยคเปิด (Open Sentence) เพราะไม่ทราบค่า x
- (ค) ไม่เป็นประพจน์ — เป็นประโยคคำสั่ง (Imperative)
- (ง) เป็นประพจน์ — จริง (T)
- (จ) เป็นประพจน์ — จริง (T)

แบบฝึกหัด 1.27 1.11 (แนวคำตอบ)

- (ก) Modus Ponens — Valid $\square$ (ยืนยันเหตุ ผลตามมา)
- (ข) Affirming the Consequent — Invalid (Fallacy) เพราะ q อาจเกิดจากสาเหตุอื่น
- (ค) Modus Tollens — Valid $\square$ (ปฏิเสธผล นิเสธเหตุ)

แบบฝึกหัด 1.28 1.14 (แนวคำตอบ)

- (ก) Tautology (Law of Excluded Middle — จริงเสมอ)
- (ข) Contradiction (Law of Non-Contradiction — เท็จเสมอ)
- (ค) Tautology (เพราะ $p \rightarrow q \equiv \neg p \vee q$)
- (ง) Tautology (เพราะ $p \wedge q \rightarrow p$ จะเป็นจริงเสมอ)

แบบฝึกหัด 1.29 1.22 (แนวคำตอบ)

- (ก) Counterexample คือ $n = 2$ เพราะ 2 เป็นจำนวนเฉพาะแต่ 2 เป็นจำนวนคู่ (ไม่ใช่คี่)
- (ข) ใช้ Contraposition: ถ้า n เป็นจำนวนคี่ แล้ว n^2 เป็นจำนวนคี่ $\rightarrow$ พิสูจน์ได้โดย Direct (ให้ $n = 2k + 1$, $n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$ ซึ่งเป็นจำนวนคี่)

แบบฝึกหัด 1.30 1.24 (แนวคำตอบ)

(ก) $\forall x \exists y : P(x, y)$ — **จริง** เพราะทุก x มี $y = 10 - x$ ที่ทำให้ $x + y = 10$

(ข) $\exists x \forall y : P(x, y)$ — **เท็จ** เพราะไม่มี x ที่ทำให้ทุก y มี $x + y = 10$ (เช่น ถ้า $x = 3$ แล้ว $y = 8$ ทำให้ $3 + 8 = 11 \neq 10$)

(ค) $\forall x \forall y : P(x, y)$ — **เท็จ** เพราะเช่น $1 + 1 = 2 \neq 10$

(ง) $\exists x \exists y : P(x, y)$ — **จริง** เพราะเช่น $x = 3, y = 7$ ทำให้ $3 + 7 = 10$

Bibliography

- [1] Rosen, K. H. (2019). *Discrete Mathematics and Its Applications* (8th ed.). McGraw-Hill. บทที่ 1-2.
- [2] Grimaldi, R. P. (2003). *Discrete and Combinatorial Mathematics* (5th ed.). Addison-Wesley. บทที่ 2.
- [3] Huth, M., & Ryan, M. (2004). *Logic in Computer Science: Modelling and Reasoning about Systems* (2nd ed.). Cambridge University Press.
- [4] Klement, E. C. (2003). *The Logic Book* (4th ed.). McGraw-Hill.
- [5] Epp, S. S. (2020). *Discrete Mathematics with Applications* (5th ed.). Cengage Learning.
- [6] สมศักดิ์ ประวัติดาว. (2554). *คณิตศาสตร์เชิงการของคอมพิวเตอร์*. บริษัท เท็กซ์แอนด์เจอร์นัล พับลิเคชัน.
- [7] Cormen, T. H., et al. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (เรื่อง Loop Invariant และ Algorithm Correctness)
- [8] Skeet, J. (2011). *C# in Depth* (3rd ed.). Manning Publications. (เรื่อง Logic ในภาษาโปรแกรม)

1.12 เอกสารอ้างอิง

แผนการสอนประจำบทที่ 2

รหัสวิชา	4121701
ชื่อวิชา	คณิตศาสตร์สำหรับคอมพิวเตอร์
หน่วยกิต	3 (3-0-6)
บทที่	2
ชื่อบท	ทฤษฎีเซต (Set Theory)
จำนวนชั่วโมง	6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- อธิบายความหมายและสัญลักษณ์ของเซตได้
- ระบุประเภทของเซต เช่น เซตว่าง เซตจำกัด เซตอนันต์ได้
- ใช้การดำเนินการบนเซต เช่น ยูเนียน อินเตอร์เซกชัน ผลต่างเซต และคอมพลีเมนต์ได้
- พิสูจน์สมบัติของเซตโดยใช้แผนภาพเวนน์และพีชคณิตเซตได้
- ประยุกต์ใช้ทฤษฎีเซตในการแก้ปัญหาทางคอมพิวเตอร์ได้

หัวข้อที่สอน

- นิยามและสัญลักษณ์ของเซต
- ประเภทของเซต
- การดำเนินการบนเซต
- กฎพีชคณิตเซต (Set Algebra Laws)
- แผนภาพเวนน์ (Venn Diagram)
- การประยุกต์ทฤษฎีเซตในวิทยาการคอมพิวเตอร์

สื่อการสอน

- สไลด์ประกอบการสอน
- แผนภาพเวนน์
- แบบฝึกหัดท้ายบท

ทฤษฎีเซต (Set Theory)

วัตถุประสงค์การเรียนรู้

หลังจากศึกษาจบบทนี้ ผู้เรียนจะสามารถ:

- อธิบายความหมายและความสำคัญของเซต สมาชิก และการเป็นสมาชิกของเซตได้
- ดำเนินการระหว่างเซต (เพิ่ม ตัด ผลต่าง คอมพลีเมนต์) และอธิบายความหมายของผลลัพธ์ได้
- พิสูจน์และประยุกต์ใช้กฎของเดอมอร์แกน (De Morgan's Laws) สำหรับเซตได้
- เขียนและตีความเพาเวอร์เซต (Power Set) และผลคูณคาร์ทีเซียน (Cartesian Product) ได้
- อธิบายความสัมพันธ์ระหว่างเซตกับโครงสร้างข้อมูลและฐานข้อมูลในคอมพิวเตอร์ได้
- พิสูจน์ข้อความทางเซตโดยใช้เทคนิคการพิสูจน์ที่เหมาะสมได้

แนะนำ: ความเป็นมาและความสำคัญของทฤษฎีเซต

2.2.1 ประวัติและที่มาของทฤษฎีเซต

ทฤษฎีเซต (Set Theory) เป็นรากฐานทางคณิตศาสตร์ที่พัฒนาขึ้นอย่างเป็นระบบโดย Georg Cantor (1845–1918) นักคณิตศาสตร์ชาวเยอรมัน ในช่วงปลายศตวรรษที่ 19 การค้นพบของ Cantor ถือเป็นการปฏิวัติวงการคณิตศาสตร์ เพราะก่อนหน้านี้ไม่เคยมีใครตั้งคำถามเกี่ยวกับธรรมชาติของความไม่สิ้นสุด (infinity) อย่างจริงจังมาก่อน

Cantor เริ่มศึกษาเรื่องเซตเมื่อเขาพยายามทำความเข้าใจเรื่องอนุกรมฟูรีเยร์ (Fourier series) และค่าความไม่ต่อเนื่อง (discontinuity) ในการวิเคราะห์ทางคณิตศาสตร์ เขาสร้างทฤษฎีที่สามารถเปรียบเทียบขนาดของเซตอนันต์ได้ ผลลัพธ์ที่โด่งดังที่สุดของ Cantor คือ การที่เซตของจำนวนเต็ม $\mathbb{Z}$ และเซตของจำนวนตรรกยะ $\mathbb{Q}$ มีขนาดเท่ากัน (countable infinite) ในขณะที่เซตของจำนวนจริง $\mathbb{R}$ มีขนาดใหญ่กว่า (uncountable infinite) แสดงให้เห็นว่าความอนันต์มีหลายระดับ

แม้ทฤษฎีเซตจะถูกตั้งคำถามในช่วงแรก แต่ปัจจุบันเป็นพื้นฐานของคณิตศาสตร์ทั้งหมด และเป็นภาษาที่ใช้กำหนดโครงสร้างของวิทยาการคอมพิวเตอร์อย่างแท้จริง

2.2.2 ทฤษฎีเซตกับวิทยาการคอมพิวเตอร์

ทฤษฎีเซตมีความสำคัญอย่างยิ่งในวิทยาการคอมพิวเตอร์ ได้แก่:

1. **โครงสร้างข้อมูล (Data Structures)** คือ Arrays เป็นการจัดเรียงสมาชิกในเซตตามลำดับ List คือเซตที่มีลำดับและอนุญาตให้ซ้ำกันได้ Hash Table ใช้หลักการของการจับคู่ระหว่างสมาชิกกับตำแหน่ง Tree และ Graph ก็มีความเกี่ยวข้องอย่างลึกซึ้งกับทฤษฎีเซต
 2. **ฐานข้อมูล (Database Systems)** คือระบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) มีพื้นฐานมาจากทฤษฎีเซตโดยตรง Relational Algebra ซึ่งเป็นพื้นฐานทางคณิตศาสตร์ของ SQL คือการดำเนินการระหว่างเซต ได้แก่ Union, Intersection, Difference และ Cartesian Product
 3. **ทฤษฎีภาษา (Language Theory)** คือในการศึกษาภาษาการเขียนโปรแกรมและทฤษฎีการคำนวณ แนวคิดเรื่องตัวอักษร (Alphabet) สายอักขระ (String) และภาษา (Language) ล้วนเป็นเซตทั้งสิ้น
 4. **การ พิสูจน์ ความ ถูก ต้อง ของ โปรแกรม (Program Verification)** คือ การ พิสูจน์ ว่า โปรแกรมทำงานถูกต้องตามข้อกำหนด ต้องใช้การดำเนินการระหว่างเซตเพื่ออธิบายสถานะของโปรแกรม Precondition และ Postcondition เป็นเซตของสถานะที่เป็นไปได้
 5. **ทฤษฎีความน่าจะเป็น (Probability Theory)** คือ sample space คือเซตของผลลัพธ์ที่เป็นไปได้ทั้งหมด และ events คือเซตย่อยของ sample space การดำเนินการระหว่าง events เช่น union, intersection, complement ตรงกับการดำเนินการระหว่างเซต
 6. **Logic และ AI** คือระบบฐานความรู้ (Knowledge Base) ในปัญญาประดิษฐ์ใช้ทฤษฎีเซตในการจัดการข้อเท็จจริงและกฎเกณฑ์ การอนุมาน (Inference) ก็คือการดำเนินการระหว่างเซตของข้อเท็จจริง
- หมายเหตุ 2.1 ในการเขียนโปรแกรม เรามักใช้เซตในความหมายที่ไม่เคร่งครัดนัก เช่น "เซตของตัวเลข" อาจหมายถึง list หรือ array ที่มีข้อมูลซ้ำกันได้ แต่ในทางคณิตศาสตร์ เซตตามนิยามไม่มีสมาชิกซ้ำกันและไม่มีลำดับ ความแตกต่างนี้สำคัญเมื่อต้องการพิสูจน์คุณสมบัติทางคณิตศาสตร์

นิยามพื้นฐานของเซต

ในบทนี้ เราจะศึกษาหัวใจสำคัญของทฤษฎีเซต นั่นคือความหมายของเซต สมาชิก และวิธีการเขียนเซตในรูปแบบต่างๆ ความเข้าใจที่ลึกซึ้งในพื้นฐานเหล่านี้จะเป็นรากฐานให้เราสามารถศึกษาหัวข้อที่ซับซ้อนขึ้นในบทต่อไป เช่น การดำเนินการระหว่างเซต ความสัมพันธ์ระหว่างเซต และการพิสูจน์ทางเซต

ทฤษฎีเซตเป็นภาษาพื้นฐานของคณิตศาสตร์ทั้งหมด เหมือนกับที่ตัวอักษรเป็นพื้นฐานของภาษา เราต้องเรียนรู้ตัวอักษรและไวยากรณ์ก่อนจึงจะสามารถเขียนประโยคและเรียนรู้วรรณกรรมได้ ในทำนองเดียวกัน เราต้องเข้าใจเซตและสมาชิกอย่างลึกซึ้งก่อนจึงจะสามารถศึกษาคณิตศาสตร์ขั้นสูงได้

ในชีวิตประจำวัน เรามักจัดกลุ่มของสิ่งต่างๆ อยู่เสมอ เช่น "กลุ่มเพื่อนสนิท" "กลุ่มวิชาที่ชอบ" "กลุ่มของใช้บนโต๊ะ" ในทางคณิตศาสตร์ เราเรียกกลุ่มของสิ่งต่างๆ ว่า "เซต" และเรียกสิ่งที่อยู่ในกลุ่มว่า "สมาชิก"

2.3.1 นิยามของเซตและสมาชิก

ทฤษฎีบท 2.2 **เซต (Set)** คือ กลุ่มของวัตถุที่มีคุณสมบัติร่วมกันหรือมีการระบุไว้อย่างชัดเจน โดยเรียกวัตถุแต่ละชิ้นว่า **สมาชิก (Element)** ของเซต ถ้า a เป็นสมาชิกของเซต A เขียนแทนด้วย $a \in A$ อ่านว่า " a เป็นสมาชิกของ A " ถ้า a ไม่เป็นสมาชิกของ A เขียนแทนด้วย $a \notin A$ อ่านว่า " a ไม่เป็นสมาชิกของ A " สัญลักษณ์ $\in$ มาจากภาษากรีก "epsilon" (ϵ) ซึ่ง Cantor นำมาใช้เพื่อแสดงการ "เป็นสมาชิกของ" (element of) ความสัมพันธ์ $a \in A$ เป็นความสัมพันธ์ระหว่างวัตถุกับเซต ไม่ใช่ความสัมพันธ์ระหว่างเซตกับเซต

ตัวอย่าง 2.3 พิจารณาตัวอย่างต่อไปนี้ซึ่งแสดงเซตในบริบทที่แตกต่างกัน:

(ก) $A = \{1, 2, 3, 4, 5\}$ — เซตของจำนวนเต็มบวกตั้งแต่ 1 ถึง 5 สมาชิกของ A คือ 1, 2, 3, 4, 5 สังเกตว่าเราสามารถตรวจสอบได้ทันทีว่า $3 \in A$ เป็นจริง แต่ $7 \notin A$ ก็เป็นจริงเช่นกัน

(ข) $B = \{\text{แดง, เขียว, น้ำเงิน}\}$ — เซตของสีพื้นฐานในระบบสี RGB ลำดับไม่สำคัญ ดังนั้น $\{\text{แดง, เขียว, น้ำเงิน}\} = \{\text{เขียว, แดง, น้ำเงิน}\}$

(ค) $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ — เซตของจำนวนนับ (Natural Numbers) สัญลักษณ์ $\mathbb{N}$ เป็นมาตรฐาน ในตำราเล่มนี้เราจะถือว่า $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ คือเริ่มที่ 0

(ง) $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ — เซตของจำนวนเต็ม (Integers) สัญลักษณ์ $\mathbb{Z}$ มาจากภาษาเยอรมัน "Zahlen"

(จ) $\mathbb{Q} = \{\frac{a}{b} \mid a, b \in \mathbb{Z}, b \neq 0\}$ — เซตของจำนวนตรรกยะ (Rational Numbers) สัญลักษณ์ $\mathbb{Q}$ มาจาก "quotient"

(ฉ) $\mathbb{R}$ — เซตของจำนวนจริง (Real Numbers) รวมถึงจำนวนตรรกยะและจำนวนอตรรกยะ (irrational numbers) เช่น $\sqrt{2}$, π , e

เมื่อเรากล่าวว่า $a \in A$ เรากำลังยืนยันว่าวัตถุ a มีอยู่ในเซต A ความจริงนี้มีค่าความจริง (truth value) เป็นจริง (True) หรือเท็จ (False) อย่างใดอย่างหนึ่งเสมอ ไม่มีทางที่ $a \in A$ จะเป็น "ครึ่งหนึ่งจริง" — นี่คือหลักการที่เรียกว่า Law of Excluded Middle ในตรรกศาสตร์

หมายเหตุ 2.4 ความสัมพันธ์ระหว่างเซตของจำนวนต่างๆ มีความสำคัญมาก:

$$\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}$$

จำนวนนับเป็นเซตย่อยของจำนวนเต็ม จำนวนเต็มเป็นเซตย่อยของจำนวนตรรกยะ และจำนวนตรรกยะเป็นเซตย่อยของจำนวนจริง ตัวอย่างเช่น $\frac{1}{2} \in \mathbb{Q}$ เป็นจริง และ $\frac{1}{2} \in \mathbb{R}$ ก็เป็นจริงด้วย แต่ $\sqrt{2} \notin \mathbb{Q}$ แต่ $\sqrt{2} \in \mathbb{R}$

ตัวอย่าง 2.5 พิจารณาความจริงของประโยคต่อไปนี้:

1. $3 \in \{1, 2, 3, 4, 5\}$ เป็นจริง เพราะ 3 อยู่ในเซต
2. $7 \in \{1, 2, 3, 4, 5\}$ เป็นเท็จ เพราะ 7 ไม่อยู่ในเซต
3. $\sqrt{2} \in \mathbb{Q}$ เป็นเท็จ เพราะ $\sqrt{2}$ เป็นจำนวนอตรรกยะ
4. $\sqrt{2} \in \mathbb{R}$ เป็นจริง เพราะ $\sqrt{2}$ เป็นจำนวนจริง
5. $0.75 \in \mathbb{Q}$ เป็นจริง เพราะ $0.75 = \frac{3}{4}$
6. $0.\bar{3} \in \mathbb{Q}$ เป็นจริง เพราะ $0.\bar{3} = \frac{1}{3}$

หมายเหตุ 2.6 ในการเขียนโปรแกรม การทดสอบ $a \in A$ อาจมีความหมายต่างกันขึ้นอยู่กับโครงสร้างข้อมูล: ใน Array/List ต้องตรวจสอบทุก element ที่ละตัว ใช้เวลา $O(n)$ ใน Hash Set ใช้ hash function ใช้เวลา $O(1)$ โดยเฉลี่ย ใน Tree ใช้เวลา $O(\log n)$ ถ้า balanced

2.3.2 วิธีการเขียนเซต

ตัวอย่าง 2.7 มี 2 วิธีหลักในการเขียนเซต:

1. **แจกแจงสมาชิก (Roster Method)** — เขียนสมาชิกทั้งหมดภายในวงเล็บปีกกา $\{\}$ โดยคั่นด้วยเครื่องหมายจุลภาค วิธีนี้เหมาะกับเซตที่มีจำนวนสมาชิกน้อยและสามารถระบุได้ทั้งหมด

$A = \{1, 2, 3\}$ แสดงเซตที่มีสมาชิกสามตัว $B = \{a, e, i, o, u\}$ แสดงเซตของสระในภาษาอังกฤษ สังเกตว่าลำดับไม่สำคัญ $\{a, e, i, o, u\} = \{e, a, i, o, u\}$ และไม่มีสมาชิกซ้ำ $\{1, 1, 2, 2, 3\} = \{1, 2, 3\}$

การใช้ ... (ellipsis): เมื่อเซตมีสมาชิกมากแต่มีรูปแบบที่ชัดเจน เช่น $\{1, 2, 3, \dots, 100\}$ แสดงเซตของจำนวนเต็มตั้งแต่ 1 ถึง 100 ข้อควรระวัง: ต้องมีสมาชิกอย่างน้อย 2 ตัวก่อน ellipsis เพื่อให้ผู้อ่านเข้าใจรูปแบบ

ตัวอย่าง 2.8 2. แจกแจง โดย กำหนด คุณสมบัติ (Set-builder Notation) — เขียน ใน รูป $\{x \mid \text{เงื่อนไขที่ } x \text{ ต้อง satisfy}\}$ วิธีนี้เหมาะกับเซตที่มีจำนวนสมาชิกมากหรือนันต์ พิจารณาตัวอย่างต่อไปนี้:

- $A = \{x \in \mathbb{Z} \mid x > 0\}$ — เซตของจำนวนเต็มบวก เขียนแบบย่อได้เป็น $\mathbb{Z}^+$
- $B = \{n^2 \mid n \in \mathbb{N} \text{ และ } n < 5\} = \{0, 1, 4, 9, 16\}$ — สมาชิกคือกำลังสองของจำนวนนับที่น้อยกว่า 5
- $E = \{x \in \mathbb{Z} \mid x \text{ หารด้วย 2 ลงตัว}\} = 2\mathbb{Z}$ — เซตของจำนวนคู่
- $O = \{x \in \mathbb{Z} \mid x \text{ หารด้วย 2 ไม่ลงตัว}\}$ — เซตของจำนวนคี่
- $C = \{x \in \mathbb{R} \mid x^2 - 5x + 6 = 0\} = \{2, 3\}$ — เซตของคำตอบของสมการ $x^2 - 5x + 6 = 0$
- $D = \{x \in \mathbb{R} \mid x^2 + 1 = 0\} = \emptyset$ — ไม่มีจำนวนจริงที่ยกกำลังสองแล้วบวก 1 ได้ศูนย์

หมายเหตุ 2.9 คุณสมบัติสำคัญของเซต:

1. ไม่มีลำดับ (Unordered) คือลำดับของสมาชิกไม่สำคัญ $\{1, 2\} = \{2, 1\}$ หากต้องการลำดับต้องใช้ tuple หรือ sequence แทน
2. ไม่มีสมาชิกซ้ำ (No Duplicates) คือ $\{1, 1, 2\} = \{1, 2\}$ หากต้องซ้ำ ต้องใช้ multiset หรือ list แทน

List (ordered and allows duplicates)

```
my_list = [1, 2, 2, 3]
```

Set (unordered and no duplicates)

```
my_set = {1, 2, 2, 3} # = {1, 2, 3}
```

นี้แสดงให้เห็นว่าในทางคณิตศาสตร์ $\{1, 2, 2, 3\} = \{1, 2, 3\}$ แต่ในการเขียนโปรแกรม list $[1, 2, 2, 3] \neq [2, 1, 2, 3]$ เมื่อต้องการเปลี่ยนจาก list ที่มีซ้ำไปเป็นเซต เราต้อง "ลบซ้ำ" ซึ่งในทางคณิตศาสตร์ทำได้โดยการกำหนดเซตใหม่ $\{1, 2, 2, 3\} = \{x \mid x \in \{1, 2, 2, 3\}\}$ แต่ในทางปฏิบัติ เราอาจใช้ฟังก์ชัน "unique" หรือ "distinct"

2.3.3 เซตพิเศษ

หมายเหตุ 2.10 ความเข้าใจผิดที่พบบ่อย: $\emptyset$ คือเซตที่ไม่มีสมาชิกเลย แต่ $\{\emptyset\}$ คือเซตที่มีสมาชิก 1 ตัว คือ $\emptyset$ ดังนั้น $\emptyset \neq \{\emptyset\} \neq \{\{\emptyset\}\}$ และ $|\emptyset| = 0, |\{\emptyset\}| = 1$

ทฤษฎีบท 2.11 เซตพิเศษที่สำคัญ ได้แก่: (1) **เซตว่าง (Empty Set)** เขียนแทนด้วย $\emptyset$ หรือ $\{\}$ คือเซตที่ไม่มีสมาชิกเลย (2) **Singleton Set** คือเซตที่มีสมาชิกเพียงตัวเดียว เช่น $\{5\}$, $\{a\}$, $\{\emptyset\}$ (3) **Universal Set** เขียนแทนด้วย U คือเซตที่ประกอบด้วยสมาชิกทุกตัวที่สนใจในบริบทหนึ่ง เช่น ในการศึกษาเรื่องจำนวน $\mathbb{R}$ อาจเป็น Universal Set เป็นเหมือน "กรอบอ้างอิง" ที่กำหนดขอบเขตของการพิจารณา นอกจากนี้ เซตจำกัด (Finite Set) คือเซตที่มีจำนวนสมาชิกจำกัด ในขณะที่ $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ เป็น **เซตอนันต์ (Infinite Set)** ที่มีจำนวนสมาชิกไม่สิ้นสุด และเซตว่างเป็นเซตย่อยของทุกเซต: $\emptyset \subseteq A$ สำหรับทุกเซต A เพราะ "ทุกสมาชิกของ $\emptyset$ มีคุณสมบัติ P " เป็นจริงโดย vacuous truth

หมายเหตุ 2.12 **ความเข้าใจผิดที่พบบ่อย:** $\emptyset \in \{\emptyset\}$ เป็นจริง เพราะ $\emptyset$ เป็นสมาชิกของ $\{\emptyset\}$ แต่ $\emptyset \subseteq \{\emptyset\}$ ก็เป็นจริงด้วย (เซตว่างเป็นเซตย่อยของทุกเซต) แสดงว่า $\in$ (เป็นสมาชิก) และ $\subseteq$ (เป็นเซตย่อย) เป็นความสัมพันธ์ที่ต่างกัน แต่สามารถเป็นจริงทั้งคู่พร้อมกันได้

ตัวอย่าง 2.13 เพื่อเข้าใจความแตกต่างระหว่าง $\in$ และ $\subseteq$: $1 \in \{1, 2, 3\}$ ถูกต้อง แต่ $\{1\} \in \{1, 2, 3\}$ ไม่ถูกต้อง ส่วน $\{1\} \subseteq \{1, 2, 3\}$ ถูกต้องเพราะทุกสมาชิกของ $\{1\}$ (คือ 1) อยู่ใน $\{1, 2, 3\}$ และ $\emptyset \subseteq \{1, 2, 3\}$ ถูกต้อง (เซตว่างเป็นเซตย่อยของทุกเซต) แต่ $\emptyset \in \{1, 2, 3\}$ ไม่ถูกต้อง สรุปคือ $\in$ และ $\subseteq$ เป็นความสัมพันธ์ที่ต่างกันแต่สามารถเป็นจริงทั้งคู่พร้อมกันได้ และในชีวิตจริง: ฐานข้อมูลนักศึกษา U = นักศึกษาทั้งหมด, A = นักศึกษาคณิตศาสตร์, B = นักศึกษาฟิสิกส์ แล้ว $A \cap B$ = นักศึกษาที่เรียนทั้งสองวิชา

ความสัมพันธ์ระหว่างเซต

2.4.1 เซตย่อย (Subset) และเซตย่อยแท้ (Proper Subset)

การเข้าใจนิยามนี้เป็นสิ่งสำคัญมาก เพราะเป็นพื้นฐานของการพิสูจน์ทางเซตทั้งหมด เมื่อต้องการพิสูจน์ว่า $A \subseteq B$ เราต้องสมมติว่า $x \in A$ แล้วแสดงว่า $x \in B$ ด้วย — นี่คือรูปแบบ direct proof สำหรับ subset

ทฤษฎีบท 2.14 เซต A เป็น **เซตย่อย (Subset)** ของเซต B เขียนแทนด้วย $A \subseteq B$ ก็ต่อเมื่อทุกสมาชิกของ A เป็นสมาชิกของ B ด้วย กล่าวคือ $\forall x, (x \in A \rightarrow x \in B)$ นอกจากนี้ $\emptyset \subseteq A$ และ $A \subseteq A$ สำหรับทุกเซต A (เซตว่างเป็นเซตย่อยของทุกเซตโดย vacuous truth และเซตทุกเซตเป็นเซต

ย่อยของตัวเอง) ความสัมพันธ์ระหว่าง $\subseteq$ และ $=$ มีความสำคัญมาก: $A \subseteq B$ และ $B \subseteq A$ ก็ต่อเมื่อ $A = B$ นี่คือการ Extensionality Axiom และ $A \subseteq B$ และ $B \not\subseteq A$ ก็ต่อเมื่อ $A \subset B$ (เซตย่อยแท้ / Proper Subset) ตัวอย่างเช่น $\{1, 2\} \subset \{1, 2, 3\}$ เพราะ $\{1, 2\} \subseteq \{1, 2, 3\}$ แต่ $\{1, 2, 3\} \not\subseteq \{1, 2\}$ ในขณะที่ $\{1, 2, 3\} \not\subset \{1, 2, 3\}$

ตัวอย่าง 2.15 $\{1, 2\} \subseteq \{1, 2, 3\}$ ถูกต้องเพราะ 1 และ 2 ต่างก็อยู่ใน $\{1, 2, 3\}$ และ $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R}$ เป็นความจริงพื้นฐานที่แสดงความสัมพันธ์ระหว่างเซตของจำนวน $\emptyset \subseteq \{1, 2\}$ ถูกต้องเพราะเซตว่างเป็นเซตย่อยของทุกเซต ส่วน $\{1, 2\} \not\subseteq \{2, 3\}$ ไม่ถูกต้องเพราะ $1 \in \{1, 2\}$ แต่ $1 \notin \{2, 3\}$

ทฤษฎีบท 2.16 ในการพิสูจน์ว่า $A = B$ เราต้องพิสูจน์ทั้ง $A \subseteq B$ และ $B \subseteq A$ นั่นคือต้องแสดงว่า:
 (1) ทุก $x \in A \Rightarrow x \in B$ (แสดง $A \subseteq B$) และ (2) ทุก $x \in B \Rightarrow x \in A$ (แสดง $B \subseteq A$) วิธีนี้เรียกว่า
 "proof by double inclusion" เป็นเทคนิคมาตรฐานในการพิสูจน์ความเท่ากันของเซต

ตัวอย่าง 2.17 พิสูจน์ว่า $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ (Distributive Law): ต้องแสดง $\subseteq$ และ $\supseteq$ ($\subseteq$) สมมติ $x \in A \cap (B \cup C)$ โดยนิยามของ intersection และ union: $x \in A$ และ $x \in B \cup C$ ซึ่งหมายความว่า $x \in A$ และ ($x \in B$ หรือ $x \in C$) ใช้ distributive law ของตรรกศาสตร์: ($x \in A$ และ $x \in B$) หรือ ($x \in A$ และ $x \in C$) กลับไปใช้นิยามของ intersection และ union: $x \in (A \cap B) \cup (A \cap C)$ ($\supseteq$) พิสูจน์กลับกันโดยทำตามขั้นตอนย้อนกลับ สรุปได้ว่า $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

2.5 การดำเนินการระหว่างเซต (Set Operations)

การดำเนินการระหว่างเซตเป็นเครื่องมือพื้นฐานในการสร้างเซตใหม่จากเซตที่มีอยู่ เราจะศึกษาการดำเนินการ 4 อย่างหลักๆ ได้แก่ union ($\cup$) intersection ($\cap$) difference ($\setminus$) และ complement (c)

2.5.1 ยูเนียน (Union)

ทฤษฎีบท 2.18 ยูเนียน (Union) ของเซต A และ B เขียนแทนด้วย $A \cup B$ คือเซตที่ประกอบด้วยสมาชิกที่อยู่ใน A หรืออยู่ใน B หรืออยู่ทั้งสองเซต กล่าวคือ $A \cup B = \{x \mid x \in A \text{ หรือ } x \in B\}$ สมบัติสำคัญ ได้แก่: (1) $A \cup A = A$ (Idempotent Law) (2) $A \cup \emptyset = A$ (Identity Law) (3) $A \cup U = U$ (Domination Law) (4) $A \cup B = B \cup A$ (Commutative Law) (5) $(A \cup B) \cup C = A \cup (B \cup C)$ (Associative Law) และ $A \subseteq A \cup B$, $B \subseteq A \cup B$ สำหรับทุกเซต A, B, C สมาชิกใหม่จะเข้ามาในผลลัพธ์ก็ต่อเมื่อมันอยู่ใน A หรืออยู่ใน B อย่างน้อยที่หนึ่ง และสมาชิกที่อยู่ทั้งสองเซตจะปรากฏในผลลัพธ์เพียงครั้งเดียว (เพราะเซตไม่มีสมาชิกซ้ำ)

ตัวอย่าง 2.19 พิจารณาตัวอย่างต่อไปนี้เพื่อเข้าใจ union อย่างลึกซึ้ง: $\{1, 2\} \cup \{2, 3\} = \{1, 2, 3\}$ — สังเกตว่า 2 อยู่ทั้งสองเซต แต่ในผลลัพธ์ 2 ปรากฏเพียงครั้งเดียว $\{1, 2\} \cup \emptyset = \{1, 2\}$ — union กับเซตว่างไม่เปลี่ยนเซตเดิม $\{x \mid x \text{ เป็นจำนวนคู่}\} \cup \{x \mid x \text{ เป็นจำนวนคี่}\} = \mathbb{Z}$ — ทุกจำนวนเต็มเป็นได้ทั้งคู่หรือคี่ อย่างใดอย่างหนึ่ง $A \cup A = A$ และ $A \cup B = B \cup A$ แสดงสมบัติ idempotent และ commutative ตามลำดับ

2.5.2 อินเตอร์เซกชัน (Intersection)

ในการคิดถึง intersection ให้นึกภาพว่าเป็นส่วนที่ซ้อนทับกันของเซตสองเซต สมาชิกต้องอยู่ในทั้งสองเซตจึงจะอยู่ใน intersection การที่ $x \in A \cap B$ หมายความว่า x เป็นสมาชิกของ A และ x เป็นสมาชิกของ B ทั้งคู่ ในทางตรรกศาสตร์สอดคล้องกับ "AND" และถ้า $A \cap B = \emptyset$ เราจะเรียก A และ B ว่าเซตแยกกัน (Disjoint Sets)

ทฤษฎีบท 2.20 อินเตอร์เซกชัน (Intersection) ของเซต A และ B เขียนแทนด้วย $A \cap B$ คือเซตที่ประกอบด้วยสมาชิกที่อยู่ทั้งใน A และใน B พร้อมกัน นั่นคือ $A \cap B = \{x \mid x \in A \text{ และ } x \in B\}$ สมบัติสำคัญ ได้แก่ $A \cap A = A$ (Idempotent), $A \cap \emptyset = \emptyset$ (Domination), $A \cap U = A$ (Identity), $A \cap B = B \cap A$ (Commutative), $(A \cap B) \cap C = A \cap (B \cap C)$ (Associative) และ $A \cap B \subseteq A$, $A \cap B \subseteq B$

ตัวอย่าง 2.21 $\{1, 2\} \cap \{2, 3\} = \{2\}$ — 2 เป็นสมาชิกเพียงตัวเดียวที่อยู่ทั้งสองเซต $\{1, 2\} \cap \{3, 4\} = \emptyset$ — ไม่มีสมาชิกใดที่อยู่ทั้งสองเซตจึงเป็น disjoint $\mathbb{Z} \cap \mathbb{R} = \mathbb{Z}$ เพราะทุกจำนวนเต็มเป็นจำนวนจริง และ

$\{x \mid x \text{ เป็นจำนวนคู่}\} \cap \{x \mid x \text{ เป็นจำนวนคี่}\} = \emptyset$ เพราะไม่มีจำนวนใดเป็นได้ทั้งคู่และคี่พร้อมกัน

ทฤษฎีบท 2.22 (Algebraic Laws for Set Operations) สำหรับเซต A, B, C ใดๆ ใน universal set U การดำเนินการระหว่างเซตมีสมบัติดังต่อไปนี้:

1. **Idempotent Laws:** $A \cup A = A$ และ $A \cap A = A$
2. **Identity Laws:** $A \cup \emptyset = A$ และ $A \cap U = A$
3. **Domination Laws:** $A \cup U = U$ และ $A \cap \emptyset = \emptyset$
4. **Commutative Laws:** $A \cup B = B \cup A$ และ $A \cap B = B \cap A$
5. **Associative Laws:** $(A \cup B) \cup C = A \cup (B \cup C)$ และ $(A \cap B) \cap C = A \cap (B \cap C)$
6. **Distributive Laws:** $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ และ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
7. **Absorption Laws:** $A \cup (A \cap B) = A$ และ $A \cap (A \cup B) = A$
8. **Complement Laws:** $A \cup A^c = U$, $A \cap A^c = \emptyset$, $(A^c)^c = A$, $\emptyset^c = U$, และ $U^c = \emptyset$

ตัวอย่าง 2.23 การพิสูจน์ Absorption Law $A \cup (A \cap B) = A$ โดย double inclusion:

($\subseteq$) ถ้า $x \in A \cup (A \cap B)$ แล้ว $x \in A$ หรือ $x \in A \cap B$

ถ้า $x \in A$ ก็เสร็จ ถ้า $x \in A \cap B$ แล้ว $x \in A$ โดยนิยามของ $\cap$

ดังนั้น $x \in A$ ในทุกกรณี

($\supseteq$) ถ้า $x \in A$ แล้ว $x \in A \cup (A \cap B)$ เพราะ $x \in A \Rightarrow x \in A \cup X$ สำหรับทุกเซต X

สรุปได้ว่า $A \cup (A \cap B) = A$

2.5.3 ผลต่าง (Set Difference)

ทฤษฎีบท 2.24 **ผลต่าง (Difference)** ของเซต A และ B เขียนแทนด้วย $A - B$ หรือ $A \setminus B$ คือเซตที่ประกอบด้วยสมาชิกที่อยู่ใน A แต่ไม่อยู่ใน B กล่าวคือ $A - B = \{x \mid x \in A \text{ และ } x \notin B\}$ ในทางตรรกศาสตร์สอดคล้องกับ "A และ ไม่ B" และโดยทั่วไป $A - B \neq B - A$ ไม่มีสมบัติสลับที่ ตัวอย่างเช่น $\{1, 2\} - \{2, 3\} = \{1\}$ ในขณะที่ $\{2, 3\} - \{1, 2\} = \{3\}$

ตัวอย่าง 2.25 พิจารณาตัวอย่างต่อไปนี้: $\{1, 2, 3\} - \{2, 4\} = \{1, 3\}$ — นำ 2 และ 4 ออกจากเซตแรก

2 อยู่ในเซตแรกดังนั้นถูกนำออก 4 ไม่อยู่ในเซตแรกดังนั้นไม่กระทบ $\mathbb{Z} - \mathbb{N} = \{x \in \mathbb{Z} \mid x < 0\} = \mathbb{Z}^-$ (จำนวนเต็มลบ) — จำนวนเต็มที่ไม่เป็นจำนวนนับคือจำนวนเต็มลบ $\{1, 2\} - \{1, 2\} = \emptyset$ — นำสมาชิกทั้งหมดออกไม่เหลืออะไร และ $\{1, 2\} - \{3, 4\} = \{1, 2\}$ — ไม่มีสมาชิกใดในเซตที่สองที่อยู่ในเซตแรก ในบริบทต่างๆ: ให้ U เป็นเซตของนักศึกษาทั้งหมด และ A เป็นเซตของนักศึกษาที่สอบผ่าน แล้ว $U - A$ คือนักศึกษาที่สอบตก

2.5.4 คอมพลิเมนต์ (Complement)

Complement เป็นการดำเนินการที่ให้ "สิ่งที่ไม่อยู่ใน A " โดยอ้างอิงกับ Universal Set สิ่งสำคัญคือต้องกำหนด Universal Set ก่อนเสมอ เพราะ complement ขึ้นอยู่กับบริบท

ทฤษฎีบท 2.26 **คอมพลิเมนต์ (Complement)** ของเซต A เขียนแทนด้วย A^c หรือ $\bar{A}$ คือเซตที่ประกอบด้วยสมาชิกทั้งหมดใน Universal Set U ที่ไม่อยู่ใน A นั่นคือ $A^c = \{x \in U \mid x \notin A\} = U - A$ สมบัติสำคัญ: (1) $A \cup A^c = U$ (Complement Law) (2) $A \cap A^c = \emptyset$ (Non-Contradiction) (3) $(A^c)^c = A$ (Double Complement) (4) $A - B = A \cap B^c$ (Set Difference as Intersection with Complement) ซึ่งเป็นประโยชน์มากในการเปลี่ยนผลต่างเป็น intersection กับ complement ทำให้สามารถใช้ distributive law ได้

ตัวอย่าง 2.27 ถ้า $U = \{1, 2, 3, 4, 5\}$ และ $A = \{1, 2\}$ แล้ว $A^c = \{3, 4, 5\}$ เพราะ $3 \in U$ และ $3 \notin A$ ในขณะที่ $1 \notin A^c$ เพราะ $1 \in A$ แต่ถ้าเปลี่ยน Universal Set เป็น $U = \{1, 2, 3, 4, 5, 6, 7\}$ แล้ว $A^c = \{3, 4, 5, 6, 7\}$ — เซตเดียวกัน A มี complement ต่างกันเพราะ Universal Set ต่างกัน พิสูจน์ $A - B = A \cap B^c$: สมมติ $x \in A - B$ แล้ว $x \in A$ และ $x \notin B$ ซึ่งหมายความว่า $x \in A$ และ $x \in B^c$ ดังนั้น $x \in A \cap B^c$ ได้ $A - B \subseteq A \cap B^c$ และพิสูจน์กลับได้ $A \cap B^c \subseteq A - B$ ดังนั้น $A - B = A \cap B^c$

กฎของเดมอร์แกน (De Morgan's Laws)

กฎของเดมอร์แกนตั้งชื่อตาม Augustus De Morgan (1806–1871) นักคณิตศาสตร์ชาวอังกฤษ ซึ่งเป็นหนึ่งในผู้ก่อตั้งสาขาคณิตศาสตร์คณิตศาสตร์ (mathematical logic) กฎเหล่านี้แสดงความสัมพันธ์ระหว่าง complement กับ union และ intersection และมีความสำคัญมากในการพิสูจน์ทางคณิตศาสตร์และการออกแบบวงจรดิจิทัล

ทฤษฎีบท 2.28 (De Morgan's Laws สำหรับเซต) สำหรับทุกเซต A และ B :

1. $(A \cup B)^c = A^c \cap B^c$ (Complement ของ Union = Intersection ของ Complements)

2. $(A \cap B)^c = A^c \cup B^c$ (Complement ของ Intersection = Union ของ Complements)

กฎข้อ 1 พูดว่า "ไม่ (A หรือ B)" เท่ากับ "(ไม่ A) และ (ไม่ B)"

กฎข้อ 2 พูดว่า "ไม่ (A และ B)" เท่ากับ "(ไม่ A) หรือ (ไม่ B)"

หมายเหตุ 2.29 สังเกตความสอดคล้องกับ De Morgan's Laws ในตรรกศาสตร์:

1. $\neg(p \wedge q) \equiv \neg p \vee \neg q$ สอดคล้องกับ $(A \cap B)^c = A^c \cup B^c$

2. $\neg(p \vee q) \equiv \neg p \wedge \neg q$ สอดคล้องกับ $(A \cup B)^c = A^c \cap B^c$

นี่เป็นตัวอย่างที่แสดงความเชื่อมโยงระหว่างตรรกศาสตร์และทฤษฎีเซตอย่างลึกซึ้ง ทั้งสองระบบมีโครงสร้างทางคณิตศาสตร์ที่เหมือนกัน

De Morgan's Laws มีการประยุกต์ใช้มากในวงจรดิจิทัล ในการออกแบบวงจร เรามักต้องการแปลง AND gates เป็น OR gates หรือในทางกลับกัน โดยใช้ NOT gates De Morgan's Laws บอกเราว่าวิธีที่ถูกต้องคืออะไร

ตัวอย่าง 2.30 พิสูจน์ $(A \cup B)^c = A^c \cap B^c$

สมมติ $x \in (A \cup B)^c$

$x \notin A \cup B$

$\neg(x \in A \cup B)$

$\neg(x \in A \text{ หรือ } x \in B)$

$\neg(x \in A) \text{ และ } \neg(x \in B)$

(De Morgan ในตรรกศาสตร์)

$x \notin A \text{ และ } x \notin B$

$x \in A^c \text{ และ } x \in B^c$

$x \in A^c \cap B^c$

ดังนั้น $(A \cup B)^c \subseteq A^c \cap B^c$

สมมติ $x \in A^c \cap B^c$

$$x \in A^c \text{ และ } x \in B^c$$

$$x \notin A \text{ และ } x \notin B$$

$$\neg(x \in A) \text{ และ } \neg(x \in B)$$

$$\neg(x \in A \text{ หรือ } x \in B)$$

(De Morgan ในตรรกศาสตร์)

$$x \notin A \cup B$$

$$x \in (A \cup B)^c$$

$$\text{ดังนั้น } A^c \cap B^c \subseteq (A \cup B)^c$$

เนื่องจาก $\subseteq$ และ $\supseteq$ ต่างก็เป็นจริง ดังนั้น $(A \cup B)^c = A^c \cap B^c$

2.6.1 เพาเวอร์เซต (Power Set)

ทฤษฎีบท 2.31 เพาเวอร์เซต (Power Set) ของเซต A เขียนแทนด้วย $\mathcal{P}(A)$ หรือ 2^A คือเซตที่ประกอบด้วยเซตย่อยทั้งหมดของ A กล่าวคือ $\mathcal{P}(A) = \{B \mid B \subseteq A\}$ เพาเวอร์เซตเป็น "เซตของเซต" เพราะสมาชิกของมันคือเซตย่อยของ A สมบัติสำคัญ: ถ้า $|A| = n$ แล้ว $|\mathcal{P}(A)| = 2^n$ เพราะแต่ละสมาชิกใน A สามารถตัดสินใจได้ 2 ทางคือ "อยู่" หรือ "ไม่อยู่" ในเซตย่อย สัญลักษณ์ 2^A จึงสื่อความหมายเพราะ $|\mathcal{P}(A)| = 2^{|A|}$

ตัวอย่าง 2.32 พิจารณาตัวอย่างต่อไปนี้: $\mathcal{P}(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ — เซตย่อยทั้งหมดได้แก่ เซตว่าง, เซตที่มี 1, เซตที่มี 2, และเซตเต็ม $\mathcal{P}(\emptyset) = \{\emptyset\}$ — สังเกตว่าเพาเวอร์เซตของเซตว่างไม่ใช่เซตว่าง เพราะมันมีสมาชิก 1 ตัว คือ $\emptyset$ เอง $\mathcal{P}(\{a\}) = \{\emptyset, \{a\}\}$ — มีเซตย่อย 2 ตัว และ $\mathcal{P}(\{1, 2, 3\}) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$ — มีเซตย่อย 8 ตัว ซึ่งตรงกับ $2^3 = 8$

2.6.2 ผลคูณคาร์ทีเซียน (Cartesian Product)

คู่อันดับ (a, b) แตกต่างจากเซต $\{a, b\}$ ตรงที่มีลำดับ กล่าวคือ $(a, b) \neq (b, a)$ ยกเว้นกรณี $a = b$ และ $(a, b) = (c, d)$ ก็ต่อเมื่อ $a = c$ และ $b = d$ สัญลักษณ์ Cartesian Product มาจากชื่อนักคณิตศาสตร์ Rene Descartes ผู้ที่พัฒนาเรขาคณิตวิเคราะห์ $\mathbb{R} \times \mathbb{R} = \mathbb{R}^2$ คือระนาบ 2 มิติที่เราใช้กันทั่วไป

ทฤษฎีบท 2.33 ผลคูณคาร์ทีเซียน (Cartesian Product) ของเซต A และ B เขียนแทนด้วย $A \times B$ คือเซตของคู่อันดับ (ordered pairs) (a, b) โดยที่ $a \in A$ และ $b \in B$ นั่นคือ $A \times B = \{(a, b) \mid a \in A \text{ และ } b \in B\}$ สัญลักษณ์ $\times$ อ่านว่า "คูณ" หรือ "cross"

ตัวอย่าง 2.34 $\{1, 2\} \times \{a, b\} = \{(1, a), (1, b), (2, a), (2, b)\}$ — สมาชิกตัวแรกมาจากเซตแรก สมาชิกตัวหลังมาจากเซตหลัง ทุกคู่ที่เป็นไปได้จะอยู่ในผลลัพธ์ สังเกตว่าลำดับสำคัญ $(1, 3) \neq (3, 1)$ นอกจากนี้ $\mathbb{R} \times \mathbb{R} = \mathbb{R}^2$ คือระนาบ 2 มิติ และ $\emptyset \times A = A \times \emptyset = \emptyset$ สำหรับทุกเซต A เพราะไม่มีคู่อันดับใดที่มาจากเซตว่าง

ทฤษฎีบท 2.35 ถ้า $|A| = m$ และ $|B| = n$ แล้ว $|A \times B| = m \cdot n$ เพราะทุกสมาชิกใน A สามารถจับคู่กับทุกสมาชิกใน B ได้ จึงมี $m \times n$ คู่ และ $\mathbb{R} \times \mathbb{R}$ มี cardinality เท่ากับ $|\mathbb{R}|^2$ ซึ่งเป็น uncountable เช่นเดียวกับ $\mathbb{R}$

ทฤษฎีบท 2.36 สมบัติของ Cartesian Product:

1. $A \times (B \cup C) = (A \times B) \cup (A \times C)$
2. $A \times (B \cap C) = (A \times B) \cap (A \times C)$
3. $(A \cup B) \times C = (A \times C) \cup (B \times C)$
4. $(A \cap B) \times C = (A \times C) \cap (B \times C)$

สังเกตว่า Cartesian Product กระจายเหนือ union และ intersection ในทางที่คล้ายกับการกระจายการคูณเหนือการบวก

การนับจำนวนสมาชิกของเซต (Cardinality)

ทฤษฎีบท 2.37 **คาร์ดินัลลิตี้ (Cardinality)** ของเซต A เขียนแทนด้วย $|A|$ หมายถึงจำนวนสมาชิกในเซต A

สำหรับเซตจำกัด คาร์ดินัลลิตี้คือจำนวนสมาชิกที่นับได้ สำหรับเซตอนันต์จะต้องใช้แนวคิดที่ซับซ้อนกว่านี้ (Cantor's diagonal argument) ซึ่งเราจะศึกษาในบทต่อไป

การนับจำนวนสมาชิกเป็นพื้นฐานของการนับ (combinatorics) และมีการประยุกต์ใช้มากในการนับโอกาสทั้งหมดที่เป็นไปได้

ทฤษฎีบท 2.38 (หลักการรวม-การแยก (Inclusion-Exclusion Principle)) สำหรับเซตจำกัด A และ B :

1. $|A \cup B| = |A| + |B| - |A \cap B|$
2. ถ้า A และ B แยกกัน (disjoint, คือ $A \cap B = \emptyset$) แล้ว $|A \cup B| = |A| + |B|$

หลักการรวม-การแยกใช้เมื่อต้องการนับจำนวนสมาชิกของ union ถ้านับ $|A| + |B|$ โดยตรง สมาชิกที่อยู่ทั้งสองเซต (intersection) จะถูกนับซ้ำ จึงต้องลบออก

ตัวอย่าง 2.39 ถ้า $A = \{1, 2, 3, 4\}$, $B = \{3, 4, 5, 6\}$ แล้ว $|A| = 4$, $|B| = 4$, $|A \cap B| = 2$ (คือ $\{3, 4\}$)

$$|A \cup B| = |\{1, 2, 3, 4, 5, 6\}| = 6 = 4 + 4 - 2$$

นี้แสดงให้เห็นว่าถ้านับ $4 + 4 = 8$ โดยไม่หัก intersection จะได้ 8 ซึ่งมากกว่าความจริง (6) เพราะ 3 และ 4 ถูกนับซ้ำ

การประยุกต์ใช้ในชีวิตจริง:

1. นักศึกษา 30 คน 15 คนเรียนภาษาอังกฤษ 20 คนเรียนคณิตศาสตร์ มีนักศึกษากี่คนที่เรียนอย่างน้อยหนึ่งวิชา? ถ้าไม่มีนักศึกษาคนใดเรียนทั้งสองวิชา (disjoint) ก็ $15 + 20 = 35$ ซึ่งมากกว่า 30 แสดงว่าต้องมีนักศึกษาที่เรียนทั้งสองวิชา

ทฤษฎีบท 2.40 สำหรับเซต 3 เซต:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$$

สูตรนี้ขยายจาก 2 เซต โดยลบ intersection ของทุกคู่ แล้วบวก intersection ของทั้งสามกลับ (เพราะถูกลบเกินไป)

ตัวอย่าง 2.41 ในห้องเรียน 20 คน มี 12 คนเรียนคณิตศาสตร์ 15 คนเรียนฟิสิกส์ และ 10 คนเรียนเคมี นอกจากนี้ 6 คนเรียนคณิตศาสตร์และฟิสิกส์ 5 คนเรียนคณิตศาสตร์และเคมี 4 คนเรียนฟิสิกส์และเคมี และ 2 คนเรียนทั้งสามวิชา มีกี่คนที่เรียนอย่างน้อยหนึ่งวิชา?

ใช้สูตร:

$$|M \cup P \cup C| = 12 + 15 + 10 - 6 - 5 - 4 + 2 = 24$$

ตรวจสอบ: $24 \leq 20$ เป็นไปไม่ได้! นี่บอกว่าข้อมูลขัดแย้งกัน หรืออาจมีนักเรียนที่ไม่ได้เรียนวิชาใดเลย 24 คนเรียนอย่างน้อย 1 วิชา แต่มีนักเรียนทั้งหมด 20 คน แสดงว่าต้องมีคนซ้ำซ้อนในการนับ หรือ

ข้อมูลผิด

ทฤษฎีบท 2.42 สำหรับ Cartesian Product: $|A \times B| = |A| \cdot |B|$

ทฤษฎีบท 2.43 $|\mathcal{P}(A)| = 2^{|A|}$ สำหรับเซตจำกัด A

นี่เป็นเหตุผลว่าทำไมเพาเวอร์เซตเติบโตเร็วมาก เมื่อ $|A|$ เพิ่มขึ้น 1 $|\mathcal{P}(A)|$ จะเพิ่มเป็น 2 เท่า

ประยุกต์ในวิทยาการคอมพิวเตอร์

ทฤษฎีเซตไม่ได้เป็นเพียงนามธรรมทางคณิตศาสตร์ แต่เป็นรากฐานของวิทยาการคอมพิวเตอร์หลายด้าน ในส่วนนี้เราจะศึกษาการประยุกต์ใช้ที่สำคัญโดยเน้นความเชื่อมโยงทางคณิตศาสตร์

2.8.1 การแทนเซตในโครงสร้างข้อมูล

ในการเขียนโปรแกรม มีหลายวิธีในการแทนเซต แต่ละวิธีมีข้อดีและข้อเสียต่างกัน

ตัวอย่าง 2.44 (Bit Vector Representation) ในการแทนเซตด้วย bit vector เราใช้ bit ที่ position ที่ i แทนว่าสมาชิกที่ i อยู่ในเซตหรือไม่

ถ้า Universal Set $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ เราสามารถแทนเซต $A = \{2, 5, 7\}$ ด้วย bit vector:

$U: 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9$

$A: 0 \ 1 \ 0 \ 0 \ 1 \ 0 \ 1 \ 0 \ 0$

ซึ่งแทนด้วย integer คือ $010010100_2 = 148$ ในฐาน 10

ประโยชน์ของ bit vector:

1. Union = bitwise OR ($\vee$)
2. Intersection = bitwise AND ($\wedge$)
3. Complement = bitwise NOT ($\neg$)
4. Membership test = ตรวจสอบ bit ที่ position ที่เกี่ยวข้อง

การแทนแบบนี้มีประสิทธิภาพมากในหน่วยความจำและเวลา เพราะทุก operation ทำได้ใน $O(1)$

บน hardware

ทฤษฎีบท 2.45 (Set Operations as Boolean Algebra) เซตบน universal set ที่มีขนาด n สามารถแทนด้วย bit vector ความยาว n ซึ่งทำให้การดำเนินการระหว่างเซตสอดคล้องกับ Boolean algebra:

1. $A \cup B \Leftrightarrow$ bitwise OR ของ bit vectors
2. $A \cap B \Leftrightarrow$ bitwise AND ของ bit vectors
3. $A^c \Leftrightarrow$ bitwise NOT ของ bit vector
4. $A \subseteq B \Leftrightarrow (A \cap B^c) = \emptyset$ หรือ $(A \wedge \neg B) = 0$ ทุก bit

นี่คือความเชื่อมโยงระหว่าง set theory และ Boolean algebra ซึ่งมีพื้นฐานมาจาก propositional logic

2.8.2 การประยุกต์พื้นฐานข้อมูล

ในระบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ตารางแต่ละตารางเป็นเซตของ tuples (แถว) การ query ด้วย SQL ก็คือการดำเนินการระหว่างเซตอย่างเป็นทางการ

ทฤษฎีบท 2.46 (Relational Algebra as Set Operations) การดำเนินการบนตารางใน relational database มีความสัมพันธ์โดยตรงกับ set operations:

ตารางที่ 9 ความสัมพันธ์ระหว่าง Relational Algebra และ Set Theory

Relational Algebra	Set Theory
Union ($\cup$)	Union
Selection (σ)	Filtering / Subset
Projection (π)	Domain restriction
Cartesian Product ($\times$)	Cartesian Product
Join ($\bowtie$)	Complex combination

ตัวอย่าง 2.47 (Database Relations as Mathematical Sets) ในทฤษฎีฐานข้อมูลเชิงสัมพันธ์ การเข้าใจความสัมพันธ์ระหว่าง relational algebra และ set theory ช่วยให้เขียน SQL ที่มีประสิทธิภาพและเข้าใจการทำงานของ query optimizer

ตัวอย่าง:

1. **ตาราง (Relation)** R เป็นเซตของ tuples นั่นคือ $R \subseteq D_1 \times D_2 \times \dots \times D_n$ โดยที่ D_i คือ domain ของ attribute ที่ i

2. **Primary Key** คือ subset ของ attributes ที่ uniquely identify tuples ใน relation

3. **Foreign Key** คือ subset ของ attributes ที่อ้างอิงไปยัง primary key ของอีก relation หนึ่ง (เป็น subset relationship ระหว่าง schemas)

ถ้ากำหนดให้ S แทนเซตของนักศึกษา และ C แทนเซตของรายวิชา และ $E \subseteq S \times C$ แทนเซตของการลงทะเบียน

แล้ว E คือเซตของ tuples (s, c) โดยที่ $s \in S$ และ $c \in C$ ซึ่งแสดงว่านักศึกษา s ลงทะเบียนเรียนวิชา c — นี่คือ Cartesian Product ในทางคณิตศาสตร์

2.8.3 Hash Tables และ Functions

Hash Table เป็นโครงสร้างข้อมูลที่ใช้ฟังก์ชันแฮช (hash function) ซึ่งมีความเชื่อมโยงกับทฤษฎีเซตอย่างลึกซึ้ง

ทฤษฎีบท 2.48 (Hash Function as Mathematical Function) Hash Function $h : U \rightarrow \{0, 1, \dots, m-1\}$ เป็นฟังก์ชันจาก universe of keys U ไปยัง set of addresses $\{0, 1, \dots, m-1\}$ สมบัติที่พึงประสงค์:

1. **Deterministic:** ถ้า $k_1 = k_2$ แล้ว $h(k_1) = h(k_2)$

2. **Uniformly Distributed:** สำหรับ keys ที่สุ่มมา $h(k)$ ควรกระจาย uniformly ใน $\{0, 1, \dots, m-1\}$

3. **Efficiently Computable:** คำนวณได้ใน $O(1)$ time

เมื่อเกิด collision $h(k_1) = h(k_2)$ สำหรับ $k_1 \neq k_2$ มีวิธีการแก้ไขได้แก่:

1. **Chaining:** $T[i] = \{k \in K \mid h(k) = i\}$ — แต่ละ slot เก็บเซตของ keys ที่ collide

2. **Open Addressing:** ใช้ probe sequence $h(k, i) = (h'(k) + i) \bmod m$ สำหรับ $i = 0, 1, 2, \dots$

ทฤษฎีบท 2.49 (Complexity of Set Operations) ประสิทธิภาพของ set operations บน hash-based structures:

ตารางที่ 10 ประสิทธิภาพของ Hash Set Operations

Operation	Time Complexity
Membership Test ($x \in A$)	$O(1)$
Add ($A \cup \{x\}$)	$O(1)$
Remove ($A \setminus \{x\}$)	$O(1)$
Union ($A \cup B$)	$O(A + B)$
Intersection ($A \cap B$)	$O(\min(A , B))$
Subset Test ($A \subseteq B$)	$O(A)$

แบบฝึกหัด

2.9.1 ส่วนที่ 1: นิยามพื้นฐาน

1. เขียนเซตต่อไปนี้โดยใช้ทั้ง roster method และ set-builder notation:

- 1.1. เซตของจำนวนเต็มบวกที่น้อยกว่า 10
- 1.2. เซตของจำนวนเฉพาะที่น้อยกว่า 20
- 1.3. เซตของสระในภาษาอังกฤษ
- 1.4. เซตของจำนวนเต็ม x ที่ $x^2 - 4 = 0$
- 1.5. เซตของจำนวนเต็ม x ที่ $x^2 - 5x + 6 = 0$

2. กำหนดให้ $A = \{1, 2, 3\}$, $B = \{3, 4, 5\}$, $C = \{5, 6\}$ จงหา:

- 2.1. $A \cup B$
- 2.2. $A \cap B$
- 2.3. $A - B$
- 2.4. $B - A$
- 2.5. $(A \cup B) - C$

$$2.6. A \cap (B \cup C)$$

$$2.7. (A - B) \cup (B - A)$$

$$2.8. A \cap B \cap C$$

$$2.9. (A \cup B) \cap (B \cup C)$$

3. พิจารณาว่าข้อความต่อไปนี้จริงหรือเท็จ พร้อมอธิบายเหตุผล:

$$3.1. \emptyset \in \{\emptyset\}$$

$$3.2. \emptyset \subseteq \{\emptyset\}$$

$$3.3. \{\emptyset\} \in \{\emptyset, \{\emptyset\}\}$$

$$3.4. \{1, 2\} \subseteq \{\{1\}, \{2\}\}$$

$$3.5. \{1, 2\} \in \{\{1, 2\}\}$$

$$3.6. |\mathcal{P}(\{1, 2, 3\})| = 8$$

$$3.7. \mathcal{P}(\emptyset) = \emptyset$$

$$3.8. \{1, 2\} \times \{3, 4\} = \{3, 4\} \times \{1, 2\}$$

4. อธิบายความแตกต่างระหว่าง $\in$ และ $\subseteq$ พร้อมยกตัวอย่างที่แสดงความแตกต่างนั้น

2.9.2 ส่วนที่ 2: การพิสูจน์

1. พิสูจน์โดยตรง (Direct Proof):

$$1.1. \text{ ถ้า } A \subseteq B \text{ และ } B \subseteq C \text{ แล้ว } A \subseteq C \text{ (Transitivity ของ } \subseteq \text{)}$$

$$1.2. \text{ ถ้า } A \subseteq B \text{ แล้ว } A \cup C \subseteq B \cup C$$

$$1.3. \text{ ถ้า } A \subseteq B \text{ แล้ว } A \cap C \subseteq B \cap C$$

$$1.4. A \cup (B \cap C) = (A \cup B) \cap (A \cup C) \text{ (Distributive Law ข้อ 2)}$$

2. พิสูจน์โดย contraposition:

$$2.1. \text{ ถ้า } A \cup B \neq A \text{ แล้ว } B \not\subseteq A$$

$$2.2. \text{ ถ้า } A \cap B = \emptyset \text{ แล้ว } A \subseteq B^c$$

$$2.3. \text{ ถ้า } A \not\subseteq B \cup C \text{ แล้ว } A \not\subseteq B \text{ และ } A \not\subseteq C$$

3. พิสูจน์โดยขัดแย้ง (Proof by Contradiction):

$$3.1. \text{ ถ้า } A \subseteq B \text{ แล้ว } B^c \subseteq A^c$$

3.2. ถ้า $A \cup B = A \cap B$ แล้ว $A = B$

3.3. ไม่มีเซต A ที่ทำให้ $A \subsetneq \mathcal{P}(A)$

4. พิสูจน์ว่าเซตต่อไปนี้เท่ากัน:

4.1. $A - (B \cup C) = (A - B) \cap (A - C)$

4.2. $A \Delta B = (A \cup B) - (A \cap B)$ (Symmetric Difference)

4.3. $(A - B) \cap C = (A \cap C) - B$

2.9.3 ส่วนที่ 3: กฎของเดมอร์แกนและการดำเนินการ

1. พิสูจน์ De Morgan's Laws:

1.1. $(A \cap B)^c = A^c \cup B^c$

1.2. $(A \cup B)^c = A^c \cap B^c$

2. ลดรูปเซตต่อไปนี้ (แสดงว่าเท่ากับเซตที่ง่ายกว่า):

2.1. $(A \cup B) \cap (A \cup B^c)$

2.2. $(A - B) \cup (A \cap B)$

2.3. $A \cup (B - A)$

2.4. $(A \cap B) \cup (A - B)$

2.5. $A \cap (A \cup B)$

2.6. $(A^c)^c$

3. กำหนดให้ A, B, C เป็นเซตย่อยของ universal set U พิจารณาว่าข้อความต่อไปนี้จริงหรือเท็จ พร้อมพิสูจน์หรือหักล้าง:

3.1. $A - B = B - A$

3.2. $A \cup B = A \cap B$ แก่สมการนี้

3.3. $(A - B) - C = A - (B - C)$

2.9.4 ส่วนที่ 4: เพาเวอร์เซตและคาร์ทีเซียน

1. จงหา:

1.1. $\mathcal{P}(\{a, b, c\})$

1.2. $|\mathcal{P}(\mathcal{P}(\emptyset))|$

- 1.3. $\{1, 2\} \times \{3, 4, 5\}$
- 1.4. $|\{1, 2\} \times \{3, 4\} \times \{5, 6\}|$
- 1.5. $\mathcal{P}(\{1, 2\}) \cap \mathcal{P}(\{2, 3\})$
- 1.6. $\mathcal{P}(\{1, 2\}) \cup \mathcal{P}(\{2, 3\})$

2. พิสูจน์:

- 2.1. $\mathcal{P}(A) \cup \mathcal{P}(B) \subseteq \mathcal{P}(A \cup B)$
- 2.2. พิสูจน์หรือหักล้าง: $\mathcal{P}(A \cup B) \subseteq \mathcal{P}(A) \cup \mathcal{P}(B)$
- 2.3. $(A \times B) \cap (C \times D) = (A \cap C) \times (B \cap D)$

3. ถ้า $|A| = 3$ และ $|B| = 4$ จงหา:

- 3.1. $|A \times B|$
- 3.2. $|\mathcal{P}(A)|$
- 3.3. $|\mathcal{P}(A \times B)|$
- 3.4. $|\mathcal{P}(A) \times \mathcal{P}(B)|$

4. อธิบายว่าทำไม $|\mathcal{P}(A)| > |A|$ สำหรับทุกเซต A ที่ไม่ใช่เซตว่าง (Cantor's Theorem)

2.9.5 ส่วนที่ 5: การประยุกต์และการเชื่อมโยงทางคณิตศาสตร์

1. กำหนดให้ $A = \{10, 20, 30, 40\}$ และ $B = \{25, 30, 35, 40\}$ ในฐานข้อมูล จงเขียนในรูปแบบคณิตศาสตร์:

- 1.1. จงหา $A \cup B$, $A \cap B$, $A \setminus B$, $A \Delta B$ (Symmetric Difference)
- 1.2. แสดงว่า $A \Delta B = (A \cup B) \setminus (A \cap B)$

2. กำหนดให้ $U = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ และ $A = \{1, 2, 3, 4, 5\}$, $B = \{4, 5, 6, 7\}$

- 2.1. เขียน bit vector ของ A , B , $A \cup B$, $A \cap B$, A^c
- 2.2. แสดงว่า bitwise operations บน bit vectors ให้ผลลัพธ์ที่ถูกต้องสำหรับ set operations

3. อธิบายความสัมพันธ์ระหว่าง:

3.1. De Morgan's Laws ใน ทฤษฎีเซต และ De Morgan's Laws ใน ตรรกศาสตร์ (Propositional Logic)

- 3.2. Cartesian Product $A \times B$ และ relations ระหว่างเซต
- 3.3. Power Set $\mathcal{P}(A)$ และ cardinality $|\mathcal{P}(A)| = 2^{|A|}$
4. พิสูจน์โดยใช้ทฤษฎีเซต:
- 4.1. ถ้า $A \subseteq B$ แล้ว $\mathcal{P}(A) \subseteq \mathcal{P}(B)$
- 4.2. ถ้า $A \times B = A \times C$ และ $A \neq \emptyset$ แล้ว $B = C$

เฉลยแบบฝึกหัด

2.10.1 เฉลยส่วนที่ 1

1.
 - 1.1. $\{1, 2, 3, 4, 5, 6, 7, 8, 9\} = \{x \in \mathbb{N} \mid x < 10\}$
 - 1.2. $\{2, 3, 5, 7, 11, 13, 17, 19\} = \{p \in \mathbb{N} \mid p \text{ เป็นจำนวนเฉพาะ และ } p < 20\}$
 - 1.3. $\{a, e, i, o, u\}$
 - 1.4. $\{-2, 2\} = \{x \in \mathbb{Z} \mid x^2 = 4\}$
 - 1.5. $\{2, 3\} = \{x \in \mathbb{Z} \mid x^2 - 5x + 6 = 0\}$
2.
 - 2.1. $\{1, 2, 3, 4, 5\}$
 - 2.2. $\{3\}$
 - 2.3. $\{1, 2\}$
 - 2.4. $\{4, 5\}$
 - 2.5. $\{1, 2, 3, 4\}$
 - 2.6. $\{3, 5\}$
 - 2.7. $\{1, 2, 4, 5\}$
 - 2.8. $\{5\}$
 - 2.9. $\{3, 4, 5\}$
3.
 - 3.1. จริง ($\emptyset$ เป็นสมาชิกของ $\{\emptyset\}$ เพราะ $\emptyset$ อยู่ในนั้น)
 - 3.2. จริง (เซตว่างเป็นเซตย่อยของทุกเซต)
 - 3.3. จริง ($\{\emptyset\}$ เป็นสมาชิกของ $\{\emptyset, \{\emptyset\}\}$)
 - 3.4. เท็จ ($\{1\}$ ไม่เท่ากับ $\{1, 2\}$ ดังนั้น $\{1, 2\}$ ไม่เป็นเซตย่อย)

- 3.5. เท็จ ($\{1, 2\}$ ไม่เท่ากับ $\{\{1, 2\}\}$ สมาชิกของ $\{\{1, 2\}\}$ คือ $\{1, 2\}$ ไม่ใช่ 1 หรือ 2)
- 3.6. จริง ($|\{1, 2, 3\}| = 3, |\mathcal{P}(A)| = 2^3 = 8$)
- 3.7. เท็จ ($\mathcal{P}(\emptyset) = \{\emptyset\} \neq \emptyset$)
- 3.8. เท็จ ($(1, 3) \neq (3, 1)$ ดังนั้น Cartesian Product ไม่มีสมบัติสลับที่)

เซตที่นับได้และเซตที่นับไม่ได้

วัตถุประสงค์การเรียนรู้:

- ✓ เข้าใจความแตกต่างระหว่างเซตที่นับได้และเซตที่นับไม่ได้
- ✓ รู้จักภาวะเชิงการนับ $\aleph_0$ และ c
- ✓ เข้าใจการพิสูจน์ว่าเซตจำนวนจริงนับไม่ได้

ในการจำแนกประเภทของเซตอนันต์ เราแบ่งออกเป็น "เซตที่นับได้" (Countable) และ "เซตที่นับไม่ได้" (Uncountable) เซตที่นับได้คือเซตที่สามารถจับคู่สมาชิกกับจำนวนนับได้ทุกตัว กล่าวคือมีฟังก์ชันหนึ่งต่อหนึ่งจากเซตไปยัง $\mathbb{N}$

นิยาม 2.50 (เซตที่นับได้และเซตที่นับไม่ได้) เซต A เป็น **เซตที่นับได้** (countable) ก็ต่อเมื่อ $|A| \leq |\mathbb{N}|$ หรือกล่าวอีกนัยหนึ่งคือมีฟังก์ชันหนึ่งต่อหนึ่งจาก A ไปยังเซตย่อยของ $\mathbb{N}$ ถ้า A ไม่นับได้ เราเรียกว่า A เป็น **เซตที่นับไม่ได้** (uncountable)

Cantor ได้พิสูจน์ว่าเซตที่นับได้มีขนาดเล็กกว่าเซตจำนวนจริงอย่างเด็ดขาด โดยใช้วิธีการที่เรียกว่า "การให้เหตุผลแบบ diagonal" (Diagonal Argument) พิสูจน์นี้แสดงให้เห็นว่าแม้แต่ช่วง $(0, 1)$ ของจำนวนจริงก็นับไม่ได้

ทฤษฎีบท 2.51 (Cantor) ช่วงเปิด $(0, 1)$ ของจำนวนจริงเป็นเซตที่นับไม่ได้

ตัวอย่าง 2.52 ตัวอย่างเซตที่นับได้และเซตที่นับไม่ได้:

(ก) $\mathbb{N}$ นับได้ เพราะ $|\mathbb{N}| = \aleph_0$

(ข) $\mathbb{Z}$ นับได้ เพราะสามารถจับคู่กับ $\mathbb{N}$ ได้ เช่น $0 \leftrightarrow 1, 1 \leftrightarrow 2, -1 \leftrightarrow 3, 2 \leftrightarrow 4, -2 \leftrightarrow 5, \dots$

- (ค) $\mathbb{Q}$ (จำนวนตรรกยะ) นับได้ แม้ว่าจะ dense อยู่ระหว่างจำนวนจริงทุกตัว
- (ง) $\mathbb{R}$ นับไม่ได้ เพราะ $|\mathbb{R}| = \mathfrak{c} > \aleph_0$

สรุป

ในบทนี้เราได้ศึกษาพื้นฐานของทฤษฎีเซตอย่างครอบคลุม ซึ่งประกอบด้วย:

1. **นิยามของเซต** — เซต สมาชิก วิธีเขียนเซต (roster method, set-builder notation) เซตพิเศษ (empty set, universal set)
2. **ความสัมพันธ์ระหว่างเซต** — เซตย่อย (subset), เซตย่อยแท้ (proper subset), ความเท่ากันของเซต, การพิสูจน์โดย double inclusion
3. **การดำเนินการระหว่างเซต** — Union, Intersection, Difference, Complement และสมบัติต่างๆ
4. **กฎของเดมอร์แกน** — ความสัมพันธ์ระหว่าง complement กับ union/intersection
5. **เพาเวอร์เซตและคาร์ทีเซียน** — $\mathcal{P}(A)$, $A \times B$, ความสัมพันธ์กับ functions
6. **หลักการรวม-การแยก (Inclusion-Exclusion)** — $|A \cup B| = |A| + |B| - |A \cap B|$
7. **การประยุกต์ในวิทยาการคอมพิวเตอร์** — โครงสร้างข้อมูล, ฐานข้อมูล, Hash Tables

ทฤษฎีเซต เป็น เครื่องมือ ที่ จำเป็น สำหรับ การ ศึกษา บท ต่อ ไป โดยเฉพาะ **ความสัมพันธ์ (Relations)** ซึ่งใช้ Cartesian Product เป็นพื้นฐาน และ **ฟังก์ชัน (Functions)** ซึ่งเป็น special case ของ relations

ความเข้าใจที่ลึกซึ้งใน ทฤษฎีเซต จะ ช่วยให้ เข้าใจ แนวคิด ชั้น สูง ใน คณิตศาสตร์ และ วิทยาการคอมพิวเตอร์ได้ดียิ่งขึ้น เพราะภาษาของเซตเป็นภาษาพื้นฐานที่ใช้ในทุกสาขาของคณิตศาสตร์ทั้งด้านบริสุทธิ์และประยุกต์

Bibliography

- [1] Rosen, Kenneth H. *Discrete Mathematics and Its Applications*. 8th ed., McGraw-Hill, 2019.

- [2] Epp, Susanna S. *Discrete Mathematics with Applications*. 5th ed., Cengage Learning, 2019.
- [3] Halmos, Paul R. *Naive Set Theory*. Springer, 1974.
- [4] Devlin, Keith. *Fundamentals of Contemporary Set Theory*. Springer, 1979.
- [5] Enderton, Herbert B. *Elements of Set Theory*. Academic Press, 1977.
- [6] Cormen, Thomas H., et al. *Introduction to Algorithms*. 3rd ed., MIT Press, 2009.
- [7] Elmasri, Ramez, and Shamkant B. Navathe. *Fundamentals of Database Systems*. 7th ed., Pearson, 2016.
- [8] Grimaldi, Ralph P. *Discrete and Combinatorial Mathematics: An Applied Introduction*. 5th ed., Pearson, 2003.
- [9] Cantini, Andrea, and Erik W. Baruser. *Paradoxes, Self-Reference and Truth*. Springer, 1999.

เอกสารอ้างอิง

แผนการสอนประจำบทที่ 3

รหัสวิชา	4121701
ชื่อวิชา	คณิตศาสตร์สำหรับคอมพิวเตอร์
หน่วยกิต	3 (3-0-6)
บทที่	3
ชื่อบท	ความสัมพันธ์และฟังก์ชัน (Relations and Functions)
จำนวนชั่วโมง	6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- อธิบายความหมายของความสัมพันธ์ (Relation) และผลคูณคาร์ทีเซียนได้
- ระบุคุณสมบัติของความสัมพันธ์ เช่น Reflexive, Symmetric, Transitive ได้
- ระบุและจำแนกประเภทของฟังก์ชัน เช่น Injective, Surjective, Bijective ได้
- ใช้การประกอบฟังก์ชัน (Composition) และหาฟังก์ชันผกผันได้
- ประยุกต์ใช้ความสัมพันธ์และฟังก์ชันในบริบทของโปรแกรมคอมพิวเตอร์ได้

หัวข้อที่สอน

- ผลคูณคาร์ทีเซียนและความสัมพันธ์
- คุณสมบัติของความสัมพันธ์
- ความสัมพันธ์สมมูล (Equivalence Relation)
- นิยามและประเภทของฟังก์ชัน
- การประกอบฟังก์ชันและฟังก์ชันผกผัน

สื่อการสอน

- สไลด์ประกอบการสอน
- ตัวอย่างโค้ดและ Diagram
- แบบฝึกหัดท้ายบท

ความสัมพันธ์ (Relations)

วัตถุประสงค์การเรียนรู้

หลังจากศึกษาจบบทนี้ ผู้เรียนจะสามารถดำเนินการต่างๆ ได้อย่างครบถ้วน ได้แก่ การอธิบายความหมายและความสำคัญของความสัมพันธ์ในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ การบอกนิยามของความสัมพันธ์และความสัมพันธ์แบบทวิภาค การตรวจสอบคุณสมบัติสรีการของความสัมพันธ์ การอธิบายและพิสูจน์คุณสมบัติของความสัมพันธ์สมมูลพร้อมทั้งหาคลาสสมมูล การอธิบายและใช้ความสัมพันธ์สมมาตรเชิงส่วนย่อย การดำเนินการรวมและหาความสัมพันธ์ผกผัน และการประยุกต์ใช้ความสัมพันธ์ในงานต่างๆ ทางคอมพิวเตอร์

บทนำ

ความสัมพันธ์เป็นแนวคิดพื้นฐานที่มีความสำคัญยิ่งในวิชาคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ในชีวิตประจำวัน ผู้คนต่างมีความสัมพันธ์ซึ่งกันและกัน ไม่ว่าจะเป็นความสัมพันธ์ระหว่างบิดามารดากับบุตร ความสัมพันธ์ระหว่างนายจ้างกับลูกจ้าง หรือความสัมพันธ์ระหว่างผู้ส่งอีเมลกับผู้รับอีเมล

ในทางคณิตศาสตร์ ความสัมพันธ์ช่วยให้เราจัดระเบียบและอธิบายความเชื่อมโยงระหว่างสิ่งต่างๆ ได้อย่างเป็นระบบ ความสัมพันธ์ถูกนำไปใช้ในหลากหลายสาขา ได้แก่ ฐานข้อมูลซึ่งใช้ความสัมพันธ์ระหว่างตาราง การวิเคราะห์โครงสร้างเว็บไซต์ซึ่งใช้ความสัมพันธ์แบบไฮเปอร์ลิงก์ และการจัดลำดับการทำงานของโปรแกรมซึ่งใช้ความสัมพันธ์เพื่อกำหนดลำดับก่อนหลัง

3.2.1 ประวัติความเป็นมาของความสัมพันธ์

แนวคิดเรื่องความสัมพันธ์มีรากฐานมาจากทฤษฎีเซตที่ Georg Cantor พัฒนาขึ้นในช่วงปลายศตวรรษที่ 19 ในปี ค.ศ. 1908 นักคณิตศาสตร์เบอร์ทรันด์ รัสเซลล์ และ อัลเฟรด นอร์ธ ไวทเฮด ได้เผยแพร่ผลงาน Principia Mathematica ซึ่งรวมแนวคิดเรื่องความสัมพันธ์เข้าไว้ในระบบตรรกศาสตร์อย่างเป็นทางการ พวกเขาได้นิยามความสัมพันธ์ว่าเป็นเซตของคู่อันดับและพัฒนาพีชคณิตของความสัมพันธ์ที่มีความสำคัญอย่างยิ่ง

ในสาขาวิทยาการคอมพิวเตอร์ เอ็ดการ์ คอดด์ ได้พัฒนา Relational Model สำหรับฐานข้อมูลในปี ค.ศ. 1970 ซึ่งเป็นรากฐานของระบบฐานข้อมูลสมัยใหม่ทุกระบบ

หมายเหตุ 3.1 ความสัมพันธ์แตกต่างจากฟังก์ชันตรงที่ความสัมพันธ์อนุญาตให้มีความสัมพันธ์แบบหลายต่อหนึ่งและแบบหลายต่อหลายได้ ในขณะที่ฟังก์ชันกำหนดให้แต่ละค่าในโดเมนส่งไปยังค่าเดียวในเรนจ์เท่านั้น ทุกฟังก์ชันเป็นความสัมพันธ์ชนิดหนึ่ง แต่ไม่ใช่ทุกความสัมพันธ์จะเป็นฟังก์ชัน

นิยามพื้นฐานของความสัมพันธ์

ในส่วนนี้เราจะศึกษานิยามทางคณิตศาสตร์ของความสัมพันธ์ ซึ่งเป็นรากฐานสำคัญสำหรับการเข้าใจเนื้อหาทั้งหมดในบทนี้ ความสัมพันธ์ในทางคณิตศาสตร์ถูกนิยามอย่างเป็นทางการโดยอาศัยแนวคิดของผลคูณคาร์ทีเซียน และความสัมพันธ์ทุกชนิดล้วนเป็นเซตทั้งสิ้น ดังนั้นการดำเนินการบนเซตทุกอย่างจึงสามารถนำมาประยุกต์ใช้กับความสัมพันธ์ได้

3.3.1 นิยามของความสัมพันธ์

ก่อนที่จะเข้าสู่นิยามของความสัมพันธ์ ผู้อ่านควรเข้าใจแนวคิดของผลคูณคาร์ทีเซียนเสียก่อน ผลคูณคาร์ทีเซียนของเซต A และ B เขียนแทนด้วย $A \times B$ หมายถึงเซตของคู่อันดับทั้งหมด (a, b) โดยที่ $a \in A$ และ $b \in B$ ความสัมพันธ์คือเซตย่อยของผลคูณคาร์ทีเซียนนี้ ทำให้สามารถเลือกได้ว่าจะระบุความสัมพันธ์ใดบ้างที่ต้องการ

ทฤษฎีบท 3.2 ความสัมพันธ์ (Relation) จากเซต A ไปยังเซต B คือเซตย่อยของผลคูณคาร์ทีเซียน $A \times B$ กล่าวคือ $R \subseteq A \times B$ เรียก R ว่าเป็นความสัมพันธ์จาก A ไป B

ถ้า $(a, b) \in R$ เราก็กเขียน $a R b$ และอ่านว่า “ a มีความสัมพันธ์กับ b โดย R ” หรือ “ a สัมพันธ์กับ b ”

ถ้า $(a, b) \notin R$ เราก็กเขียน $a \not R b$ และอ่านว่า “ a ไม่มีความสัมพันธ์กับ b โดย R ”

จากนิยาม ความสัมพันธ์คือเซต ดังนั้นเราสามารถใช้การดำเนินการของเซต ได้แก่ ยูเนียน อินเตอร์เซกชัน และคอมพลีเมนต์ กับความสัมพันธ์ได้อย่างเต็มที่ การที่ความสัมพันธ์เป็นเซตทำให้เราสามารถนำทฤษฎีเซตทั้งหมดมาประยุกต์ใช้ได้

ตัวอย่าง 3.3 กำหนดเซต $A = \{1, 2, 3\}$ และ $B = \{a, b\}$ ผลคูณคาร์ทีเซียน $A \times B$ ประกอบด้วยคู่อันดับทั้งหมดหกคู่ ได้แก่ $(1, a), (1, b), (2, a), (2, b), (3, a), (3, b)$ ความสัมพันธ์ $R_1 = \{(1, a), (2, b), (3, a)\}$ จาก A ไป B หมายความว่า 1 สัมพันธ์กับ a โดยเลือกเฉพาะคู่ $(1, a)$ มาจากผลคูณคาร์ทีเซียน 2 สัมพันธ์กับ b และ 3 สัมพันธ์กับ a ตามลำดับ ความสัมพันธ์ $R_2 = \{(1, a), (1, b), (2, a)\}$ จาก A ไป B หมายความว่า 1 สัมพันธ์กับ a และ b พร้อมกัน แต่ 2 สัมพันธ์กับ a เท่านั้น และ 3 ไม่สัมพันธ์กับทั้ง a และ b เลย ตัวอย่าง

นี้แสดงให้เห็นว่าความสัมพันธ์สามารถเลือกคู่อันดับย่อยๆ จากผลคูณคาร์ทีเซียนได้ตามต้องการ

ในการตรวจสอบว่าความสัมพันธ์ที่กำหนดให้ถูกต้องหรือไม่ สิ่งสำคัญคือต้องตรวจสอบว่าทุกคู่อันดับในความสัมพันธ์เป็นสมาชิกของผลคูณคาร์ทีเซียน กล่าวคือ สมาชิกตัวแรกของแต่ละคู่อันดับต้องอยู่ในเซต A และสมาชิกตัวที่สองต้องอยู่ในเซต B

3.3.2 ความสัมพันธ์แบบทวิภาค

ในหลายกรณีทางคณิตศาสตร์ เราต้องการพิจารณาความสัมพันธ์ระหว่างสมาชิกในเซตเดียวกัน ตัวอย่างเช่น ความสัมพันธ์ "มากกว่า" ระหว่างจำนวนเต็ม หรือความสัมพันธ์ "เป็นเพื่อนกับ" ระหว่างผู้คน ในเครือข่ายสังคม กรณีพิเศษนี้เรียกว่าความสัมพันธ์แบบทวิภาค ซึ่งมีความสำคัญและถูกใช้งานมากที่สุดในทางคณิตศาสตร์และวิทยาการคอมพิวเตอร์

ทฤษฎีบท 3.4 ความสัมพันธ์แบบทวิภาค R บนเซต A คือเซตย่อยของ $A \times A$ กล่าวคือ $R \subseteq A \times A$ ในกรณีนี้ เราจะเขียน $a R b$ แทน $(a, b) \in R$ สำหรับ $a, b \in A$

ความสัมพันธ์แบบทวิภาคมีความพิเศษตรงที่ทั้งต้นทางและปลายทางเป็นเซตเดียวกัน ทำให้สามารถวิเคราะห์คุณสมบัติบางอย่างได้ที่ไม่สามารถทำได้กับความสัมพันธ์ทั่วไป คุณสมบัติเหล่านี้รวมถึงความสะท้อน ความสมมาตร และความถ่ายทอด ซึ่งจะกล่าวถึงในรายละเอียดในส่วนถัดไป

ตัวอย่าง 3.5 กำหนดเซต $A = \{1, 2, 3, 4\}$ เราพิจารณาความสัมพันธ์แบบทวิภาคหลายชนิดบนเซตนี้ ความสัมพันธ์ $R_1 = \{(1, 1), (1, 2), (2, 2), (2, 3), (3, 3), (3, 4), (4, 4)\}$ เป็นความสัมพันธ์แบบทวิภาคบน A เพราะทุกคู่อันดับมีสมาชิกจาก A ทั้งคู่ ความสัมพันธ์ $R_2 = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$ เป็นความสัมพันธ์แบบทวิภาคบน A เช่นกัน และมีคุณสมบัติสมมาตร ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ถ้า $(1, 2)$ อยู่ใน R_2 แล้ว $(2, 1)$ ก็อยู่ใน R_2 ด้วย ในทางกลับกัน ความสัมพันธ์ $R_3 = \emptyset$ เป็นความสัมพันธ์ว่างบน A ซึ่งไม่มีคู่อันดับใดๆ เลย และความสัมพันธ์ $R_4 = A \times A$ เป็นความสัมพันธ์สากลบน A ซึ่งมีทุกคู่อันดับที่เป็นไปได้

ความสัมพันธ์ว่าง และ ความสัมพันธ์ สากลเป็นกรณีพิเศษที่สำคัญ ความสัมพันธ์ว่างไม่มีความสัมพันธ์ใดๆ เลย ขณะที่ความสัมพันธ์สากลมีความสัมพันธ์ครบทุกคู่ที่เป็นไปได้ ความสัมพันธ์ทั้งสองนี้มีประโยชน์ในการให้ตัวอย่างและพิสูจน์ทฤษฎีบท

3.3.3 โดเมนและเรนจ์ของความสัมพันธ์

เมื่อเรามีความสัมพันธ์จากเซต A ไปยังเซต B แล้ว สิ่งที่สำคัญคือต้องรู้ว่าสมาชิกใดใน A บ้างที่มีความสัมพันธ์กับสมาชิกใน B และมีสมาชิกใดใน B บ้างที่ถูกสัมพันธ์โดยสมาชิกใน A แนวคิดนี้นำไปสู่นิยาม

ของโดเมนและเรนจ์ ซึ่งเป็นเซตย่อยของ A และ B ตามลำดับ

ทฤษฎีบท 3.6 กำหนดความสัมพันธ์ R จากเซต A ไปยังเซต B โดเมนของ R คือเซตของทุกสมาชิกใน A ที่มีความสัมพันธ์กับบางสมาชิกใน B เขียนแทนด้วย $\text{dom}(R)$ หรือ $\mathcal{D}(R)$ โดย $\text{dom}(R) = \{a \in A \mid \exists b \in B, (a, b) \in R\}$ เรนจ์ของ R คือเซตของทุกสมาชิกใน B ที่ถูกสัมพันธ์โดยบางสมาชิกใน A เขียนแทนด้วย $\text{ran}(R)$ หรือ $\mathcal{R}(R)$ โดย $\text{ran}(R) = \{b \in B \mid \exists a \in A, (a, b) \in R\}$

โดเมนและเรนจ์อาจเป็นเซตย่อยของ A และ B ตามลำดับ หรืออาจเท่ากับ A และ B ก็ได้ ขึ้นอยู่กับว่ามีสมาชิกใดบ้างที่ปรากฏในคู่อันดับของความสัมพันธ์ สมาชิกใน A ที่ไม่ปรากฏในตำแหน่งแรกของคู่อันดับใดๆ จะไม่อยู่ในโดเมน และสมาชิกใน B ที่ไม่ปรากฏในตำแหน่งที่สองจะไม่อยู่ในเรนจ์

ตัวอย่าง 3.7 กำหนดความสัมพันธ์ $R = \{(1, a), (1, b), (2, b), (4, a)\}$ จาก $A = \{1, 2, 3, 4\}$ ไป $B = \{a, b, c\}$ ผู้อ่านจะเห็นว่าคู่อันดับใน R มีสมาชิกตัวแรกคือ 1, 1, 2, 4 และสมาชิกตัวที่สองคือ a, b, b, a ตามลำดับ ดังนั้นโดเมนของ R คือเซต $\{1, 2, 4\}$ เพราะสมาชิก 3 ไม่ปรากฏในตำแหน่งแรกของคู่อันดับใดๆ ใน R เรนจ์ของ R คือเซต $\{a, b\}$ เพราะสมาชิก c ไม่ปรากฏในตำแหน่งที่สองของคู่อันดับใดๆ ใน R ตัวอย่างนี้แสดงให้เห็นว่าโดเมนและเรนจ์อาจเป็นเซตย่อยของเซตเดิมได้

หมายเหตุ 3.8 อย่าสับสนระหว่าง $\text{dom}(R)$ กับเซต A และ $\text{ran}(R)$ กับเซต B โดเมนของความสัมพันธ์อาจเป็นเซตย่อยของ A และเรนจ์อาจเป็นเซตย่อยของ B ก็ได้ ขึ้นอยู่กับว่ามีสมาชิกใดบ้างที่ปรากฏในคู่อันดับของความสัมพันธ์ การตรวจสอบโดเมนและเรนจ์จึงต้องดูจากคู่อันดับจริงๆ ในความสัมพันธ์

คุณสมบัติของความสัมพันธ์แบบทวิภาค

ความสัมพันธ์แบบทวิภาคบนเซต A มีคุณสมบัติพื้นฐานที่สำคัญสี่ประการ ได้แก่ สะท้อน สมมาตร เป็นผลาด และถ่ายทอด คุณสมบัติเหล่านี้ช่วยให้เราจำแนกและวิเคราะห์ความสัมพันธ์ได้อย่างเป็นระบบ การเข้าใจคุณสมบัติทั้งสี่อย่างลึกซึ้งเป็นพื้นฐานสำคัญสำหรับการศึกษาความสัมพันธ์สมมูลและความสัมพันธ์สมมาตรเชิงส่วนย่อยในส่วนถัดไป

คุณสมบัติเหล่านี้มีความหมายทางปฏิบัติมากมาย ความสะท้อนหมายความว่าทุกสมาชิกมีความสัมพันธ์กับตัวมันเอง ความสมมาตรหมายความว่าความสัมพันธ์เป็นแบบสองทาง ความเป็นผลาดหมายความว่าถ้าทั้งสองทางสัมพันธ์กันแล้วต้องเป็นสมาชิกเดียวกัน และความถ่ายทอดหมายความว่าความสัมพันธ์สามารถต่อกันได้

3.4.1 คุณสมบัติสะท้อน (Reflexive Property)

ความสะท้อนเป็นคุณสมบัติที่สำคัญที่สุดอย่างหนึ่งของความสัมพันธ์ ความหมายโดยธรรมชาติของความสะท้อนคือ ทุกสมาชิกในเซตมีความสัมพันธ์กับตัวมันเอง ตัวอย่างเช่น ความสัมพันธ์ "มีอายุเท่ากับ" เป็นความสะท้อนเพราะคนทุกคนมีอายุเท่ากับตัวมันเอง ในทางคณิตศาสตร์ ความสัมพันธ์ "เท่ากับ" และ "มากกว่าหรือเท่ากับ" ล้วนเป็นความสะท้อน

ทฤษฎีบท 3.9 ความสัมพันธ์ R บนเซต A กล่าวว่า เป็นความสัมพันธ์สะท้อน ก็ต่อเมื่อสำหรับทุกสมาชิก $a \in A$ จะได้ว่า $(a, a) \in R$ หรือเขียนแทนด้วย $\forall a \in A, a R a$

นั่นหมายความว่า ทุกสมาชิกในเซต A ต้องมีความสัมพันธ์กับตัวมันเองโดย R การพิสูจน์ว่าความสัมพันธ์เป็นความสะท้อนต้องตรวจสอบทุกสมาชิกในเซต แต่การพิสูจน์ว่าความสัมพันธ์ไม่เป็นความสะท้อนเพียงแค่หาสมาชิกตัวเดียวที่ไม่มีคู่ (a, a) ในความสัมพันธ์ก็เพียงพอ

ตัวอย่าง 3.10 พิจารณาความสัมพันธ์ต่อไปนี้บนเซต $A = \{1, 2, 3\}$ เพื่อตรวจสอบคุณสมบัติสะท้อน สำหรับความสัมพันธ์ $R_1 = \{(1, 1), (2, 2), (3, 3)\}$ เราตรวจพบว่าทุกสมาชิกใน A มีคู่อันดับที่สัมพันธ์กับตัวมันเอง กล่าวคือ $(1, 1), (2, 2), (3, 3)$ ล้วนอยู่ใน R_1 ดังนั้น R_1 เป็นความสัมพันธ์สะท้อน สำหรับความสัมพันธ์ $R_2 = \{(1, 1), (2, 2)\}$ เราตรวจพบว่าสมาชิก 3 ไม่มีคู่ $(3, 3)$ ใน R_2 ดังนั้น R_2 ไม่เป็นความสัมพันธ์สะท้อน เราเพียงแค่หาตัวอย่างตรงข้ามเพียงตัวเดียวก็เพียงพอที่จะพิสูจน์ว่าความสัมพันธ์ไม่เป็นความสะท้อน สำหรับความสัมพันธ์ $R_3 = \{(1, 1), (1, 2), (2, 2), (3, 3)\}$ ทุกสมาชิกมีคู่กับตัวมันเอง จึงเป็นความสัมพันธ์สะท้อน สำหรับความสัมพันธ์ $R_4 = \{(1, 2), (2, 1), (2, 3)\}$ เราตรวจพบว่าไม่มี $(1, 1)$ และไม่มี $(3, 3)$ ใน R_4 ดังนั้น R_4 ไม่เป็นความสัมพันธ์สะท้อน

ตัวอย่าง 3.11 ในความสัมพันธ์ในชีวิตจริง ความสัมพันธ์ "มีอายุเท่ากับ" บนเซตของมนุษย์เป็นความสัมพันธ์สะท้อน เพราะคนทุกคนมีอายุเท่ากับตัวมันเองโดยประยุกต์ใช้ ความสัมพันธ์ "เป็นลูกของ" บนเซตของมนุษย์ไม่เป็นความสัมพันธ์สะท้อน เพราะคนไม่เป็นลูกของตัวเอง ความสัมพันธ์ " $\leq$ " บนเซต $\mathbb{Z}$ เป็นความสัมพันธ์สะท้อน เพราะ $a \leq a$ สำหรับทุกจำนวนเต็ม a ความสัมพันธ์ " $<$ " บน $\mathbb{Z}$ ไม่เป็นความสะท้อน เพราะ $a < a$ ไม่เป็นจริงสำหรับจำนวนเต็มใดๆ

3.4.2 คุณสมบัติสมมาตร (Symmetric Property)

ความสมมาตรเป็นคุณสมบัติที่มีความหมายว่า ถ้า a สัมพันธ์กับ b แล้ว b ก็ต้องสัมพันธ์กับ a ด้วย ตัวอย่างเช่น ความสัมพันธ์ "เป็นเพื่อนกับ" เป็นความสมมาตร เพราะถ้า a เป็นเพื่อนกับ b แล้ว b ก็ต้องเป็น

เพื่อนกับ a ในทางกลับกัน ความสัมพันธ์ "เป็นลูกของ" ไม่เป็นความสัมพันธ์สมมาตร เพราะถ้า a เป็นลูกของ b แล้ว b ไม่จำเป็นต้องเป็นลูกของ a

ทฤษฎีบท 3.12 ความสัมพันธ์ R บนเซต A กล่าวว่าเป็นความสัมพันธ์สมมาตร ก็ต่อเมื่อทุกครั้งที่ $a R b$ แล้ว $b R a$ ด้วย กล่าวคือ $\forall a, b \in A, (a R b \rightarrow b R a)$

นั่นหมายความว่า ถ้า a สัมพันธ์กับ b แล้ว b ก็ต้องสัมพันธ์กับ a ด้วย การพิสูจน์ว่าความสัมพันธ์เป็นความสัมพันธ์สมมาตรต้องตรวจสอบทุกคู่ (a, b) ที่อยู่ในความสัมพันธ์ว่ามี (b, a) อยู่ด้วยหรือไม่ ส่วนการพิสูจน์ว่าไม่เป็นความสัมพันธ์สมมาตรเพียงแค่ว่าคู่ (a, b) ที่อยู่ในความสัมพันธ์แต่ (b, a) ไม่อยู่

ตัวอย่าง 3.13 พิจารณาความสัมพันธ์ต่อไปนี้บนเซต $A = \{1, 2, 3\}$ เพื่อตรวจสอบคุณสมบัติสมมาตร สำหรับความสัมพันธ์ $R_1 = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$ เราตรวจสอบพบว่าถ้า $(1, 2) \in R_1$ แล้ว $(2, 1) \in R_1$ ด้วย และถ้า $(2, 3) \in R_1$ แล้ว $(3, 2) \in R_1$ ด้วย ดังนั้น R_1 เป็นความสัมพันธ์สมมาตร สำหรับความสัมพันธ์ $R_2 = \{(1, 1), (1, 2), (2, 1)\}$ เราตรวจสอบพบว่า $(1, 2) \in R_2$ และ $(2, 1) \in R_2$ ด้วย ดังนั้น R_2 เป็นความสัมพันธ์สมมาตร สำหรับความสัมพันธ์ $R_3 = \{(1, 2), (2, 3)\}$ เราตรวจสอบพบว่า $(1, 2) \in R_3$ แต่ $(2, 1) \notin R_3$ ดังนั้น R_3 ไม่เป็นความสัมพันธ์สมมาตร นี่คือนิยามตรงข้ามที่ชัดเจน สำหรับความสัมพันธ์ $R_4 = \{(1, 1), (2, 2), (3, 3)\}$ ทุกคู่ใน R_4 เป็นคู่ของสมาชิกเดียวกัน ดังนั้น R_4 เป็นความสัมพันธ์สมมาตร

ตัวอย่าง 3.14 ในความสัมพันธ์ในชีวิตจริง ความสัมพันธ์ "มีอายุเท่ากับ" เป็นความสัมพันธ์สมมาตร เพราะถ้า a มีอายุเท่ากับ b แล้ว b ก็มีอายุเท่ากับ a ด้วย ความสัมพันธ์ "เป็นเพื่อนกับ" เป็นความสัมพันธ์สมมาตร เพราะถ้า a เป็นเพื่อนกับ b แล้ว b ก็เป็นเพื่อนกับ a ด้วย ความสัมพันธ์ "เป็นลูกของ" ไม่เป็นความสัมพันธ์สมมาตร เพราะถ้า a เป็นลูกของ b แล้ว b ไม่จำเป็นต้องเป็นลูกของ a ในทางกลับกัน b เป็นบิดาหรือมารดาของ a ความสัมพันธ์ " $\leq$ " บน $\mathbb{Z}$ ไม่เป็นความสัมพันธ์สมมาตร เพราะ $1 \leq 2$ เป็นจริง แต่ $2 \not\leq 1$ เป็นเท็จ

3.4.3 คุณสมบัติเป็นผลาด (Antisymmetric Property)

ความเป็นผลาดเป็นคุณสมบัติที่ตรงข้ามกับความสมมาตรในแง่หนึ่ง แต่ไม่ใช่คุณสมบัติตรงข้ามกัน อย่างสมบูรณ์ ความหมายของความเป็นผลาดคือ ถ้า a สัมพันธ์กับ b และ b สัมพันธ์กับ a แล้ว a และ b ต้องเป็นสมาชิกตัวเดียวกัน ความสัมพันธ์ " $\leq$ " บนจำนวนเป็นตัวอย่างที่ดีของความเป็นผลาด เพราะถ้า $a \leq b$ และ $b \leq a$ แล้ว $a = b$

ทฤษฎีบท 3.15 ความสัมพันธ์ R บนเซต A กล่าวว่า เป็นความสัมพันธ์เป็นผลัด ก็ต่อเมื่อทุกครั้งที่ $a R b$ และ $b R a$ แล้ว $a = b$ กล่าวคือ $\forall a, b \in A, (a R b \wedge b R a) \rightarrow a = b$ นั้นหมายความว่า ถ้ามีทั้ง (a, b) และ (b, a) ในความสัมพันธ์ แล้ว a กับ b ต้องเป็นตัวย่อยกัน ไม่ใช่คนละตัว

ตัวอย่าง 3.16 พิจารณาความสัมพันธ์ต่อไปนี้บนเซต $A = \{1, 2, 3\}$ เพื่อตรวจสอบคุณสมบัติเป็นผลัด สำหรับความสัมพันธ์ $R_1 = \{(1, 1), (2, 2), (3, 3)\}$ เงื่อนไข $a R b \wedge b R a$ เกิดขึ้นเฉพาะเมื่อ $a = b$ เท่านั้น กล่าวคือ เมื่อ $a = b = 1$ จะได้ $(1, 1)$ และเมื่อ $a = b = 2$ จะได้ $(2, 2)$ ในทุกกรณีที่เงื่อนไขเป็นจริง $a = b$ เป็นจริงเสมอ ดังนั้น R_1 เป็นความสัมพันธ์เป็นผลัด สำหรับความสัมพันธ์ $R_2 = \{(1, 1), (1, 2), (2, 2)\}$ เงื่อนไข $a R b \wedge b R a$ เกิดขึ้นเฉพาะเมื่อ $a = b = 1$ หรือ $a = b = 2$ เท่านั้น ดังนั้น R_2 เป็นความสัมพันธ์เป็นผลัด สำหรับความสัมพันธ์ $R_3 = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$ เราตรวจสอบพบว่า $(1, 2) \in R_3$ และ $(2, 1) \in R_3$ แต่ $1 \neq 2$ ดังนั้นเงื่อนไข $(a R b \wedge b R a) \rightarrow a = b$ เป็นเท็จ เพราะมีตัวอย่างที่ $a = 1, b = 2$ ทำให้เงื่อนไขเป็นจริงแต่ผลสรุป $a = b$ เป็นเท็จ ดังนั้น R_3 ไม่เป็นความสัมพันธ์เป็นผลัด

ตัวอย่าง 3.17 ในความสัมพันธ์ในชีวิตจริง ความสัมพันธ์ " $\leq$ " บน $\mathbb{Z}$ เป็นความสัมพันธ์เป็นผลัด เพราะถ้า $a \leq b$ และ $b \leq a$ แล้ว $a = b$ ความสัมพันธ์ " $\subseteq$ " บนเซตของเซตเป็นความสัมพันธ์เป็นผลัด เพราะถ้า $A \subseteq B$ และ $B \subseteq A$ แล้ว $A = B$ ความสัมพันธ์ " $<$ " บน $\mathbb{Z}$ เป็นความสัมพันธ์เป็นผลัดด้วย เพราะเงื่อนไข $a < b \wedge b < a$ ไม่เคยเกิดขึ้นสำหรับ $a \neq b$ ใดๆ ความสัมพันธ์ "เป็นเพื่อนกับ" ไม่เป็นความสัมพันธ์เป็นผลัด เพราะถ้า a เป็นเพื่อนกับ b และ b เป็นเพื่อนกับ a แล้ว a กับ b อาจเป็นคนละคนกันได้

หมายเหตุ 3.18 **Remark 3.2:** ความสัมพันธ์สมมาตรและความสัมพันธ์เป็นผลัดไม่ใช่คุณสมบัติตรงข้ามกัน ความสัมพันธ์บางตัวอาจมีทั้งสองคุณสมบัติพร้อมกัน เช่น $R = \{(1, 1), (2, 2)\}$ เป็นทั้งสมมาตรและเป็นผลัด ในทางกลับกัน ความสัมพันธ์บางตัวไม่มีทั้งสองคุณสมบัติ เช่น $R = \{(1, 2), (2, 3), (1, 3)\}$

3.4.4 คุณสมบัติถ่ายทอด (Transitive Property)

ความถ่ายทอดเป็นคุณสมบัติที่มีความหมายว่า ความสัมพันธ์สามารถ "ต่อกัน" ได้ กล่าวคือ ถ้า a สัมพันธ์กับ b และ b สัมพันธ์กับ c แล้ว a ก็ต้องสัมพันธ์กับ c ด้วย ตัวอย่างเช่น ความสัมพันธ์ "มากกว่า" เป็นความถ่ายทอด เพราะถ้า $a > b$ และ $b > c$ แล้ว $a > c$ ในทางกลับกัน ความสัมพันธ์ "เป็นเพื่อนกับ" ไม่เป็นความถ่ายทอด เพราะถ้า a เป็นเพื่อนกับ b และ b เป็นเพื่อนกับ c แล้ว a กับ c อาจไม่เป็นเพื่อนกัน

ทฤษฎีบท 3.19 ความสัมพันธ์ R บนเซต A กล่าวว่าเป็นความสัมพันธ์ถ่ายทอด ก็ต่อเมื่อทุกครั้งที่ $a R b$ และ $b R c$ แล้ว $a R c$ ด้วย กล่าวคือ $\forall a, b, c \in A, (a R b \wedge b R c) \rightarrow a R c$

นั่นหมายความว่า ความสัมพันธ์สามารถ "ถ่ายทอด" จาก a ไป c ผ่าน b ได้ การตรวจสอบความถ่ายทอดต้องพิจารณาทุกสามสมาชิก a, b, c ที่ทำให้ $(a, b) \in R$ และ $(b, c) \in R$ แล้วตรวจสอบว่า $(a, c) \in R$ หรือไม่

ตัวอย่าง 3.20 พิจารณาความสัมพันธ์ต่อไปนี้บนเซต $A = \{1, 2, 3, 4\}$ เพื่อตรวจสอบคุณสมบัติถ่ายทอดสำหรับความสัมพันธ์ $R_1 = \{(1, 2), (2, 3), (1, 3)\}$ เราตรวจสอบพบว่าถ้า $1 R_1 2$ และ $2 R_1 3$ แล้ว $1 R_1 3$ ก็อยู่ใน R_1 ด้วย ไม่มีสามสมาชิกอื่นที่ต้องตรวจสอบ ดังนั้น R_1 เป็นความสัมพันธ์ถ่ายทอด สำหรับความสัมพันธ์ $R_2 = \{(1, 2), (2, 3)\}$ เราตรวจสอบพบว่า $1 R_2 2$ และ $2 R_2 3$ แต่ $1 R_2 3$ ไม่อยู่ใน R_2 ดังนั้น R_2 ไม่เป็นความสัมพันธ์ถ่ายทอด นี่คือตัวอย่างตรงข้ามที่ชัดเจน สำหรับความสัมพันธ์ $R_3 = \{(1, 2), (2, 1)\}$ เราตรวจสอบพบว่า $1 R_3 2$ และ $2 R_3 1$ แต่ไม่มี $(1, 1)$ ใน R_3 ดังนั้น R_3 ไม่เป็นความสัมพันธ์ถ่ายทอด เราต้องตรวจสอบทุกคู่ (a, b) และ (b, c) ที่อยู่ใน R_3 ว่ามี (a, c) ใน R_3 หรือไม่

ตัวอย่าง 3.21 ในความสัมพันธ์ในชีวิตจริง ความสัมพันธ์ " $\leq$ " บน $\mathbb{Z}$ เป็นความสัมพันธ์ถ่ายทอด เพราะถ้า $a \leq b$ และ $b \leq c$ แล้ว $a \leq c$ ความสัมพันธ์ "เป็นบิดาของ" ไม่เป็นความสัมพันธ์ถ่ายทอดในความหมายของ "บิดาของ" เพราะถ้า a เป็นบิดาของ b และ b เป็นบิดาของ c แล้ว a เป็นปู่ของ c ไม่ใช่บิดาของ c ความสัมพันธ์ "มีอายุนานกว่า" บน $\mathbb{Z}$ เป็นความสัมพันธ์ถ่ายทอด เพราะถ้า $a > b$ และ $b > c$ แล้ว $a > c$ ความสัมพันธ์ "เป็นเพื่อนกับ" ไม่เป็นความสัมพันธ์ถ่ายทอด เพราะถ้า a เป็นเพื่อนกับ b และ b เป็นเพื่อนกับ c แล้ว a กับ c อาจไม่เป็นเพื่อนกัน

3.4.5 ตารางสรุปคุณสมบัติของความสัมพันธ์บางชนิด

ในส่วนนี้เราจะสรุปคุณสมบัติของความสัมพันธ์ที่พบบ่อยในคณิตศาสตร์และชีวิตจริง การเข้าใจตารางนี้จะช่วยให้ผู้อ่านจดจำคุณสมบัติของความสัมพันธ์แต่ละชนิดได้ดีขึ้น

ตารางที่ 11 สรุปคุณสมบัติของความสัมพันธ์บางชนิด

ความสัมพันธ์	สะท้อน	สมมาตร	เป็นผลาด	ถ่ายทอด
"=" บนเซตใดๆ	✓	✓	✓	✓
"≤" บน $\mathbb{Z}$	✓	□	✓	✓
"<" บน $\mathbb{Z}$	□	□	✓	✓
"⊆" บนเซตของเซต	✓	□	✓	✓
"เป็นเพื่อนกับ"	□	✓	□	□
"เป็นลูกของ"	□	□	✓	□
ความสัมพันธ์ว่าง $\emptyset$	□	✓	✓	✓
ความสัมพันธ์สากล $A \times A$	✓	✓	□	✓

ตารางข้างต้นแสดงให้เห็นว่าความสัมพันธ์แต่ละชนิดมีคุณสมบัติไม่เหมือนกัน ความสัมพันธ์ "=" มีทั้งสี่คุณสมบัติครบถ้วน ความสัมพันธ์ "≤" และ "⊆" มีสามคุณสมบัติยกเว้นความสมมาตร ความสัมพันธ์ว่างมีคุณสมบัติสมมาตร เป็นผลาด และถ่ายทอด แต่ไม่มีคุณสมบัติสะท้อน

ตัวอย่าง 3.22 จงพิจารณาคุณสมบัติของ ความสัมพันธ์ต่อไปนี้บนเซตของจำนวนเต็ม $\mathbb{Z}$ สำหรับ ความสัมพันธ์ $R_1 = \{(a, b) \mid a = b\}$ ซึ่งก็คือความสัมพันธ์ "เท่ากับ" เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่า สำหรับทุก $a \in \mathbb{Z}$ จะได้ $a = a$ ดังนั้น $(a, a) \in R_1$ ดังนั้น R_1 เป็นความสัมพันธ์สะท้อน เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าถ้า $a = b$ แล้ว $b = a$ ดังนั้นถ้า $(a, b) \in R_1$ แล้ว $(b, a) \in R_1$ ดังนั้น R_1 เป็นความสัมพันธ์สมมาตร เราตรวจสอบคุณสมบัติเป็นผลาดโดยพบว่าถ้า $a = b$ และ $b = a$ แล้ว $a = b$ ดังนั้นถ้า $(a, b) \in R_1$ และ $(b, a) \in R_1$ แล้ว $a = b$ ดังนั้น R_1 เป็นความสัมพันธ์เป็นผลาด เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $a = b$ และ $b = c$ แล้ว $a = c$ ดังนั้นถ้า $(a, b) \in R_1$ และ $(b, c) \in R_1$ แล้ว $(a, c) \in R_1$ ดังนั้น R_1 เป็นความสัมพันธ์ถ่ายทอด ดังนั้น R_1 เป็นความสัมพันธ์สมมูล

สำหรับความสัมพันธ์ $R_2 = \{(a, b) \mid a \leq b\}$ ซึ่งก็คือความสัมพันธ์ "น้อยกว่าหรือเท่ากับ" เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าสำหรับทุก $a \in \mathbb{Z}$ จะได้ $a \leq a$ ดังนั้น $(a, a) \in R_2$ ดังนั้น R_2 เป็นความสัมพันธ์สะท้อน เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าพิจารณา $a = 1, b = 2$ จะได้ $1 \leq 2$ แต่ $2 \not\leq 1$ ดังนั้น $(1, 2) \in R_2$ แต่ $(2, 1) \notin R_2$ ดังนั้น R_2 ไม่เป็นความสัมพันธ์สมมาตร เราตรวจสอบคุณสมบัติเป็นผลาดโดยพบว่าถ้า $a \leq b$ และ $b \leq a$ แล้ว $a = b$ ดังนั้นถ้า $(a, b) \in R_2$ และ $(b, a) \in R_2$ แล้ว $a = b$

ดังนั้น R_2 เป็นความสัมพันธ์เป็นผลัด เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $a \leq b$ และ $b \leq c$ แล้ว $a \leq c$ ดังนั้นถ้า $(a, b) \in R_2$ และ $(b, c) \in R_2$ แล้ว $(a, c) \in R_2$ ดังนั้น R_2 เป็นความสัมพันธ์ถ่ายทอด ดังนั้น R_2 เป็นความสัมพันธ์สมมาตรเชิงส่วนย่อย

ความสัมพันธ์สมมูล (Equivalence Relations)

ความสัมพันธ์สมมูลเป็นความสัมพันธ์ที่มีความสำคัญอย่างยิ่งในคณิตศาสตร์ เพราะมันจำแนกสมาชิกของเซตออกเป็นกลุ่มๆ ที่มีคุณสมบัติเหมือนกัน แนวคิดนี้ปรากฏมาแล้วซ้ำแล้วซ้ำเล่าในหลากหลายสาขาของคณิตศาสตร์ ตั้งแต่ทฤษฎีจำนวนไปจนถึงพีชคณิต □□ ความสัมพันธ์สมมูลมีการใช้งานมากมาย ได้แก่ การจัดกลุ่มข้อมูล การจัดระเบียบเซต และการนิยามโครงสร้างใหม่บนเซต

ความสัมพันธ์สมมูลมีชื่อเรียกนี้เพราะมันกำหนดว่าสมาชิกใดในเซต "สมมูล" กันในแง่ของคุณสมบัติบางอย่าง สมาชิกสองตัวที่อยู่ในความสัมพันธ์สมมูลกันมีคุณสมบัติเหมือนกันในบริบทที่พิจารณา ทำให้สามารถจัดกลุ่มได้

3.5.1 นิยามของความสัมพันธ์สมมูล

การเข้าใจนิยามของความสัมพันธ์สมมูลต้องอาศัยความเข้าใจคุณสมบัติทั้งสามประการของความสัมพันธ์ คือ ความสะท้อน ความสมมาตร และความถ่ายทอด ความสัมพันธ์สมมูลคือความสัมพันธ์ที่มีทั้งสามคุณสมบัตินี้พร้อมกัน การที่ความสัมพันธ์ต้องมีทั้งสามคุณสมบัติทำให้มันสามารถจำแนกสมาชิกของเซตออกเป็นกลุ่มที่ไม่ซ้ำกันได้อย่างชัดเจน

ทฤษฎีบท 3.23 ความสัมพันธ์ R บนเซต A กล่าวว่าเป็นความสัมพันธ์สมมูล ก็ต่อเมื่อ R มีคุณสมบัติทั้งสามประการต่อไปนี้พร้อมกัน ประการที่หนึ่ง ความสะท้อน กล่าวคือ $\forall a \in A, a R a$ ประการที่สอง ความสมมาตร กล่าวคือ $\forall a, b \in A, (a R b \rightarrow b R a)$ และประการที่สาม ความถ่ายทอด กล่าวคือ $\forall a, b, c \in A, ((a R b \wedge b R c) \rightarrow a R c)$ เมื่อความสัมพันธ์มีคุณสมบัติทั้งสามนี้พร้อมกัน จะเกิดผลที่สำคัญคือ ความสัมพันธ์จะแบ่งเซต A ออกเป็นส่วนย่อยที่ไม่ทับซ้อนกัน เรียกว่าคลาสสมมูล โดยสมาชิกสองตัวใดๆ ที่อยู่ในคลาสสมมูลเดียวกันจะมีความสัมพันธ์กัน และสมาชิกที่อยู่คนละคลาสสมมูลจะไม่มีความสัมพันธ์กัน

ตัวอย่าง 3.24 พิจารณาความสัมพันธ์ต่อไปนี้บนเซต $A = \{1, 2, 3, 4, 5, 6\}$ สำหรับความสัมพันธ์ $R_1 = \{(1, 1), (2, 2), (3, 3), (4, 4), (5, 5), (6, 6)\}$ เราตรวจสอบพบว่ามีความสัมพันธ์สะท้อน สมมาตร และถ่ายทอด

ครบถ้วน ดังนั้น R_1 เป็นความสัมพันธ์สมมูล นี่คือความสัมพันธ์เอกลักษณ์ I_A ซึ่งเป็นความสัมพันธ์สมมูลที่เล็กที่สุด สำหรับความสัมพันธ์ $R_2 = A \times A$ เราตรวจสอบพบว่ามีความสัมพันธ์ทุกคู่อันดับที่เป็นไปได้ จึงมีคุณสมบัติทั้งสามครบถ้วน ดังนั้น R_2 เป็นความสัมพันธ์สมมูล นี่คือความสัมพันธ์สากล ซึ่งเป็นความสัมพันธ์สมมูลที่ใหญ่ที่สุด ตัวอย่างเหล่านี้แสดงให้เห็นว่ามีความสัมพันธ์สมมูลหลายชนิดที่เป็นไปได้บนเซตเดียวกัน

ตัวอย่าง 3.25 พิจารณาความสัมพันธ์ R บนเซตของจำนวนเต็ม $\mathbb{Z}$ โดย aRb ก็ต่อเมื่อ $a \equiv b \pmod{n}$ สำหรับ n ที่กำหนดให้ กล่าวคือ a และ b ให้เศษเหลือเท่ากันเมื่อหารด้วย n เราตรวจสอบคุณสมบัติสะท้อน โดยพบว่าสำหรับทุก $a \in \mathbb{Z}$ จะได้ $a \equiv a \pmod{n}$ เพราะ $a - a = 0$ หารด้วย n ลงตัว ดังนั้น $(a, a) \in R$ เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าถ้า $a \equiv b \pmod{n}$ แล้ว $a - b$ หารด้วย n ลงตัว แสดงว่า $b - a = -(a - b)$ ก็หารด้วย n ลงตัว ดังนั้น $b \equiv a \pmod{n}$ ดังนั้นถ้า $(a, b) \in R$ แล้ว $(b, a) \in R$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $a \equiv b \pmod{n}$ และ $b \equiv c \pmod{n}$ แล้ว $a - b$ และ $b - c$ หารด้วย n ลงตัว แสดงว่า $a - c = (a - b) + (b - c)$ ก็หารด้วย n ลงตัว ดังนั้น $a \equiv c \pmod{n}$ ดังนั้นถ้า $(a, b) \in R$ และ $(b, c) \in R$ แล้ว $(a, c) \in R$ ดังนั้น R เป็นความสัมพันธ์สมมูลบน $\mathbb{Z}$ ความสัมพันธ์นี้เรียกว่า "ความสัมพันธ์สมมูลแบบมอดุโล n " และเป็นตัวอย่างที่สำคัญมากของความสัมพันธ์สมมูลในทฤษฎีจำนวน

ตัวอย่าง 3.26 พิจารณาความสัมพันธ์ R บนเซตของเส้นตรงในระนาบ โดย $l_1 R l_2$ ก็ต่อเมื่อ l_1 ขนานกับ l_2 รวมถึงกรณีที่ $l_1 = l_2$ เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าทุกเส้นตรงขนานกับตัวมันเอง ดังนั้น $l R l$ เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าถ้า l_1 ขนานกับ l_2 แล้ว l_2 ก็ขนานกับ l_1 ดังนั้นถ้า $(l_1, l_2) \in R$ แล้ว $(l_2, l_1) \in R$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า l_1 ขนานกับ l_2 และ l_2 ขนานกับ l_3 แล้ว l_1 ก็ขนานกับ l_3 ดังนั้นถ้า $(l_1, l_2) \in R$ และ $(l_2, l_3) \in R$ แล้ว $(l_1, l_3) \in R$ ดังนั้น R เป็นความสัมพันธ์สมมูลบนเซตของเส้นตรง ความสัมพันธ์นี้จำแนกเส้นตรงออกเป็นกลุ่มๆ ตามความชันของเส้นตรง

ตัวอย่าง 3.27 พิจารณาความสัมพันธ์ R บนเซตของมนุษย์ทุกคน โดย aRb ก็ต่อเมื่อ a และ b เกิดในปีเดียวกัน เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าทุกคนเกิดในปีเดียวกับตัวมันเอง ดังนั้น $a R a$ เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าถ้า a และ b เกิดในปีเดียวกัน แล้ว b และ a ก็เกิดในปีเดียวกัน ดังนั้นถ้า $(a, b) \in R$ แล้ว $(b, a) \in R$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า a และ b เกิดในปีเดียวกัน และ b และ c เกิดในปีเดียวกัน แล้ว a และ c ก็เกิดในปีเดียวกัน เพราะทั้งคู่เกิดในปีเดียวกับ b ดังนั้นถ้า $(a, b) \in R$ และ $(b, c) \in R$ แล้ว $(a, c) \in R$ ดังนั้น R เป็นความสัมพันธ์สมมูล ความสัมพันธ์นี้จำแนกมนุษย์ออกเป็นกลุ่มๆ ตามปีเกิด แต่ละคลาสสมมูลคือเซตของคนที่เกิดในปีเดียวกัน

3.5.2 คลาสสมมูล (Equivalence Classes)

เมื่อเรามีความสัมพันธ์สมมูลบนเซต A แล้ว สมาชิกแต่ละตัวใน A จะอยู่ในกลุ่มที่เรียกว่าคลาสสมมูล คลาสสมมูลของ a คือเซตของทุกสมาชิกใน A ที่สมมูลกับ a โดยความสัมพันธ์นั้น สมาชิกทุกตัวในคลาสสมมูลเดียวกันมีความสัมพันธ์กัน และสมาชิกในคลาสสมมูลต่างกันไม่มีความสัมพันธ์กัน

ทฤษฎีบท 3.28 กำหนดให้ R เป็นความสัมพันธ์สมมูลบนเซต A สำหรับแต่ละสมาชิก $a \in A$ เรานิยามคลาสสมมูลของ a โดย R ว่าเป็นเซตของทุกสมาชิกใน A ที่สมมูลกับ a โดย R เขียนแทนด้วย $[a]_R$ หรือ $[a]$ กล่าวคือ $[a]_R = \{x \in A \mid a R x\}$ สัญลักษณ์ $[a]_R$ อ่านว่า "คลาสสมมูลของ a โดย R " และ a เรียกว่าตัวแทนของคลาสสมมูล

ทฤษฎีบท 3.29 **ทฤษฎีบทเกี่ยวกับคลาสสมมูล:** กำหนดให้ R เป็นความสัมพันธ์สมมูลบนเซต A จะได้ว่า ประการแรก ทุกคลาสสมมูลไม่เป็นเซตว่าง กล่าวคือ $[a]_R \neq \emptyset$ สำหรับทุก $a \in A$ เพราะจากคุณสมบัติสะท้อน $a \in [a]_R$ เสมอ ประการที่สอง ถ้า $a R b$ แล้ว $[a]_R = [b]_R$ กล่าวคือสมาชิกสองตัวที่สมมูลกันจะอยู่ในคลาสสมมูลเดียวกัน ประการที่สาม ถ้า $a \not R b$ แล้ว $[a]_R \cap [b]_R = \emptyset$ กล่าวคือสมาชิกสองตัวที่ไม่สมมูลกันจะอยู่ในคลาสสมมูลที่ไม่ intersect กัน และประการที่สี่ เซตของคลาสสมมูลทั้งหมดของ A โดย R เรียกว่าเซตผัณ เขียนแทนด้วย A/R กล่าวคือ $A/R = \{[a]_R \mid a \in A\}$ และเซต A/R เป็นการแบ่งแยกของเซต A กล่าวคือ สมาชิกทุกตัวใน A อยู่ในคลาสสมมูลที่มากกว่าหนึ่งเซต และคลาสสมมูลต่างๆ ไม่มีสมาชิกร่วมกัน

ตัวอย่าง 3.30 พิจารณาความสัมพันธ์สมมูล R บนเซต $A = \{1, 2, 3, 4, 5, 6\}$ โดย $a R b$ ก็ต่อเมื่อ $a \equiv b \pmod{3}$ กล่าวคือ a และ b ให้เศษเหลือเท่ากันเมื่อหารด้วย 3 เราพบว่าสมาชิกที่สมมูลกันเมื่อหารด้วย 3 ให้เศษเท่ากัน กลุ่มแรกคือสมาชิกที่ให้เศษ 1 เมื่อหารด้วย 3 ได้แก่ 1 และ 4 ดังนั้น $[1]_R = \{1, 4\}$ กลุ่มที่สองคือสมาชิกที่ให้เศษ 2 เมื่อหารด้วย 3 ได้แก่ 2 และ 5 ดังนั้น $[2]_R = \{2, 5\}$ กลุ่มที่สามคือสมาชิกที่ให้เศษ 0 เมื่อหารด้วย 3 ได้แก่ 3 และ 6 ดังนั้น $[3]_R = \{3, 6\}$ หรือ $[6]_R = \{3, 6\}$ สังเกตว่า $[1]_R = [4]_R$ และ $[2]_R = [5]_R$ และ $[3]_R = [6]_R$ เพราะสมาชิกในคลาสสมมูลเดียวกันสมมูลกัน เซตผัณ $A/R = \{\{1, 4\}, \{2, 5\}, \{3, 6\}\}$ ซึ่งเป็นการแบ่งแยกเซต A ออกเป็นสามส่วนที่ไม่มีสมาชิกร่วมกัน

ตัวอย่าง 3.31 พิจารณาความสัมพันธ์สมมูล R บนเซตของเส้นตรงในระนาบ โดย $l_1 R l_2$ ก็ต่อเมื่อ l_1 ขนานกับ l_2 สมาชิกในแต่ละคลาสสมมูลคือเส้นตรงที่ขนานกัน คลาสสมมูลแต่ละคลาสสามารถระบุได้โดยความชัน

ของเส้นตรง ตัวอย่างเช่น $[l_1]_R$ เป็นเซตของเส้นตรงทั้งหมดที่ขนานกับ l_1 ถ้า l_2 ไม่ขนานกับ l_1 แล้ว $[l_2]_R$ เป็นเซตของเส้นตรงทั้งหมดที่ขนานกับ l_2 และ $[l_1]_R \cap [l_2]_R = \emptyset$ เซตผัณ L/R คือเซตของกลุ่มเส้นตรงที่ขนานกันทั้งหมด ซึ่งสามารถระบุได้โดยความชันของเส้นตรงในแต่ละกลุ่ม

ตัวอย่าง 3.32 พิจารณาความสัมพันธ์ R บนเซตของสตริงบนอักษร $\{a, b\}$ โดย $s R t$ ก็ต่อเมื่อ $|s| = |t|$ กล่าวคือสตริง s และ t มีความยาวเท่ากัน เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าสำหรับทุกสตริง s จะได้ $|s| = |s|$ ดังนั้น $s R s$ เราตรวจสอบคุณสมบัติสมมาตรโดยพบว่าถ้า $|s| = |t|$ แล้ว $|t| = |s|$ ดังนั้นถ้า $(s, t) \in R$ แล้ว $(t, s) \in R$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $|s| = |t|$ และ $|t| = |u|$ แล้ว $|s| = |u|$ ดังนั้นถ้า $(s, t) \in R$ และ $(t, u) \in R$ แล้ว $(s, u) \in R$ ดังนั้น R เป็นความสัมพันธ์สมมูล คลาสสมมูล $[s]_R$ คือเซตของสตริงทั้งหมดที่มีความยาวเท่ากับ $|s|$ เซตผัณ S/R คือเซตของกลุ่มสตริงที่มีความยาวเท่ากัน

ความสัมพันธ์สมมาตรเชิงส่วนย่อย (Partial Ordering Relations)

ความสัมพันธ์สมมาตรเชิงส่วนย่อยเป็นความสัมพันธ์ที่มีคุณสมบัติสะท้อน เป็นผลาด และถ่ายทอด ความสัมพันธ์ชนิดนี้ใช้ในการจัดลำดับสมาชิกในเซต โดยไม่จำเป็นต้องเปรียบเทียบสมาชิกทุกคู่ได้ ความสัมพันธ์สมมาตรเชิงส่วนย่อยมีความสำคัญมากในวิทยาการคอมพิวเตอร์ โดยเฉพาะในการจัดลำดับ การทำงานของโปรแกรมและการจัดโครงสร้างข้อมูล

ความแตกต่างระหว่างความสัมพันธ์สมมูลและความสัมพันธ์สมมาตรเชิงส่วนย่อยอยู่ที่คุณสมบัติสมมาตร ความสัมพันธ์สมมูลต้องมีคุณสมบัติสมมาตร แต่ความสัมพันธ์สมมาตรเชิงส่วนย่อยต้องมีคุณสมบัติเป็นผลาดแทน ความสัมพันธ์สมมูลใช้ในการจัดกลุ่มสมาชิกที่เทียบเท่ากัน ขณะที่ความสัมพันธ์สมมาตรเชิงส่วนย่อยใช้ในการจัดลำดับสมาชิก

3.6.1 นิยามของความสัมพันธ์สมมาตรเชิงส่วนย่อย

การเข้าใจนิยามของความสัมพันธ์สมมาตรเชิงส่วนย่อยต้องเปรียบเทียบกับความสัมพันธ์สมมูล ความสัมพันธ์สมมาตรเชิงส่วนย่อยใช้คุณสมบัติเป็นผลาดแทนคุณสมบัติสมมาตร ทำให้เหมาะสำหรับการจัดลำดับ แต่ไม่เหมาะสำหรับการจัดกลุ่ม เซตที่มีความสัมพันธ์สมมาตรเชิงส่วนย่อยเรียกว่าเซตอันดับบางส่วน หรือ Poset

ทฤษฎีบท 3.33 ความสัมพันธ์ R บนเซต A กล่าวว่าเป็นความสัมพันธ์สมมาตรเชิงส่วนย่อย ก็ต่อเมื่อ R มีคุณสมบัติทั้งสามประการต่อไปนี้พร้อมกัน ประการที่หนึ่ง ความสะท้อน กล่าวคือ $\forall a \in A, a R a$ ประการที่สอง ความเป็นผลาด กล่าวคือ $\forall a, b \in A, (a R b \wedge b R a) \rightarrow a = b$ และประการที่สาม ความถ่ายทอด กล่าวคือ $\forall a, b, c \in A, ((a R b \wedge b R c) \rightarrow a R c)$ เราเรียกเซต A ที่มีความสัมพันธ์สมมาตรเชิงส่วนย่อย R ว่าเป็นเซตอันดับบางส่วน เขียนแทนด้วย (A, R) หรือ $(A, \leq)$ โดยสัญลักษณ์ $\leq$ มักใช้แทนความสัมพันธ์สมมาตรเชิงส่วนย่อยในทางทั่วไป

ตัวอย่าง 3.34 พิจารณาความสัมพันธ์ " $\leq$ " บนเซต $\mathbb{Z}$ ของจำนวนเต็ม เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าสำหรับทุก $a \in \mathbb{Z}$ จะได้ $a \leq a$ เราตรวจสอบคุณสมบัติเป็นผลาดโดยพบว่าถ้า $a \leq b$ และ $b \leq a$ แล้ว $a = b$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $a \leq b$ และ $b \leq c$ แล้ว $a \leq c$ ดังนั้น $(\mathbb{Z}, \leq)$ เป็น Poset ความสัมพันธ์นี้เป็นความสัมพันธ์สมมาตรเชิงส่วนย่อยที่สำคัญที่สุดในคณิตศาสตร์ พิจารณาความสัมพันธ์ " $\subseteq$ " บนเซตของเซตทั้งหมด $\mathcal{P}(A)$ สำหรับเซต A ใดๆ เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าสำหรับทุกเซต X จะได้ $X \subseteq X$ เราตรวจสอบคุณสมบัติเป็นผลาดโดยพบว่าถ้า $X \subseteq Y$ และ $Y \subseteq X$ แล้ว $X = Y$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $X \subseteq Y$ และ $Y \subseteq Z$ แล้ว $X \subseteq Z$ ดังนั้น $(\mathcal{P}(A), \subseteq)$ เป็น Poset ถ้า $A = \{1, 2\}$ แล้ว $\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ และ $(\mathcal{P}(A), \subseteq)$ เป็น Poset ที่มีโครงสร้างเป็นตาราง

ตัวอย่าง 3.35 พิจารณาความสัมพันธ์ " $|$ " บนเซต $\mathbb{Z}^+$ ของจำนวนเต็มบวก โดย $a | b$ ก็ต่อเมื่อ a หาร b ลงตัว กล่าวคือมี $k \in \mathbb{Z}^+$ ที่ทำให้ $b = ak$ เราตรวจสอบคุณสมบัติสะท้อนโดยพบว่าสำหรับทุก $a \in \mathbb{Z}^+$ จะได้ $a | a$ เพราะ $a = a \cdot 1$ เราตรวจสอบคุณสมบัติเป็นผลาดโดยพบว่าถ้า $a | b$ และ $b | a$ แล้ว $a = b$ ให้ $b = ak_1$ และ $a = bk_2 = ak_1k_2$ ดังนั้น $k_1k_2 = 1$ และเนื่องจาก $k_1, k_2 \in \mathbb{Z}^+$ แสดงว่า $k_1 = k_2 = 1$ จึงได้ $a = b$ เราตรวจสอบคุณสมบัติถ่ายทอดโดยพบว่าถ้า $a | b$ และ $b | c$ แล้ว $b = ak_1$ และ $c = bk_2 = ak_1k_2$ ดังนั้น $a | c$ ดังนั้น $(\mathbb{Z}^+, |)$ เป็น Poset ความสัมพันธ์นี้มีความสำคัญมากในทฤษฎีจำนวน พิจารณาความสัมพันธ์ " $\leq$ " บนเซต $\{1, 2, 3\}$ จะได้ $(\{1, 2, 3\}, \leq)$ เป็น Poset โดยมีสมาชิกน้อยที่สุดคือ 1 และสมาชิกมากที่สุดคือ 3 สำหรับ Poset ที่มีสมาชิกน้อยที่สุดและมากที่สุด เราเรียกว่า Lattice

หมายเหตุ 3.36 ความสัมพันธ์สมมาตรเชิงส่วนย่อยบางตัวไม่สามารถเปรียบเทียบสมาชิกบางคู่ได้ เช่น ใน Poset $(\mathcal{P}(\{1, 2\}), \subseteq)$ เรามี $\{1\} \subseteq \{2\}$ เป็นเท็จ และ $\{2\} \subseteq \{1\}$ ก็เป็นเท็จเช่นกัน ดังนั้น $\{1\}$ และ $\{2\}$ ไม่สามารถเปรียบเทียบกันได้ ใน Poset นี้ เราเรียกว่า "บางส่วน" เพราะไม่ใช่ทุกคู่ของสมาชิกจะสามารถ

เปรียบเทียบกันได้

3.6.2 สมาชิกพิเศษใน Poset

ในเซตอันดับบางส่วน อาจมีสมาชิกพิเศษที่มีบทบาทสำคัญ ได้แก่ สมาชิกน้อยที่สุด สมาชิกมากที่สุด สมาชิกต่ำสุด และสมาชิกสูงสุด สมาชิกเหล่านี้มีความหมายที่ต่างกัน และไม่ใช่ทุก Poset จะมีสมาชิกเหล่านี้

สมาชิกน้อยที่สุดคือสมาชิกที่มีความสัมพันธ์กับทุกสมาชิกในเซต สมาชิกมากที่สุดคือสมาชิกที่ทุกสมาชิกในเซตมีความสัมพันธ์กับมัน สมาชิกต่ำสุดคือสมาชิกที่ไม่มีสมาชิกอื่นที่เล็กกว่ามัน และสมาชิกสูงสุดคือสมาชิกที่ไม่มีสมาชิกอื่นที่ใหญ่กว่ามัน

ทฤษฎีบท 3.37 กำหนด Poset (A, R) สมาชิก $a \in A$ เป็นสมาชิกน้อยที่สุด ก็ต่อเมื่อ $\forall x \in A, a R x$ สมาชิก $a \in A$ เป็นสมาชิกมากที่สุด ก็ต่อเมื่อ $\forall x \in A, x R a$ สมาชิก $a \in A$ เป็นสมาชิกต่ำสุด ก็ต่อเมื่อไม่มีสมาชิก $x \in A$ ที่ $x \neq a$ และ $x R a$ สมาชิก $a \in A$ เป็นสมาชิกสูงสุด ก็ต่อเมื่อไม่มีสมาชิก $x \in A$ ที่ $x \neq a$ และ $a R x$ ความแตกต่างระหว่างสมาชิกน้อยที่สุดกับสมาชิกต่ำสุดอยู่ที่นิยาม สมาชิกน้อยที่สุดต้องมีความสัมพันธ์กับทุกสมาชิก แต่สมาชิกต่ำสุดเพียงแค่ว่าไม่มีสมาชิกอื่นที่น้อยกว่ามัน ในกรณีส่วนใหญ่ ถ้ามีสมาชิกน้อยที่สุดจะเท่ากับสมาชิกต่ำสุด แต่ไม่เสมอไป

ตัวอย่าง 3.38 พิจารณา Poset $(\mathcal{P}(\{1, 2\}), \subseteq)$ โดย $\mathcal{P}(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ สมาชิกน้อยที่สุดคือ $\emptyset$ เพราะ $\emptyset \subseteq X$ สำหรับทุก $X \in \mathcal{P}(\{1, 2\})$ สมาชิกมากที่สุดคือ $\{1, 2\}$ เพราะ $X \subseteq \{1, 2\}$ สำหรับทุก $X \in \mathcal{P}(\{1, 2\})$ สมาชิกต่ำสุดคือ $\emptyset$ เท่านั้น เพราะไม่มีสมาชิกอื่นที่เป็นสมาชิกย่อยของ $\emptyset$ สมาชิกสูงสุดคือ $\{1, 2\}$ เท่านั้น เพราะไม่มีสมาชิกอื่นที่มี $\{1, 2\}$ เป็นสมาชิกย่อย ในกรณีนี้ สมาชิกน้อยที่สุดเท่ากับสมาชิกต่ำสุด และสมาชิกมากที่สุดเท่ากับสมาชิกสูงสุด

ตัวอย่าง 3.39 พิจารณา Poset $(\mathbb{Z}^+, |)$ โดย $a | b$ ก็ต่อเมื่อ a หหาร b ลงตัว สมาชิกน้อยที่สุดคือ 1 เพราะสำหรับทุก $x \in \mathbb{Z}^+$ เรามี $1 | x$ สมาชิกมากที่สุดไม่มี เพราะสำหรับทุก $a \in \mathbb{Z}^+$ จะมี $b = 2a$ ที่ $a | b$ แต่ $b | a$ ไม่เป็นจริง สมาชิกต่ำสุดคือ 1 เท่านั้น เพราะไม่มี $x < 1$ ใน $\mathbb{Z}^+$ สมาชิกสูงสุดไม่มี

ตัวอย่าง 3.40 พิจารณา Poset $(\{1, 2, 3, 4, 6, 12\}, |)$ โดย $a | b$ ก็ต่อเมื่อ a หหาร b ลงตัว ความสัมพันธ์คือ $R = \{(1, 1), (1, 2), (1, 3), (1, 4), (1, 6), (1, 12), (2, 2), (2, 4), (2, 6), (2, 12), (3, 3), (3, 6), (3, 12), (4, 4), (4, 12), (6, 6), (6, 12)\}$ สมาชิกน้อยที่สุดคือ 1 เพราะ $1 | x$ สำหรับทุก x ในเซต สมาชิกมากที่สุดคือ 12 เพราะ $x | 12$ สำหรับทุก x ในเซต นี่คือตัวอย่างของ Bounded Poset ซึ่งมีทั้งสมาชิกน้อยที่สุดและมากที่สุด

อธิบายการดำเนินการบนความสัมพันธ์

ในส่วนนี้เราจะศึกษาการดำเนินการที่สำคัญบนความสัมพันธ์ ได้แก่ การรวมความสัมพันธ์และการย้อนกลับความสัมพันธ์ การดำเนินการเหล่านี้มีความสำคัญในการวิเคราะห์ความสัมพันธ์ที่ซับซ้อนและการประยุกต์ใช้ในวิทยาการคอมพิวเตอร์

การรวมความสัมพันธ์ช่วยให้เราสามารถต่อความสัมพันธ์เข้าด้วยกัน ซึ่งมีความหมายทางปฏิบัติมาก เช่น ในการหาเส้นทางในกราฟ การย้อนกลับความสัมพันธ์ช่วยให้เราสามารถมองความสัมพันธ์ในทิศทางตรงกันข้าม การดำเนินการทั้งสองมีคุณสมบัติทางพีชคณิตที่น่าสนใจ

3.7.1 การรวมความสัมพันธ์ (Composition of Relations)

การรวมความสัมพันธ์เป็นการดำเนินการที่ต่อความสัมพันธ์เข้าด้วยกัน ถ้า R เป็นความสัมพันธ์จาก A ไป B และ S เป็นความสัมพันธ์จาก B ไป C แล้วการรวม $S \circ R$ คือความสัมพันธ์จาก A ไป C ที่มีคู่ (a, c) ก็ต่อเมื่อมี $b \in B$ ที่ทำให้ $(a, b) \in R$ และ $(b, c) \in S$ การรวมความสัมพันธ์มีความหมายโดยธรรมชาติในหลายบริบท เช่น ในการหาเส้นทางในกราฟ ถ้า R คือเส้นทางจากเมือง A ไปเมือง B และ S คือเส้นทางจากเมือง B ไปเมือง C แล้ว $S \circ R$ คือเส้นทางจากเมือง A ไปเมือง C ผ่านเมือง B

ทฤษฎีบท 3.41 กำหนดความสัมพันธ์ R จาก A ไป B และความสัมพันธ์ S จาก B ไป C การรวมของ R และ S เขียนแทนด้วย $S \circ R$ หรือ $R; S$ คือความสัมพันธ์จาก A ไป C โดย $S \circ R = \{(a, c) \in A \times C \mid \exists b \in B, (a, b) \in R \wedge (b, c) \in S\}$

นั่นหมายความว่า $(a, c) \in S \circ R$ ก็ต่อเมื่อ มี $b \in B$ บางตัว ที่ทำให้ $(a, b) \in R$ และ $(b, c) \in S$

การรวมความสัมพันธ์มีความสำคัญมากในการวิเคราะห์เส้นทางในกราฟ และการติดตามการเชื่อมโยงข้อมูล ในบริบทของกราฟ ถ้าเรามีเส้นเชื่อมจาก a ไป b และจาก b ไป c แล้วการรวมความสัมพันธ์จะให้เส้นเชื่อมจาก a ไป c ผ่าน b

ตัวอย่าง 3.42 กำหนด $A = \{1, 2, 3\}$, $B = \{a, b, c\}$, $C = \{x, y, z\}$ ความสัมพันธ์ $R = \{(1, a), (1, b), (2, b), (3, c)\}$ จาก A ไป B และความสัมพันธ์ $S = \{(a, x), (a, y), (b, y), (c, z)\}$ จาก B ไป C การหา $S \circ R$ ทำได้โดยพิจารณาทุกคู่ (a, c) ใน $A \times C$ ว่ามี $b \in B$ ที่ทำให้ $(a, b) \in R$ และ $(b, c) \in S$ หรือไม่ สำหรับ $1 \in A$ เรามี $(1, a) \in R$ และ $(a, x) \in S$ ดังนั้น $(1, x) \in S \circ R$ สำหรับ $1 \in A$ เรามี $(1, a) \in R$ และ $(a, y) \in S$ ดังนั้น $(1, y) \in S \circ R$ สำหรับ $1 \in A$ เรามี $(1, b) \in R$ และ $(b, y) \in S$ ดังนั้น $(1, y) \in S \circ R$ แต่ $(1, y)$ มีอยู่แล้วจากกรณีก่อน สำหรับ $2 \in A$ เรามี $(2, b) \in R$ และ $(b, y) \in S$

ดังนั้น $(2, y) \in S \circ R$ สำหรับ $3 \in A$ เรามี $(3, c) \in R$ และ $(c, z) \in S$ ดังนั้น $(3, z) \in S \circ R$ รวมแล้ว $S \circ R = \{(1, x), (1, y), (2, y), (3, z)\}$

การหาการรวมความสัมพันธ์ทำได้โดยระบุทุกเส้นทางจาก A ไป C ผ่าน B ในตัวอย่างนี้ เราพบว่า มีเส้นทางจาก 1 ไป x ผ่าน a , เส้นทางจาก 1 ไป y ผ่าน a และผ่าน b , เส้นทางจาก 2 ไป y ผ่าน b , และเส้นทางจาก 3 ไป z ผ่าน c

ทฤษฎีบท 3.43 คุณสมบัติ การ รวม ความ สัมพันธ์: กำหนด ความ สัมพันธ์ R, S, T และ เซต A, B, C, D จะได้ว่า ประการแรก การรวมเป็นสมาชิกได้ กล่าวคือ $(T \circ S) \circ R = T \circ (S \circ R)$ ถ้าความสัมพันธ์ทั้งสามมีโดเมนและเรนจ์ที่เหมาะสม ประการที่สอง การรวมกับยูเนียน กล่าวคือ $(R \cup S) \circ T = (R \circ T) \cup (S \circ T)$ และประการที่สาม การรวมกับอินเตอร์เซกชัน กล่าวคือ $(R \cap S) \circ T \subseteq (R \circ T) \cap (S \circ T)$

คุณสมบัติการเป็นสมาชิกได้ของการรวมความสัมพันธ์เป็นสิ่งสำคัญ เพราะทำให้เราสามารถจัดกลุ่มการรวมได้โดยไม่ต้องกังวลว่าลำดับจะมีผลต่อผลลัพธ์ อย่างไรก็ตาม การรวมความสัมพันธ์ไม่สลับที่ได้ กล่าวคือ $R \circ S$ ไม่จำเป็นต้องเท่ากับ $S \circ R$

ตัวอย่าง 3.44 ตรวจสอบคุณสมบัติการรวมเป็นสมาชิกได้ กำหนด $A = \{1\}$, $B = \{2\}$, $C = \{3\}$, $D = \{4\}$ ความสัมพันธ์ $R = \{(1, 2)\}$ จาก A ไป B , $S = \{(2, 3)\}$ จาก B ไป C , และ $T = \{(3, 4)\}$ จาก C ไป D การหา $(T \circ S) \circ R$ ทำได้โดยก่อนหา $T \circ S = \{(2, 4)\}$ แล้ว $(T \circ S) \circ R = \{(1, 4)\}$ การหา $T \circ (S \circ R)$ ทำได้โดยก่อนหา $S \circ R = \{(1, 3)\}$ แล้ว $T \circ (S \circ R) = \{(1, 4)\}$ ดังนั้น $(T \circ S) \circ R = T \circ (S \circ R) = \{(1, 4)\}$ ซึ่งพิสูจน์คุณสมบัติการรวมเป็นสมาชิกได้

3.7.2 การย้อนกลับความสัมพันธ์ (Converse Relation)

การย้อนกลับความสัมพันธ์เป็นการดำเนินการที่สลับตำแหน่งของสมาชิกในแต่ละคู่อันดับ ถ้า R เป็นความสัมพันธ์จาก A ไป B แล้ว R^{-1} เป็นความสัมพันธ์จาก B ไป A ที่มีคู่ (b, a) ก็ต่อเมื่อ $(a, b) \in R$ การย้อนกลับมีความหมายโดยธรรมชาติในหลายบริบท เช่น ถ้า R คือความสัมพันธ์ "เป็นบิดาของ" แล้ว R^{-1} คือความสัมพันธ์ "เป็นบุตรของ"

ทฤษฎีบท 3.45 กำหนดความสัมพันธ์ R จาก A ไป B ความสัมพันธ์ผกผันของ R เขียนแทนด้วย R^{-1} คือความสัมพันธ์จาก B ไป A โดย $R^{-1} = \{(b, a) \in B \times A \mid (a, b) \in R\}$

การย้อนกลับความสัมพันธ์ช่วยให้เรามองความสัมพันธ์ในทิศทางตรงกันข้าม ถ้าเรามีความสัมพันธ์ “มากกว่า” แล้วการย้อนกลับจะให้ความสัมพันธ์ “น้อยกว่า”

ตัวอย่าง 3.46 กำหนด $R = \{(1, a), (1, b), (2, b), (3, c)\}$ จาก $A = \{1, 2, 3\}$ ไป $B = \{a, b, c\}$ การหา R^{-1} ทำได้โดยสลับตำแหน่งของสมาชิกในแต่ละคู่อันดับ จาก $(1, a)$ ได้ $(a, 1)$ จาก $(1, b)$ ได้ $(b, 1)$ จาก $(2, b)$ ได้ $(b, 2)$ และจาก $(3, c)$ ได้ $(c, 3)$ ดังนั้น $R^{-1} = \{(a, 1), (b, 1), (b, 2), (c, 3)\}$ จาก B ไป A

ทฤษฎีบท 3.47 คุณสมบัติของ ความสัมพันธ์ ผกผัน: กำหนด ความสัมพันธ์ R ใดๆ จะได้ว่า $(R^{-1})^{-1} = R$ เพราะการย้อนกลับสองครั้งจะได้ความสัมพันธ์เดิม $(R \cup S)^{-1} = R^{-1} \cup S^{-1}$ เพราะการย้อนกลับกระจายไปบนยูเนียน $(R \cap S)^{-1} = R^{-1} \cap S^{-1}$ เพราะการย้อนกลับกระจายไปบนอินเตอร์เซกชัน $(R \circ S)^{-1} = S^{-1} \circ R^{-1}$ เพราะการย้อนกลับการรวมจะสลับลำดับ ถ้า R เป็นความสัมพันธ์สมมาตร แล้ว $R^{-1} = R$ เพราะทุกคู่มีสองทิศทาง และถ้า R เป็นความสัมพันธ์สะท้อน แล้ว R^{-1} ก็เป็นความสัมพันธ์สะท้อนด้วย

คุณสมบัติเหล่านี้มีประโยชน์ในการพิสูจน์และการคำนวณ สูตร $(R \circ S)^{-1} = S^{-1} \circ R^{-1}$ มีความสำคัญเป็นพิเศษ เพราะการย้อนกลับการรวมจะสลับลำดับของความสัมพันธ์

ตัวอย่าง 3.48 ตรวจสอบ $(R \circ S)^{-1} = S^{-1} \circ R^{-1}$ กำหนด $R = \{(1, a)\}$ จาก $A = \{1\}$ ไป $B = \{a\}$ และ $S = \{(a, x)\}$ จาก $B = \{a\}$ ไป $C = \{x\}$ การหา $R \circ S$ ได้ $\{(1, x)\}$ และ $(R \circ S)^{-1} = \{(x, 1)\}$ การหา S^{-1} ได้ $\{(x, a)\}$ และ $R^{-1} = \{(a, 1)\}$ การหา $S^{-1} \circ R^{-1}$ ได้ $\{(x, 1)\}$ ดังนั้น $(R \circ S)^{-1} = S^{-1} \circ R^{-1}$ ซึ่งพิสูจน์สูตรนี้

สมบัติปิดคุณสมบัติ (Closure Properties)

ในหลายกรณี เราต้องการขยายความสัมพันธ์ให้มีคุณสมบัติบางอย่างโดยเพิ่มคู่อันดับให้น้อยที่สุดเท่าที่จะเป็นไปได้ กระบวนการนี้เรียกว่าการปิด แนวคิดนี้มีความสำคัญในการสร้างความสัมพันธ์ที่มีคุณสมบัติตามต้องการโดยเพิ่มสมาชิกให้น้อยที่สุด

การปิดมีสามชนิดหลัก ได้แก่ การปิดสะท้อน การปิดสมมาตร และการปิดถ่ายทอด แต่ละชนิดเพิ่มคู่อันดับที่จำเป็นเพื่อให้ความสัมพันธ์มีคุณสมบัตินั้นๆ

3.8.1 การปิดสะท้อน (Reflexive Closure)

การปิดสะท้อนของความสัมพันธ์ R คือความสัมพันธ์สะท้อนที่น้อยที่สุดที่ประกอบด้วย R การหาการปิดสะท้อนทำได้โดยเพิ่มคู่อันดับ (a, a) สำหรับทุก $a \in A$ ที่ยังไม่มีใน R การปิดสะท้อนมีความหมายในหลายบริบท เช่น ในการเพิ่มความสัมพันธ์ "เท่ากับตัวมันเอง" ให้กับความสัมพันธ์ที่ยังไม่มี

ทฤษฎีบท 3.49 กำหนดความสัมพันธ์ R บนเซต A ความสัมพันธ์สะท้อนน้อยที่สุดที่ประกอบด้วย R คือ $R \cup \{(a, a) \mid a \in A\}$ เรียกว่าการปิดสะท้อนของ R เขียนแทนด้วย $r(R)$

การปิดสะท้อนเป็นการเพิ่มเฉพาะคู่อันดับที่จำเป็นเพื่อให้ทุกสมาชิกมีความสัมพันธ์กับตัวมันเอง คู่อันดับที่มีอยู่แล้วใน R จะไม่ถูกลบออก

ตัวอย่าง 3.50 กำหนด $A = \{1, 2, 3\}$ และ $R = \{(1, 2), (2, 3)\}$ การปิดสะท้อนหาได้โดยเพิ่มคู่อันดับ (a, a) สำหรับทุก $a \in A$ ที่ยังไม่มีใน R ในกรณีนี้ ยังไม่มี $(1, 1), (2, 2), (3, 3)$ ใน R ดังนั้น $r(R) = R \cup \{(1, 1), (2, 2), (3, 3)\} = \{(1, 1), (1, 2), (2, 2), (2, 3), (3, 3)\}$ สังเกตว่า $r(R)$ เป็นความสัมพันธ์สะท้อนและเป็นความสัมพันธ์สะท้อนที่น้อยที่สุดที่ประกอบด้วย R เพราะถ้าเราลบคู่อันดับใดๆ ออก ความสัมพันธ์ก็จะไม่เป็นสะท้อน

3.8.2 การปิดสมมาตร (Symmetric Closure)

การปิดสมมาตรของความสัมพันธ์ R คือความสัมพันธ์สมมาตรที่น้อยที่สุดที่ประกอบด้วย R การหาการปิดสมมาตรทำได้โดยเพิ่มคู่อันดับ (b, a) สำหรับทุก $(a, b) \in R$ ที่ (b, a) ยังไม่มีใน R การปิดสมมาตรมีความหมายในหลายบริบท เช่น ในการเพิ่มความสัมพันธ์ "เป็นเพื่อนกับ" ในทั้งสองทิศทาง

ทฤษฎีบท 3.51 กำหนดความสัมพันธ์ R บนเซต A ความสัมพันธ์สมมาตรน้อยที่สุดที่ประกอบด้วย R คือ $R \cup R^{-1}$ เรียกว่าการปิดสมมาตรของ R เขียนแทนด้วย $s(R)$

การปิดสมมาตรเป็นการเพิ่มเฉพาะคู่อันดับที่จำเป็นเพื่อให้ถ้า $(a, b) \in R$ แล้ว $(b, a) \in R$ ด้วย การใช้ R^{-1} ทำให้การคำนวณง่ายขึ้น

ตัวอย่าง 3.52 กำหนด $A = \{1, 2, 3\}$ และ $R = \{(1, 2), (2, 3)\}$ การปิดสมมาตรหาได้โดยเพิ่มคู่อันดับ (b, a) สำหรับทุก $(a, b) \in R$ ที่ (b, a) ยังไม่มีใน R จาก $(1, 2)$ ได้ $(2, 1)$ จาก $(2, 3)$ ได้ $(3, 2)$ ดังนั้น $s(R) = R \cup R^{-1} = \{(1, 2), (2, 3)\} \cup \{(2, 1), (3, 2)\} = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$ สังเกตว่า $s(R)$ เป็น

ความสัมพันธ์สมมาตร และเป็นความสัมพันธ์สมมาตรที่น้อยที่สุดที่ประกอบด้วย R

3.8.3 การปิดถ่ายทอด (Transitive Closure)

การปิดถ่ายทอดของความสัมพันธ์ R คือความสัมพันธ์ถ่ายทอดที่น้อยที่สุดที่ประกอบด้วย R การหาการปิดถ่ายทอดทำได้โดยเพิ่มคู่อันดับ (a, c) สำหรับทุก $(a, b) \in R$ และ $(b, c) \in R$ การปิดถ่ายทอดมีความหมายในหลายบริบท เช่น ในการหาเส้นทางทั้งหมดในกราฟ

ทฤษฎีบท 3.53 กำหนดความสัมพันธ์ R บนเซต A ความสัมพันธ์ถ่ายทอดน้อยที่สุดที่ประกอบด้วย R คือ $t(R) = R \cup R^2 \cup R^3 \cup \dots = \bigcup_{n=1}^{\infty} R^n$ โดย R^n หมายถึงการรวมความสัมพันธ์ R กับตัวมันเอง n ครั้ง เรียกว่าการปิดถ่ายทอดของ R เขียนแทนด้วย $t(R)$

สำหรับเซตจำกัด A จะมี n ที่ทำให้ $t(R) = \bigcup_{i=1}^n R^i$

การปิดถ่ายทอดอาจต้องเพิ่มคู่อันดับมาก เพราะต้องเพิ่มทุกเส้นทางที่เป็นไปได้ ในเซตจำกัด การปิดถ่ายทอดจะหยุดเมื่อไม่มีคู่อันดับใหม่ที่ต้องเพิ่ม

ตัวอย่าง 3.54 กำหนด $A = \{1, 2, 3, 4\}$ และ $R = \{(1, 2), (2, 3), (3, 4)\}$ การหา $R^2 = R \circ R$ ทำได้โดยหาทุกคู่ (a, c) ที่มี b ที่ทำให้ $(a, b) \in R$ และ $(b, c) \in R$ จาก $(1, 2) \in R$ และ $(2, 3) \in R$ ได้ $(1, 3) \in R^2$ จาก $(2, 3) \in R$ และ $(3, 4) \in R$ ได้ $(2, 4) \in R^2$ ดังนั้น $R^2 = \{(1, 3), (2, 4)\}$ การหา $R^3 = R^2 \circ R$ จาก $(1, 3) \in R^2$ และ $(3, 4) \in R$ ได้ $(1, 4) \in R^3$ ดังนั้น $R^3 = \{(1, 4)\}$ การหา $R^4 = R^3 \circ R$ ไม่มีคู่อันดับใหม่ เพราะไม่มี $(a, b) \in R^3$ และ $(b, c) \in R$ ที่ให้ (a, c) ใหม่ ดังนั้น $t(R) = R \cup R^2 \cup R^3 = \{(1, 2), (2, 3), (3, 4), (1, 3), (2, 4), (1, 4)\}$

สังเกตว่า $t(R)$ เป็นความสัมพันธ์ถ่ายทอด ซึ่งหมายความว่าถ้า $1 R 2$ และ $2 R 3$ แล้ว $1 R 3$ เป็นจริงใน $t(R)$ ด้วย ความสัมพันธ์ $t(R)$ บอกว่ามีเส้นทางจาก 1 ไป 4 โดยผ่าน 2 และ 3

ตัวอย่าง 3.55 ในทางทฤษฎีกราฟ การปิดถ่ายทอดของความสัมพันธ์ R บน A หมายถึงการมีเส้นทางจาก a ไป c ในกราฟที่มีเส้นเชื่อมเป็นคู่อันดับใน R ถ้า $R = \{(1, 2), (2, 3), (3, 4)\}$ แสดงว่ามีเส้นทางจาก 1 ไป 4 โดยผ่าน 2 และ 3 ดังนั้นใน $t(R)$ จะมี $(1, 4)$ เพิ่มเข้ามา เพราะมีเส้นทางจาก 1 ไป 4 ผ่านคนกลาง

8. ประยุกต์ใช้ความสัมพันธ์ในวิทยาการคอมพิวเตอร์

ความสัมพันธ์มีการประยุกต์ใช้มากมายในวิทยาการคอมพิวเตอร์ ในส่วนนี้เราจะศึกษาตัวอย่างการประยุกต์ใช้ที่สำคัญ ซึ่งจะช่วยให้ผู้อ่านเข้าใจความสำคัญของความสัมพันธ์ในทางปฏิบัติ

ความสัมพันธ์เป็นพื้นฐานของระบบฐานข้อมูลเชิงสัมพันธ์ การวิเคราะห์เครือข่ายสังคม การตรวจสอบความสมดุลของวงเล็บ และการวิเคราะห์การพึ่งพาของโปรแกรม การเข้าใจการประยุกต์ใช้เหล่านี้จะช่วยให้ผู้อ่านเห็นคุณค่าของความสัมพันธ์ในงานจริง

3.9.1 ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

ระบบฐานข้อมูลเชิงสัมพันธ์ที่พัฒนาโดยเอ็ดการ์ คอดด์ ใช้ความสัมพันธ์เป็นพื้นฐานในการจัดเก็บข้อมูล ตารางในฐานข้อมูลเชิงสัมพันธ์คือความสัมพันธ์ในทางคณิตศาสตร์ โดยแต่ละแถวคือทูเปิล และแต่ละคอลัมน์คือแอตทริบิวต์ ระบบฐานข้อมูลเชิงสัมพันธ์ใช้ทฤษฎีความสัมพันธ์ในการดำเนินการบนข้อมูล เช่น การเลือกข้อมูล การฉายข้อมูล และการรวมตาราง

Edgar F. Codd ได้เสนอ Relational Model ในปี 1970 ซึ่งเป็นรากฐานของระบบฐานข้อมูลสมัยใหม่ทุกระบบ ทฤษฎีความสัมพันธ์ทำให้การจัดการข้อมูลเป็นระบบที่มีความสอดคล้องและถูกต้อง

ตัวอย่าง 3.56 พิจารณาฐานข้อมูลของนักศึกษาที่ประกอบด้วยตาราง Student ซึ่งมีแอตทริบิวต์ ID, Name, และ Major ตารางนี้สามารถมองได้ว่าเป็นความสัมพันธ์ $R \subseteq \text{ID} \times \text{Name} \times \text{Major}$ โดยแต่ละทูเปิล (1001, สมชาย, CS) เป็นสมาชิกของ R การดำเนินการบนฐานข้อมูล เช่น Selection คือการเลือกทูเปิลที่ตรงตามเงื่อนไข Projection คือการเลือกแอตทริบิวต์บางตัว และ Join คือการรวมความสัมพันธ์สองตารางโดยมีเงื่อนไขการรวม การดำเนินการเหล่านี้สามารถอธิบายได้ด้วยการดำเนินการบนเซตและความสัมพันธ์

ในฐานข้อมูลเชิงสัมพันธ์ มีการดำเนินการบนความสัมพันธ์หลายชนิดที่สำคัญ Selection คือการเลือกทูเปิลที่ตรงตามเงื่อนไข Projection คือการเลือกแอตทริบิวต์บางตัว Join คือการรวมความสัมพันธ์สองตารางโดยมีเงื่อนไขการรวม Union คือการรวมทูเปิลจากสองตาราง Intersection คือการหาทูเปิลที่อยู่ในทั้งสองตาราง และ Difference คือการหาทูเปิลที่อยู่ในตารางหนึ่งแต่ไม่อยู่ในอีกตารางหนึ่ง

3.9.2 การตรวจสอบความสมดุลของวงเล็บ

การตรวจสอบว่าวงเล็บในสตริงมีความสมดุลหรือไม่เป็นปัญหาพื้นฐานในการเขียนคอมไพเลอร์ และการประมวลผลภาษา สตริงที่มีวงเล็บสมดุลเช่น "((()))" มีจำนวนวงเล็บเปิดเท่ากับจำนวนวงเล็บปิด และวงเล็บปิดแต่ละตัวจับคู่กับวงเล็บเปิดที่ถูกต้อง สตริงที่ไม่สมดุลเช่น "())" มีวงเล็บปิดเกิน

เราสามารถใช้ความสัมพันธ์ในการแก้ปัญหานี้ได้โดยกำหนดความสูงสะสม ซึ่งบอกจำนวนวงเล็บเปิดที่ยังไม่ได้ปิด ณ แต่ละตำแหน่งในสตริง และกำหนดความสัมพันธ์ระหว่างวงเล็บเปิดและวงเล็บปิดที่จับคู่กัน

ตัวอย่าง 3.57 พิจารณาปัญหาการตรวจสอบว่าวงเล็บในสตริงมีความสมดุลหรือไม่ กำหนดสตริง s ของ

วงเล็บเปิดและปิด สำหรับแต่ละตำแหน่ง i ใน s กำหนดความสูงสะสม $h(i)$ คือจำนวนวงเล็บเปิดที่ยังไม่ได้ปิด ณ ตำแหน่งนั้น ความสัมพันธ์ $R = \{(i, j) \mid \text{วงเล็บเปิดที่ตำแหน่ง } i \text{ จับคู่กับวงเล็บปิดที่ตำแหน่ง } j\}$ สตรีงมีวงเล็บสมดุล ก็ต่อเมื่อทุกวงเล็บเปิดจับคู่กับวงเล็บปิดที่ถูกต้อง และไม่มีการซ้อนทับกัน กล่าวคือ ถ้า $(i, j) \in R$ แล้วไม่มี $(i', j') \in R$ ที่ $i < i' < j < j'$ หรือ $i' < i < j' < j$

การตรวจสอบความสมดุลของวงเล็บสามารถทำได้โดยใช้ Stack โดยผลักวงเล็บเปิดลง Stack และเมื่อพบวงเล็บปิด ให้ตรวจสอบว่า Stack ไม่ว่างและวงเล็บเปิดบนสุดจับคู่กับวงเล็บปิดนี้ได้หรือไม่

3.9.3 การจัดการเครือข่ายสังคม (Social Network Analysis)

การวิเคราะห์เครือข่ายสังคมเป็นสาขาที่สำคัญในวิทยาการคอมพิวเตอร์และสังคมวิทยา เครือข่ายสังคมประกอบด้วยผู้ใช้และความสัมพันธ์ระหว่างผู้ใช้ เช่น ความสัมพันธ์ "เป็นเพื่อนกับ" หรือ "ติดตาม" การวิเคราะห์เครือข่ายสังคมช่วยให้เข้าใจโครงสร้างและพลวัตของสังคม

ตัวอย่าง 3.58 พิจารณาเครือข่ายสังคมที่มีผู้ใช้ n คน โดยมีความสัมพันธ์ "เป็นเพื่อนกับ" ระหว่างผู้ใช้ กำหนดเซต $V = \{1, 2, 3, \dots, n\}$ เป็นเซตของผู้ใช้ และความสัมพันธ์ $R = \{(a, b) \mid a \text{ และ } b \text{ เป็นเพื่อนกัน}\}$ คุณสมบัติของ R ได้แก่ ไม่เป็นความสัมพันธ์สะท้อน เพราะคนไม่เป็นเพื่อนกับตัวเอง เป็นความสัมพันธ์สมมาตร เพราะถ้า a เป็นเพื่อนกับ b แล้ว b ก็เป็นเพื่อนกับ a และไม่เป็นความสัมพันธ์ถ่ายทอด เพราะถ้า a เป็นเพื่อนกับ b และ b เป็นเพื่อนกับ c แล้ว a อาจไม่เป็นเพื่อนกับ c เราสามารถใช้การปิดถ่ายทอด $t(R)$ เพื่อหา "เพื่อนของเพื่อน" ได้ ถ้า $(a, b) \in t(R)$ แสดงว่ามีเส้นทางจาก a ไป b ในเครือข่ายสังคม ไม่ว่าจะผ่านกี่คน

การปิดถ่ายทอดของความสัมพันธ์ "เป็นเพื่อนกับ" จะบอกว่ามีเส้นทางเชื่อมต่อระหว่างผู้ใช้สองคนหรือไม่ แม้ว่าพวกเขาจะไม่ได้เป็นเพื่อนกันโดยตรง

3.9.4 การวิเคราะห์การพึ่งพาของโปรแกรม (Program Dependency Analysis)

ในการวิเคราะห์โปรแกรม เราสนใจความสัมพันธ์ระหว่างตัวแปรและคำสั่ง การวิเคราะห์การพึ่งพาช่วยให้เข้าใจว่าคำสั่งหนึ่งๆ ขึ้นอยู่กับคำสั่งใด ซึ่งมีความสำคัญในการจัดลำดับการทำงานของโปรแกรมและการตรวจสอบความปลอดภัย

ตัวอย่าง 3.59 ในการวิเคราะห์โปรแกรม กำหนดความสัมพันธ์ $R = \{(v, s) \mid \text{ตัวแปร } v \text{ ถูกใช้ในคำสั่ง } s\}$ เราสามารถหาการพึ่งพาระหว่างคำสั่งได้โดยใช้การปิดถ่ายทอด ถ้า $(s_1, s_2) \in t(R)$ แสดงว่าการพึ่งพาข้อมูลจาก s_1 ไป s_2 ข้อมูลนี้สามารถใช้ในการจัดลำดับการทำงานของโปรแกรม หรือการตรวจสอบความปลอดภัย

การวิเคราะห์การพึ่งพามีความสำคัญในการทำให้โปรแกรมทำงานขนานได้ ถ้าไม่มีการพึ่งพาระหว่างคำสั่ง คำสั่งเหล่านั้นสามารถทำงานพร้อมกันได้

ตัวอย่างและแบบฝึกหัดท้ายบท

ตัวอย่าง 3.60 จงพิจารณาความสัมพันธ์ต่อไปนี้บนเซต $\{1, 2, 3, 4\}$ โดย $R = \{(1, 1), (1, 3), (2, 2), (2, 4), (3, 1), (3, 3), (4, 2), (4, 4)\}$ จงตรวจสอบว่า R มีคุณสมบัติใดต่อไปนี้ สะท้อน สมมาตร เป็นผลาด ถ่ายทอด และเป็นความสัมพันธ์สมมูลหรือไม่ และเป็นความสัมพันธ์สมมาตรเชิงส่วนย่อยหรือไม่

วิธีทำ: เราจะตรวจสอบคุณสมบัติทีละข้อ

สำหรับคุณสมบัติสะท้อน เราต้องตรวจสอบว่า $(a, a) \in R$ สำหรับทุก $a \in \{1, 2, 3, 4\}$ หรือไม่ จากการพิจารณาพบว่า $(1, 1) \in R$, $(2, 2) \in R$, $(3, 3) \in R$, และ $(4, 4) \in R$ ดังนั้น R เป็นความสัมพันธ์สะท้อน

สำหรับคุณสมบัติสมมาตร เราต้องตรวจสอบว่าถ้า $(a, b) \in R$ แล้ว $(b, a) \in R$ หรือไม่ จากการพิจารณาพบว่า $(1, 1) \in R$ และ $(1, 1) \in R$, $(1, 3) \in R$ และ $(3, 1) \in R$, $(2, 2) \in R$ และ $(2, 2) \in R$, $(2, 4) \in R$ และ $(4, 2) \in R$, $(3, 1) \in R$ และ $(1, 3) \in R$, $(3, 3) \in R$ และ $(3, 3) \in R$, $(4, 2) \in R$ และ $(2, 4) \in R$, $(4, 4) \in R$ และ $(4, 4) \in R$ ดังนั้น R เป็นความสัมพันธ์สมมาตร

สำหรับคุณสมบัติเป็นผลาด เราต้องตรวจสอบว่าถ้า $(a, b) \in R$ และ $(b, a) \in R$ แล้ว $a = b$ หรือไม่ จากการพิจารณาพบว่า $(1, 3) \in R$ และ $(3, 1) \in R$ แต่ $1 \neq 3$ ดังนั้นเงื่อนไข $(a R b \wedge b R a) \rightarrow a = b$ เป็นเท็จ เพราะมีตัวอย่างที่ $a = 1$, $b = 3$ ทำให้เงื่อนไขเป็นจริงแต่ผลสรุป $a = b$ เป็นเท็จ ดังนั้น R ไม่เป็นความสัมพันธ์เป็นผลาด

สำหรับคุณสมบัติถ่ายทอด เราต้องตรวจสอบว่าถ้า $(a, b) \in R$ และ $(b, c) \in R$ แล้ว $(a, c) \in R$ หรือไม่ จากการพิจารณาพบว่า $(1, 3) \in R$ และ $(3, 1) \in R$ ดังนั้นต้องมี $(1, 1) \in R$ ซึ่งมีอยู่ พบว่า $(1, 3) \in R$ และ $(3, 3) \in R$ ดังนั้นต้องมี $(1, 3) \in R$ ซึ่งมีอยู่ พบว่า $(2, 4) \in R$ และ $(4, 2) \in R$ ดังนั้นต้องมี $(2, 2) \in R$ ซึ่งมีอยู่ พบว่า $(2, 4) \in R$ และ $(4, 4) \in R$ ดังนั้นต้องมี $(2, 4) \in R$ ซึ่งมีอยู่ พบว่า $(4, 2) \in R$ และ $(2, 2) \in R$ ดังนั้นต้องมี $(4, 2) \in R$ ซึ่งมีอยู่ พบว่า $(4, 2) \in R$ และ $(2, 4) \in R$ ดังนั้นต้องมี $(4, 4) \in R$ ซึ่งมีอยู่ ดังนั้น R เป็นความสัมพันธ์ถ่ายทอด

เนื่องจาก R เป็นความสัมพันธ์สะท้อน สมมาตร และถ่ายทอด ดังนั้น R เป็นความสัมพันธ์สมมูล เนื่องจาก R ไม่เป็นความสัมพันธ์เป็นผลาด ดังนั้น R ไม่เป็นความสัมพันธ์สมมาตรเชิงส่วนย่อย

ตัวอย่าง 3.61 จงหาคลาสสมมูลของความสัมพันธ์สมมูล R ในตัวอย่างก่อนหน้า

วิธีทำ: จากการตรวจสอบคุณสมบัติ พบว่า R เป็นความสัมพันธ์สมมูล โดยมีคู่ $(1, 3)$ และ $(3, 1)$ แสดงว่า 1 สมมูลกับ 3 และมี $(2, 4)$ และ $(4, 2)$ แสดงว่า 2 สมมูลกับ 4 คลาสสมมูลของ R ได้แก่ $[1]_R = \{1, 3\}$ เพราะ 1 สมมูลกับ 1 และ 3, $[2]_R = \{2, 4\}$ เพราะ 2 สมมูลกับ 2 และ 4, $[3]_R = \{1, 3\}$ เหมือนกับ $[1]_R$ เพราะ 3 สมมูลกับ 1 และ 3, และ $[4]_R = \{2, 4\}$ เหมือนกับ $[2]_R$ เพราะ 4 สมมูลกับ 2 และ 4 เซตผัณ $A/R = \{\{1, 3\}, \{2, 4\}\}$ ซึ่งเป็นการแบ่งแยกเซต A ออกเป็นสองส่วนที่ไม่มีสมาชิกร่วมกัน

ตัวอย่าง 3.62 จงพิจารณาความสัมพันธ์ R บนเซตของจำนวนเต็ม $\mathbb{Z}$ โดย $a R b$ ก็ต่อเมื่อ $|a| = |b|$ จงพิสูจน์ว่า R เป็นความสัมพันธ์สมมูล หาคาสสมมูลของ 0, 1, 2, -3 และหาเซตผัณ $\mathbb{Z}/R$

วิธีทำ: เราตรวจสอบคุณสมบัติทั้งสามของความสัมพันธ์สมมูล

สำหรับคุณสมบัติสะท้อน สำหรับทุก $a \in \mathbb{Z}$ จะได้ $|a| = |a|$ ดังนั้น $a R a$ ดังนั้น R เป็นความสัมพันธ์สะท้อน

สำหรับคุณสมบัติสมมาตร ถ้า $|a| = |b|$ แล้ว $|b| = |a|$ ดังนั้นถ้า $a R b$ แล้ว $b R a$ ดังนั้น R เป็นความสัมพันธ์สมมาตร

สำหรับคุณสมบัติถ่ายทอด ถ้า $|a| = |b|$ และ $|b| = |c|$ แล้ว $|a| = |c|$ ดังนั้นถ้า $a R b$ และ $b R c$ แล้ว $a R c$ ดังนั้น R เป็นความสัมพันธ์ถ่ายทอด

ดังนั้น R เป็นความสัมพันธ์สมมูล

สำหรับคลาสสมมูล $[0]_R = \{x \in \mathbb{Z} \mid |x| = |0|\} = \{x \in \mathbb{Z} \mid |x| = 0\} = \{0\}$ $[1]_R = \{x \in \mathbb{Z} \mid |x| = |1|\} = \{x \in \mathbb{Z} \mid |x| = 1\} = \{1, -1\}$ $[2]_R = \{x \in \mathbb{Z} \mid |x| = |2|\} = \{x \in \mathbb{Z} \mid |x| = 2\} = \{2, -2\}$ $[-3]_R = \{x \in \mathbb{Z} \mid |x| = |-3|\} = \{x \in \mathbb{Z} \mid |x| = 3\} = \{3, -3\}$ สังเกตว่า $[-3]_R = [3]_R$ เพราะ 3 และ -3 มีค่าสัมบูรณ์เท่ากัน

เซตผัณ $\mathbb{Z}/R = \{\{0\}, \{1, -1\}, \{2, -2\}, \{3, -3\}, \dots\}$ ซึ่งประกอบด้วย คลาส สมมูล ของจำนวนเต็มที่มีค่าสัมบูรณ์เท่ากัน แต่ละคลาสสมมูลมีสมาชิกสองตัวยกเว้น $[0]_R$ ที่มีสมาชิกตัวเดียว

หมายเหตุ 3.63 เมื่อพิสูจน์ว่า ความสัมพันธ์ไม่มี คุณสมบัติ บาง อย่าง ต้องหาตัวอย่างตรงข้ามที่ชัดเจน ตัวอย่างเช่น เมื่อพิสูจน์ว่า R ไม่เป็นความสัมพันธ์สมมาตร ต้องหา a, b ที่ $(a, b) \in R$ แต่ $(b, a) \notin R$

สรุป

ในบทนี้ เราได้ศึกษาเกี่ยวกับความสัมพันธ์ซึ่งเป็นแนวคิดพื้นฐานที่สำคัญในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ความสัมพันธ์คือเซตย่อยของผลคูณคาร์ทีเซียน และความสัมพันธ์แบบทวิภาคคือ

ความสัมพันธ์บนเซตเดียวกัน คุณสมบัติพื้นฐานของความสัมพันธ์ประกอบด้วยสะท้อน สมมาตร เป็นผลาด และถ่ายทอด ความสัมพันธ์สมมูลมีทั้งสามคุณสมบัติแรก และแบ่งเซตออกเป็นคลาสสมมูลที่ไม่ปะติดปะต่อกัน ความสัมพันธ์สมมาตรเชิงส่วนย่อยมีสะท้อน เป็นผลาด และถ่ายทอด ใช้ในการจัดลำดับสมาชิกในเซต การดำเนินการบนความสัมพันธ์ ได้แก่ การรวมและการผกผัน มีคุณสมบัติที่สำคัญในการวิเคราะห์ความสัมพันธ์ซับซ้อน การปิดคุณสมบัติประกอบด้วยการปิดสะท้อน การปิดสมมาตร และการปิดถ่ายทอด ความสัมพันธ์มีการใช้ในฐานะข้อมูลเชิงสัมพันธ์ การวิเคราะห์เครือข่ายสังคม การตรวจสอบความสมดุลของวงเล็บ และการวิเคราะห์การพึ่งพาของโปรแกรม

ความเข้าใจในความสัมพันธ์เป็นรากฐานของแนวคิดที่สำคัญมากมายในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ ได้แก่ ฟังก์ชันซึ่งเป็นความสัมพันธ์ชนิดพิเศษ กราฟซึ่งใช้ความสัมพันธ์ในการเชื่อมโยงโหนด และฐานข้อมูลเชิงสัมพันธ์ซึ่งใช้ความสัมพันธ์ในการจัดเก็บและค้นหาข้อมูล การเข้าใจความสัมพันธ์อย่างลึกซึ้งจะช่วยให้เข้าใจแนวคิดเหล่านี้ได้ดียิ่งขึ้น

หมายเหตุ 3.64 **Remark 3.3:** ความสัมพันธ์เป็นรากฐานของแนวคิดที่สำคัญมากมายในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ เช่น ฟังก์ชันเป็นความสัมพันธ์ชนิดพิเศษ กราฟใช้ความสัมพันธ์ในการเชื่อมโยงโหนด และฐานข้อมูลเชิงสัมพันธ์ใช้ความสัมพันธ์ในการจัดเก็บและค้นหาข้อมูล การเข้าใจความสัมพันธ์อย่างลึกซึ้งจะช่วยให้เข้าใจแนวคิดเหล่านี้ได้ดียิ่งขึ้น

แบบฝึกหัดท้ายบท

- กำหนดเซต $A = \{1, 2, 3, 4, 5\}$ และความสัมพันธ์ $R = \{(1, 1), (1, 2), (2, 3), (3, 4), (4, 5)\}$ จงหาโดเมนและเรนจ์ของ R การปิดสะท้อนของ R การปิดสมมาตรของ R และการปิดถ่ายทอดของ R
- พิจารณาความสัมพันธ์ต่อไปนี้บนเซตของจำนวนจริง $\mathbb{R}$ จงระบุคุณสมบัติของแต่ละความสัมพันธ์ ได้แก่ สะท้อน สมมาตร เป็นผลาด และถ่ายทอด สำหรับความสัมพันธ์ $a R b$ ก็ต่อเมื่อ $a = b$ $a R b$ ก็ต่อเมื่อ $a < b$ $a R b$ ก็ต่อเมื่อ $a \leq b + 1$ และ $a R b$ ก็ต่อเมื่อ $|a - b| \leq 1$
- จงพิสูจน์ว่าความสัมพันธ์ R บนเซตใดๆ เป็นความสัมพันธ์สมมูล ก็และก็ต่อเมื่อ R สามารถเขียนเป็นการรวมของคลาสสมมูลที่ไม่ปะติดปะต่อกัน
- กำหนดความสัมพันธ์ $R = \{(1, 2), (2, 1), (2, 3), (3, 2)\}$ บนเซต $\{1, 2, 3\}$ จงหา R^{-1} $R \circ R$ คุณสมบัติทั้ง 4 ของ R และความสัมพันธ์สมมูลที่ใกล้เคียงที่สุดที่ประกอบด้วย R
- พิจารณาความสัมพันธ์ R บนเซตของสตริงบนอักษร $\{0, 1\}$ โดย $s R t$ ก็ต่อเมื่อ s และ t มี

จำนวน 1 เท่ากัน จงพิสูจน์ว่า R เป็นความสัมพันธ์สมมูล หากคลาสสมมูลของ "001" "110" "000" และหาจำนวนคลาสสมมูลทั้งหมดสำหรับสตริงที่มีความยาว n

6. ให้ (A, R) เป็น Poset จงพิสูจน์ว่า (A, R^{-1}) ก็เป็น Poset เช่นกัน
7. จงอธิบายการประยุกต์ใช้ความสัมพันธ์ในโปรแกรมหนึ่งที่คุณรู้จัก โดยระบุเซตที่เกี่ยวข้อง ความสัมพันธ์ที่ใช้ คุณสมบัติของความสัมพันธ์นั้น และประโยชน์ที่ได้จากการใช้ความสัมพันธ์นี้
8. กำหนดความสัมพันธ์ R บนเซต $\{1, 2, 3, 4, 5, 6\}$ โดย $a R b$ ก็ต่อเมื่อ a หาร b ลงตัว จงพิสูจน์ว่า R เป็นความสัมพันธ์สมมาตรเชิงส่วนย่อย หาสมาชิกต่ำสุดและสมาชิกสูงสุดถ้ามี และหาสมาชิกน้อยที่สุดและสมาชิกมากที่สุดถ้ามี
9. ให้ R และ S เป็นความสัมพันธ์สมมูลบนเซต A จงพิสูจน์หรือหักล้างว่า $R \cap S$ เป็นความสัมพันธ์สมมูล และจงพิสูจน์หรือหักล้างว่า $R \cup S$ เป็นความสัมพันธ์สมมูล
10. จงเขียนโปรแกรมตรวจสอบว่าความสัมพันธ์ R บนเซต $\{1, 2, \dots, n\}$ มีคุณสมบัติใดต่อไปนี้ สะท้อน สมมาตร เป็นผลาด ถ่ายทอด สมมูล และสมมาตรเชิงส่วนย่อย

เอกสารอ้างอิง

Bibliography

- [1] Rosen, Kenneth H. *Discrete Mathematics and Its Applications*. 8th ed., McGraw-Hill, 2019.
- [2] Epp, Susanna S. *Discrete Mathematics with Applications*. 5th ed., Cengage Learning, 2019.
- [3] Grimaldi, Ralph P. *Discrete and Combinatorial Mathematics: An Applied Introduction*. 5th ed., Pearson, 2003.
- [4] Lean, Marcus. *Relations and Functions*. University of Sydney, 2019.

๑๑๔ การอ้างอิง

แผนการสอนประจำบทที่ 4

รหัสวิชา	4121701
ชื่อวิชา	คณิตศาสตร์สำหรับคอมพิวเตอร์
หน่วยกิต	3 (3-0-6)
บทที่	4
ชื่อบท	ระบบเลขฐาน (Number Bases)
จำนวนชั่วโมง	6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- แปลงตัวเลขระหว่างระบบเลขฐาน 2, 8, 10 และ 16 ได้อย่างถูกต้อง
- ทำการบวก ลบ คูณ หาร ตัวเลขในระบบเลขฐาน 2 ได้
- อธิบายการแทนค่าจำนวนลบในระบบดิจิทัล (Two's Complement) ได้
- อธิบายระบบเลขทศนิยมทวิ (Floating Point) เบื้องต้นได้
- ประยุกต์ใช้ความรู้เรื่องระบบเลขฐานในการทำงานกับข้อมูลคอมพิวเตอร์ได้

หัวข้อที่สอน

- ระบบเลขฐาน 10, 2, 8, 16
- การแปลงฐาน (Base Conversion)
- เลขคณิตฐานสอง (Binary Arithmetic)
- การแทนค่าจำนวนลบ: Two's Complement
- จำนวนทศนิยมทวิ (Floating Point Basics)

สื่อการสอน

- สไลด์ประกอบการสอน
- ตัวอย่างการคำนวณเลขฐาน
- แบบฝึกหัดท้ายบท

ระบบเลขฐาน (Number Bases)

วัตถุประสงค์การเรียนรู้

หลังจากศึกษาจบบทนี้ ผู้เรียนจะสามารถ:

1. อธิบายความหมายและความสำคัญของระบบเลขฐานต่างๆ ได้
2. แปลงเลขฐานสิบเป็นฐานอื่นและในทางกลับกันได้
3. ดำเนินการบวก ลบ คูณ หาร เลขฐานสองและฐานสิบหกได้
4. เข้าใจการแทนจำนวนลบในฐานสอง (1's complement และ 2's complement)
5. อธิบายมาตรฐาน IEEE 754 สำหรับการแทนเลขทศนิยมในคอมพิวเตอร์ได้
6. เข้าใจการแทนข้อมูลในคอมพิวเตอร์ (ASCII, Unicode)

บทนำ

ระบบเลขฐาน (Number Base System) คือวิธีการเขียนและแทนจำนวนโดยใช้ตัวเลขหลายตัวมาประกอบกัน ค่าของแต่ละตำแหน่งจะขึ้นอยู่กับฐาน (base หรือ radix) ของระบบนั้นๆ ระบบเลขฐานมีความสำคัญอย่างยิ่งในวิทยาการคอมพิวเตอร์ เนื่องจากคอมพิวเตอร์ทำงานด้วยสัญญาณไฟฟ้าสองสถานะ (เปิด/ปิด) จึงใช้ระบบเลขฐานสอง (binary) เป็นภาษาพื้นฐาน

ในชีวิตประจำวัน มนุษย์ใช้ระบบเลขฐานสิบ (decimal) ซึ่งมีรากฐานจากการนับนิ้วมือทั้งสิบนิ้ว อารยธรรมโบราณใช้ระบบฐานที่แตกต่างกัน เช่น ชาวบาบิโลเนียใช้ฐาน 60 (sexagesimal) ซึ่งยังคงใช้อยู่ในการวัดเวลา (60 วินาที 60 นาที) และการวัดมุม (360 องศา)

ระบบเลขฐานสิบ (Decimal System)

ระบบเลขฐานสิบใช้ตัวเลข 10 ตัว คือ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 และมีฐานเท่ากับ 10

4.3.1 ค่าประจำตำแหน่ง

ในระบบเลขฐานสิบ ค่าของตัวเลขในแต่ละตำแหน่งจะคูณด้วย 10^n โดย n คือตำแหน่งจากขวาสุด (เริ่มที่ 0)

ทฤษฎีบท 4.1 จำนวนใดๆ ในระบบเลขฐานสิบสามารถเขียนในรูปผลบวกของค่าประจำตำแหน่งได้:

$$d_n d_{n-1} \dots d_1 d_0 . d_{-1} d_{-2} \dots d_{-m} = \sum_{i=-m}^n d_i \times 10^i$$

โดย d_i คือเลขโดด (digit) ในแต่ละตำแหน่ง และ 10^i คือค่าประจำตำแหน่ง (place value)

ตัวอย่าง 4.2 4729 ในฐานสิบ หมายความว่า:

$$4729 = 4 \times 10^3 + 7 \times 10^2 + 2 \times 10^1 + 9 \times 10^0 = 4000 + 700 + 20 + 9$$

สำหรับจำนวนทศนิยม 53.847:

$$53.847 = 5 \times 10^1 + 3 \times 10^0 + 8 \times 10^{-1} + 4 \times 10^{-2} + 7 \times 10^{-3}$$

ระบบเลขฐานสอง (Binary System)

ระบบเลขฐานสองใช้ตัวเลขเพียง 2 ตัว คือ 0 และ 1 และมีฐานเท่ากับ 2 คอมพิวเตอร์ใช้ระบบนี้ เพราะวงจรอิเล็กทรอนิกส์สามารถแสดงสองสถานะได้ง่าย (เช่น มีไฟ/ไม่มีไฟ, ปิด/เปิด)

ทฤษฎีบท 4.3 จำนวนในระบบเลขฐานสองสามารถเขียนในรูปผลบวกของ 2^i ได้:

$$(b_n b_{n-1} \dots b_1 b_0)_2 = \sum_{i=0}^n b_i \times 2^i$$

โดย $b_i \in \{0, 1\}$

หมายเหตุ 4.4 เลขฐานสอง $(1101)_2$ อ่านว่า "หนึ่งหนึ่งศูนย์หนึ่งฐานสอง" ไม่ใช่ "พันหนึ่งร้อยเอ็ด" เพราะแต่ละหลักมีค่าประจำตำแหน่งเป็น 2^n ไม่ใช่ 10^n

ตัวอย่าง 4.5 แปลง $(1101)_2$ เป็นฐานสิบ:

$$\begin{aligned} (1101)_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 \\ &= 8 + 4 + 0 + 1 = 13 \end{aligned}$$

แปลง $(10110)_2$ เป็นฐานสิบ:

$$\begin{aligned} (10110)_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\ &= 16 + 0 + 4 + 2 + 0 = 22 \end{aligned}$$

ทฤษฎีบท 4.6 สำหรับจำนวนทศนิยมในฐานสอง:

$$(0.b_1b_2\dots b_m)_2 = \sum_{i=1}^m b_i \times 2^{-i}$$

ตัวอย่าง 4.7 แปลง $(0.101)_2$ เป็นฐานสิบ:

$$\begin{aligned}(0.101)_2 &= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} \\ &= \frac{1}{2} + 0 + \frac{1}{8} = 0.5 + 0.125 = 0.625\end{aligned}$$

แปลง $(10.011)_2$ เป็นฐานสิบ:

$$\begin{aligned}(10.011)_2 &= 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} \\ &= 2 + 0 + 0 + 0.25 + 0.125 = 2.375\end{aligned}$$

ระบบเลขฐานแปด (Octal System)

ระบบเลขฐานแปดใช้ตัวเลข 8 ตัว คือ 0, 1, 2, 3, 4, 5, 6, 7 และมีฐานเท่ากับ 8

ทฤษฎีบท 4.8 ความสัมพันธ์ระหว่างฐานสองและฐานแปด: กลุ่มตัวเลขฐานสอง 3 ตัว สามารถแทนด้วยตัวเลขฐานแปด 1 ตัว เนื่องจาก $2^3 = 8$

ตัวอย่าง 4.9 แปลง $(110101)_2$ เป็นฐานแปด:

จัดกลุ่มตัวเลขฐานสองครั้งละ 3 ตัวจากขวาสุด:

$$110101_2 = 110\ 101_2$$

แปลงแต่ละกลุ่ม:

$$1. \ 110_2 = 6_8$$

$$2. \ 101_2 = 5_8$$

$$\text{ดังนั้น } (110101)_2 = (65)_8$$

ตัวอย่าง 4.10 แปลง $(472)_8$ เป็นฐานสิบ:

$$\begin{aligned}(472)_8 &= 4 \times 8^2 + 7 \times 8^1 + 2 \times 8^0 \\ &= 4 \times 64 + 7 \times 8 + 2 \times 1 \\ &= 256 + 56 + 2 = 314\end{aligned}$$

หมายเหตุ 4.11 ฐานแปดเคยใช้กันแพร่หลายในยุคแรกของคอมพิวเตอร์เนื่องจากการแปลงระหว่างฐาน

สองและฐานแปดทำได้ง่าย ปัจจุบันฐานสิบหกมีการใช้มากกว่า

ระบบเลขฐานสิบหก (Hexadecimal System)

ระบบเลขฐานสิบหกใช้ตัวเลขและตัวอักษร 16 ตัว คือ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F โดย $A = 10, B = 11, C = 12, D = 13, E = 14, F = 15$ และมีฐานเท่ากับ 16

ทฤษฎีบท 4.12 ความสัมพันธ์ระหว่างฐานสองและฐานสิบหก: กลุ่มตัวเลขฐานสอง 4 ตัว สามารถแทนด้วยตัวเลขฐานสิบหก 1 ตัว เนื่องจาก $2^4 = 16$

ตารางที่ 12 ตารางเปรียบเทียบฐานสิบ ฐานสอง และฐานสิบหก

ฐานสิบ	ฐานสอง (4 หลัก)	ฐานสิบหก
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

ตัวอย่าง 4.13 แปลง $(11010110)_2$ เป็นฐานสิบหก:

จัดกลุ่มตัวเลขฐานสองครั้งละ 4 ตัวจากขวาสุด:

$$11010110_2 = 1101 \ 0110_2$$

แปลงแต่ละกลุ่ม:

$$1. \ 1101_2 = D_{16}$$

$$2. \ 0110_2 = 6_{16}$$

$$\text{ดังนั้น } (11010110)_2 = (D6)_{16}$$

ตัวอย่าง 4.14 แปลง $(3A.F)_{16}$ เป็นฐานสิบ:

$$\begin{aligned} (3A.F)_{16} &= 3 \times 16^1 + 10 \times 16^0 + 15 \times 16^{-1} \\ &= 3 \times 16 + 10 \times 1 + 15 \times \frac{1}{16} \\ &= 48 + 10 + 0.9375 = 58.9375 \end{aligned}$$

4.7 แปลงฐานทั่วไป

4.7.1 การแปลงจากฐานใดๆ เป็นฐานสิบ

วิธีที่ง่ายที่สุดในการแปลงจากฐานใดๆ เป็นฐานสิบคือการกระจายตามค่าประจำตำแหน่ง

ทฤษฎีบท 4.15 ถ้า $(d_n d_{n-1} \dots d_1 d_0)_b$ เป็นจำนวนในฐาน b แล้ว:

$$(d_n d_{n-1} \dots d_1 d_0)_b = \sum_{i=0}^n d_i \times b^i$$

ตัวอย่าง 4.16 แปลง $(2B3)_{16}$ เป็นฐานสิบ:

$$\begin{aligned} (2B3)_{16} &= 2 \times 16^2 + 11 \times 16^1 + 3 \times 16^0 \\ &= 2 \times 256 + 11 \times 16 + 3 \times 1 \\ &= 512 + 176 + 3 = 691 \end{aligned}$$

แปลง $(57)_8$ เป็นฐานสิบ:

$$(57)_8 = 5 \times 8^1 + 7 \times 8^0 = 40 + 7 = 47$$

4.7.2 การแปลงจากฐานสิบเป็นฐานใดๆ

ใช้วิธีการต่อเนื่อง (successive division) โดยหารจำนวนด้วยฐานที่ต้องการแล้วบันทึกเศษ

ทฤษฎีบท 4.17 ขั้นตอนการแปลงจำนวนเต็มจากฐานสิบเป็นฐาน b :

1. หารจำนวนด้วย b จดเศษไว้
2. นำผลหารไปหารด้วย b อีก จดเศษ
3. ทำซ้ำจนกว่าผลหารเป็น 0
4. เศษที่ได้อ่านจากล่างขึ้นบนคือคำตอบ

ตัวอย่าง 4.18 แปลง 45 ฐานสิบเป็นฐานสอง:

หาร	ผลหาร	เศษ
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

อ่านเศษจากล่างขึ้นบน: 101101_2

$$\begin{aligned} \text{ตรวจสอบ: } 101101_2 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 32 + 0 + 8 + 4 + 0 + 1 = 45 \checkmark \end{aligned}$$

ตัวอย่าง 4.19 แปลง 158 ฐานสิบเป็นฐานแปด:

หาร	ผลหาร	เศษ
$158 \div 8$	19	6
$19 \div 8$	2	3
$2 \div 8$	0	2

ดังนั้น $158_{10} = 236_8$

$$\text{ตรวจสอบ: } 236_8 = 2 \times 8^2 + 3 \times 8^1 + 6 \times 8^0 = 128 + 24 + 6 = 158 \checkmark$$

ตัวอย่าง 4.20 แปลง 254_3 ฐานสิบเป็นฐานสิบหก:

หาร	ผลหาร	เศษ
$2543 \div 16$	158	15 (F)
$158 \div 16$	9	14 (E)
$9 \div 16$	0	9

ดังนั้น $2543_{10} = 9EF_{16}$

$$\begin{aligned}\text{ตรวจสอบ: } 9EF_{16} &= 9 \times 16^2 + 14 \times 16^1 + 15 \times 16^0 \\ &= 9 \times 256 + 14 \times 16 + 15 = 2304 + 224 + 15 = 2543\checkmark\end{aligned}$$

4.8 แปลงจำนวนทศนิยมระหว่างฐาน

4.8.1 การแปลงส่วนทศนิยมจากฐานสิบเป็นฐานอื่น

ใช้วิธีคูณต่อเนื่อง (successive multiplication) กับส่วนทศนิยม

ทฤษฎีบท 4.21 ขั้นตอนการแปลงส่วนทศนิยมจากฐานสิบเป็นฐาน b :

1. คูณส่วนทศนิยมด้วย b
2. บันทึกผลคูณที่เป็นจำนวนเต็ม (0 หรือ 1 สำหรับฐานสอง) เป็นเลขหลักถัดไป
3. นำส่วนทศนิยมของผลคูณไปคูณต่อ
4. ทำซ้ำจนกว่าส่วนทศนิยมเป็น 0 หรือได้จำนวนหลักตามต้องการ

ตัวอย่าง 4.22 แปลง 0.625 ฐานสิบเป็นฐานสอง:

คูณ	ผลลัพธ์	เลขหลัก
0.625×2	1.25	1
0.25×2	0.5	0
0.5×2	1.0	1

ดังนั้น $0.625_{10} = 0.101_2$

$$\text{ตรวจสอบ: } 0.101_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 0.5 + 0 + 0.125 = 0.625\checkmark$$

ตัวอย่าง 4.23 แปลง 0.7 ฐานสิบเป็นฐานสอง (แสดง 8 หลัก):

คูณ	ผลลัพธ์	เลขหลัก
0.7×2	1.4	1
0.4×2	0.8	0
0.8×2	1.6	1
0.6×2	1.2	1
0.2×2	0.4	0
0.4×2	0.8	0
0.8×2	1.6	1
0.6×2	1.2	1

สังเกตว่า 0.4 ปรากฏซ้ำอีกแล้ว ดังนั้น $0.7_{10} \approx 0.10110011_2$ (repeating)

ค่าจริงคือ $0.7 = 0.\overline{1011}_2$ (repeating block "1011")

หมายเหตุ 4.24 ไม่ใช่ทุกจำนวนทศนิยมที่แปลงได้ลงตัวในฐานใดฐานหนึ่ง การแปลง 0.1_{10} เป็นฐานสองจะได้ทศนิยมซ้ำไม่รู้จบ นี่คือสาเหตุของปัญหา floating-point error ในคอมพิวเตอร์

ตัวอย่าง 4.25 แปลง 0.3125 ฐานสิบเป็นฐานแปด:

คูณ	ผลลัพธ์	เลขหลัก
0.3125×8	2.5	2
0.5×8	4.0	4

ดังนั้น $0.3125_{10} = 0.24_8$

ตรวจสอบ: $0.24_8 = 2 \times 8^{-1} + 4 \times 8^{-2} = \frac{2}{8} + \frac{4}{64} = 0.25 + 0.0625 = 0.3125 \checkmark$

4.9.1 การบวกและลบเลขฐานสอง

4.9.1 การบวกเลขฐานสอง

การบวกเลขฐานสองมีกฎพื้นฐานดังนี้:

ทฤษฎีบท 4.26 การบวกเลขฐานสอง:

- $0 + 0 = 0$

- $0 + 1 = 1$

$$3. \ 1 + 0 = 1$$

$$4. \ 1 + 1 = 10 \text{ (เป็น 0 ทด 1)}$$

$$5. \ 1 + 1 + 1 = 11 \text{ (เป็น 1 ทด 1)}$$

ตัวอย่าง 4.27 บวก $(1011)_2$ กับ $(1101)_2$:

$$\begin{array}{r} 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

ขั้นตอน:

1. $1 + 1 = 10$ คือ เขียน 0 ทด 1
2. $1 + 0 + 1 = 10$ คือ เขียน 0 ทด 1
3. $0 + 1 + 1 = 10$ คือ เขียน 0 ทด 1
4. $1 + 1 + 1 = 11$ คือ เขียน 1 ทด 1
5. ทด 1 ลงมาเป็น 1

ดังนั้น $1011_2 + 1101_2 = 11000_2$

ตรวจสอบ: $1011_2 = 11_{10}$, $1101_2 = 13_{10}$, $11000_2 = 24_{10}$ ซึ่ง $11 + 13 = 24 \checkmark$

ตัวอย่าง 4.28 บวก $(1111)_2$ กับ $(0001)_2$:

$$\begin{array}{r} 1111 \\ + 0001 \\ \hline 10000 \end{array}$$

ดังนั้น $1111_2 + 0001_2 = 10000_2$ (เป็น $2^4 = 16$)

4.9.2 การลบเลขฐานสอง

การลบเลขฐานสองใช้หลักการยืม (borrowing)

ทฤษฎีบท 4.29 การลบเลขฐานสอง:

1. $0 - 0 = 0$
2. $1 - 0 = 1$
3. $1 - 1 = 0$
4. $0 - 1 = 1$ (ยืม 1 จากหลักซ้าย ทำให้หลักซ้ายลดลง 1)

ตัวอย่าง 4.30 ลบ $(1100)_2$ ด้วย $(0101)_2$:

$$\begin{array}{r} 1100 \\ - 0101 \\ \hline 0111 \end{array}$$

ขั้นตอน:

1. $0 - 1$ คือ ยืม 1 จากหลักซ้าย ได้ $10 - 1 = 1$
2. 0 (หลักที่ยืมไปแล้วลดเหลือ 0) $- 0 = 0$
3. $1 - 0 = 1$

ดังนั้น $1100_2 - 0101_2 = 0111_2$

ตรวจสอบ: $1100_2 = 12_{10}$, $0101_2 = 5_{10}$, $0111_2 = 7_{10}$ ซึ่ง $12 - 5 = 7 \checkmark$

4.10 การคูณและการหารเลขฐานสอง

4.10.1 การคูณเลขฐานสอง

การคูณเลขฐานสองมีกฎง่ายๆ เพราะตัวคูณมีแค่ 0 กับ 1

ทฤษฎีบท 4.31 การคูณเลขฐานสอง:

1. $0 \times 0 = 0$
2. $0 \times 1 = 0$
3. $1 \times 0 = 0$

$$4. \quad 1 \times 1 = 1$$

ตัวอย่าง 4.32 คูณ $(1011)_2$ ด้วย $(1101)_2$:

$$\begin{array}{r}
 1011 \\
 \times 1101 \\
 \hline
 1011 \quad (1011 \times 1) \\
 0000 \quad (1011 \times 0 \text{ ทดไป 1 หลัก}) \\
 1011 \quad (1011 \times 1 \text{ ทดไป 2 หลัก}) \\
 1011 \quad (1011 \times 1 \text{ ทดไป 3 หลัก}) \\
 \hline
 10001111
 \end{array}$$

ดังนั้น $1011_2 \times 1101_2 = 10001111_2$

ตรวจสอบ: $1011_2 = 11_{10}$, $1101_2 = 13_{10}$, $10001111_2 = 143_{10}$ ซึ่ง $11 \times 13 = 143 \checkmark$

หมายเหตุ 4.33 การคูณเลขฐานสองในคอมพิวเตอร์ใช้วิธี shift-and-add ซึ่งประสิทธิภาพกว่าการคูณแบบธรรมดา

4.4.1 แทนจำนวนเต็มลบในฐานสอง

ในคอมพิวเตอร์ จำนวนเต็มลบต้องมีการแทนที่พิเศษ เนื่องจากไม่มีเครื่องหมายลบในฐานสอง มี 3 วิธีหลักที่ใช้กัน:

4.11.1 Signed Magnitude (Sign-Magnitude)

ใช้หลักซ้ายสุดเป็น bit สำหรับเครื่องหมาย (0 = บวก, 1 = ลบ) และหลักที่เหลือเป็นค่าสัมบูรณ์

ทฤษฎีบท 4.34 ในระบบ signed magnitude ขนาด n bits:

1. bit ซ้ายสุด = เครื่องหมาย (0 บวก, 1 ลบ)
2. bits ที่เหลือ = ค่าสัมบูรณ์
3. ช่วง: $-(2^{n-1} - 1)$ ถึง $+(2^{n-1} - 1)$
4. มี 0 สองแบบ คือ $+0$ และ -0

ตัวอย่าง 4.35 ในระบบ signed magnitude 8 bits:

1. $+45 = 00101101_2$

2. $-45 = 10101101_2$

หมายเหตุ 4.36 วิธี signed magnitude ไม่ค่อยใช้ในปัจุบันเพราะมี 0 สองแบบและวงจรลำบาก

4.11.2 1's Complement

การแทนจำนวนลบโดยการกลับ bit ทุก bit ของจำนวนบวก

ทฤษฎีบท 4.37 ในระบบ 1's complement ขนาด n bits:

1. ค่าบวก เหมือน unsigned
2. ค่าลบ คือ กลับ bit ทุก bit ของค่าบวก
3. ช่วง: $-(2^{n-1} - 1)$ ถึง $+(2^{n-1} - 1)$
4. ยังมี 0 สองแบบ คือ $+0 = 00000000$, $-0 = 11111111$

ตัวอย่าง 4.38 ในระบบ 1's complement 8 bits:

1. $+45 = 00101101_2$

2. $-45 = 11010010_2$ (กลับ bit ของ 00101101)

การบวกใน 1's complement ต้องมี end-around carry:

$$00101101 + 11010010 = 11111111 = -0$$

4.11.3 2's Complement

วิธีที่ใช้กันมากที่สุดในคอมพิวเตอร์ปัจจุบัน

ทฤษฎีบท 4.39 ในระบบ 2's complement ขนาด n bits:

1. ค่าบวก เหมือน unsigned
2. ค่าลบ คือ กลับ bit ทุก bit แล้วบวก 1 หรือเทียบเท่ากับ $2^n - |x|$
3. ช่วง: -2^{n-1} ถึง $+(2^{n-1} - 1)$
4. มี 0 แบบเดียว คือ 00000000
5. การบวกปกติใช้ได้เลยโดยไม่ต้อง special case

ตัวอย่าง 4.40 ในระบบ 2's complement 8 bits:

$$1. +45 = 00101101_2$$

$$2. -45 = 11010011_2 \text{ (กลับ bit แล้วบวก 1)}$$

$$\text{ตรวจสอบ: } 00101101 + 11010011 = 100000000_2 = 00000000_2 \text{ (ลบ bit ซ้ายสุด)}$$

ตัวอย่าง 4.41 หาค่าของ 11010011_2 ในระบบ 2's complement 8 bits:

bit ซ้ายสุดคือ 1 ดังนั้นเป็นจำนวนลบ หาค่าสัมบูรณ์โดยกลับ bit แล้วบวก 1:

$$11010011 \rightarrow 00101100 \rightarrow 00101101_2 = 45$$

$$\text{ดังนั้น } 11010011_2 = -45$$

ทฤษฎีบท 4.42 ช่วงค่าในระบบ 2's complement n bits:

1. ค่าบวกมากที่สุด คือ $2^{n-1} - 1$ (เช่น 127 ใน 8 bits)
2. ค่าลบน้อยที่สุด คือ -2^{n-1} (เช่น -128 ใน 8 bits)
3. ไม่มี -0 เพราะ -128 ใช้ 10000000_2

การบวกและลบในฐานสิบหก

การบวกในฐานสิบหกทำได้โดยบวกตามปกติแล้วทดเมื่อผลลัพธ์เกิน 15 (F)

ทฤษฎีบท 4.43 ถ้าผลบวกของแต่ละหลักเกิน F_{16} ให้ทด 1 ไปหลักซ้าย และเขียนผลลัพธ์ลบ 16

ตัวอย่าง 4.44 บวก $(3A5)_{16}$ กับ $(1F7)_{16}$:

$$\begin{array}{r} 3A5 \\ + 1F7 \\ \hline 59C \end{array}$$

ขั้นตอน:

1. $5 + 7 = 12$ เขียน C ไม่ทด
2. $A + F = 10 + 15 = 25 = 16 + 9$ เขียน 9 ทด 1
3. $3 + 1 + 1 = 5$ เขียน 5

$$\text{ดังนั้น } 3A5_{16} + 1F7_{16} = 59C_{16}$$

$$\text{ตรวจสอบ: } 3A5_{16} = 933_{10}, 1F7_{16} = 503_{10}, 59C_{16} = 1436_{10} \text{ ซึ่ง } 933 + 503 = 1436 \checkmark$$

ตัวอย่าง 4.45 ลบ $(9F2)_{16}$ ด้วย $(3A4)_{16}$:

$$\begin{array}{r} 9F2 \\ - 3A4 \\ \hline 64E \end{array}$$

ขั้นตอน:

1. $2 - 4$ ยืม 1 คือ $12 - 4 = 8$ เขียน 8
2. $F - 1$ (ยืมไปแล้วลดเหลือ E) $- A = 14 - 10 = 4$ เขียน 4
3. $9 - 3 = 6$ เขียน 6

$$\text{ดังนั้น } 9F2_{16} - 3A4_{16} = 64E_{16}$$

$$\text{ตรวจสอบ: } 9F2_{16} = 2546_{10}, 3A4_{16} = 932_{10}, 64E_{16} = 1614_{10} \text{ ซึ่ง } 2546 - 932 = 1614 \checkmark$$

IEEE 754: การแทนทศนิยมในคอมพิวเตอร์

มาตรฐาน IEEE 754 (Institute of Electrical and Electronics Engineers) กำหนดวิธีการแทนจำนวนทศนิยมแบบ floating-point ในคอมพิวเตอร์ ปัจจุบันใช้กันแพร่หลายในฮาร์ดแวร์และซอฟต์แวร์คอมพิวเตอร์ทั่วโลก

4.13.1 โครงสร้างของ IEEE 754 Single Precision (32 bits)

ทฤษฎีบท 4.46 จำนวน single precision 32 bits แบ่งเป็น 3 ส่วน:

1. **Sign (S)** คือ 1 bit เครื่องหมาย (0 = บวก, 1 = ลบ)
2. **Exponent (E)** คือ 8 bits เลขยกกำลัง (biased)
3. **Fraction (F)** คือ 23 bits ส่วนเศษ (mantissa)

$$\text{ค่าจริงคือ: } (-1)^S \times 1.F \times 2^{E-127}$$

ตัวอย่าง 4.47 จำนวน 3.14 ในระบบ IEEE 754 single precision:

1. แปลงเป็นฐานสอง: $3.14 \approx 11.00100011\dots_2$

2. Normalize: $1.100100011... \times 2^1$
3. Sign = 0
4. Exponent = $1 + 127 = 128 = 10000000_2$
5. Fraction = 100100011... (23 bits)

4.13.2 Special Values ใน IEEE 754

ทฤษฎีบท 4.48 ค่าพิเศษใน IEEE 754:

1. **Zero** คือ Exponent = 0, Fraction = 0
2. **Denormalized** คือ Exponent = 0, Fraction $\neq 0$ (very small numbers)
3. **Infinity** คือ Exponent = 255, Fraction = 0
4. **NaN (Not a Number)** คือ Exponent = 255, Fraction $\neq 0$

ตัวอย่าง 4.49 ใน single precision:

1. $+0 = 00000000_00000000000000000000000_2$
2. $-0 = 10000000_000000000000000000000000_2$
3. $+\infty = 01111111_000000000000000000000000_2$
4. $-\infty = 11111111_000000000000000000000000_2$

หมายเหตุ 4.50 **Machine Epsilon** คือค่าที่เล็กที่สุดที่บวกกับ 1 แล้วได้ค่าต่างจาก 1 ใน floating-point สำหรับ single precision คือ $2^{-23} \approx 1.19 \times 10^{-7}$ นี่คือขอบเขตของความแม่นยำในการคำนวณทศนิยม

4.14 แทนข้อมูลในคอมพิวเตอร์

คอมพิวเตอร์แทนทุกสิ่งด้วย bits ข้อมูลประเภทต่างๆ เช่น ตัวอักษร รูปภาพ เสียง ล้วนแปลงเป็นตัวเลขก่อนประมวลผล

4.14.1 ASCII (American Standard Code for Information Interchange)

ASCII ใช้ 7 bits (หรือ 8 bits รวม parity) แทนอักขระ 128 ตัว

ทฤษฎีบท 4.51 ASCII แบ่งออกเป็น:

1. 0–31 คือ ตัวควบคุม (control characters)
2. 32–47 คือ ช่องว่างและเครื่องหมายพิเศษ (., ", #, ...)
3. 48–57 คือ ตัวเลข 0–9
4. 65–90 คือ ตัวอักษรภาษาอังกฤษพิมพ์ใหญ่ A–Z
5. 97–122 คือ ตัวอักษรภาษาอังกฤษพิมพ์เล็ก a–z

ตารางที่ 13 ตาราง ASCII Code บางส่วน

อักขระ	ASCII (decimal)	ASCII (hex)
'A'	65	41
'B'	66	42
'0'	48	30
'/'	47	2F
'a'	97	61
'{'	123	7B

ตัวอย่าง 4.52 คำ "Hi" ใน ASCII hexadecimal:

1. 'H' = 0x48
2. 'i' = 0x69
3. ดังนั้น "Hi" = 0x4869

หมายเหตุ 4.53 ASCII มีข้อจำกัดในการแทนอักขระภาษาอื่น เช่น ภาษาไทย จีน ญี่ปุ่น จึงพัฒนา Unicode ขึ้นมาเพื่อแก้ปัญหานี้

4.14.2 Unicode

Unicode ใช้หลาย bytes แทนอักขระ รองรับอักขระจากทั่วโลก

ทฤษฎีบท 4.54 Unicode มีหลาย encoding:

1. **UTF-8** คือ 1–4 bytes ใช้กันแพร่หลายที่สุดบนอินเทอร์เน็ต
2. **UTF-16** คือ 2–4 bytes ใช้ใน Windows และ Java
3. **UTF-32** คือ 4 bytes ต่ออักขระ

ตัวอย่าง 4.55 อักขระไทยใน UTF-8:

1. 'ก' (U+0E01) = 0xE0 0xB8 0x81 (3 bytes)
 2. 'ข' (U+0E02) = 0xE0 0xB8 0x82 (3 bytes)
- ดังนั้นคำว่า "กข" = 0xE0B881E0B882

แบบฝึกหัด

4.15.1 ส่วนที่ 1: การแปลงฐาน

1. แปลงต่อไปนี้เป็นฐานสิบ:
 - 1.1. $(1011)_2$
 - 1.2. $(10010)_2$
 - 1.3. $(72)_8$
 - 1.4. $(5F)_{16}$
 - 1.5. $(1001.101)_2$
 - 1.6. $(3A.8)_{16}$
2. แปลงต่อไปนี้เป็นฐานสอง:
 - 2.1. 45_{10}
 - 2.2. 127_{10}
 - 2.3. 0.625_{10}
 - 2.4. 0.8125_{10}
 - 2.5. 73_{10}
 - 2.6. 255_{10}

3. แปลงต่อไปนี้เป็นฐานแปดและฐานสิบหก:

3.1. $(11010110)_2$

3.2. $(10101010)_2$

4. แปลงต่อไปนี้เป็นฐานสิบ:

4.1. $(A1B)_{16}$

4.2. $(777)_8$

4.3. $(3CD.2A)_{16}$

4.15.2 ส่วนที่ 2: การดำเนินการฐานสอง

1. จงหา:

1.1. $(1011)_2 + (1101)_2$

1.2. $(10001)_2 - (111)_2$

1.3. $(1101)_2 \times (101)_2$

1.4. $(11000)_2 \div (100)_2$

2. จงหา 2's complement ของ $(45)_{10}$ ใน 8 bits

3. กำหนดให้ในระบบ 2's complement 8 bits จงหาค่าของ:

3.1. 01101001_2

3.2. 10110110_2

4.15.3 ส่วนที่ 3: การดำเนินการฐานสิบหก

1. จงหา:

1.1. $(3A7)_{16} + (1B9)_{16}$

1.2. $(FF)_{16} - (A1)_{16}$

1.3. $(2F)_{16} \times (4)_{16}$

4.15.4 ส่วนที่ 4: การแทนข้อมูลและ IEEE 754

1. เขียนคำต่อไปนี้ในรูป hexadecimal (ASCII):

1.1. "Hi"

1.2. "OK"

2. อธิบายข้อดีข้อเสียของ signed magnitude, 1's complement และ 2's complement
3. จำนวน 1.0 ใน IEEE 754 single precision มีค่า hex คือ 3F800000 จงอธิบายว่าแต่ละ field คืออะไร

เฉลยแบบฝึกหัด

4.16.1 เฉลยส่วนที่ 1

1.
 - 1.1. $1011_2 = 1 \times 8 + 1 \times 2 + 1 \times 1 = 11$
 - 1.2. $10010_2 = 1 \times 16 + 1 \times 2 = 18$
 - 1.3. $72_8 = 7 \times 8 + 2 = 58$
 - 1.4. $5F_{16} = 5 \times 16 + 15 = 95$
 - 1.5. $1001.101_2 = 9.625$
 - 1.6. $3A.8_{16} = 58.5$
2.
 - 2.1. $45 = 101101_2$
 - 2.2. $127 = 1111111_2$
 - 2.3. $0.625 = 0.101_2$
 - 2.4. $0.8125 = 0.1101_2$
 - 2.5. $73 = 1001001_2$
 - 2.6. $255 = 11111111_2$

สรุป

ในบทนี้เราได้ศึกษาระบบเลขฐานอย่างครอบคลุม ซึ่งประกอบด้วย:

1. ระบบเลขฐานสิบ คือ ระบบที่มนุษย์ใช้ทั่วไป ฐาน 10
2. ระบบเลขฐานสอง คือ ภาษาของคอมพิวเตอร์ ฐาน 2
3. ระบบเลขฐานแปด คือ ฐาน 8 ง่ายต่อการแปลงกับฐานสอง
4. ระบบเลขฐานสิบหก คือ ฐาน 16 ใช้แทนข้อมูลในคอมพิวเตอร์
5. การแปลงฐาน คือ ทารต่อเนื่องสำหรับจำนวนเต็ม คู่ต่อเนื่องสำหรับทศนิยม

6. การดำเนินการฐานสองและฐานสิบหก คือ บวก ลบ คูณ หาร
7. การแทนจำนวนลบ คือ Signed Magnitude, 1's complement, 2's complement
8. IEEE 754 คือ มาตรฐานการแทนทศนิยมในคอมพิวเตอร์
9. ASCII และ Unicode คือ การแทนอักขระในคอมพิวเตอร์

ความเข้าใจในระบบเลขฐานเป็นพื้นฐานที่จำเป็นสำหรับการศึกษาวิทยาการคอมพิวเตอร์ โดยเฉพาะอย่างยิ่งในเรื่อง สถาปัตยกรรมคอมพิวเตอร์ (Computer Architecture) การเขียนโปรแกรมระดับต่ำ (Low-level Programming) และการเข้ารหัสข้อมูล (Data Encoding)

Bibliography

- [1] Stallings, William. *Computer Organization and Architecture*. 11th ed., Pearson, 2020.
- [2] Patterson, David A., and John L. Hennessy. *Computer Organization and Design*. 5th ed., Morgan Kaufmann, 2017.
- [3] Rosen, Kenneth H. *Discrete Mathematics and Its Applications*. 8th ed., McGraw-Hill, 2019.
- [4] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, IEEE, 2019.
- [5] The Unicode Consortium. *The Unicode Standard*. Version 14.0, Unicode Inc., 2021.

ผลการอ้างอิง

แผนการสอนประจำบทที่ 5

รหัสวิชา	4121701
ชื่อวิชา	คณิตศาสตร์สำหรับคอมพิวเตอร์
หน่วยกิต	3 (3-0-6)
บทที่	5
ชื่อบท	เมทริกซ์และดีเทอร์มิแนนต์ (Matrices & Determinants)
จำนวนชั่วโมง	6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- อธิบายโครงสร้างและประเภทของเมทริกซ์ได้
- ทำการบวก ลบ คูณเมทริกซ์ได้อย่างถูกต้อง
- คำนวณดีเทอร์มิแนนต์ของเมทริกซ์ขนาด 2×2 และ 3×3 ได้
- หาเมทริกซ์ผกผัน (Inverse Matrix) ได้
- ประยุกต์ใช้เมทริกซ์ในการแก้ระบบสมการเชิงเส้น และงานด้านกราฟิกคอมพิวเตอร์ได้

หัวข้อที่สอน

- ความรู้เบื้องต้นเกี่ยวกับเมทริกซ์
- การดำเนินการบนเมทริกซ์
- ดีเทอร์มิแนนต์ (Determinants)
- เมทริกซ์ผกผัน (Inverse Matrix)
- การประยุกต์ใช้เมทริกซ์ในคอมพิวเตอร์

สื่อการสอน

- สไลด์ประกอบการสอน
- ตัวอย่างการคำนวณ
- แบบฝึกหัดท้ายบท

เมทริกซ์และดีเทอร์มิแนนต์ (Matrices & Determinants)

วัตถุประสงค์การเรียนรู้

หลังจากศึกษาจบบทนี้ ผู้เรียนจะสามารถ:

- อธิบายความหมายและความสำคัญของเมทริกซ์ รวมถึงประเภทต่างๆ ของเมทริกซ์ได้
- ดำเนินการระหว่างเมทริกซ์ (บวก ลบ คูณ สลับเปลี่ยน) และอธิบายความหมายของผลลัพธ์ได้
- คำนวณดีเทอร์มิแนนต์ของเมทริกซ์ขนาดต่างๆ โดยวิธีการต่างๆ ได้
- หาอินเวอร์สของเมทริกซ์และประยุกต์ใช้ในการแก้ระบบสมการเชิงเส้นได้
- พิสูจน์และประยุกต์ใช้สมบัติของดีเทอร์มิแนนต์ได้
- อธิบายการประยุกต์ของเมทริกซ์ในวิทยาการคอมพิวเตอร์ เช่น คอมพิวเตอร์กราฟิกส์ และการเรียนรู้ของเครื่องได้

บทนำ

เมทริกซ์ (Matrix) เป็นโครงสร้างข้อมูลทางคณิตศาสตร์ที่ใช้จัดระเบียบข้อมูลในรูปแบบตารางสองมิติ เมทริกซ์ประกอบด้วยแถว (rows) และคอลัมน์ (columns) โดยแต่ละช่องเก็บค่าที่เรียกว่า สมาชิก (element) หรือ ค่าประจำตำแหน่ง (entry) ของเมทริกซ์

เมทริกซ์มีความสำคัญอย่างยิ่งในวิทยาการคอมพิวเตอร์และคณิตศาสตร์ประยุกต์ เนื่องจากเป็นพื้นฐานของ คอมพิวเตอร์กราฟิกส์ (การแปลงภาพ 2D และ 3D) การเรียนรู้ของเครื่อง (Neural Networks) การเข้ารหัส (Hill Cipher) และการแก้ระบบสมการเชิงเส้น

ประวัติศาสตร์ของเมทริกซ์เริ่มต้นในช่วงต้นศตวรรษที่ 19 โดย Arthur Cayley และ William Rowan Hamilton ถึงแม้ว่าแนวคิดเรื่องดีเทอร์มิแนนต์จะมีมาก่อนหน้านี้โดย mathematicians ชาวญี่ปุ่นและยุโรป เช่น Seki Kowa และ Gottfried Leibniz

นิยามพื้นฐานของเมทริกซ์

5.3.1 นิยามของเมทริกซ์

ทฤษฎีบท 5.1 **เมทริกซ์ (Matrix)** คือ การจัดเรียงข้อมูลในรูปแบบตารางสี่เหลี่ยมผืนผ้า ซึ่งประกอบด้วย m แถว และ n คอลัมน์ เมทริกซ์ขนาด $m \times n$ จะมีสมาชิกทั้งหมด $m \cdot n$ ตัว เรียกว่า **เมทริกซ์ขนาด $m \times n$** เขียนแทนด้วย $A_{m \times n}$ หรือ $A = [a_{ij}]$ โดยที่ a_{ij} คือสมาชิกที่อยู่ในแถวที่ i และคอลัมน์ที่ j

ตัวอย่าง 5.2 เมทริกซ์ $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ มีขนาด 2×3 (2 แถว 3 คอลัมน์)

สมาชิก $a_{11} = 1, a_{12} = 2, a_{13} = 3, a_{21} = 4, a_{22} = 5, a_{23} = 6$

ตัวอย่าง 5.3 เมทริกซ์ $B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$ มีขนาด 3×2 (3 แถว 2 คอลัมน์)

เมทริกซ์ขนาด $m \times n$ สามารถเขียนในรูปทั่วไปได้ดังนี้:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

5.3.2 ประเภทของเมทริกซ์

ทฤษฎีบท 5.4 เมทริกซ์มีหลายประเภทที่สำคัญ:

1. **เมทริกซ์ศูนย์ (Zero Matrix) $O_{m \times n}$** คือเมทริกซ์ที่ทุกสมาชิกเป็น 0

$$O = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

2. **เมทริกซ์แถว (Row Matrix)** คือเมทริกซ์ที่มีเพียง 1 แถว ($1 \times n$)

$$R = \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix}$$

3. เมทริกซ์คอลัมน์ (Column Matrix) คือเมทริกซ์ที่มีเพียง 1 คอลัมน์ ($m \times 1$)

$$C = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

4. เมทริกซ์จัตุรัส (Square Matrix) คือเมทริกซ์ที่มีจำนวนแถวเท่ากับจำนวนคอลัมน์ ($n \times n$) เรียก n ว่า **ขนาด** (order) ของเมทริกซ์

$$S = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \text{ มีขนาด } 2 \times 2$$

5. เมทริกซ์ทแยงมุม (Diagonal Matrix) คือเมทริกซ์จัตุรัสที่สมาชิกนอกเส้นทแยงมุมหลักเป็น 0 ทั้งหมด

$$D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

6. เมทริกซ์เอกลักษณ์ (Identity Matrix) I_n คือเมทริกซ์ทแยงมุมที่สมาชิกบนเส้นทแยงมุมหลักเป็น 1 ทั้งหมด

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

7. เมทริกซ์สามเหลี่ยมบน (Upper Triangular Matrix) คือเมทริกซ์จัตุรัสที่สมาชิกใต้เส้นทแยงมุมหลักเป็น 0

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$$

8. เมทริกซ์สามเหลี่ยมล่าง (Lower Triangular Matrix) คือเมทริกซ์จัตุรัสที่สมาชิกเหนือเส้นทแยงมุมหลักเป็น 0

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 4 & 5 & 6 \end{bmatrix}$$

9. เมทริกซ์สมมาตร (Symmetric Matrix) คือเมทริกซ์จัตุรัสที่มีคุณสมบัติ $a_{ij} = a_{ji}$

สำหรับทุก i, j

$$S = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$$

10. เมทริกซ์เส้นทแยงมุมเฉียง (Skew-Symmetric / Antisymmetric Matrix) คือเม

ทริกซ์จัตุรัสที่มีคุณสมบัติ $a_{ij} = -a_{ji}$ สำหรับทุก i, j และ $a_{ii} = 0$

$$K = \begin{bmatrix} 0 & 2 & -3 \\ -2 & 0 & 4 \\ 3 & -4 & 0 \end{bmatrix}$$

หมายเหตุ 5.5 เมทริกซ์สมมาตรและเมทริกซ์เส้นทแยงมุมเฉียงมีความสำคัญในการประยุกต์ เช่น ในทฤษฎีสถิติ เมทริกซ์ความแปรปรวน (covariance matrix) เป็นเมทริกซ์สมมาตร และในฟิสิกส์ เมทริกซ์เส้นทแยงมุมเฉียงใช้ในการแสดงปริมาณเวกเตอร์

5.4 ดำเนินการระหว่างเมทริกซ์

5.4.1 ความเท่ากันของเมทริกซ์

ทฤษฎีบท 5.6 เมทริกซ์ $A = [a_{ij}]$ และ $B = [b_{ij}]$ มีขนาด $m \times n$ เท่ากัน ก็ต่อเมื่อ $a_{ij} = b_{ij}$ สำหรับทุก $i = 1, 2, \dots, m$ และ $j = 1, 2, \dots, n$

ตัวอย่าง 5.7 ถ้า $\begin{bmatrix} x & 2 \\ 3 & y \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ แล้ว $x = 1$ และ $y = 4$

5.4.2 การบวกและการลบเมทริกซ์

ทฤษฎีบท 5.8 กำหนดเมทริกซ์ $A = [a_{ij}]$ และ $B = [b_{ij}]$ ขนาด $m \times n$:

- การบวกเมทริกซ์ $A + B = [a_{ij} + b_{ij}]$
- การลบเมทริกซ์ $A - B = [a_{ij} - b_{ij}]$

หมายเหตุ: เมทริกซ์สองเมทริกซ์จะบวกหรือลบกันได้ก็ต่อเมื่อมีขนาดเท่ากัน

ตัวอย่าง 5.9

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} - \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

ทฤษฎีบท 5.10 (สมบัติของการบวกเมทริกซ์) สำหรับเมทริกซ์ A, B, C ขนาด $m \times n$:

1. $A + B = B + A$ (Commutative Law)
2. $(A + B) + C = A + (B + C)$ (Associative Law)
3. $A + O = A$ (Identity Law, O คือเมทริกซ์ศูนย์)
4. $A + (-A) = O$ (Inverse Law)
5. $A - B = A + (-B)$

หมายเหตุ 5.11 การลบเมทริกซ์ไม่มีสมบัติการสลับที่ กล่าวคือ $A - B \neq B - A$ โดยทั่วไป

5.4.3 การคูณเมทริกซ์ด้วยสเกลาร์

ทฤษฎีบท 5.12 กำหนดเมทริกซ์ $A = [a_{ij}]$ ขนาด $m \times n$ และสเกลาร์ c (ค่าคงที่จำนวนจริง) การคูณสเกลาร์ cA คือเมทริกซ์ $[c \cdot a_{ij}]$

ตัวอย่าง 5.13

$$3 \cdot \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

ทฤษฎีบท 5.14 (สมบัติของการคูณสเกลาร์) สำหรับเมทริกซ์ A, B และสเกลาร์ c, d :

1. $c(A + B) = cA + cB$ (Distributive Law)
2. $(c + d)A = cA + dA$ (Distributive Law)
3. $c(dA) = (cd)A$ (Associative Law)
4. $1A = A$ (Identity Law)
5. $0A = O$ (Zero Law)

5.4.4 การคูณเมทริกซ์ด้วยเมทริกซ์

การคูณเมทริกซ์เป็นการดำเนินการที่ซับซ้อนกว่าการบวกและการคูณสเกลาร์ สมาชิกของเมทริกซ์ ผลคูณคำนวณจากผลรวมของผลคูณ

ทฤษฎีบท 5.15 กำหนดเมทริกซ์ $A = [a_{ij}]$ ขนาด $m \times n$ และเมทริกซ์ $B = [b_{jk}]$ ขนาด $n \times p$ การคูณ AB จะได้เมทริกซ์ $C = [c_{ik}]$ ขนาด $m \times p$ โดยที่:

$$c_{ik} = a_{i1}b_{1k} + a_{i2}b_{2k} + a_{i3}b_{3k} + \cdots + a_{in}b_{nk} = \sum_{j=1}^n a_{ij}b_{jk}$$

หมายเหตุ: การคูณ AB กระทำได้ก็ต่อเมื่อจำนวนคอลัมน์ของ A เท่ากับจำนวนแถวของ B (เรียกว่า **conformable**)

ตัวอย่าง 5.16

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$AB = \begin{bmatrix} (1)(5) + (2)(7) & (1)(6) + (2)(8) \\ (3)(5) + (4)(7) & (3)(6) + (4)(8) \end{bmatrix} = \begin{bmatrix} 5 + 14 & 6 + 16 \\ 15 + 28 & 18 + 32 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

ตัวอย่าง 5.17

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = [1(4) + 2(5) + 3(6)] = [32]$$

$$\begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 8 & 12 \\ 5 & 10 & 15 \\ 6 & 12 & 18 \end{bmatrix}$$

สังเกตว่า $AB \neq BA$ ในกรณีทั่วไป!

หมายเหตุ 5.18 การคูณเมทริกซ์ไม่มีสมบัติการสลับที่ กล่าวคือ $AB \neq BA$ โดยทั่วไป นอกจากนี้ $AB = O$ ไม่ได้หมายความว่า $A = O$ หรือ $B = O$

ทฤษฎีบท 5.19 (สมบัติของการคูณเมทริกซ์) สำหรับเมทริกซ์ A, B, C ที่ขนาดเหมาะสม และสเกลาร์ c :

1. $(AB)C = A(BC)$ (Associative Law)
2. $A(B + C) = AB + AC$ (Left Distributive Law)
3. $(A + B)C = AC + BC$ (Right Distributive Law)
4. $AI = IA = A$ (Identity Law, I คือเมทริกซ์เอกลักษณ์)
5. $c(AB) = (cA)B = A(cB)$ (Scalar Multiplication)
6. $(AB)^T = B^T A^T$ (Transpose of Product)

ตัวอย่าง 5.20 พิสูจน์ $(AB)^T = B^T A^T$

ให้ $A = [a_{ij}]$, $B = [b_{jk}]$, และ $(AB)^T = [c_{ki}]$ โดย $c_{ki} = (AB)_{ik} = \sum_j a_{ij} b_{jk}$

แต่ $(AB)^T$ ที่ตำแหน่ง (k, i) คือ $c_{ki} = \sum_j a_{ij} b_{jk}$

ในขณะที่ $B^T A^T$ ที่ตำแหน่ง (k, i) คือ $\sum_j b_{kj}^T \cdot a_{ji}^T = \sum_j b_{jk} \cdot a_{ij}$

ดังนั้น $(AB)^T = B^T A^T$

5.4.5 เมทริกซ์สลับเปลี่ยน (Transpose)

ทฤษฎีบท 5.21 เมทริกซ์สลับเปลี่ยน (Transpose) ของเมทริกซ์ $A = [a_{ij}]$ ขนาด $m \times n$ คือเมทริกซ์ $A^T = [a_{ji}]$ ขนาด $n \times m$ โดยการสลับแถวเป็นคอลัมน์

ตัวอย่าง 5.22

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

ทฤษฎีบท 5.23 (สมบัติของ Transpose) สำหรับเมทริกซ์ A, B :

1. $(A^T)^T = A$
2. $(A + B)^T = A^T + B^T$
3. $(cA)^T = cA^T$
4. $(AB)^T = B^T A^T$

ทฤษฎีบท 5.24 เมทริกซ์จัตุรัส A เป็น:

- สมมาตร (Symmetric) ก็ต่อเมื่อ $A^T = A$
- เส้นทแยงมุมเฉียง (Skew-Symmetric) ก็ต่อเมื่อ $A^T = -A$

ตัวอย่าง 5.25 เมทริกซ์สมมาตร: $A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$ เพราะ $A^T = A$

เมทริกซ์เส้นทแยงมุมเฉียง: $A = \begin{bmatrix} 0 & 2 & -3 \\ -2 & 0 & 4 \\ 3 & -4 & 0 \end{bmatrix}$ เพราะ $A^T = -A$

ทฤษฎีบท 5.26 ทุกเมทริกซ์จัตุรัส A สามารถเขียนได้ในรูปผลบวกของเมทริกซ์สมมาตรและเมทริกซ์เส้นทแยงมุมเฉียง:

$$A = \frac{1}{2}(A + A^T) + \frac{1}{2}(A - A^T)$$

โดย $\frac{1}{2}(A + A^T)$ เป็นเมทริกซ์สมมาตร และ $\frac{1}{2}(A - A^T)$ เป็นเมทริกซ์เส้นทแยงมุมเฉียง

ดีเทอร์มิแนนต์ (Determinants)

ดีเทอร์มิแนนต์เป็นค่าจำเพาะ (scalar value) ที่คำนวณได้จากเมทริกซ์จัตุรัส ดีเทอร์มิแนนต์มีความสำคัญในการกำหนดคุณสมบัติต่างๆ ของเมทริกซ์ เช่น การหาอินเวอร์ส และการแก้ระบบสมการเชิงเส้น

5.5.1 นิยามของดีเทอร์มิแนนต์

ทฤษฎีบท 5.27 ดีเทอร์มิแนนต์ (Determinant) ของเมทริกซ์จัตุรัส A ขนาด $n \times n$ เขียนแทนด้วย $\det(A)$ หรือ $|A|$

ทฤษฎีบท 5.28 สำหรับเมทริกซ์ขนาด 1×1 : $\det([a]) = a$

ทฤษฎีบท 5.29 สำหรับเมทริกซ์ขนาด 2×2 :

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

ตัวอย่าง 5.30

$$\det \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = (1)(4) - (2)(3) = 4 - 6 = -2$$

ทฤษฎีบท 5.31 สำหรับเมทริกซ์ขนาด 3×3 สามารถคำนวณได้โดย กฎของ Sarrus:

$$\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bfg + cdh - ceg - bdi - afh$$

ตัวอย่าง 5.32

$$\begin{aligned} \det \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} &= (1)(5)(9) + (2)(6)(7) + (3)(4)(8) - (3)(5)(7) - (2)(4)(9) - (1)(6)(8) \\ &= 45 + 84 + 96 - 105 - 72 - 48 = 0 \end{aligned}$$

หมายเหตุ 5.33 กฎของ Sarrus ใช้ได้เฉพาะกับเมทริกซ์ขนาด 3×3 เท่านั้น สำหรับเมทริกซ์ขนาดใหญ่มากกว่า

ต้องใช้วิธีการอื่น เช่น cofactor expansion หรือการลดรูป (row reduction)

5.5.2 Cofactor Expansion

สำหรับเมทริกซ์ขนาด $n \times n$ สามารถคำนวณดีเทอร์มิแนนต์ได้โดยใช้การกระจาย cofactor ตามแถวหรือคอลัมน์ใดก็ได้

ทฤษฎีบท 5.34 กำหนดเมทริกซ์ $A = [a_{ij}]$ ขนาด $n \times n$ **minor** ของ a_{ij} คือดีเทอร์มิแนนต์ของเมทริกซ์ย่อยที่ได้จากการลบแถว i และคอลัมน์ j เขียนแทนด้วย M_{ij}

Cofactor ของ a_{ij} คือ $C_{ij} = (-1)^{i+j}M_{ij}$

ดีเทอร์มิแนนต์ของ A สามารถคำนวณได้โดยการกระจายตามแถวที่ i :

$$\det(A) = a_{i1}C_{i1} + a_{i2}C_{i2} + \cdots + a_{in}C_{in}$$

หรือการกระจายตามคอลัมน์ที่ j :

$$\det(A) = a_{1j}C_{1j} + a_{2j}C_{2j} + \cdots + a_{nj}C_{nj}$$

ตัวอย่าง 5.35 คำนวณ $\det \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ โดยการกระจายตามแถวแรก

$$M_{11} = \det \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix} = 45 - 48 = -3, C_{11} = (-1)^{1+1}(-3) = -3$$

$$M_{12} = \det \begin{pmatrix} 4 & 6 \\ 7 & 9 \end{pmatrix} = 36 - 42 = -6, C_{12} = (-1)^{1+2}(-6) = 6$$

$$M_{13} = \det \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} = 32 - 35 = -3, C_{13} = (-1)^{1+3}(-3) = -3$$

$$\det(A) = 1(-3) + 2(6) + 3(-3) = -3 + 12 - 9 = 0$$

ตัวอย่าง 5.36 คำนวณ $\det \begin{pmatrix} 2 & 1 & 3 \\ 0 & 4 & 1 \\ 5 & 6 & 0 \end{pmatrix}$ โดยการกระจายตามคอลัมน์ที่ 3

$$M_{13} = \det \begin{pmatrix} 0 & 4 \\ 5 & 6 \end{pmatrix} = 0 - 20 = -20, C_{13} = (-1)^{1+3}(-20) = -20$$

$$M_{23} = \det \begin{pmatrix} 2 & 1 \\ 5 & 6 \end{pmatrix} = 12 - 5 = 7, C_{23} = (-1)^{2+3}(7) = -7$$

$$M_{33} = \det \begin{pmatrix} 2 & 1 \\ 0 & 4 \end{pmatrix} = 8 - 0 = 8, C_{33} = (-1)^{3+3}(8) = 8$$

$$\det(A) = 3(-20) + 1(-7) + 0(8) = -60 - 7 + 0 = -67$$

หมายเหตุ 5.37 ในการคำนวณดีเทอร์มิแนนต์ ควรเลือกแถวหรือคอลัมน์ที่มีศูนย์มากที่สุด เพื่อลดจำนวนการคำนวณ ในตัวอย่างข้างต้น การกระจายตามคอลัมน์ที่ 3 มีสมาชิก $a_{33} = 0$ ทำให้มีเพียง 2 cofactor ที่ต้องคำนวณ

5.5.3 สมบัติของดีเทอร์มิแนนต์

ทฤษฎีบท 5.38 สมบัติสำคัญของดีเทอร์มิแนนต์:

1. $\det(I) = 1$ โดยที่ I คือเมทริกซ์เอกลักษณ์
2. ถ้ามีแถวหรือคอลัมน์เป็นศูนย์ทั้งหมด $\Rightarrow \det(A) = 0$
3. ถ้าสลับแถวสองแถว $\Rightarrow$ ดีเทอร์มิแนนต์เปลี่ยนเครื่องหมาย
4. ถ้าคูณแถวด้วยสเกลาร์ $c \Rightarrow \det(cA) = c^n \det(A)$ สำหรับเมทริกซ์ $n \times n$
5. ถ้าแถวหนึ่งเป็นผลรวมของสองแถว $\Rightarrow$ ดีเทอร์มิแนนต์แยกได้
6. $\det(AB) = \det(A) \det(B)$ (Multiplicative Property)
7. $\det(A^T) = \det(A)$
8. เมทริกซ์สามเหลี่ยม: $\det(A) = a_{11} \cdot a_{22} \cdots a_{nn}$ (ผลคูณของสมาชิกบนเส้นทแยงมุมหลัก)

หลัก)

ตัวอย่าง 5.39

$$\det \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{pmatrix} = 2 \times 3 \times 5 = 30$$

ตัวอย่าง 5.40 พิสูจน์ $\det(AB) = \det(A) \det(B)$ สำหรับเมทริกซ์ 2×2

$$\text{ให้ } A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, B = \begin{pmatrix} e & f \\ g & h \end{pmatrix}$$

$$AB = \begin{pmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{pmatrix}$$

$$\begin{aligned} \det(AB) &= (ae + bg)(cf + dh) - (af + bh)(ce + dg) = ae \cdot cf + ae \cdot dh + bg \cdot cf + bg \cdot dh - \\ &af \cdot ce - af \cdot dg - bh \cdot ce - bh \cdot dg = ae \cdot cf - af \cdot ce + ae \cdot dh - bh \cdot ce + bg \cdot cf - af \cdot dg + bg \cdot dh - bh \cdot dg \\ &= a \cdot e \cdot (cf - dh) - b \cdot g \cdot (af - ce) + \dots \end{aligned}$$

ใช้ความจริงที่ว่าทุกพจน์จัดรูปได้เป็น $(ad - bc)(eh - fg) = \det(A) \det(B)$

หมายเหตุ 5.41 $\det(A + B) \neq \det(A) + \det(B)$ โดยทั่วไป

ทฤษฎีบท 5.42 เมทริกซ์จัตุรัส A จะมีอินเวอร์สก็ต่อเมื่อ $\det(A) \neq 0$ (เรียกว่า **nonsingular** หรือ **invertible**)

อินเวอร์สของเมทริกซ์ (Matrix Inverse)

อินเวอร์สของเมทริกซ์เปรียบเสมือนตัวผกผันการคูณในระบบจำนวน กล่าวคือ $A \cdot A^{-1} = A^{-1} \cdot$

$$A = I$$

5.6.1 นิยามของอินเวอร์ส

ทฤษฎีบท 5.43 กำหนดเมทริกซ์จัตุรัส A ขนาด $n \times n$ ถ้ามีเมทริกซ์ B ที่ทำให้ $AB = BA = I_n$ แล้ว B คือ **อินเวอร์ส** (inverse) ของ A เขียนแทนด้วย A^{-1} และเรียก A ว่าเป็น **เมทริกซ์ที่มีอินเวอร์ส** (invertible หรือ nonsingular)

ทฤษฎีบท 5.44 เมทริกซ์จัตุรัส A มีอินเวอร์สก็ต่อเมื่อ $\det(A) \neq 0$

ตัวอย่าง 5.45 สำหรับเมทริกซ์ 2×2 :

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad \text{เมื่อ } ad - bc \neq 0$$

$$\text{ตรวจสอบ: } \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^{-1} = \frac{1}{1(4) - 2(3)} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \frac{1}{-2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$$

$$\text{ตรวจสอบ: } \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

ตัวอย่าง 5.46 หาอินเวอร์สของ $A = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$

ใช้สูตร $A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$

$$\det(A) = 2(1 - 0) - 1(0 - 1) + 1(0 - 1) = 2 + 1 - 1 = 2$$

$$C_{ij} \text{ (cofactor): } C_{11} = +\det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = 1 \quad C_{12} = -\det \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} = -(-1) = 1 \quad C_{13} =$$

$$+\det \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = -1$$

$$C_{21} = -\det \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = -1 \quad C_{22} = +\det \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} = 1 \quad C_{23} = -\det \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix} = -(-1) =$$

1

$$C_{31} = +\det \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = 0 \quad C_{32} = -\det \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix} = -2 \quad C_{33} = +\det \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix} = 2$$

$$\text{adj}(A) = \begin{pmatrix} 1 & 1 & -1 \\ -1 & 1 & -2 \\ 0 & 1 & 2 \end{pmatrix}^T = \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 1 \\ -1 & -2 & 2 \end{pmatrix}$$

$$A^{-1} = \frac{1}{2} \begin{pmatrix} 1 & -1 & 0 \\ 1 & 1 & 1 \\ -1 & -2 & 2 \end{pmatrix} = \begin{pmatrix} 0.5 & -0.5 & 0 \\ 0.5 & 0.5 & 0.5 \\ -0.5 & -1 & 1 \end{pmatrix}$$

ทฤษฎีบท 5.47 (สมบัติของอินเวอร์ส) สำหรับเมทริกซ์ invertible A, B :

$$1. (A^{-1})^{-1} = A$$

$$2. (AB)^{-1} = B^{-1}A^{-1}$$

(สังเกตการสลับที่)

$$3. (A^T)^{-1} = (A^{-1})^T$$

$$4. \det(A^{-1}) = \frac{1}{\det(A)}$$

หมายเหตุ 5.48 $(AB)^{-1} = B^{-1}A^{-1}$ ไม่ใช่ $A^{-1}B^{-1}$ ต้องระวังลำดับการคูณ!

ตัวอย่าง 5.49 พิสูจน์ $(AB)^{-1} = B^{-1}A^{-1}$

ต้องแสดงว่า $(AB)(B^{-1}A^{-1}) = I$ และ $(B^{-1}A^{-1})(AB) = I$

$$(AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AIA^{-1} = AA^{-1} = I$$

$$(B^{-1}A^{-1})(AB) = B^{-1}(A^{-1}A)B = B^{-1}IB = B^{-1}B = I$$

ดังนั้น $B^{-1}A^{-1}$ เป็นอินเวอร์สของ AB

ระบบสมการเชิงเส้น (Systems of Linear Equations)

ระบบสมการเชิงเส้นเป็นเครื่องมือพื้นฐานในคณิตศาสตร์ประยุกต์ เมทริกซ์ช่วยให้การแก้ระบบสมการเชิงเส้นมีประสิทธิภาพมากขึ้น

5.7.1 การเขียนระบบสมการในรูปเมทริกซ์

ทฤษฎีบท 5.50 ระบบสมการเชิงเส้น n ตัวแปร m สมการ สามารถเขียนในรูปเมทริกซ์ได้ดังนี้:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

หรือเขียนย่อว่า $Ax = b$ โดยที่ A คือเมทริกซ์สัมประสิทธิ์, x คือเวกเตอร์ตัวแปร, และ b คือเวกเตอร์ค่าคงที่

ตัวอย่าง 5.51 ระบบสมการ:

$$\begin{cases} x + 2y + 3z = 14 \\ 2x + 3y + z = 10 \\ 3x + y + 2z = 14 \end{cases}$$

เขียนในรูปเมทริกซ์ได้เป็น:

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 3 & 1 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 14 \\ 10 \\ 14 \end{pmatrix}$$

5.7.2 การแก้ระบบสมการโดยใช้อินเวอร์สเมทริกซ์

ทฤษฎีบท 5.52 ถ้า A มีอินเวอร์ส ($Ax = b$) แล้ว $x = A^{-1}b$

ตัวอย่าง 5.53 แก้ระบบสมการโดยใช้อินเวอร์ส:

$$\begin{cases} x + 2y = 5 \\ 3x + 4y = 11 \end{cases}$$

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, b = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$$

$$A^{-1} = \frac{1}{1(4) - 2(3)} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \frac{1}{-2} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$$

$$x = A^{-1}b = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix} \begin{pmatrix} 5 \\ 11 \end{pmatrix} = \begin{pmatrix} -10 + 11 \\ 7.5 - 5.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

ดังนั้น $x = 1, y = 2$

5.7.3 การแก้ระบบสมการโดยใช้กฎของคราเมอร์ (Cramer's Rule)

ทฤษฎีบท 5.54 (กฎของคราเมอร์) สำหรับระบบสมการเชิงเส้น $Ax = b$ โดยที่ A ขนาด $n \times n$ และ $\det(A) \neq 0$ คำตอบคือ:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

โดยที่ A_i คือเมทริกซ์ที่ได้จากการแทนคอลัมน์ที่ i ของ A ด้วยเวกเตอร์ b

ตัวอย่าง 5.55 แก้ระบบโดยใช้กฎของคราเมอร์:

$$\begin{cases} x + 2y = 5 \\ 3x + 4y = 11 \end{cases}$$

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, b = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$$

$$\det(A) = 1(4) - 2(3) = -2$$

$$A_1 = \begin{pmatrix} 5 & 2 \\ 11 & 4 \end{pmatrix}, x_1 = \frac{\det(A_1)}{\det(A)} = \frac{5(4) - 2(11)}{-2} = \frac{20 - 22}{-2} = \frac{-2}{-2} = 1$$

$$A_2 = \begin{pmatrix} 1 & 5 \\ 3 & 11 \end{pmatrix}, x_2 = \frac{\det(A_2)}{\det(A)} = \frac{1(11)-5(3)}{-2} = \frac{11-15}{-2} = \frac{-4}{-2} = 2$$

ดังนั้น $x = 1, y = 2$

หมายเหตุ 5.56 กฎของคราเมอร์มีประสิทธิภาพในการแสดงทฤษฎี แต่ในทางปฏิบัติการคำนวณมีความซับซ้อน $O(n!)$ สำหรับระบบขนาดใหญ่ วิธี Gaussian elimination มีประสิทธิภาพมากกว่าคือ $O(n^3)$

5.7.4 การแก้ระบบสมการโดยวิธีการกำจัดแบบเกาส์ (Gaussian Elimination)

วิธีการกำจัดแบบเกาส์ใช้การแปลงเมทริกซ์ขยาย (augmented matrix) ให้เป็นรูปแบบ row echelon form

ทฤษฎีบท 5.57 **Augmented Matrix** ของระบบ $Ax = b$ คือ $[A|b]$ ขนาด $n \times (n+1)$

การดำเนินการแถว (Row Operations):

1. สลับแถว ($R_i \leftrightarrow R_j$)
2. คูณแถวด้วยสเกลาร์ไม่เป็นศูนย์ (cR_i)
3. แทน R_i ด้วย $R_i + cR_j$ ($i \neq j$)

การดำเนินการเหล่านี้ไม่เปลี่ยนคำตอบของระบบสมการ

ตัวอย่าง 5.58 แก้ระบบสมการโดยวิธี Gaussian elimination:

$$\begin{cases} x + 2y + z = 9 \\ 2x + 5y + 3z = 23 \\ x + 3z = 7 \end{cases}$$

Augmented matrix:

$$\begin{bmatrix} 1 & 2 & 1 & | & 9 \\ 2 & 5 & 3 & | & 23 \\ 1 & 0 & 3 & | & 7 \end{bmatrix}$$

$$R_2 \leftarrow R_2 - 2R_1: \begin{bmatrix} 1 & 2 & 1 & | & 9 \\ 0 & 1 & 1 & | & 5 \\ 1 & 0 & 3 & | & 7 \end{bmatrix}$$

$$R_3 \leftarrow R_3 - R_1: \begin{bmatrix} 1 & 2 & 1 & | & 9 \\ 0 & 1 & 1 & | & 5 \\ 0 & -2 & 2 & | & -2 \end{bmatrix}$$

$$R_3 \leftarrow R_3 + 2R_2: \begin{bmatrix} 1 & 2 & 1 & | & 9 \\ 0 & 1 & 1 & | & 5 \\ 0 & 0 & 4 & | & 8 \end{bmatrix}$$

จาก $4z = 8 \Rightarrow z = 2$

จาก $y + z = 5 \Rightarrow y + 2 = 5 \Rightarrow y = 3$

จาก $x + 2y + z = 9 \Rightarrow x + 6 + 2 = 9 \Rightarrow x = 1$

ดังนั้น $x = 1, y = 3, z = 2$

ทฤษฎีบท 5.59 ระบบสมการ $Ax = b$ มี:

- คำตอบเดียว ก็ต่อเมื่อ $\det(A) \neq 0$ (เมทริกซ์ A invertible)
- ไม่มีคำตอบ หรือ มีคำตอบไม่จำกัด ก็ต่อเมื่อ $\det(A) = 0$ (เมทริกซ์ A singular)

ทฤษฎีบท 5.60 ถ้า $\det(A) = 0$:

- ถ้า b อยู่ใน column space ของ $A \Rightarrow$ มีคำตอบไม่จำกัด
- ถ้า b ไม่อยู่ใน column space ของ $A \Rightarrow$ ไม่มีคำตอบ

๕.๘ประยุกต์ในวิทยาการคอมพิวเตอร์

5.8.1 การแปลงในคอมพิวเตอร์กราฟิกส์ (Computer Graphics)

เมทริกซ์ใช้ในการแปลงภาพกราฟิก 2 มิติ และ 3 มิติ อย่างมีประสิทธิภาพ

ตัวอย่าง 5.61 (การแปลง 2 มิติ) การแปลงจุด (x, y) ในระบบ 2 มิติ สามารถเขียนในรูปเมทริกซ์ได้ดังนี้:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = T \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

โดยใช้ homogeneous coordinates

การเลื่อน (Translation): $T = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$

การขยาย/หด (Scaling): $S = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$

การหมุน (Rotation): $R = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$

การแปลงผสมสามารถทำได้โดยการคูณเมทริกซ์: $P' = T \cdot R \cdot S \cdot P$

ตัวอย่าง 5.62 หมุนจุด $(1, 0)$ รอบจุดกำเนิด 90° :

$$P = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, R = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P' = R \cdot P = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$

จุดใหม่คือ $(0, 1)$

5.8.2 การเข้ารหัส Hill Cipher

ทฤษฎีบท 5.63 Hill Cipher เป็นระบบเข้ารหัสแบบบล็อกที่ใช้เมทริกซ์ ข้อความธรรมดาถูกแปลงเป็นเวกเตอร์ตัวเลข แล้วคูณด้วยเมทริกซ์กุญแจ K ผลลัพธ์คือข้อความเข้ารหัส

การเข้ารหัส: $C = K \cdot P \pmod{26}$

การถอดรหัส: $P = K^{-1} \cdot C \pmod{26}$

ตัวอย่าง 5.64 ใช้เมทริกซ์กุญแจ $K = \begin{pmatrix} 3 & 2 \\ 1 & 1 \end{pmatrix}$ เข้ารหัสข้อความ "HI"

$H = 7, I = 8$ ($A=0, B=1, \dots, Z=25$)

$$P = \begin{pmatrix} 7 \\ 8 \end{pmatrix}, C = K \cdot P = \begin{pmatrix} 3 & 2 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 7 \\ 8 \end{pmatrix} = \begin{pmatrix} 37 \\ 15 \end{pmatrix} \pmod{26} = \begin{pmatrix} 11 \\ 15 \end{pmatrix}$$

$$C = 11 = L, C = 15 = P$$

ดังนั้น "HI" เข้ารหัสเป็น "LP"

หมายเหตุ: การถอดรหัสต้องใช้ K^{-1} ซึ่งมีอยู่ก็ต่อเมื่อ $\det(K) \neq 0 \pmod{26}$ นั่นคือ $\gcd(\det(K), 26) = 1$

5.8.3 เมทริกซ์ในการเรียนรู้ของเครื่อง

ทฤษฎีบท 5.65 โมเดล Linear Regression สามารถเขียนในรูปเมทริกซ์ได้:

$$y = X\beta + \epsilon$$

โดยที่:

- X คือ design matrix ขนาด $n \times p$
- β คือเวกเตอร์ของ parameters ขนาด $p \times 1$
- y คือเวกเตอร์ของ target values ขนาด $n \times 1$
- ϵ คือเวกเตอร์ของ errors ขนาด $n \times 1$

ค่า β ที่เหมาะสมที่สุดคำนวณได้จาก:

$$\beta = (X^T X)^{-1} X^T y$$

ตัวอย่าง 5.66 สำหรับข้อมูล 3 ตัวอย่าง 2 features: $X = \begin{pmatrix} 1 & 2 \\ 1 & 4 \\ 1 & 6 \end{pmatrix}, y = \begin{pmatrix} 3 \\ 5 \\ 7 \end{pmatrix}$

$$X^T X = \begin{pmatrix} 3 & 12 \\ 12 & 56 \end{pmatrix}, \det(X^T X) = 3(56) - 12(12) = 168 - 144 = 24 \neq 0$$

$$(X^T X)^{-1} = \frac{1}{24} \begin{pmatrix} 56 & -12 \\ -12 & 3 \end{pmatrix} = \begin{pmatrix} 2.33 & -0.5 \\ -0.5 & 0.125 \end{pmatrix}$$

$$X^T y = \begin{pmatrix} 3 + 5 + 7 \\ 6 + 20 + 42 \end{pmatrix} = \begin{pmatrix} 15 \\ 68 \end{pmatrix}$$

$$\beta = (X^T X)^{-1} X^T y = \begin{pmatrix} 2.33 & -0.5 \\ -0.5 & 0.125 \end{pmatrix} \begin{pmatrix} 15 \\ 68 \end{pmatrix} = \begin{pmatrix} 35 - 34 \\ -7.5 + 8.5 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

ดังนั้น $y = 1 + 1x$

ทฤษฎีบท 5.67 Neural Networks ใช้เมทริกซ์ในการคำนวณ forward propagation:

$$A^{(l)} = f(Z^{(l)})$$

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$$

โดยที่ $W^{(l)}$ คือเมทริกซ์น้ำหนัก (weight matrix) และ $b^{(l)}$ คือ bias vector

5.8.4 เมทริกซ์ประชิดในทฤษฎีกราฟ

ทฤษฎีบท 5.68 **Adjacency Matrix** ของกราฟ G ที่มี n โหนด คือเมทริกซ์ A ขนาด $n \times n$ โดยที่ $a_{ij} = 1$ ถ้ามีเส้นเชื่อมระหว่างโหนด i และ j และ $a_{ij} = 0$ ในกรณีอื่น

สมบัติที่สำคัญ:

- $(A^k)_{ij}$ = จำนวนเส้นทางความยาว k จากโหนด i ไปยังโหนด j
- ผลรวมของแถวที่ i = degree ของโหนด i

ตัวอย่าง 5.69 กราฟรูปสามเหลี่ยม (3 โหนด เชื่อมกันทุกคู่):

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

$$A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

$(A^2)_{ij}$ บอกจำนวนเส้นทาง 2 ชั้นจาก i ไป j เช่น $(A^2)_{12} = 1$ หมายความว่า มี 1 เส้นทาง 2 ชั้น

จากโหนด 1 ไปโหนด 2 (คือ $1 \rightarrow 3 \rightarrow 2$)

5.8.5 PageRank Algorithm

ทฤษฎีบท 5.70 **PageRank** เป็นอัลกอริทึมที่ใช้เมทริกซ์ในการจัดอันดับเว็บเพจ โดยพิจารณาจำนวนและคุณภาพของลิงก์ที่เชื่อมมายังหน้านั้น

สมมติว่ามี n เว็บเพจ กำหนดเมทริกซ์การเชื่อมโยง (Link Matrix) L ขนาด $n \times n$ โดยที่:

$$L_{ij} = \begin{cases} 1 & \text{ถ้าหน้า } j \text{ มีลิงก์ไปหน้า } i \\ 0 & \text{ในกรณีอื่น} \end{cases}$$

PageRank vector $\mathbf{r}$ หาได้จาก:

$$\mathbf{r} = c \cdot L^T \cdot \mathbf{r} + \frac{1-c}{n} \cdot \mathbf{1}$$

โดยที่ c คือ damping factor (ปกติ $c = 0.85$) และ $\mathbf{1}$ คือเวกเตอร์ของ 1 ทั้งหมด

ตัวอย่าง 5.71 พิจารณาเว็บ 3 หน้า A, B, C:

- A มีลิงก์ไป B และ C
- B มีลิงก์ไป C
- C มีลิงก์ไป A

เมทริกซ์การเชื่อมโยง:

$$L^T = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \end{pmatrix}$$

ค่า PageRank ของแต่ละหน้าคือ eigenvector ที่สำคัญที่สุดของ L^T

5.8.6 การประมวลผลภาพ (Image Processing)

ทฤษฎีบท 5.72 ในการประมวลผลภาพ เมทริกซ์คอนโวลูชัน (Convolution Matrix) หรือ **Kernel** ใช้ในการทำ operations ต่างๆ เช่น blur, sharpen, edge detection

สำหรับภาพขาวดำ ภาพถูกแทนด้วยเมทริกซ์ I ของ pixel values

การคอนโวลูชัน: $(I * K)_{ij} = \sum_m \sum_n I_{i+m, j+n} \cdot K_{m,n}$

ตัวอย่าง 5.73 (Edge Detection Kernel) เมทริกซ์ Sobel สำหรับ edge detection ในแนวนอน:

$$K_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

และในแนวตั้ง:

$$K_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

$$\text{Edge magnitude: } E = \sqrt{(K_x * I)^2 + (K_y * I)^2}$$

ตัวอย่าง 5.74 (Gaussian Blur Kernel) เมทริกซ์ Gaussian blur ขนาด 3×3 :

$$K = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

เมทริกซ์นี้ใช้ในการทำให้ภาพเบลอ โดยค่าในเมทริกซ์บวกกันได้ 1 เพื่อรักษาความสว่างรวมของ

ภาพ

5.8.7 Principal Component Analysis (PCA)

ทฤษฎีบท 5.75 PCA เป็นเทคนิคในการลดมิติของข้อมูลโดยใช้ค่าเฉพาะ (Eigenvalues) และเวกเตอร์เฉพาะ (Eigenvectors) ของเมทริกซ์ความแปรปรวนร่วม (Covariance Matrix)

กำหนดข้อมูล X ขนาด $n \times p$ (n ตัวอย่าง, p features):

1. คำนวณ Covariance Matrix: $\Sigma = \frac{1}{n-1} X^T X$
2. หา eigenvectors และ eigenvalues ของ Σ
3. เรียง eigenvectors ตาม eigenvalues จากมากไปน้อย
4. เลือก k eigenvectors แรกเป็น principal components

ข้อมูลที่ลดมิติแล้ว: $Y = X \cdot W_k$ โดยที่ W_k คือเมทริกซ์ของ k eigenvectors

ตัวอย่าง 5.76 สำหรับข้อมูล 2 มิติ:

$$X = \begin{pmatrix} 2 & 3 \\ 3 & 4 \\ 4 & 5 \\ 5 & 6 \end{pmatrix}$$

Covariance Matrix:

$$\Sigma = \begin{pmatrix} 1.67 & 1.67 \\ 1.67 & 1.67 \end{pmatrix}$$

Eigenvalues: $\lambda_1 = 3.33, \lambda_2 = 0$

Eigenvector ที่สำคัญ: $\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

ข้อมูลที่ลดมิติเป็น 1 มิติ: $Y = X \cdot \mathbf{v}_1$

สรุป

ในบทนี้เราได้ศึกษาเกี่ยวกับเมทริกซ์และดีเทอร์มิแนนต์ ซึ่งเป็นเครื่องมือพื้นฐานที่สำคัญในคณิตศาสตร์และวิทยาการคอมพิวเตอร์ สารสำคัญที่ควรจำ:

1. **เมทริกซ์** คือการจัดเรียงข้อมูลในรูปตารางสองมิติ มีหลายประเภท เช่น จตุรัส, เส้นทแยงมุม, สมมาตร, สามเหลี่ยม
 2. **การดำเนินการระหว่างเมทริกซ์** ประกอบด้วย การบวก ลบ คูณด้วยสเกลาร์ คูณเมทริกซ์ และ transpose การคูณเมทริกซ์ไม่มีสมบัติสลับที่
 3. **ดีเทอร์มิแนนต์** เป็นค่าจำเพาะของเมทริกซ์จตุรัส คำนวณได้โดย cofactor expansion หรือ กฎของ Sarrus (สำหรับ 3×3) สมบัติสำคัญคือ $\det(AB) = \det(A) \det(B)$
 4. **อินเวอร์สของเมทริกซ์** A^{-1} ทำให้ $AA^{-1} = A^{-1}A = I$ มีอินเวอร์สก็ต่อเมื่อ $\det(A) \neq 0$
 5. **การแก้ระบบสมการเชิงเส้น** สามารถทำได้โดยอินเวอร์สเมทริกซ์ ($x = A^{-1}b$), กฎของคราเมอร์, หรือ Gaussian elimination
 6. **การประยุกต์** ในคอมพิวเตอร์กราฟิกส์, การเข้ารหัส, การเรียนรู้ของเครื่อง, และทฤษฎีกราฟ
- เมทริกซ์เป็นพื้นฐานที่จำเป็นสำหรับการศึกษาต่อในสาขาคณิตศาสตร์ชั้นสูง เช่น พีชคณิตเชิงเส้น (Linear Algebra) และการวิเคราะห์เชิงตัวเลข (Numerical Analysis)

Bibliography

- [1] Rosen, Kenneth H. *Discrete Mathematics and Its Applications*. 8th ed., McGraw-Hill, 2019.
- [2] Anton, Howard. *Elementary Linear Algebra*. 12th ed., Wiley, 2019.
- [3] Lay, David C., Lay, Steven R., and McDonald, Judi J. *Linear Algebra and Its Applications*. 5th ed., Pearson, 2015.
- [4] Strang, Gilbert. *Linear Algebra and Its Applications*. 4th ed., Cengage Learning, 2005.

ผลการอ้างอิง

แผนการสอนประจำบทที่ 6

รหัสวิชา	4121701
ชื่อวิชา	คณิตศาสตร์สำหรับคอมพิวเตอร์
หน่วยกิต	3 (3-0-6)
บทที่	6
ชื่อบท	พีชคณิตบูลีน (Boolean Algebra)
จำนวนชั่วโมง	6 ชั่วโมง

วัตถุประสงค์การเรียนรู้ประจำบท

เมื่อนักศึกษาเรียนบทนี้จบแล้ว นักศึกษาจะสามารถ:

- อธิบายสัจพจน์และกฎของพีชคณิตบูลีนได้
- ลดรูปสมการบูลีนโดยใช้กฎพีชคณิตบูลีนได้
- สร้างและวิเคราะห์วงจรลอจิกเกตจากนิพจน์บูลีนได้
- ใช้แผนที่การโน (Karnaugh Map) ในการลดรูปฟังก์ชันบูลีนได้
- เชื่อมโยงพีชคณิตบูลีนกับการออกแบบวงจรดิจิทัลและการเขียนโปรแกรมได้

หัวข้อที่สอน

- สัจพจน์และกฎของพีชคณิตบูลีน
- การลดรูปนิพจน์บูลีน
- วงจรลอจิกเกต (Logic Gates)
- แผนที่การโน (Karnaugh Map)
- การประยุกต์ใช้พีชคณิตบูลีนในวงจรดิจิทัล

สื่อการสอน

- สไลด์ประกอบการสอน
- ซอฟต์แวร์จำลองวงจรลอจิก
- แบบฝึกหัดท้ายบท

พีชคณิตบูลีน (Boolean Algebra)

วัตถุประสงค์การเรียนรู้

หลังจากศึกษาจบบทนี้ ผู้เรียนจะสามารถ:

- อธิบายความหมายและความสำคัญของพีชคณิตบูลีนได้
- ดำเนินการทางบูลีน (AND, OR, NOT) และอธิบายความหมายของผลลัพธ์ได้
- พิสูจน์และประยุกต์ใช้กฎของเดอมอร์แกน (De Morgan's Laws) สำหรับพีชคณิตบูลีนได้
- ลดรูปนิพจน์บูลีนโดยใช้กฎและทฤษฎีบทต่างๆ ได้
- เขียนและตีความนิพจน์บูลีนในรูปแบบ Sum of Products และ Product of Sums ได้
- ใช้แผนที่คาร์นอ (Karnaugh Map) ในการลดรูปฟังก์ชันบูลีนได้
- อธิบายความสัมพันธ์ระหว่างพีชคณิตบูลีนกับวงจรดิจิทัลและตรรกศาสตร์ในคอมพิวเตอร์ได้
- พิสูจน์ข้อความทางพีชคณิตบูลีนโดยใช้เทคนิคการพิสูจน์ที่เหมาะสมได้

บทนำ

พีชคณิตบูลีน (Boolean Algebra) เป็นสาขาของคณิตศาสตร์ที่ศึกษาเกี่ยวกับค่าตรรกะ (logical values) และการดำเนินการทางตรรกะ (logical operations) พีชคณิตบูลีนถูกพัฒนาขึ้นโดย George Boole (1815–1864) นักคณิตศาสตร์ชาวอังกฤษในปี ค.ศ. 1854 ผ่านงานวิจัยเรื่อง "An Investigation of the Laws of Thought"

พีชคณิตบูลีนมีความสำคัญอย่างยิ่งในวิทยาการคอมพิวเตอร์ เนื่องจากเป็นพื้นฐานของ วงจรดิจิทัล (การออกแบบ logic gates) ตรรกศาสตร์ในการเขียนโปรแกรม (เงื่อนไข if-else, loops) ฐานข้อมูล (boolean queries ใน SQL) และทฤษฎีการคำนวณ (boolean circuits)

ความเชื่อมโยงระหว่างพีชคณิตบูลีนกับทฤษฎีเซตและตรรกศาสตร์เป็นสิ่งสำคัญ โดยมีความสัมพันธ์ดังนี้:

Boolean Algebra	Set Theory	Propositional Logic
0 (False)	$\emptyset$ (Empty Set)	$\perp$ (False)
1 (True)	U (Universal Set)	$\top$ (True)
OR (+)	Union ($\cup$)	OR ($\vee$)
AND ($\cdot$)	Intersection ($\cap$)	AND ($\wedge$)
NOT ($\bar{x}$)	Complement (A^c)	NOT ($\neg$)

นิยามพื้นฐานของพีชคณิตบูลีน

6.3.1 นิยามของ Boolean Algebra

ทฤษฎีบท 6.1 **Boolean Algebra** คือ algebraic structure $(B, +, \cdot, ', 0, 1)$ โดยที่ B เป็นเซตที่ประกอบด้วยสองค่า $\{0, 1\}$ และ $+$, $\cdot$ เป็น binary operations และ $'$ เป็น unary operation โดยมี axioms ดังนี้:

1. **Closure:** สำหรับทุก $a, b \in B$:

- $a + b \in B$
- $a \cdot b \in B$

2. **Identity Elements:**

- $a + 0 = a$ (0 เป็น identity ของ OR)
- $a \cdot 1 = a$ (1 เป็น identity ของ AND)

3. **Commutative Laws:**

- $a + b = b + a$
- $a \cdot b = b \cdot a$

4. **Associative Laws:**

- $(a + b) + c = a + (b + c)$
- $(a \cdot b) \cdot c = a \cdot (b \cdot c)$

5. **Distributive Laws:**

- $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$
- $a + (b \cdot c) = (a + b) \cdot (a + c)$

6. Complement Laws:

- $a + a' = 1$
- $a \cdot a' = 0$

6.3.2 ค่าคงที่บูลีน

ทฤษฎีบท 6.2 ในพีชคณิตบูลีน มีค่าคงที่สองค่า:

1. **0 (False/ปลอม)** คือ แทนค่าความจริงว่างเปล่าหรือเท็จ
2. **1 (True/จริง)** คือ แทนค่าความจริงว่าถูกต้องหรือจริง

ตัวอย่าง 6.3 ในบริบทของวงจรดิจิทัล:

- 0 อาจแทนแรงดัน 0V หรือสัญญาณปิด (off)
- 1 อาจแทนแรงดัน 5V หรือสัญญาณเปิด (on)

ในบริบทของตรรกศาสตร์:

- 0 อาจแทน “ไม่จริง” หรือ “เท็จ”
- 1 อาจแทน “จริง” หรือ “ถูกต้อง”

6. การดำเนินการทางบูลีน (Boolean Operations)

6.4.1 การดำเนินการ OR (Disjunction)

ทฤษฎีบท 6.4 การดำเนินการ OR ของ a และ b เขียนแทนด้วย $a + b$ หรือ $a \vee b$ ให้ผลลัพธ์เป็น 1 ถ้าอย่างน้อยหนึ่งใน a หรือ b เป็น 1 และให้ผลลัพธ์เป็น 0 ก็ต่อเมื่อ $a = b = 0$

Truth Table ของ OR:

a	b	$a + b$
0	0	0
0	1	1
1	0	1
1	1	1

ตัวอย่าง 6.5

$$\bullet 0 + 0 = 0$$

$$\bullet 0 + 1 = 1$$

$$\bullet 1 + 0 = 1$$

$$\bullet 1 + 1 = 1$$

ทฤษฎีบท 6.6 (Properties ของ OR) สำหรับทุก $a, b, c \in \{0, 1\}$:

1. $a + a = a$ (Idempotent Law)
2. $a + 0 = a$ (Identity Law)
3. $a + 1 = 1$ (Domination Law)
4. $a + a' = 1$ (Complement Law)
5. $a + b = b + a$ (Commutative Law)
6. $(a + b) + c = a + (b + c)$ (Associative Law)

6.4.2 การดำเนินการ AND (Conjunction)

ทฤษฎีบท 6.7 การดำเนินการ AND ของ a และ b เขียนแทนด้วย $a \cdot b$ หรือ ab หรือ $a \wedge b$ ให้ผลลัพธ์เป็น 1 ก็ต่อเมื่อ $a = b = 1$ และให้ผลลัพธ์เป็น 0 ถ้าอย่างน้อยหนึ่งใน a หรือ b เป็น 0

Truth Table ของ AND:

a	b	$a \cdot b$
0	0	0
0	1	0
1	0	0
1	1	1

ตัวอย่าง 6.8

$$\bullet 0 \cdot 0 = 0$$

$$\bullet 0 \cdot 1 = 0$$

$$\bullet 1 \cdot 0 = 0$$

$$\bullet 1 \cdot 1 = 1$$

ทฤษฎีบท 6.9 (Properties ของ AND) สำหรับทุก $a, b, c \in \{0, 1\}$:

1. $a \cdot a = a$ (Idempotent Law)
2. $a \cdot 0 = 0$ (Domination Law)
3. $a \cdot 1 = a$ (Identity Law)
4. $a \cdot a' = 0$ (Complement Law)
5. $a \cdot b = b \cdot a$ (Commutative Law)
6. $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ (Associative Law)

6.4.3 การดำเนินการ NOT (Negation)

ทฤษฎีบท 6.10 การดำเนินการ NOT ของ a เขียนแทนด้วย a' หรือ $\bar{a}$ หรือ $\neg a$ ให้ผลลัพธ์ตรงข้ามกับ a นั่นคือ ถ้า $a = 0$ แล้ว $a' = 1$ และถ้า $a = 1$ แล้ว $a' = 0$

Truth Table ของ NOT:

a	a'
0	1
1	0

ตัวอย่าง 6.11 $\bullet 0' = 1$

$$\bullet 1' = 0$$

$$\bullet (0')' = 0$$

$$\bullet (1')' = 1$$

ทฤษฎีบท 6.12 (Properties ของ NOT) สำหรับทุก $a \in \{0, 1\}$:

$$1. a + a' = 1 \quad (\text{Complement Law ข้อ 1})$$

$$2. a \cdot a' = 0 \quad (\text{Complement Law ข้อ 2})$$

$$3. (a')' = a \quad (\text{Double Negation})$$

$$4. 0' = 1 \text{ และ } 1' = 0$$

หมายเหตุ 6.13 อย่าสับสนระหว่างการดำเนินการทางคณิตศาสตร์ทั่วไปกับพีชคณิตบูลีน:

1. ในพีชคณิตบูลีน: $1 + 1 = 1$ (ไม่ใช่ 2)
2. ในพีชคณิตบูลีน: $1 \cdot 1 = 1$ (เหมือนการคูณปกติ)
3. ในพีชคณิตบูลีน: $1 + 0 = 1$ (เหมือนการบวกปกติ)

สัญลักษณ์ $+$ และ $\cdot$ ในบริบทของ Boolean Algebra มีความหมายแตกต่างจากใน ordinary algebra

กฎของพีชคณิตบูลีน (Boolean Identities)

6.5.1 Distributive Laws

ทฤษฎีบท 6.14 (Distributive Laws) สำหรับทุก $a, b, c \in \{0, 1\}$:

$$1. a \cdot (b + c) = (a \cdot b) + (a \cdot c) \quad (\text{AND กระจายเหนือ OR})$$

$$2. a + (b \cdot c) = (a + b) \cdot (a + c) \quad (\text{OR กระจายเหนือ AND})$$

ตัวอย่าง 6.15 พิสูจน์ $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ โดยใช้ truth table:

a	b	c	$b + c$	$a \cdot (b + c)$	$a \cdot b$	$a \cdot c$	$(a \cdot b) + (a \cdot c)$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

เนื่องจากคอลัมน์ $a \cdot (b + c)$ และ $(a \cdot b) + (a \cdot c)$ มีค่าเท่ากันทุกแถว ดังนั้น $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

ตัวอย่าง 6.16 พิสูจน์ $a + (b \cdot c) = (a + b) \cdot (a + c)$ โดยใช้ truth table:

a	b	c	$b \cdot c$	$a + (b \cdot c)$	$a + b$	$a + c$	$(a + b) \cdot (a + c)$
0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0
0	1	1	1	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	0	1	1	1	1
1	1	1	1	1	1	1	1

เนื่องจากคอลัมน์ $a + (b \cdot c)$ และ $(a + b) \cdot (a + c)$ มีค่าเท่ากันทุกแถว ดังนั้น $a + (b \cdot c) = (a + b) \cdot (a + c)$

6.5.2 Absorption Laws

ทฤษฎีบท 6.17 (Absorption Laws) สำหรับทุก $a, b \in \{0, 1\}$:

1. $a + (a \cdot b) = a$ (Absorption Law ข้อ 1)
2. $a \cdot (a + b) = a$ (Absorption Law ข้อ 2)

ตัวอย่าง 6.18 พิสูจน์ $a + (a \cdot b) = a$ โดยใช้ truth table:

a	b	$a \cdot b$	$a + (a \cdot b)$	a
0	0	0	0	0
0	1	0	0	0
1	0	0	1	1
1	1	1	1	1

ดังนั้น $a + (a \cdot b) = a$

ตัวอย่าง 6.19 พิสูจน์ $a \cdot (a + b) = a$ โดยใช้ truth table:

a	b	$a + b$	$a \cdot (a + b)$	a
0	0	0	0	0
0	1	1	0	0
1	0	1	1	1
1	1	1	1	1

ดังนั้น $a \cdot (a + b) = a$

กฎของเดมอร์แกน (De Morgan's Laws)

กฎของเดมอร์แกนตั้งชื่อตาม Augustus De Morgan (1806–1871) นักคณิตศาสตร์ชาวอังกฤษ เช่นเดียวกับในทฤษฎีเซต

ทฤษฎีบท 6.20 (De Morgan's Laws สำหรับพีชคณิตบูลีน) สำหรับทุก $a, b \in \{0, 1\}$:

1. $(a + b)' = a' \cdot b'$ (Complement ของ OR = AND ของ Complements)
2. $(a \cdot b)' = a' + b'$ (Complement ของ AND = OR ของ Complements)

หมายเหตุ 6.21 สังเกตความสอดคล้องกับ De Morgan's Laws ในตรรกศาสตร์และทฤษฎีเซต:

1. $\neg(p \vee q) \equiv \neg p \wedge \neg q$ สอดคล้องกับ $(a + b)' = a' \cdot b'$
2. $\neg(p \wedge q) \equiv \neg p \vee \neg q$ สอดคล้องกับ $(a \cdot b)' = a' + b'$

นี่เป็นตัวอย่างที่แสดงความเชื่อมโยงระหว่างตรรกศาสตร์ ทฤษฎีเซต และพีชคณิตบูลีนอย่างลึกซึ้ง

ตัวอย่าง 6.22 พิสูจน์ $(a + b)' = a' \cdot b'$ โดยใช้ truth table:

a	b	$a + b$	$(a + b)'$	a'	b'	$a' \cdot b'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

เนื่องจากคอลัมน์ $(a + b)'$ และ $a' \cdot b'$ มีค่าเท่ากันทุกแถว ดังนั้น $(a + b)' = a' \cdot b'$

ตัวอย่าง 6.23 พิสูจน์ $(a \cdot b)' = a' + b'$ โดยใช้ truth table:

a	b	$a \cdot b$	$(a \cdot b)'$	a'	b'	$a' + b'$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

เนื่องจากคอลัมน์ $(a \cdot b)'$ และ $a' + b'$ มีค่าเท่ากันทุกแถว ดังนั้น $(a \cdot b)' = a' + b'$

นิพจน์บูลีนและฟังก์ชันบูลีน (Boolean Expressions and Functions)

6.7.1 นิยามของนิพจน์บูลีน

ทฤษฎีบท 6.24 นิพจน์บูลีน (Boolean Expression) คือ การรวมกันของ:

- ตัวแปรบูลีน (Boolean variables) เช่น x, y, z
- ค่าคงที่บูลีน 0 และ 1
- การดำเนินการบูลีน (+, ·, ')
- วงเล็บ (และ = เพื่อกำหนดลำดับการดำเนินการ

ตัวอย่าง 6.25 นิพจน์บูลีนที่ถูกต้อง:

- $x + y$
- $x \cdot y'$

- $(x + y) \cdot z$
- $x' + y \cdot z$
- $(a + b) \cdot (a' + c) + (a \cdot b \cdot c)'$

6.7.2 ฟังก์ชันบูลีน

ทฤษฎีบท 6.26 ฟังก์ชันบูลีน (Boolean Function) $f(x_1, x_2, \dots, x_n)$ คือ ฟังก์ชันที่รับค่าบูลีน n ตัวแปรและให้ผลลัพธ์เป็นค่าบูลีน

ฟังก์ชันบูลีนสามารถแทนด้วย **truth table** ที่แสดงผลลัพธ์สำหรับการผสมผสานของค่าอินพุต

ตัวอย่าง 6.27 ฟังก์ชันบูลีน $f(x, y) = x \cdot y + x' \cdot y'$

x	y	x'	y'	$x \cdot y$	$x \cdot y + x' \cdot y'$
0	0	1	1	0	1
0	1	1	0	0	0
1	0	0	1	0	0
1	1	0	0	1	1

ฟังก์ชันนี้เรียกว่า **XNOR** ซึ่งให้ผลลัพธ์เป็น 1 เมื่อ $x = y$ และให้ผลลัพธ์เป็น 0 เมื่อ $x \neq y$

ตัวอย่าง 6.28 ฟังก์ชันบูลีน $f(x, y, z) = (x + y') \cdot z + x \cdot y$

x	y	z	y'	$x + y'$	$(x + y') \cdot z$	$x \cdot y$	f
0	0	0	1	1	0	0	0
0	0	1	1	1	1	0	1
0	1	0	0	0	0	0	0
0	1	1	0	0	0	0	0
1	0	0	1	1	0	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	1	1
1	1	1	0	1	1	1	1

รูปแบบปกติ (Canonical Forms)

6.8.1 Minterms และ Maxterms

ทฤษฎีบท 6.29 **Minterm** ของตัวแปร n ตัวคือ AND term ที่มีตัวแปรทุกตัวปรากฏ exactly once ในรูปแบบปกติ (ถ้าตัวแปรเป็น 1 ใช้ตัวแปรเดียว ถ้าตัวแปรเป็น 0 ใช้ complement)

สำหรับ 2 ตัวแปร x, y :

- $m_0 = x' \cdot y'$ (ทั้ง $x = 0, y = 0$)
- $m_1 = x' \cdot y$ (ทั้ง $x = 0, y = 1$)
- $m_2 = x \cdot y'$ (ทั้ง $x = 1, y = 0$)
- $m_3 = x \cdot y$ (ทั้ง $x = 1, y = 1$)

ทฤษฎีบท 6.30 **Maxterm** ของตัวแปร n ตัวคือ OR term ที่มีตัวแปรทุกตัวปรากฏ exactly once ในรูปแบบปกติ (ถ้าตัวแปรเป็น 0 ใช้ตัวแปรเดียว ถ้าตัวแปรเป็น 1 ใช้ complement)

สำหรับ 2 ตัวแปร x, y :

- $M_0 = x + y$ (ทั้ง $x = 0, y = 0$)
- $M_1 = x + y'$ (ทั้ง $x = 0, y = 1$)
- $M_2 = x' + y$ (ทั้ง $x = 1, y = 0$)
- $M_3 = x' + y'$ (ทั้ง $x = 1, y = 1$)

ทฤษฎีบท 6.31 ความสัมพันธ์ระหว่าง Minterm และ Maxterm:

- $m_i = M'_i$
- $M_i = m'_i$
- $\sum$ (sigma) ใช้แทน sum ของ minterms
- $\prod$ (pi) ใช้แทน product ของ maxterms

6.8.2 Sum of Products (SOP) — ผลบวกของผลคูณ

ทฤษฎีบท 6.32 **Sum of Products (SOP)** คือ รูปแบบของนิพจน์บูลีนที่เขียนเป็น OR ของ AND terms

ฟังก์ชันบูลีนใดๆ สามารถเขียนเป็น SOP ได้โดยใช้ **minterm expansion** หรือ **canonical SOP form**

ตัวอย่าง 6.33 เขียนฟังก์ชัน $f(x, y) = x \cdot y + x'$ ในรูปแบบ canonical SOP

จาก truth table:

x	y	$x \cdot y$	x'	f
0	0	0	1	1
0	1	0	1	1
1	0	0	0	0
1	1	1	0	1

$f = 1$ เมื่อ $(x, y) = (0, 0)$ หรือ $(0, 1)$ หรือ $(1, 1)$

ดังนั้น $f(x, y) = x'y' + x'y + xy = \sum(0, 1, 3)$

ตัวอย่าง 6.34 จาก truth table ของ $f(x, y, z)$ ที่ $f = 1$ สำหรับ minterms m_1, m_3, m_5, m_7 :

$$f(x, y, z) = x'y'z + x'yz' + xy'z + xyz' = \sum(1, 3, 5, 7)$$

6.8.3 Product of Sums (POS) — ผลคูณของผลบวก

ทฤษฎีบท 6.35 **Product of Sums (POS)** คือ รูปแบบของนิพจน์บูลีนที่เขียนเป็น AND ของ OR terms

ฟังก์ชันบูลีนใดๆ สามารถเขียนเป็น POS ได้โดยใช้ **maxterm expansion** หรือ **canonical POS form**

ตัวอย่าง 6.36 เขียนฟังก์ชัน $f(x, y) = (x + y) \cdot (x' + y')$ ในรูปแบบ canonical POS

จาก truth table:

x	y	$x + y$	$x' + y'$	f
0	0	0	1	0
0	1	1	0	0
1	0	1	0	0
1	1	1	1	1

$f = 0$ เมื่อ $(x, y) = (0, 0)$ หรือ $(0, 1)$ หรือ $(1, 0)$

ดังนั้น $f(x, y) = M_0 \cdot M_1 \cdot M_2 = \prod(0, 1, 2)$

หรือ $f(x, y) = (x + y) \cdot (x + y') \cdot (x' + y)$

ทฤษฎีบท 6.37 ความสัมพันธ์ระหว่าง SOP และ POS: ถ้า $f(x_1, \dots, x_n) = \sum(m_{i_1}, m_{i_2}, \dots)$ แล้ว
 $f(x_1, \dots, x_n) = \prod(M_{j_1}, M_{j_2}, \dots)$
โดยที่ $\{j_1, j_2, \dots\}$ คือ maxterms ที่ไม่อยู่ใน minterm list

การลดรูปนิพจน์บูลีน (Boolean Expression Simplification)

การลดรูปนิพจน์บูลีนมีความสำคัญในการออกแบบวงจรดิจิทัลที่มีประสิทธิภาพ การลดรูปสามารถทำได้หลายวิธี

6.9.1 การใช้ Boolean Identities

ตัวอย่าง 6.38 ลดรูป $f(x, y, z) = xy + x'y + xy'$

$$\begin{aligned}
 f &= xy + x'y + xy' \\
 &= y(x + x') + xy' \quad (\text{Distributive ย้อนกลับ}) \\
 &= y \cdot 1 + xy' \quad (\text{Complement: } x + x' = 1) \\
 &= y + xy' \\
 &= (y + x)(y + y') \quad (\text{Distributive}) \\
 &= (y + x) \cdot 1 \quad (\text{Complement}) \\
 &= x + y
 \end{aligned}$$

ดังนั้น $f = x + y$

ตัวอย่าง 6.39 ลดรูป $f(a, b, c) = abc + abc' + a'b + ab'c$

$$\begin{aligned}
 f &= abc + abc' + a'b + ab'c \\
 &= ab(c + c') + a'b + ab'c \quad (\text{Distributive}) \\
 &= ab \cdot 1 + a'b + ab'c \quad (\text{Complement}) \\
 &= ab + a'b + ab'c \\
 &= b(a + a') + ab'c \\
 &= b + ab'c \quad (\text{Consensus หรือ Distribution ย้อนกลับ}) \\
 &= (b + a)(b + b')(b + c) \\
 &= (b + a) \cdot 1 \cdot (b + c) \\
 &= b + ac \quad (\text{Consensus หรือ Distribution ย้อนกลับ})
 \end{aligned}$$

ดังนั้น $f = b + ac$

ตัวอย่าง 6.40 ลดรูป $f(x, y) = (x + y) \cdot (x + y')$

$$\begin{aligned}
 f &= (x + y) \cdot (x + y') \\
 &= x + y \cdot y' \quad (\text{Distribution: } x + yz = (x + y)(x + z)) \\
 &= x + 0 \quad (\text{Complement: } y \cdot y' = 0) \\
 &= x
 \end{aligned}$$

ดังนั้น $f = x$

6.9.2 การใช้ Consensus Theorem

ทฤษฎีบท 6.41 (Consensus Theorem)

$$1. \quad ab + a'c + bc = ab + a'c$$

$$2. \quad (a + b)(a' + c)(b + c) = (a + b)(a' + c)$$

ตัวอย่าง 6.42 ลดรูป $f(x, y, z) = xy + x'z + yz$

$$f = xy + x'z + yz$$

$$= xy + x'z \quad (\text{Consensus: } yz \text{ ถูกละทิ้ง})$$

ดังนั้น $f = xy + x'z$

แผนที่คาร์นอ (Karnaugh Map)

แผนที่คาร์นอ หรือ K-map เป็นเครื่องมือในการลดรูปฟังก์ชันบูลีนอย่างเป็นระบบ K-map แสดงค่าของฟังก์ชันในรูปแบบ grid โดยการจัดเรียงเซลล์ทำให้ minterms ที่อยู่ติดกัน (adjacent) อยู่ใกล้กัน

6.10.1 K-map สำหรับ 2 ตัวแปร

ทฤษฎีบท 6.43 K-map สำหรับ 2 ตัวแปร x, y :

	$y = 0$	$y = 1$
$x = 0$	m_0 $x'y'$	m_1 $x'y$
$x = 1$	m_2 xy'	m_3 xy

เซลล์ที่อยู่ติดกัน (adjacent cells) หมายถึงเซลล์ที่แชร์ edge ร่วมกัน

ตัวอย่าง 6.44 ใช้ K-map ลดรูป $f(x, y) = \sum(1, 2, 3)$

	$y = 0$	$y = 1$
$x = 0$	0 m_0	1 m_1
$x = 1$	1 m_2	1 m_3

Grouping:

- ครอบคลุม m_1 และ m_3 (คอลัมน์ $y = 1$) $\Rightarrow$ ให้ x
- ครอบคลุม m_2 และ m_3 (แถว $x = 1$) $\Rightarrow$ ให้ y'

ดังนั้น $f = x + y'$

ตัวอย่าง 6.45 ใช้ K-map ลดรูป $f(x, y) = xy + xy' + x'y'$

	$y = 0$	$y = 1$
$x = 0$	1 m_0	0 m_1
$x = 1$	1 m_2	1 m_3

Grouping:

- ครอบคลุม m_0, m_2 (คอลัมน์ $y = 0$) $\Rightarrow$ ให้ x'
- ครอบคลุม m_2, m_3 (แถว $x = 1$) $\Rightarrow$ ให้ x

ดังนั้น $f = x' + x = 1$ (ตรรกะที่เป็นจริงเสมอ)

6.10.2 K-map สำหรับ 3 ตัวแปร

ทฤษฎีบท 6.46 K-map สำหรับ 3 ตัวแปร x, y, z :

	$yz = 00$ $y'z'$	$yz = 01$ $y'z$	$yz = 11$ yz	$yz = 10$ yz'
$x = 0$	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
$x = 1$	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

หมายเหตุ: ลำดับของคอลัมน์ใช้ Gray code เพื่อให้เซลล์ข้างเคียงแตกต่างกันเพียง 1 bit

ตัวอย่าง 6.47 ใช้ K-map ลดรูป $f(x, y, z) = \sum(0, 1, 3, 4, 5, 6)$

	$yz = 00$	$yz = 01$	$yz = 11$	$yz = 10$
$x = 0$	1	1	1	0
$x = 1$	1	1	0	1

Grouping:

- ครอบคลุม m_0, m_1, m_4, m_5 (2x2 block มุมซ้าย) $\Rightarrow$ ให้ y'

- ครอบคลุม m_0, m_2 (wrapping ด้านขวา) $\Rightarrow$ ให้ $x'z'$

ตรวจสอบ: $f = y' + x'z'$

จริงๆ แล้ว:

- ครอบคลุม m_0, m_1, m_4, m_5 ทั้งแถวบนและแถวล่างของคอลัมน์แรกสองคอลัมน์ $\Rightarrow y'$
- ครอบคลุม m_6 กับ m_4 หรือ m_2 กับ m_6 (wrapping) $\Rightarrow xz'$ หรือ xy'

ดังนั้น $f = y' + xz'$

6.10.3 K-map สำหรับ 4 ตัวแปร

ทฤษฎีบท 6.48 K-map สำหรับ 4 ตัวแปร w, x, y, z :

	$xy = 00$ $y'z'$	$xy = 01$ $y'z$	$xy = 11$ yz	$xy = 10$ yz'
$wx = 00$	m_0	m_1	m_3	m_2
$wx = 01$	m_4	m_5	m_7	m_6
$wx = 11$	m_{12}	m_{13}	m_{15}	m_{14}
$wx = 10$	m_8	m_9	m_{11}	m_{10}

การ wrap สามารถทำได้ทั้งในแนวนอน แนวตั้ง และมุม (เช่น m_0 ติดกับ m_2, m_8, m_{10})

ตัวอย่าง 6.49 ใช้ K-map ลดรูป $f(w, x, y, z) = \sum(0, 1, 2, 3, 4, 6, 8, 9, 12, 14)$

	$xy = 00$	$xy = 01$	$xy = 11$	$xy = 10$
$wx = 00$	1	1	1	1
$wx = 01$	1	0	0	1
$wx = 11$	0	0	0	0
$wx = 10$	1	1	0	1

Grouping:

- ครอบคลุม m_0, m_1, m_8, m_9 (8-cell vertical) $\Rightarrow w'z'$
- ครอบคลุม m_0, m_2, m_8, m_{10} (wrapping) $\Rightarrow y'z'$
- ครอบคลุม $m_4, m_6 \Rightarrow wx'y'$

- ครออบคลุม $m_0, m_1, m_2, m_3 \Rightarrow w'y'$

- ครออบคลุม $m_8, m_9, m_{12}, m_{14} \Rightarrow x'z' + wx'y'$

ดังนั้น $f = w'z' + wx'y'$

ทฤษฎีบท 6.50 (ขั้นตอนการใช้ K-map)

1. เขียน K-map ตามจำนวนตัวแปร

2. ใส่ 1 ในเซลล์ที่ $\square\square$ minterm ที่อยู่ใน function

3. จัดกลุ่ม (group) เซลล์ที่มี 1 ร่วมกันเป็นกลุ่มขนาด 2^n ($n = 0, 1, 2, 3, \dots$)

4. กลุ่มควรมีขนาดใหญ่ที่สุดเท่าที่เป็นไปได้

5. กลุ่มสามารถ overlap ได้และสามารถ wrap around ได้

6. หาตัวแปรที่ไม่เปลี่ยนแปลงในกลุ่ม (คงที่) — ตัวแปรที่เป็น 1 ใช้ตัวแปรเดี่ยว ตัวแปรที่เป็น 0 ใช้ complement

7. OR terms ที่ได้จากแต่ละกลุ่ม

6.11 ประยุกต์ในวงจรดิจิทัล

6.11.1 Logic Gates

Logic gates เป็น building blocks ของวงจรดิจิทัล:

ทฤษฎีบท 6.51 ประเภทของ Logic Gates:

- NOT Gate (Inverter): แสดงผล a'

- AND Gate: แสดงผล $a \cdot b$

- OR Gate: แสดงผล $a + b$

- NAND Gate: แสดงผล $(a \cdot b)'$

- NOR Gate: แสดงผล $(a + b)'$

- XOR Gate: แสดงผล $a \oplus b = ab' + a'b$

- XNOR Gate: แสดงผล $(a \oplus b)' = ab + a'b'$

ตัวอย่าง 6.52 วงจรสำหรับ $f(x, y, z) = xy + x'z$:

1. NOT gate สำหรับ x ให้ x'
2. AND gate สำหรับ x และ y ให้ xy
3. AND gate สำหรับ x' และ z ให้ $x'z$
4. OR gate สำหรับ xy และ $x'z$ ให้ $xy + x'z$

ตัวอย่าง 6.53 ลดรูปและวาดวงจรสำหรับ $f(a, b, c) = (a + b) \cdot (a + c)$

ใช้ Distributive Law: $(a + b)(a + c) = a + bc$

ดังนั้นวงจรที่ลดรูปแล้ว:

1. AND gate สำหรับ b และ c ให้ bc
2. OR gate สำหรับ a และ bc ให้ $a + bc$

วงจรที่ลดรูปใช้ 1 AND gate และ 1 OR gate (ลดจาก 2 OR gates และ 1 AND gate)

6.11.2 การออกแบบวงจร Combination

ตัวอย่าง 6.54 ออกแบบวงจรที่ตรวจสอบว่าจำนวน 3-bit เป็นเลขคู่ (even)

ถ้า input เป็น x, y, z และผลลัพธ์เป็น 1 เมื่อจำนวนเป็นเลขคู่:

$f(x, y, z) = 1$ เมื่อมีจำนวน 1 ที่เป็นคู่ (0, 2)

Minterms: m_0 (000), m_3 (011), m_5 (101), m_6 (110)

$$f(x, y, z) = x'y'z' + x'yz + xy'z + xyz'$$

ลดรูปโดยใช้ K-map:

	$yz = 00$	$yz = 01$	$yz = 11$	$yz = 10$
$x = 0$	1	0	1	0
$x = 1$	0	1	0	1

$$f = x'y'z' + x'yz + xy'z + xyz'$$

สังเกตว่า $f = (x \oplus y) \oplus z$ หรือ parity function

ลดรูปเป็น $f = x \oplus y \oplus z$

Two-level Realization

ในการ realization วงจรดิจิทัล รูปแบบ two-level เป็นที่นิยมเนื่องจากให้ความเร็วในการทำงานสูงสุด

ทฤษฎีบท 6.55 Two-level Realization:

- SOP (Sum of Products): AND gates ระดับแรก แล้ว OR gate ระดับที่สอง
- POS (Product of Sums): OR gates ระดับแรก แล้ว AND gate ระดับที่สอง

ตัวอย่าง 6.56 realization SOP สำหรับ $f(x, y, z) = y' + xy + x'y'z$

วงจรประกอบด้วย:

- NOT gate สำหรับ y ให้ y'
- AND gate สำหรับ x และ y ให้ xy
- AND gate สำหรับ x', y', z ให้ $x'y'z$
- OR gate สำหรับ $y', xy, x'y'z$ ให้ f

Half Adder และ Full Adder

ตัวอย่าง 6.57 ออกแบบ Half Adder

Half Adder รับ input 2 bits x, y และให้ผลลัพธ์:

- Sum $S = x \oplus y = xy' + x'y$
- Carry $C = x \cdot y$

Truth Table:

x	y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

ตัวอย่าง 6.58 ออกแบบ Full Adder

Full Adder รับ input 3 bits x, y, z_{in} และให้ผลลัพธ์:

- Sum $S = x \oplus y \oplus z_{in}$
- Carry $C_{out} = xy + xz_{in} + yz_{in}$

Truth Table:

x	y	z_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

สำหรับ S ใช้ K-map ลดรูปเป็น $S = x \oplus y \oplus z_{in}$

สำหรับ C_{out} ใช้ K-map ลดรูปเป็น $C_{out} = xy + xz_{in} + yz_{in}$

สมมูลของพีชคณิตบูลีน (Boolean Algebra Equivalence)

ทฤษฎีบท 6.59 ตารางสรุป Boolean Identities:

Basic Operations:

Operation	Symbol	Result = 1 when	Result = 0 when
OR	+	At least one input = 1	All inputs = 0
AND	·	All inputs = 1	At least one input = 0
NOT	'	Input = 0	Input = 1

Laws:

Law	OR Form	AND Form
Identity	$x + 0 = x$	$x \cdot 1 = x$
Domination	$x + 1 = 1$	$x \cdot 0 = 0$
Idempotent	$x + x = x$	$x \cdot x = x$
Inverse	$x + x' = 1$	$x \cdot x' = 0$
Commutative	$x + y = y + x$	$x \cdot y = y \cdot x$
Associative	$(x + y) + z = x + (y + z)$	$(x \cdot y) \cdot z = x \cdot (y \cdot z)$
Distributive	$x + yz = (x + y)(x + z)$	$x(y + z) = xy + xz$
Absorption	$x + xy = x$	$x(x + y) = x$
De Morgan	$(x + y)' = x'y'$	$(xy)' = x' + y'$

แบบฝึกหัด

6.15.1 ส่วนที่ 1: การดำเนินการพื้นฐาน

1. จงหาค่าของนิพจน์ต่อไปนี้:

1.1. $1 + 0$

1.2. $1 \cdot 1$

1.3. $0'$

1.4. $1' + 0$

1.5. $1 \cdot 0 + 1$

1.6. $(1 + 0) \cdot 1$

1.7. $(1')'$

1.8. $1 \cdot 1 + 0 \cdot 1$

2. เขียน truth table สำหรับ:

2.1. $f(x, y) = x + y'$

2.2. $f(x, y) = (x + y)'$

2.3. $f(x, y) = x \cdot y + x' \cdot y'$

$$2.4. f(x, y) = (x \oplus y)'$$

3. พิสูจน์ว่าข้อความต่อไปนี้จริงหรือเท็จ:

$$3.1. 1 + 1 = 2$$

$$3.2. 1 \cdot 1 = 1$$

$$3.3. 0' = 0$$

$$3.4. (1')' = 1$$

$$3.5. x + 1 = 1 \text{ สำหรับทุก } x$$

$$3.6. x \cdot 0 = x \text{ สำหรับทุก } x$$

6.15.2 ส่วนที่ 2: Boolean Identities

1. พิสูจน์โดยใช้ truth table:

$$1.1. x \cdot (x + y) = x$$

$$1.2. x + x'y = x + y$$

$$1.3. (x + y)' = x'y'$$

2. ลดรูปนิพจน์ต่อไปนี้โดยใช้ Boolean identities:

$$2.1. x + xy$$

$$2.2. x \cdot (x' + y)$$

$$2.3. (x + y)(x + y')$$

$$2.4. xy + xy'$$

$$2.5. (x + y)' + x$$

$$2.6. x'y + xy' + xy$$

3. พิสูจน์ Consensus Laws:

$$3.1. (a \cdot b) + (a' \cdot c) + (b \cdot c) = (a \cdot b) + (a' \cdot c)$$

$$3.2. (a + b)(a' + c)(b + c) = (a + b)(a' + c)$$

6.15.3 ส่วนที่ 3: De Morgan's Laws

1. พิสูจน์ De Morgan's Laws:

$$1.1. (x + y)' = x'y'$$

$$1.2. (xy)' = x' + y'$$

2. ใช้ De Morgan's Laws เพื่อเขียน complement ของ:

$$2.1. xy + x'z$$

$$2.2. (x + y + z)(x'y'z')'$$

$$2.3. (a + b)(c + d)$$

$$2.4. a + bc'$$

6.15.4 ส่วนที่ 4: ฟังก์ชันบูลีนและ Canonical Forms

1. เขียน truth table สำหรับ:

$$1.1. f(x, y, z) = xy + yz + xz$$

$$1.2. f(x, y, z) = (x + y)(y + z)(x + z)$$

$$1.3. f(x, y, z) = x \oplus y \oplus z$$

2. เขียนในรูปแบบ canonical SOP ($\sum$ notation):

$$2.1. f(x, y) = x + y'$$

$$2.2. f(x, y, z) = xy + x'z$$

3. เขียนในรูปแบบ canonical POS ($\prod$ notation):

$$3.1. f(x, y) = x \cdot y$$

$$3.2. f(x, y, z) = (x + y')(x' + z)$$

6.15.5 ส่วนที่ 5: Karnaugh Maps

1. ใช้ K-map ลดรูป:

$$1.1. f(x, y) = \sum(1, 2, 3)$$

$$1.2. f(x, y) = \sum(0, 1, 2)$$

$$1.3. f(x, y, z) = \sum(0, 2, 4, 6)$$

$$1.4. f(x, y, z) = \sum(1, 3, 5, 7)$$

$$1.5. f(w, x, y, z) = \sum(0, 2, 4, 6, 8, 10, 12, 14)$$

2. ออกแบบวงจรสำหรับ:

$$2.1. \text{Half Adder (Sum และ Carry)}$$

2.2. 2-to-1 Multiplexer: $f = s' \cdot d_0 + s \cdot d_1$

2.3. วงจรที่ตรวจสอบว่า input 3 bits มีเพียง 2 bits ที่เป็น 1

เฉลยแบบฝึกหัด

6.16.1 เฉลยส่วนที่ 1

1. 1.1. $1 + 0 = 1$

1.2. $1 \cdot 1 = 1$

1.3. $0' = 1$

1.4. $1' + 0 = 0 + 0 = 0$

1.5. $1 \cdot 0 + 1 = 0 + 1 = 1$

1.6. $(1 + 0) \cdot 1 = 1 \cdot 1 = 1$

1.7. $(1')' = 1$

1.8. $1 \cdot 1 + 0 \cdot 1 = 1 + 0 = 1$

2. 2.1. $f(x, y) = x + y'$

x	y	y'	f
0	0	1	1
0	1	0	0
1	0	1	1
1	1	0	1

2.2. $f(x, y) = (x + y)'$

x	y	$x + y$	f
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

3. 3.1. เท็จ (ใน Boolean: $1 + 1 = 1$)

3.2. จริง

3.3. เท็จ ($0' = 1$)

3.4. จริง

3.5. จริง

3.6. เท็จ ($x \cdot 0 = 0$)

ตัวอย่างในวิทยาการคอมพิวเตอร์

6.17.1 Karnaugh Maps สำหรับการออกแบบวงจร

Karnaugh Map (K-map) เป็นเครื่องมือในการลดรูป (simplify) ฟังก์ชันบูลีน ซึ่งมีความสำคัญในการออกแบบวงจรดิจิทัลที่ใช้ทรัพยากรน้อยที่สุด

ตัวอย่าง 6.60 ลดรูปฟังก์ชันบูลีน $F(x, y, z) = \sum(0, 1, 2, 3, 5, 7)$ โดยใช้ K-map

K-map สำหรับ 3 ตัวแปร:

	$yz = 00$	$yz = 01$	$yz = 11$	$yz = 10$
$x = 0$	1	1	0	1
$x = 1$	0	1	1	0

จัดกลุ่ม:

- กลุ่มที่ 1: ครอบคลุม cells 0, 1, 2, 3 (ทั้งแถวบน) $\rightarrow x'$
- กลุ่มที่ 2: ครอบคลุม cells 1, 5 $\rightarrow yz$

ผลลัพธ์ที่ลดรูปแล้ว: $F = x' + yz$

6.17.2 Boolean Logic ในการเขียนโปรแกรม

ในภาษาโปรแกรมสมัยใหม่ Boolean operators มีความสำคัญในการควบคุมการทำงานของโปรแกรม

```
ตัวอย่าง 6.61 (Short-circuit Evaluation ในภาษา C/Java) if (ptr != NULL && ptr->value > 0) {
    // executes only when ptr is not NULL and value > 0
}
```

```
if (is_admin || has_permission) {
```

```
// grant access if is admin or has permission
}
```

AND ของเงื่อนไขทั้งสองทำให้เราตรวจสอบ NULL ก่อนเข้าถึง pointer

ตัวอย่าง 6.62 (Bitwise Operations ใน Python/C) # checking flags

```
if (flags & 0x01) != 0: # if bit 0 is set
    print("Option A enabled")
```

```
# setting bit
```

```
flags |= 0x02 # set bit 1
```

```
# creating mask
```

```
mask = (1 << n) - 1 # returns n bits as 1
```

6.17.3 Logic Gates และวงจรดิจิทัล

Logic gates เป็น building blocks ของวงจรดิจิทัล ซึ่งทำงานตาม Boolean algebra

ทฤษฎีบท 6.63 Logic gates พื้นฐาน:

Gate	Boolean Expression
NOT	$Y = A'$
AND	$Y = A \cdot B$
OR	$Y = A + B$
XOR	$Y = A \oplus B = A'B + AB'$
NAND	$Y = (AB)'$
NOR	$Y = (A + B)'$

ตัวอย่าง 6.64 (การสร้าง AND จาก NAND gates)

$$Y = ((AB)')' = AB$$

นำ output ของ NAND gate มาเข้า NOT gate (ซึ่งก็คือ NAND gate ที่ input ทั้งสองเท่ากัน)

6.17.4 Half Adder และ Full Adder

Adder circuits เป็นพื้นฐานของ arithmetic logic unit (ALU) ใน CPU

ทฤษฎีบท 6.65 (Half Adder) Half Adder รับ input 2 bits (A, B) และให้ output:

- Sum: $S = A \oplus B$
- Carry: $C = A \cdot B$

ทฤษฎีบท 6.66 (Full Adder) Full Adder รับ input 3 bits (A, B, C_{in}) และให้ output:

- Sum: $S = A \oplus B \oplus C_{in}$
- Carry: $C_{out} = AB + C_{in}(A \oplus B)$

ตัวอย่าง 6.67 Full Adder จาก Half Adders:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

สามารถสร้างได้จาก 2 Half Adders + 1 OR gate

6.17.5 Multiplexer และ Demultiplexer

ทฤษฎีบท 6.68 **Multiplexer (MUX)** เป็นวงจรที่เลือก 1 จาก n inputs ไปยัง 1 output โดยใช้ select lines

สำหรับ 4-to-1 MUX:

$$Y = I_0 \cdot S'_0 \cdot S'_1 + I_1 \cdot S_0 \cdot S'_1 + I_2 \cdot S'_0 \cdot S_1 + I_3 \cdot S_0 \cdot S_1$$

S_1	S_0	Output
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

ทฤษฎีบท 6.69 **Demultiplexer (DEMUX)** ทำงานตรงข้ามกับ MUX คือรับ 1 input และส่งไปยัง 1 จาก n outputs

6.17.6 FSM Implementation ด้วย Boolean Logic

Finite State Machine (FSM) สามารถ implement ได้โดยใช้ Boolean algebra

ตัวอย่าง 6.70 พิจารณา FSM ที่ตรวจสอบลำดับบิต "10":

State	Input=0	Input=1	Output
S0 (initial)	S0	S1	0
S1 (saw 1)	S0	S1	0
S2 (saw 10)	S0	S1	1

กำหนด state encoding: S0=00, S1=01, S2=10

State transitions และ outputs สามารถเขียนเป็น Boolean expressions ได้

สรุป

ในบทนี้เราได้ศึกษาพีชคณิตบูลีนอย่างครอบคลุม ซึ่งประกอบด้วย:

1. **นิยามของพีชคณิตบูลีน** คือ Boolean algebra ($B, +, \cdot, ', 0, 1$) พร้อม axioms
2. **การดำเนินการทางบูลีน** คือ OR (+), AND ($\cdot$), NOT ($'$) พร้อม truth tables
3. **Boolean Identities** คือ Idempotent, Identity, Domination, Complement, Commutative, Associative, Distributive, Absorption
4. **กฎของเดมอร์แกน** คือ $(a + b)' = a' \cdot b'$ และ $(a \cdot b)' = a' + b'$
5. **ฟังก์ชันบูลีน** คือ นิพจน์บูลีนและ truth tables
6. **Canonical Forms** คือ Sum of Products (SOP) และ Product of Sums (POS)
7. **Karnaugh Maps** คือ เครื่องมือในการลดรูปฟังก์ชันบูลีน
8. **การประยุกต์ในวิทยาการคอมพิวเตอร์** คือ K-map, Boolean logic, Logic gates, Adders, MUX/DEMUX, FSM

พีชคณิตบูลีนเป็นเครื่องมือที่จำเป็นสำหรับการศึกษาดังต่อไปนี้ โดยเฉพาะ ทฤษฎีการคำนวณ (Theory of Computation) ซึ่งใช้ Boolean circuits เป็นพื้นฐาน และ การออกแบบวงจรดิจิทัล (Digital Circuit Design) ซึ่งใช้หลักการของ Boolean algebra โดยตรง

ความเชื่อมโยงระหว่างพีชคณิตบูลีน ตรรกศาสตร์ และทฤษฎีเซตเป็นสิ่งสำคัญที่แสดงให้เห็นว่า โครงสร้างทางคณิตศาสตร์ต่างๆ มีความสัมพันธ์กันอย่างลึกซึ้ง

Bibliography

- [1] Rosen, Kenneth H. *Discrete Mathematics and Its Applications*. 8th ed., McGraw-Hill, 2019.
- [2] Epp, Susanna S. *Discrete Mathematics with Applications*. 5th ed., Cengage Learning, 2019.
- [3] Boole, George. *An Investigation of the Laws of Thought*. Cambridge University Press, 1854.
- [4] Huffman, D.A. *Synthesis of Sequential Switching Circuits*. Journal of the Franklin Institute, 1954.
- [5] Karnaugh, Maurice. *The Map Method for Synthesis of Combinational Logic Circuits*. Transactions of the American Institute of Electrical Engineers, 1953.
- [6] Mano, Morris. *Digital Logic and Computer Design*. Prentice Hall, 1979.
- [7] Tocci, Ronald, Neal Widmer, and Greg Moss. *Digital Systems: Principles and Applications*. 12th ed., Pearson, 2017.

ผลการอ้างอิง